Notes on Machine Learning for Kira IBP Reduction

Seed selection, elimination ordering and equation selection

October 12, 2026

Abstract

Feynman integral calculations are an important component of multiloop amplitude calculations in quantum field theory. In Quantum Chromodynamics (QCD), multiloop amplitudes and their associated Feynman integrals arise in higher-order perturbative calculations and can become increasingly complex at higher loop orders. Recent progress has automated these calculations to some extent using the framework Kira, which organizes target integrals into integral families and sectors, generate algebraic relations among the integrals by using integration by parts (IBP) identities, and reduce the resulting system to expressions in terms of a finite set of master integrals. We aim to explore the application of machine learning models to accelerate the IBP reduction process by investigating if learned models can improve seed selection. These models can predict the most useful integrals from which to generate IBP relations while minimizing redundant calculations. We will also explore machine learning models for learning efficient reduction and elimination sequences to minimize the number of algebraic operations required to express target integrals in terms of master integrals. Because the machine learning models will serve as decision-makers while the underlying IBP reduction and verification remain exact, these models can improve computational efficiency without compromising the correctness of the resulting reductions.

Current measured method. The target-batch comparison in Section 33 gives 1.5% more time than the previous undivided workflow across 96 matched batches with up to 11 targets. Its additional change relative to fixed partitions with the same filters is 0.8% more time, with ratio interval $[0.988, 1.035]$. Validation disables the new predictor, so this comparison establishes no additional ML gain from batch selection. Every successful reduction passes exact coefficient checks. This is a separate workload from the smaller-batch filter study. The separate analytic specialization reduces scalar massless banana targets through one corner master, with a derived coefficient valid at every positive loop count. Its 9 completed native comparisons give a speedup of 21.3–357.8 times for the stated targets. These gains come from exact mathematics and are separate from the ML gains (Section 35).

1 Introduction

Feynman integrals arise in higher-order perturbative corrections to scattering amplitudes and particle decay amplitudes. In QCD, these corrections involve quark and gluon loops and contribute to predictions for cross sections and decay rates. Calculating a loop contribution requires integration over the independent internal momenta that remain after imposing momentum conservation [1].

The scalar integrals that appear in an amplitude have the general form

$$\mathcal{J} = \left(\prod_{k=1}^{L} \int \frac{\mathrm{d}^d \ell_k}{\mathrm{i}\pi^{d/2}}\right) \frac{\mathcal{N}(\boldsymbol{\ell}, \boldsymbol{p})}{\prod_{i=1}^{n_{\mathrm{phys}}} D_i^{a_i}}, \qquad D_i = q_i^2 - m_i^2 + \mathrm{i}0. \tag{1}$$

Here L is the number of loops and $\boldsymbol{\ell} = (\ell_1, \ldots, \ell_L)$ is the list of independent loop momenta over which we integrate. The number of independent external momenta is E, and we keep these momenta, $\boldsymbol{p} = (p_1, \ldots, p_E)$, fixed during integration. The numerator $\mathcal{N}$ is a polynomial obtained from the spin and tensor algebra; we leave its mass dependence implicit in the notation. The number of physical denominators is n_{phys}, and each D_i corresponds to an internal particle line with momentum q_i and mass m_i. The positive integer a_i is the power of D_i in the denominator, and q_i is a fixed linear combination of loop and external momenta, with $q_i^2 = q_i \cdot q_i$ denoting the Lorentz scalar product.

The integration dimension is $d = 4 - 2\epsilon$, where ϵ is the regulator used to define divergent integrals, so the result depends on ϵ as well as the external momenta and masses. The factor $1/(\mathrm{i}\pi^{d/2})$ is our normalization convention for each integration measure, with $\mathrm{i}^2 = -1$. In each denominator, i0 is an infinitesimal positive imaginary term that specifies how to integrate past a propagator pole, where $q_i^2 = m_i^2$.

Higher loop orders produce collections of integrals with related numerators and denominators and algebraic identities can be used to express them in terms of an independent set of master integrals. Kira automates this reduction [2], after which the calculation requires the values of the master integrals. To explain how the reduction works and where machine learning (ML) could speed it up, we first define the notation used in this paper, then derive the identities and describe how Kira solves the resulting equations.

2 The mathematical form of Feynman integrals

To label the integrals related to Eq. (1), we specify their momentum dependence and write the numerator and denominator powers as a vector of integers. This vector identifies each integral as an unknown in the reduction equations.

2.1 Loop momenta, propagators and regularization

After choosing how the loop and external momenta flow through the internal lines, we write each physical denominator as

$$D_i = q_i^2 - m_i^2 + \mathrm{i}0, \qquad q_i = \sum_{k=1}^{L} \alpha_{ik} \ell_k + \sum_{\beta=1}^{E} \eta_{i\beta} p_\beta. \tag{2}$$

The constants α_{ik} and $\eta_{i\beta}$ specify which loop and external momenta make up q_i, with indices k, j for loop momenta and β, γ for external momenta. Squaring q_i gives the scalar products $\ell_k \cdot \ell_j$, $\ell_k \cdot p_\beta$ and $p_\beta \cdot p_\gamma$, and those containing loop momenta vary during integration. The

vector $\boldsymbol{x}$ contains the independent external scalar products and squared masses, which are the kinematic parameters we keep fixed during integration.

Loop integrals can diverge at large momenta or when massless momenta become small or parallel. Dimensional regularization defines these integrals at values of d for which they converge and extends the result by analytic continuation to $d = 4 - 2\epsilon$, using additional regulators where necessary. In an expansion around $\epsilon = 0$, the divergences appear as negative powers of ϵ, which renormalization and infrared cancellation in the observable address later in the calculation [1]. Having specified the regulated measure and momentum dependence, we can now use the powers of the numerator and denominator factors to label individual integrals.

2.2 A common notation for an integral family

We add auxiliary expressions to the physical denominators so that we can write every required loop-momentum scalar product in terms of the list $D_1, \ldots, D_N$ and the fixed parameters $\boldsymbol{x}$. Here N is the number of expressions in the list, and the auxiliary expressions allow us to represent numerator factors. Writing the numerator in terms of this list expresses $\mathcal{J}$ as a linear combination of integrals of the form

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \int_\ell \prod_{i=1}^{N} D_i^{-a_i}, \qquad \int_\ell \equiv \prod_{k=1}^{L} \int \frac{\mathrm{d}^d \ell_k}{\mathrm{i}\pi^{d/2}}, \qquad \boldsymbol{a} = (a_1, \ldots, a_N) \in \mathbb{Z}^N. \tag{3}$$

Here $\int_\ell$ is shorthand for the loop integration measure and $\boldsymbol{a}$ is the vector of N integer powers, with $\mathbb{Z}$ denoting the set of integers. These powers can be positive, zero or negative, so we can write numerator and denominator factors in the equivalent form

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \int_\ell \frac{\prod_{i:a_i<0} D_i^{-a_i}}{\prod_{i:a_i>0} D_i^{a_i}}. \tag{4}$$

A positive index places $D_i^{a_i}$ in the denominator, a negative index places $D_i^{-a_i}$ in the numerator, and a zero index removes the factor, with an empty product equal to one. Auxiliary expressions used only as numerators therefore have nonpositive indices. An irreducible scalar product is a numerator scalar product that cannot be written as a linear combination of the physical denominators and terms depending only on $\boldsymbol{x}$ [3].

An integral family contains the integrals obtained by varying $\boldsymbol{a}$ for a fixed list $D_1, \ldots, D_N$ and fixed kinematics. Within this family, the target list specifies the integrals that the amplitude requires, and grouping family members by their positive indices organizes them into sectors.

2.3 Sectors and the organization of related integrals

A sector is a set of integrals with the same positive-index positions, so they contain the same denominators. We assign a label to each sector using

$$\theta_i(\boldsymbol{a}) = \begin{cases} 1, & a_i > 0, \\ 0, & a_i \leq 0, \end{cases} \qquad S(\boldsymbol{a}) = \sum_{i=1}^{N} 2^{i-1} \theta_i(\boldsymbol{a}). \tag{5}$$

The indicator θ_i is one when denominator i is present and zero otherwise, and S is an integer that identifies this set of denominators. Integrals with different powers of the same denominators belong to the same sector because S depends only on which indices are positive. Removing one or more denominators gives a subsector containing a proper subset of the original sector's denominators.

Scaleless sectors contain no nonzero mass or kinematic scale and therefore vanish in dimensional regularization, while changes of loop integration variables can yield symmetry relations between integrals. Kira uses these zero sectors and symmetry mappings to reduce the number of unknowns [2]. We can then use the sector and index labels to write the identities that relate the remaining family members.

3 From integration to algebraic reduction

To express the targets in terms of master integrals, we generate relations between family members through integration by parts and solve the resulting equations. Evaluating the master integrals then gives the target integral values needed to calculate the amplitude.

3.1 How integration by parts produces relations

For differentiable functions $f(z)$ and $g(z)$ of a real variable z, the product rule gives the integration identity between endpoints z_0, z_1:

$$\int_{z_0}^{z_1} f'(z)g(z)\,\mathrm{d}z = [f(z)g(z)]_{z_0}^{z_1} - \int_{z_0}^{z_1} f(z)g'(z)\,\mathrm{d}z. \tag{6}$$

Primes denote derivatives with respect to z, and the brackets denote the endpoint difference $f(z_1)g(z_1) - f(z_0)g(z_0)$. To apply this product rule to loop-momentum derivatives, we derive the integration-by-parts (IBP) identity where the regulated integral has a vanishing boundary term and then extend it by analytic continuation [4, 3]. For a loop momentum ℓ_k and a vector $v \in \{\ell_1, \ldots, \ell_L, p_1, \ldots, p_E\}$, the identity is

$$0 = \int_\ell \frac{\partial}{\partial \ell_k^\mu} \left(v^\mu \prod_{i=1}^{N} D_i^{-a_i} \right). \tag{7}$$

The index μ is a spacetime index, and a repeated Lorentz index means that we sum over components using the spacetime metric. The derivative acts on both v^μ and the factors $D_i^{-a_i}$, so the product rule gives

$$0 = \int_\ell \left[\frac{\partial v^\mu}{\partial \ell_k^\mu} - \sum_{i=1}^{N} a_i \frac{v^\mu (\partial D_i / \partial \ell_k^\mu)}{D_i} \right] \prod_{j=1}^{N} D_j^{-a_j}. \tag{8}$$

The first term equals d for $v = \ell_k$ and vanishes when v is an external momentum or another independent loop momentum. Differentiating an inverse propagator produces scalar products, which we express through the family basis to convert each term into an integral with shifted indices:

$$\sum_{j=1}^{n_\lambda} c_{\lambda j}(\boldsymbol{a}; \boldsymbol{x}, \epsilon) I_{\boldsymbol{a}+\boldsymbol{\Delta}_{\lambda j}}(\boldsymbol{x}, \epsilon) = 0. \tag{9}$$

Here λ identifies the chosen pair (k, v), n_λ is the number of terms, and $\boldsymbol{\Delta}_{\lambda j}$ is the integer change in the index vector for term j. Differentiation gives the coefficient $c_{\lambda j}$, which depends on the integral indices and the parameters $\boldsymbol{x}, \epsilon$ [4].

The L choices of differentiated momentum and $L+E$ choices of v give $L(L+E)$ basic identities at each index vector, some of which may depend linearly on the others. Further relations follow from Lorentz invariance because a common rotation or Lorentz boost of all external momenta preserves a scalar integral, giving the infinitesimal identity

$$\sum_{\beta=1}^{E} \left(p_\beta^\nu \frac{\partial}{\partial p_{\beta\mu}} - p_\beta^\mu \frac{\partial}{\partial p_{\beta\nu}} \right) I_{\boldsymbol{a}} = 0. \tag{10}$$

Here μ, ν are free spacetime indices, meaning that the identity holds for each pair of components, and the derivatives act on the external-momentum dependence of the integral. Contracting these indices with external momenta gives scalar relations [5] that join the IBP and symmetry identities to form the reduction system.

3.2 From relations to master integrals

Let n_I be the number of distinct integrals in the generated equations and n_{eq} the number of equations. We arrange these integrals in a column vector so that the system takes the matrix form

$$A(\boldsymbol{x}, \epsilon)\, \boldsymbol{I} = 0, \qquad \boldsymbol{I} = (I_{\boldsymbol{a}^{(1)}}, \ldots, I_{\boldsymbol{a}^{(n_I)}})^{\mathsf{T}}. \tag{11}$$

The superscript T denotes transpose, so $\boldsymbol{I}$ is a column vector of integrals whose dependence on $\boldsymbol{x}, \epsilon$ we leave implicit here. The matrix A has n_{eq} rows and n_I columns, with each entry A_{ij} giving the coefficient of integral j in equation i. Solving this system expresses dependent integrals in terms of an independent set. This set is a master basis if the reduction relations express every required integral as a linear combination of its members. For the standard class of Feynman integrals at fixed graph, this basis contains a finite number of masters [6]. We write them as $M_1, \ldots, M_{n_M}$, where n_M is the number of master integrals, so the reduction formula is

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \sum_{m=1}^{n_M} C_{\boldsymbol{a}m}(\boldsymbol{x}, \epsilon) M_m(\boldsymbol{x}, \epsilon). \tag{12}$$

Here $C_{\boldsymbol{a}m}$ is the coefficient of master M_m in the reduction of target $I_{\boldsymbol{a}}$. The solver obtains it by adding, multiplying and dividing the identity coefficients, so it is a rational function, meaning a ratio of polynomials in $\boldsymbol{x}, \epsilon$. Different independent sets can define different master bases, and an insufficient equation set can leave additional integrals unresolved. Establishing the master basis therefore requires checking that the generated equations are enough to reduce the required integrals.

Equation (12) gives the coefficients multiplying the masters, whose evaluation requires a further calculation, for example by solving differential equations with known values or limits as boundary conditions [5]. Kira provides the reduction rules needed for this calculation by generating and processing the identities through the procedure described next.

4 How Kira performs the reduction

Kira implements the Laporta approach by generating a finite set of identities and eliminating integrals according to an ordering [7]. To obtain the coefficients in Eq. (12), Kira selects seeds, removes redundant equations and solves the retained system. We examine how Kira performs these steps to specify the seed and elimination choices that the proposed models would make.

4.1 Kira's input and seed generation

The user supplies the families in Eq. (3), relations among the parameters $\boldsymbol{x}$, the target list, and bounds on equation generation so that Kira can initialize the families and sectors. During this initialization, Kira detects scaleless sectors and identifies symmetry relations and mappings between compatible families [2].

A seed specifies the index vector $\boldsymbol{a}$ at which Kira evaluates Eq. (9) to generate several derivative identities, and the seed set may include integrals beyond the target list. The shifts $\boldsymbol{\Delta}_{\lambda j}$ in these

identities can introduce further integrals outside the seed set, so the resulting system depends on both the chosen seeds and the terms generated from them. To set limits on the indices of the seeds, Kira uses the quantities

$$t(\boldsymbol{a}) = \sum_{i=1}^{N} \theta_i(\boldsymbol{a}), \qquad r(\boldsymbol{a}) = \sum_{a_i>0} a_i, \tag{13}$$

$$s(\boldsymbol{a}) = -\sum_{a_i<0} a_i, \qquad n_{\mathrm{dot}}(\boldsymbol{a}) = r(\boldsymbol{a}) - t(\boldsymbol{a}). \tag{14}$$

Here t is the number of distinct denominators, r is the sum of their powers, and s is the sum of the numerator powers. The quantity n_{dot} is the sum of the extra denominator powers beyond one for each denominator. We use this symbol to keep d reserved for the integration dimension, and we restrict equation generation by specifying the allowed sectors and limits

$$r \le r_{\max}, \qquad s \le s_{\max}, \qquad n_{\mathrm{dot}} \le n_{\mathrm{dot,max}}, \tag{15}$$

where $r_{\max}$, $s_{\max}$ and $n_{\mathrm{dot,max}}$ are the upper limits on the corresponding quantities. The user sets $r_{\max}$ and $s_{\max}$ and may also set $n_{\mathrm{dot,max}}$. Because these choices determine which seeds contribute equations, a reduction run must establish whether the bounds supply enough relations for the targets [8].

Kira 3 changes seeding in symmetry-related sectors and provides the optional `truncate_sp` setting to restrict numerator seeding in subsectors, with parameter choices that require checks against the expected master basis [9]. The generated system can still contain redundant equations, which elimination identifies as it processes the relations into reduction rules.

4.2 Integral ordering, sector order and forward elimination

Elimination solves an identity for one integral and substitutes the result into other identities, using an integral ordering to select which unknown to eliminate from each equation. Kira defines these orderings through sector labels and the powers of the numerator and denominator factors, with choices that prioritize numerator powers or repeated denominators and thereby affect the master basis and runtime [8].

To describe this procedure, write $I_j = I_{\boldsymbol{a}^{(j)}}$ for the integrals in Eq. (11) and choose the ordering $I_1 \succ I_2 \succ \cdots \succ I_{n_I}$, where $\succ$ gives priority for elimination. In a row whose first nonzero term contains I_k, let b_j denote the coefficient of I_j, so choosing the nonzero coefficient b_k as the pivot gives

$$b_k I_k + \sum_{j>k} b_j I_j = 0 \quad \Longrightarrow \quad I_k = -\sum_{j>k} \frac{b_j}{b_k} I_j. \tag{16}$$

The coefficients b_j depend on $\boldsymbol{x}, \epsilon$, and substituting this expression into the other rows removes their dependence on I_k. Repeating this step gives forward elimination, which builds a triangular system in which each pivot integral depends on integrals later in the ordering. Back substitution then replaces the remaining pivot integrals using their own rules, completing the Laporta solution [7].

Sector scheduling sets the order in which the solver processes groups of integrals or equations, taking into account which other integrals their equations contain. Ordinary IBP identities for a sector involve that sector and its subsectors because nonpositive indices remain nonpositive when applying the derivative. Equations for one sector can therefore depend on integrals from its subsectors. The Kira 3 description starts equation generation with sectors earlier in its ordering and inserts previously obtained pivot relations into new equations [9], with the order of processing depending on the stage of the reduction.

When several rows share the same first integral, the choice of pivot row changes the substitutions that follow. Choosing row ρ to eliminate column k, for example, updates row i by

$$A_{ij} \longleftarrow A_{ij} - \frac{A_{ik}}{A_{\rho k}} A_{\rho j}, \qquad A_{\rho k} \neq 0. \tag{17}$$

Here i is the index of a row other than the pivot row, so $i \neq \rho$, and j is the column index. Subtracting the stated multiple of row ρ sets the coefficient in column k to zero and can create nonzero entries in other columns, a change called fill-in when a previously zero entry becomes nonzero. These substitutions also make the coefficient expressions longer by combining their numerators and denominators, so elimination can require more time and memory as it proceeds. Redundant equations add to this work [10], so Kira uses modular elimination to select equations before processing their full symbolic coefficients.

4.3 Removing redundant equations and selecting target dependencies

To reduce the number of equations it must solve, Kira identifies rows that follow from other equations. Elimination reduces every coefficient in such a row to zero, allowing the solver to omit that row. Kira tests these dependencies with modular arithmetic [2] by substituting sample values $\boldsymbol{x}_0, \epsilon_0$ and mapping the coefficients to a finite field $\mathbb{F}_P$ for a prime integer P:

$$A(\boldsymbol{x}, \epsilon) \longmapsto A(\boldsymbol{x}_0, \epsilon_0) \bmod P. \tag{18}$$

Integers from 0 to $P - 1$ represent the elements of $\mathbb{F}_P$, and addition and multiplication keep results in this range by taking the remainder upon division by P. Division uses the modular inverse of a nonzero element, which multiplies that element to one modulo P, so the sampling procedure excludes points where a required denominator vanishes. This arithmetic keeps field elements bounded in size while allowing the solver to identify independent rows and retain their original symbolic identities.

The matrix rank is the largest number of independent rows, where no row is a linear combination of the others in the chosen set. Some sample values can make otherwise independent rows dependent, giving a rank smaller than that of the symbolic matrix. Checks must account for this possibility when using the probabilistic equation-selection method developed by Kant [10].

Kira also tracks which independent equations contribute to the target reductions, using the dependencies of the forward-eliminated system for this selection in Kira 3 [9]. Seed selection therefore supplies the integrals from which Kira generates equations, and equation selection then retains the relations needed for the targets so that the solver can perform the full reduction on this smaller system.

4.4 Back substitution and the final symbolic result

To obtain $C_{\boldsymbol{a}m}$ from the retained equations, Kira applies back substitution to the pivot rules produced by forward elimination. Since these rules can contain other pivot integrals, back substitution inserts their expressions recursively until the right-hand sides contain only masters, giving Eq. (12). Kira's original symbolic solver carries out this calculation using rational-function arithmetic through the computer algebra system Fermat [2].

Kira also solves systems at finite-field sample points and reconstructs the rational coefficients with FireFly, the reconstruction library introduced into the solver in Kira 2.0 [11]. In this procedure, each sampled coefficient has the form

$$C_{\boldsymbol{a}m}(\boldsymbol{x}_0, \epsilon_0) \bmod P. \tag{19}$$

The modular operations use exact arithmetic, and reconstruction recovers $C_{\boldsymbol{am}}(\boldsymbol{x}, \epsilon)$ from sufficient samples before checking the result at additional points. Reconstruction therefore forms part of the calculation of the symbolic reduction coefficients, after which the master integrals still require evaluation [11].

Because seed selection determines the equations that enter this procedure and elimination decisions determine the substitutions and intermediate expressions, the proposed ML models would guide these two operations to reduce the time and memory needed for the reduction.

5 ML for seed selection and elimination ordering

We first study two machine learning models: one for seed selection and one for elimination ordering. These models learn from recorded reductions and assign scores to choices available at the current step. We select model types by complete reduction time, including inference and exact checks. Training requires targets with known master expansions, the seeds that supply their equations and the steps used to reduce them. The implemented models and their completed training appear in Sections 7 and 9, followed by their held-out tests. Section 18 then adds a third model that selects identities generated from the retained seeds for the requested targets.

The present experiments test seed-reference agreement, whether the stored identities from the predicted seeds suffice inside Kira, and reduction of a target coefficient row. We then compare complete symbolic reductions with Kira's existing strategies. A native adapter converts predicted seeds into sector bounds for equation generation, while the row adapter applies exact symbolic target-row updates before Kira finishes the reduction. Section 11 measures their costs. Section 12 then places the row model inside Kira's evolving matrix, and Section 13 tests direct seed filtering and a compiled row policy with lower prediction costs.

5.1 Selecting seeds before equation generation

The first model receives the family definition, the target terms and their index vectors $\boldsymbol{a}$, the chosen master basis and a candidate seed list within stated sectors and index bounds. It assigns each candidate a separate probability of belonging to the target's recorded sufficient seed set, which allows several seeds to receive label one.

During training, we pair each target with the seeds used by its checked reference reduction. Different reductions can use different sufficient sets, and the stored labels identify the selected reference set. They do not establish that every omitted seed is unnecessary. For use inside Kira, we supply the stored equations associated with the predicted seeds and check whether they suffice in the chosen master basis. An unresolved target requires further identities. Section 8 measures recovery of the recorded sets and then tests their algebraic sufficiency through symbolic Kira reductions. The second test also checks the full candidate pool as a fallback.

5.2 Choosing elimination steps within the solver

The second model receives the current coefficient matrix and the supplied allowed row operations. It assigns a probability to each operation, selects one operation and reports the selected probability as its confidence. An exact row update then applies Eq. (17). The next state supplies the information for the next choice.

Each training example specifies the matrix before the choice, the allowed pivots, and the recorded source row and integral to eliminate. The retained later steps permit checks of the complete path

and its arithmetic count, while predictive features use the current state and supplied actions. In the present data, the source identities stay fixed and one target row changes during reduction. The native integration follows Kira's integral ordering and supplies allowed choices from the current evolving equations. This changes the states seen by the model and requires separate exact and runtime checks.

The computational benefit depends on the total work: preparing features and evaluating the model must cost less than the algebraic work that its choices save. The tests therefore record complete-path arithmetic and timing as well as next-step agreement. The comparison in Section 11 measures complete symbolic reductions with the selected operations.

5.3 Integral data required for training and testing

To train and test both models, we need family denominators, kinematic parameters, targets and a master basis together with the exact coefficients in Eq. (12). For a target containing several integrals, these coefficients specify the master expansion of the complete expression. Each example also identifies its source equations, actual IBP seeds and derivative operators. Algebraic identities, including mass partial fractions, remain separately identified because they carry no IBP seed.

We construct targets from known master combinations and retain the operations used to form each target and reduce it back to the same basis. The elimination records give the coefficients before and after each step, all allowed source rows, the selected pivot and the operation counts. These records connect seed selection to the equations available for elimination.

The known master coefficients provide a reference for checking complete reduction in the chosen basis, as described in Section 3 [9]. We keep these answers, the selected labels and future construction histories separate from the information used to predict a choice. Section 6 explains how we obtain the bases, construct the targets and check their seed and elimination records.

6 Target Integral Generation for Machine Learning

To train and test the two models described in Section 5, we constructed targets from master bases while retaining their source identities, seeds and algebraic steps. The initial collection contains 49 master entries and 1,662 targets across 16 families. We then extended the collection to 1,000 families, giving 9,678 master entries and 240,472 targets with checked reference reductions [12, 13, 14]. We first added 100 restricted coupled-vacuum mass profiles at each loop order from two through twelve, then 1,250 further profiles at each order from one through twelve. The complete collection contains 17,100 family instances, 57,979 master entries and 2,301,272 targets. The following subsections define the earlier construction and then derive the added vacuum bases and reverse targets. Every target retains its exact symbolic definition and master expansion.

6.1 Initial one-loop families and their kinematics

We use the integral definition and normalization in Eq. (3), with $d = 4 - 2\epsilon$ and inverse propagators $D_i = q_i^2 - m_i^2 + \mathrm{i}0$. For each family, we specify the internal momenta q_i, the masses m_i and the scalar products of the external momenta, so every index vector defines a particular integral. In the equations below we suppress the common i0 prescription. We write $\mu = m^2$ for a common nonzero squared mass and $\mu_1 = m_1^2$, $\mu_2 = m_2^2$ for two independent nonzero squared masses. The symbols x_s and x_t denote momentum invariants and remain distinct from the index sums s and t in Eq. (14).

The one-loop collection begins with a massive tadpole, whose single inverse propagator is $D_1 = \ell_1^2 - \mu$ and which has no external momentum. We then include four bubble families with external momentum p and inverse propagators

$$D_1 = \ell_1^2 - m_1^2, \qquad D_2 = (\ell_1 - p)^2 - m_2^2, \qquad p^2 = x_s. \tag{20}$$

The squared-mass pairs are $(0,0)$, $(\mu,0)$, (μ,μ) and (μ_1,μ_2), corresponding to a massless bubble, a bubble with one massive line, an equal-mass bubble and an unequal-mass bubble. We keep x_s as an independent nonzero scale, with the massive families defined at generic values of their independent parameters.

To include three-point integrals, we define triangle families by

$$\begin{aligned} &D_1 = \ell_1^2 - m_1^2, \quad D_2 = (\ell_1 - p_1)^2 - m_2^2, \qquad D_3 = (\ell_1 - p_1 - p_2)^2 - m_3^2, \\ &p_1^2 = u_1, \qquad\qquad p_2^2 = u_2, \qquad\qquad (p_1 + p_2)^2 = u_3. \end{aligned} \tag{21}$$

Here u_1, u_2, u_3 are the squared external momenta, also called external virtualities, and $p_1 \cdot p_2 = (u_3 - u_1 - u_2)/2$ fixes the remaining external scalar product. We use three massless families with $(u_1,u_2,u_3) = (0,0,x_s)$, $(0,x_2,x_3)$ and (x_1,x_2,x_3). The parameters x_1, x_2, x_3 denote independent nonzero virtualities. We also use two families with $m_1^2 = m_2^2 = m_3^2 = \mu$ and virtualities $(0,0,x_s)$ and (x_1,x_2,x_3). Thus the five triangle families vary both the internal masses and the number of nonzero external virtualities.

For four-point integrals, we use three massless box families with $p_4 = -p_1 - p_2 - p_3$ and

$$\begin{aligned} &D_1 = \ell_1^2, \qquad\qquad D_2 = (\ell_1 - p_1)^2, \\ &D_3 = (\ell_1 - p_1 - p_2)^2, \quad D_4 = (\ell_1 - p_1 - p_2 - p_3)^2, \\ &x_s = (p_1 + p_2)^2, \qquad x_t = (p_2 + p_3)^2. \end{aligned} \tag{22}$$

Writing $u_i = p_i^2$ for $i = 1, \ldots, 4$, we fix the independent external scalar products through

$$p_1 \cdot p_2 = \frac{x_s - u_1 - u_2}{2}, \qquad p_2 \cdot p_3 = \frac{x_t - u_2 - u_3}{2}, \qquad p_1 \cdot p_3 = \frac{u_4 + u_2 - x_s - x_t}{2}. \tag{23}$$

The virtuality choices are $(0,0,0,0)$, $(0,0,0,x_4)$ and (x_1,x_2,x_3,x_4), where x_4 is the fourth nonzero virtuality parameter. These choices give boxes with zero, one and four off-shell legs; in this classification an on-shell external leg has $p_i^2 = 0$. We treat the remaining invariants as independent subject to the stated relations. These definitions give 13 one-loop families, which we extend with the following three two-loop families.

6.2 Initial two-loop families and auxiliary expressions

The two-loop families use loop momenta ℓ_1, ℓ_2. For a massless two-point integral with external momentum p and $p^2 = x_s$, we take

$$(D_1, D_2, D_3, D_4, D_5) = (\ell_1^2, \ell_2^2, (\ell_1 - \ell_2)^2, (\ell_1 - p)^2, (\ell_2 - p)^2). \tag{24}$$

These five expressions determine the five loop-dependent scalar products $\ell_1^2, \ell_2^2, \ell_1 \cdot \ell_2, \ell_1 \cdot p$ and $\ell_2 \cdot p$. Varying their indices therefore includes numerator insertions as well as repeated propagators.

For the equal-mass sunrise with the same external invariant, we use

$$\begin{aligned} &D_1 = \ell_1^2 - \mu, \qquad D_2 = \ell_2^2 - \mu, \qquad D_3 = (p - \ell_1 - \ell_2)^2 - \mu, \\ &D_4 = (p - \ell_1)^2, \quad D_5 = (p - \ell_2)^2, \quad a_4, a_5 \le 0. \end{aligned} \tag{25}$$

The first three expressions are the physical inverse propagators. The auxiliary expressions D_4, D_5 complete the five scalar products needed to express the derivatives in the IBP identities. Their nonpositive indices restrict them to numerator factors or their absence, so the top sector contains only D_1, D_2, D_3 as denominators. We keep x_s and μ independent.

The final family is the equal-mass two-loop vacuum sunset, which has no external momentum and uses

$$D_1 = \ell_1^2 - \mu, \qquad D_2 = \ell_2^2 - \mu, \qquad D_3 = (\ell_1 - \ell_2)^2 - \mu. \tag{26}$$

These three expressions determine all loop scalar products for this vacuum family. For every family, we include the sector containing all physical propagators and all its subsectors, so the resulting basis also accounts for integrals with removed propagators. The number of physical propagators in this top sector is denoted by t_{top}. It is one, two, three and four for the tadpole, bubble, triangle and box families, respectively, five for the massless two-loop propagator, and three for the sunrise and vacuum families. This number sets the initial range of identities used to identify the masters.

6.3 Identification and checks of the master bases

For each initial family, we used Kira 3.1 to generate IBP relations together with the applicable Lorentz-invariance and symmetry relations. Kira removes scaleless sectors and uses finite-field elimination to identify the integrals that remain independent in the equations needed for a selected integral range [2, 9]. We used Kira's integral ordering 1 throughout, which orders sectors by their labels and prefers numerator insertions over extra denominator powers when selecting representatives within a sector [8]. Keeping this ordering fixed gives a specified basis against which to compare later calculations.

We first generated identities from seeds satisfying

$$r \le t_{\text{top}} + 2, \qquad s \le 2, \qquad n_{\text{dot}} \le 2, \tag{27}$$

and selected integrals to probe the basis within

$$r \le t_{\text{top}} + 1, \qquad s \le 1, \qquad n_{\text{dot}} \le 1. \tag{28}$$

Here r is the sum of positive indices, s is minus the sum of negative indices and $n_{\text{dot}} = r - t$ is the number of extra denominator powers, as defined in Eq. (14). Both sets of limits apply to the top sector and its subsectors. The probe integrals specify the range for which Kira must identify the required masters, while the wider seed range supplies relations that can eliminate the additional integrals introduced by index shifts. These probes serve the basis calculation; the ML examples come from the subsequent reverse construction.

Kira evaluated the coefficients of these relations over a finite field using the substitution in Eq. (18). For the first calculation we used the prime $P_1 = 2147483647$ and dimension value $d_1 = 37$. To specify the kinematic substitutions, define the ordered parameter list

$$\boldsymbol{z} = (x_s, x_t, x_1, x_2, x_3, x_4, \mu, \mu_1, \mu_2). \tag{29}$$

The first sample was

$$\boldsymbol{z}^{(1)} = (-13, -19, -11, -17, -29, -43, 7, 7, 23), \tag{30}$$

using only the entries present in each family and retaining all zeros and equal-mass conditions in its definition. We interpret these assignments modulo P_1 when evaluating the identity coefficients. The dimension sample corresponds to $\epsilon_1 = (4 - d_1)/2$ in the same field. These

substitutions test algebraic relations; the master labels continue to refer to integrals at symbolic $\boldsymbol{x}, \epsilon$.

We then enlarged the seed and probe ranges to check whether the initial basis remained sufficient:

$$\begin{aligned} \text{seeds:} \quad & r \le t_{\text{top}} + 3, \qquad s \le 3, \qquad n_{\text{dot}} \le 3, \\ \text{probes:} \quad & r \le t_{\text{top}} + 2, \qquad s \le 2, \qquad n_{\text{dot}} \le 2. \end{aligned} \tag{31}$$

We repeated the calculation first at the original sample and then at a second sample with prime $P_2 = 1000000007$, dimension $d_2 = 41$ and

$$\boldsymbol{z}^{(2)} = (-31, -47, -23, -37, -53, -71, 11, 11, 29). \tag{32}$$

Changing the prime, the dimension and the independent kinematic parameters provides a second check of the linear dependencies, since particular substitutions can lower the rank of the symbolic coefficient matrix [10].

In both enlarged calculations, we supplied the original masters as the preferred basis and enabled Kira's check for additional or unused masters [9]. We required the complete list of master index vectors to agree in all three calculations, and also checked each vector against its family definition and sector label. All 16 families satisfied these requirements, giving 48 completed master-identification calculations. The agreement provides evidence for basis stability over the tested index ranges and sample points. Completeness for arbitrary integer indices would require further relations or an independent argument, and special kinematic limits can change the basis.

We stopped each calculation after master identification. This stage determines which symbolic integrals form the starting basis within the checked domains. We then obtained rational reduction coefficients for the constructed targets through the algebraic procedure described below. The numerical values and ϵ expansions of the master integrals remain separate from this basis determination.

6.4 Resulting bases and the starting coefficients

The massive tadpole gave one master. The massless, one-mass, equal-mass and unequal-mass bubbles gave one, two, two and three masters, respectively. The massive bubble bases include the necessary tadpole subsectors, with two distinct tadpoles when the internal masses differ.

The massless triangles with one, two and three nonzero external virtualities gave one, two and four masters, respectively. For the first two families, Kira chose bubble integrals in subsectors as the masters. The three-virtuality family also retained a triangle integral. The equal-mass triangles with one and three nonzero virtualities gave three and five masters. The massless boxes with zero, one and four off-shell legs gave three, four and eleven masters, respectively, including the required lower sectors in each count.

The massless two-loop propagator gave two masters, the equal-mass sunrise gave three, and the equal-mass vacuum sunset gave two. For the denominator order in Eq. (25), the sunrise basis is

$$M_1 = I_{(1,1,0,0,0)}, \qquad M_2 = I_{(1,1,1,0,0)}, \qquad M_3 = I_{(1,1,1,-2,0)}. \tag{33}$$

The first integral factorizes into a product of two massive tadpoles because removing D_3 separates the loop integrations. The other two integrals retain all three sunrise propagators, and M_3 contains the numerator factor D_4^2. This example shows how the chosen ordering can retain a numerator integral as a master.

The counts sum to 49 master entries across these 16 initial families. We count each basis within its own family, so shared tadpole and bubble subsectors can appear in more than one family.

The complete family definitions and master index vectors are available with the accompanying data [12]. Each entry specifies the denominators, kinematics, index vector, sector and mass dimension. With the normalization in Eq. (3), the latter is

$$[I_{\boldsymbol{a}}]_{\text{mass}} = Ld - 2\sum_{i=1}^{N} a_i, \tag{34}$$

where $[I_{\boldsymbol{a}}]_{\text{mass}}$ is the power of mass carried by the integral. Each loop measure contributes d powers and each factor $D_i^{-a_i}$ contributes $-2a_i$. This dimension constrains the coefficients used to combine integrals in the target construction.

For that construction, we kept the identified masters symbolic and initialized each master as itself. If $\boldsymbol{\mu}^{(m)}$ is the index vector of master M_m, the starting coefficients are

$$M_m = I_{\boldsymbol{\mu}^{(m)}}, \qquad C_{\boldsymbol{\mu}^{(m)}n} = \delta_{mn}, \qquad m, n = 1, \ldots, n_M. \tag{35}$$

Here n_M is the number of masters in the selected family, and δ_{mn} is one when $m = n$ and zero otherwise. Thus each master has coefficient one for itself and zero for the others. We held the family definition and basis fixed while constructing its targets, starting with these masters and extending the collection through the identities below. Each accepted target therefore has a specified reference basis and an exact master expansion.

6.5 Source identities and the reference integral order

To extend the starting masters into targets, we first generated relations among the integrals of each family. We used the IBP identity in Eq. (7) with seeds in every nonempty physical sector within the bounds of Eq. (27). The auxiliary sunrise indices remained nonpositive. We fixed this candidate seed set before constructing targets, and retained each equation with its seed $\boldsymbol{\sigma}$, differentiated loop momentum ℓ_k and vector v [4, 2]. The resulting equations can contain integrals beyond the seed bounds because differentiation shifts the indices.

The vectors v included loop momenta, external momenta and propagator momenta q_i. Each propagator momentum is a fixed linear combination of loop and external momenta, so its IBP equation is the corresponding linear combination of the usual IBP equations at the same seed. We retained this decomposition to specify how each equation follows from its source operators. We also obtained symmetry relations by explicit changes of loop variables whose loop transformation matrix has determinant $+1$ or -1. We accepted a transformation when it permuted the inverse propagators with their masses unchanged and preserved the physical and auxiliary index positions.

To give the constructions a reduction order, we protected the chosen masters from elimination and ordered every other integral by

$$(S(\boldsymbol{a}),\, n_{\text{dot}}(\boldsymbol{a}),\, s(\boldsymbol{a}),\, r(\boldsymbol{a}),\, \boldsymbol{a}). \tag{36}$$

Here S is the sector number in Eq. (5), and the remaining quantities are the index sums in Eq. (14). We compare the entries in the displayed order, using the first unequal entry to decide which integral is larger. We compare the components of $\boldsymbol{a}$ in the same way when all earlier entries agree. The leading integral of an equation is its largest integral outside the master basis. This reference order specifies the construction and recorded reductions; adapting the elimination model to Kira will require its actual ordering and available pivot choices.

6.6 Reverse construction from combinations of masters

To obtain target expressions with known answers, we selected subsets of masters from the same family and replaced their terms through the source identities. Let $\mathcal{J} = \{m_1, \ldots, m_h\}$ be the selected master labels, where h is the number of chosen masters. The initial expression is

$$\mathcal{T}_0 = \sum_{m \in \mathcal{J}} c_m(\boldsymbol{x}, d) M_m, \qquad 1 \le h \le \min(3, n_M). \tag{37}$$

The coefficients are rational functions of the independent kinematic parameters and $d = 4 - 2\epsilon$, with no dependence on the loop momenta. Writing them in terms of d is equivalent to writing them in terms of ϵ. Their mass dimensions must compensate for differences between the chosen masters so that every term in $\mathcal{T}_0$ has the same dimension.

For this purpose, let $B_m = \sum_i \mu_i^{(m)}$ be the sum of the indices of master M_m, and set $A = B_{m_1}$. For a fixed nonzero squared mass or momentum invariant Q^2 in the family, we used

$$c_{m_j} = u_j(d)(Q^2)^{B_{m_j} - A}, \qquad [\mathcal{T}_0]_{\text{mass}} = Ld - 2A. \tag{38}$$

Here $j = 1, \ldots, h$ identifies a selected master and u_j is dimensionless. We varied the weights through the three forms

$$\begin{gathered} u_j(d) \in \left\{ \eta_j,\ \eta_j \frac{d-2}{d-4},\ \eta_j \frac{j+1}{d-2} \right\}, \\ \eta_j \in \{-3, -2, -1, 1, 2, 3\}. \end{gathered} \tag{39}$$

Here η_j is a nonzero integer weight. For each sequence we chose one of these three forms and varied the integer weights and selected masters. The power of Q^2 in Eq. (38) gives c_m mass dimension $2(B_m - A)$, so Eq. (34) gives the stated common dimension.

To replace a term, write the current expression and a source IBP identity as

$$\mathcal{T}_n = \sum_{\boldsymbol{b}} w_{\boldsymbol{b}}^{(n)} I_{\boldsymbol{b}}, \qquad R = \sum_{\boldsymbol{b}} r_{\boldsymbol{b}} I_{\boldsymbol{b}} = 0. \tag{40}$$

Here n is the number of completed replacements, while $w_{\boldsymbol{b}}^{(n)}$ and $r_{\boldsymbol{b}}$ are the coefficients of integral $I_{\boldsymbol{b}}$ in the expression and identity. The sums contain their finitely many nonzero terms. For a present term $I_{\boldsymbol{a}}$, we selected an identity generated at seed $\boldsymbol{a}$ with $r_{\boldsymbol{a}} \neq 0$, and formed

$$\mathcal{T}_{n+1} = \mathcal{T}_n - \frac{w_{\boldsymbol{a}}^{(n)}}{r_{\boldsymbol{a}}} R. \tag{41}$$

This operation cancels the chosen term and introduces other family integrals. Since $R = 0$, the resulting expression has the same master expansion as $\mathcal{T}_0$. We required the identity to introduce a leading integral larger than every integral already present, so the recorded steps can be reversed through the order in Eq. (36).

For the initial collection, we attempted 24 sequences per family, with at most four replacements per sequence, and retained accepted intermediate expressions. A sequence stopped when no admissible replacement was found within the construction limits. We removed identical target expressions and retained only nonzero targets with complete reference reductions. When the final expression contained one integral with coefficient $q \neq 0$, we divided the expression and its master coefficients by q and recorded this normalization. Expressions containing several index vectors remain targets for reduction of the complete sum. To obtain further individual integrals, we also applied the source identities in the following way.

6.7 Constructing individual integrals from known reductions

Starting from Eq. (35), we searched for a source identity whose leading integral was the only term without a known master expansion. Such an identity has the form

$$h_0 I_{\boldsymbol{b}} + \sum_{j=1}^{n_R} h_j I_{\boldsymbol{a}^{(j)}} = 0, \qquad h_0 \neq 0, \tag{42}$$

where $I_{\boldsymbol{b}}$ is the new integral, n_R is the number of other terms and h_0, h_j are the identity coefficients. Substituting the known reductions of the other terms gives

$$I_{\boldsymbol{b}} = \sum_{m=1}^{n_M} C_{\boldsymbol{b}m} M_m, \qquad C_{\boldsymbol{b}m} = -\sum_{j=1}^{n_R} \frac{h_j}{h_0} C_{\boldsymbol{a}^{(j)}m}. \tag{43}$$

We added each resulting expansion to the known set so later identities could use it. If all its master coefficients vanished, we retained the zero-integral proof as a dependency for subsequent constructions. The target collection contains the accepted nonzero examples.

For the initial collection, we made at most five passes through the source equations and allowed at most 64 new nonzero constructions of this type per family. An equation with several still-unknown integrals became usable only after other identities supplied the missing expansions. Solving remaining coupled systems would extend this construction beyond the individual integrals obtained here. Both this procedure and the reverse replacements above preserve the source equations and their dependencies, which determine the seed and elimination records.

6.8 Seed records and elimination sequences

For each accepted target, we retained its exact master coefficients together with the source identities used to obtain them. An IBP identity contributes its seed and derivative operator, while a symmetry identity contributes its loop-variable transformation and integral mapping. Dependencies carry the earlier identities needed to reduce intermediate integrals or establish that they vanish. For an IBP step of the form in Eq. (42), the sufficient seed set therefore obeys

$$\mathcal{S}_{\boldsymbol{b}} = \{\boldsymbol{\sigma}\} \cup \bigcup_{j=1}^{n_R} \mathcal{S}_{\boldsymbol{a}^{(j)}}, \qquad \mathcal{S}_{\boldsymbol{\mu}^{(m)}} = \varnothing. \tag{44}$$

Here $\mathcal{S}_{\boldsymbol{a}}$ is the seed set recorded for $I_{\boldsymbol{a}}$, $\boldsymbol{\sigma}$ is the seed of the current IBP identity and $\varnothing$ is the empty set. Symmetry steps retain the dependency sets and their symmetry record. For reverse constructions, the seed set is the union of the seeds of the identities used along the recorded path.

We retained the full candidate seed set alongside each family so the seed model can compare possible choices. The recorded subset supplies enough equations for the known reduction, and seeds outside it can still contribute to other valid reductions. For the model inputs, we stored the family, basis, target coefficients and index information, including the sector and $t, r, s, n_{\mathrm{dot}}$. We kept the exact answer, construction history and reference seed choices separately.

To obtain elimination examples, we represented the target as a row of coefficients $T_{\boldsymbol{b}}$ multiplying $I_{\boldsymbol{b}}$. At each step, we selected its largest remaining integral outside the master basis, denoted by $I_{\boldsymbol{a}}$. The available pivot rows were the retained source identities with leading integral $I_{\boldsymbol{a}}$ and a nonzero coefficient for that integral. For the chosen row R, the update was

$$T_{\boldsymbol{b}} \longleftarrow T_{\boldsymbol{b}} - \frac{T_{\boldsymbol{a}}}{r_{\boldsymbol{a}}} r_{\boldsymbol{b}}. \tag{45}$$

This cancels $I_{\boldsymbol{a}}$ from the target and can introduce only smaller integrals or masters. Repeating the recorded choices gives the target's known master expansion. The source equations remain unchanged during this target-row reduction, so the current row, source equations and reference order specify the state seen by the elimination model.

For every step, we retained the row before and after the update, all available pivot rows, the selected equation and integral, the multiplier, and the numbers of new and cancelled nonzero entries. We kept the coefficients as exact rational functions and also stored numerical row states over a finite field. The first sample in Eq. (30), with $P_1 = 2147483647$ and $d_1 = 37$, supplied these numerical states. Substituting parameters into the coefficient functions leaves the integrals symbolic and uses exact modular arithmetic for the row operations [11].

For a selected source row with n_{row} nonzero terms, we counted one finite-field division and n_{row} multiplications and subtractions. Summing these counts gives the cost of the recorded target-row sequence. Equation generation, forming combined IBP operators, symbolic simplification, model inference and reconstruction fall outside this count. The construction gives a valid reference path and an upper bound on the minimum cost under these rules. Establishing a minimum would require a separate argument covering alternative seed sets and elimination paths.

6.9 Initial target collection and consistency checks

The initial collection contains 1,662 accepted targets across its 16 families, with 309 individual integrals, 1,353 integral expressions and 5,138 recorded elimination steps [13]. We divided the examples by complete families into 1,290 training targets, 185 validation targets and 187 test targets. Validation targets belong to the unequal-mass bubble and massless triangle with three off-shell legs. Test targets belong to the massless box with four off-shell legs and the equal-mass sunrise. These held-out targets contain terms in the full physical sector, while their basis definitions still include the required lower sectors.

In this initial collection, the validation set consists entirely of integral expressions, and the test set contains one individual integral and 186 expressions. These initial counts motivate the additional held-out individual constructions described below. This initial collection covers the specified one-loop and two-loop families and the stated construction bounds. The following sections describe seed-model training and its held-out comparison on the expanded collection. Section 9 describes the reduction model. Section 10 measures complete target-row sequences and their selection costs. Section 11 compares complete symbolic reductions, including native Kira seeding.

Before accepting the collection, we rederived all 55,654 stored source equations from their family definitions and checked the recorded symmetry transformations. We also checked 2,699 used IBP equations by independently differentiating the original quadratic expressions for the propagators in terms of loop scalar products. We then replayed every target construction and elimination sequence with exact rational functions, including intermediate zero-integral dependencies and final normalizations. Each final result agreed with its stored master expansion, and the checks also confirmed mass dimensions, seed records and allowed pivots.

To check the sampled coefficients, we repeated every elimination sequence at three finite-field points. The first two were the samples in Eqs. (30) and (32), with the primes and dimension values specified there. The third used $P_3 = 1000000009$, $d_3 = 43$ and

$$\boldsymbol{z}^{(3)} = (-59, -73, -41, -61, -83, -101, 17, 17, 47), \tag{46}$$

in the parameter order of Eq. (29). Each family used its own parameters and retained its prescribed zeros and equal masses. All required divisions were defined at the three points, and every sampled reduction reproduced the reference coefficients.

The exact symbolic checks establish the recorded reductions from their source identities at generic parameters, while the three-point checks verify the numerical data supplied with them. This construction already provides the symbolic coefficient functions, so obtaining the reference answers requires no rational-function reconstruction. The accompanying data contain the targets, equations, construction histories, sufficient seeds and elimination records needed to train and test the two proposed models [13].

6.10 Extension to different mass and kinematic conditions

To supply examples across a wider range of parameter conditions, we added 984 one-loop families with four physical propagators. For these families, we write

$$D_i = (\ell_1 - Q_i)^2 - \mu_{\alpha_i} + \mathrm{i}0, \qquad i = 1, \ldots, 4, \qquad Q_1 = 0. \tag{47}$$

Here Q_2, Q_3, Q_4 are three independent external momentum vectors, $\mu_0 = 0$ and μ_1, μ_2, μ_3 are independent nonzero squared masses. Each label $\alpha_i \in \{0, 1, 2, 3\}$ assigns the squared mass of the corresponding propagator. Repeated positive labels impose equal masses.

We specify the external kinematics through the six squared differences between the propagator shifts:

$$\Delta_{ij} = (Q_i - Q_j)^2 = y_{\beta_{ij}}, \qquad 1 \le i < j \le 4. \tag{48}$$

Here $y_0 = 0$, y_1, y_2, y_3 are independent nonzero momentum invariants, and $\beta_{ij} \in \{0, 1, 2, 3\}$. Repeated labels impose equality between the corresponding invariants. The mass parameters and momentum invariants remain independent of one another, and a family uses only the parameters that occur in its assignments.

These distances determine the external scalar products:

$$G^{\mathrm{ext}}_{ij} = Q_i \cdot Q_j = \frac{\Delta_{1i} + \Delta_{1j} - \Delta_{ij}}{2}, \qquad i, j = 2, 3, 4, \qquad \Delta_{ii} = 0. \tag{49}$$

The matrix G^{ext} is the external Gram matrix. We required it to be nonsingular at all three recorded sample points, which also excludes a determinant that vanishes identically under the stated parameter conditions. Each family then defines a symbolic integral problem through its particular zero and equality relations.

To remove equivalent definitions, we compared the assignments under all 24 permutations of the four propagators, together with independent relabelling of the nonzero mass and momentum-invariant parameters. The enumeration found 1,673 inequivalent profiles satisfying the parameter and sample requirements after excluding the existing massless box definitions. A fixed ordering selected 984 of these profiles [14]. Together with the original families, this earlier collection contained 997 one-loop and three two-loop families. The added profiles share the four-propagator box topology, so the family count describes different mass and kinematic conditions within this topology.

We identified and checked a master basis separately for every added family using the three calculations in Subsection 6.3. The four-propagator families have $t_{\mathrm{top}} = 4$, so the first seed range has $r \le 6$, $s \le 2$ and $n_{\mathrm{dot}} \le 2$. We then enlarged the seed and probe ranges, supplied the first masters as the preferred basis and repeated the basis check at a second point and prime. All 1,000 families passed their three basis calculations, giving 3,000 completed checks within the recorded domains.

For the added families, define

$$\boldsymbol{w} = (y_1, y_2, y_3, \mu_1, \mu_2, \mu_3). \tag{50}$$

We used the same three primes and dimension values specified above, with the parameter assignments

$$\begin{aligned} \boldsymbol{w}^{(1)} &= (-11, -19, -29, 7, 13, 23), \\ \boldsymbol{w}^{(2)} &= (-31, -47, -61, 11, 17, 29), \\ \boldsymbol{w}^{(3)} &= (-41, -67, -83, 17, 23, 37). \end{aligned} \tag{51}$$

Each family uses only its present parameters and retains all specified zeros and equalities. The first two points supply the basis checks, while the target validation uses all three.

6.11 Expanded targets and their verified reductions

Using the added bases, we applied the reverse construction in Eq. (41) with 48 attempted sequences per family and at most four replacements per sequence. We also allowed up to 96 nonzero individual-integral constructions per added family, using at most five passes through the source identities. The initial master weights retain the rational functions and mass-dimension conditions in Eqs. (38) and (39).

When the first construction produced no accepted targets, we extended the candidate equations to every retained IBP identity containing the selected current integral. Its seed $\boldsymbol{\sigma}$ can then differ from that integral's index vector $\boldsymbol{a}$. The substitution still uses the coefficient of $I_{\boldsymbol{a}}$ in the selected identity and preserves the known master expansion. This extension allows a reverse path to enter a higher sector through an identity generated there, including cases whose masters all lie in lower sectors. We retain the actual seed, require the new leading integral to increase under the reference order, and check every step in the resulting reduction.

To add individual integrals to the held-out sets, we also used the checked construction proofs to select integrals whose four indices are all nonzero and include at least one negative index. Each selected integral already has a known master expansion. We retained it as a separate target and rebuilt its elimination trace from the recorded identities. This step added 384 individual targets. Each has one denominator factor and three numerator factors, with their powers specified by the index vector. The complete four-expression definition therefore enters the target, while its positive indices place it in a one-propagator sector.

Before the higher-loop extension, the collection contained 9,678 master entries and 240,472 targets: 55,768 individual integrals and 184,704 integral expressions [14]. We count master entries within each family, so related lower-sector integrals can occur in several bases. Every target has its complete master expansion, source identities, sufficient seeds and recorded elimination path.

For the added families, we kept each complete kinematic zero and equality pattern in one data split across all mass assignments. This grouping also respects the original training box profiles. We assigned the original validation bubble and triangle examples to the combined training set because those integral families also occur as lower sectors of the added training families. The new grouped families supply the expanded validation set. The original data-generation split contained 771 training families with 198,527 targets, 106 validation families with 19,600 targets, and 123 test families with 22,345 targets. The reverse sequences require validation and test expressions to contain terms in their full physical sectors. The additional individual targets instead require all four family expressions to occur, including those in the numerator. Shared lower-sector masters remain part of the family definitions. The complete family profile is held out, while some lower-sector reduction structure can also occur in training.

The original data-generation validation set contained 183 individual integrals and 19,417 integral expressions, while the corresponding test set contained 202 individual integrals and 22,143 expressions. These counts describe the original construction samples. The grouped model division described below replaces this three-way data-generation division. The added individual

targets supply tests of numerator integrals in one-propagator sectors. The higher-loop construction below supplies additional individual targets and expressions in coupled vacuum families. Sections 8 and 10 measure the two models on the current grouped split.

We rederived 3,299,902 source equations and checked 227,330 used IBP equations by independent differentiation across this earlier collection. We replayed all 240,472 target constructions and 832,924 recorded elimination steps with exact rational functions. Every reduction reproduced its reference master coefficients, and the three finite-field replays checked the corresponding numerical rows. We also checked family equivalence, the grouping of data splits and the correspondence between every saved target and its records. The accompanying collection provides the family definitions, bases, targets and complete reduction data [14].

6.12 Coupled vacuum families from one through twelve loops

To supply examples at every loop order considered in the model study, we first added 100 mass profiles at each order from two through twelve. We then added 1,250 further profiles at each order from one through twelve, in groups of 250 and 1,000 profiles. These extensions provide 16,100 family instances beyond the original collection. They use coupled vacuum integrals whose master expansions follow from mass partial fractions and IBP identities, which allows us to retain exact reference answers as the number of loops increases. The profiles vary the masses within a restricted set of graphs, so the family count measures mass assignments as well as graph structure.

For this extension we use Euclidean loop momenta and define

$$V_{\boldsymbol{b}}^{(L)} = \int \prod_{i=1}^{L} \frac{d^d \ell_i}{\pi^{d/2}} \, \frac{1}{\ell_1^2 \cdots \ell_L^2} \prod_{j=1}^{R} \frac{1}{(Q^2+\mu_j)^{b_j}}, \qquad Q = \sum_{i=1}^{L} \ell_i, \qquad \mu_j = \rho_j \mu. \tag{52}$$

Here L is the number of loops, R is the number of distinct massive denominators, $\mu = m^2 > 0$ is a common squared-mass scale and ρ_j is a positive mass ratio. The nonnegative integer b_j is the power of the jth massive denominator, and $B = \sum_{j=1}^{R} b_j \geq 1$ is their total power. Every massless denominator has power one in the generated domain. The full integral index vector is $(1, \ldots, 1, b_1, \ldots, b_R)$, with $L+R$ entries. Equation (52) specifies the normalization and signs used for this Euclidean subcollection; the earlier families retain their stated Minkowski convention.

For $L > 1$, the common momentum Q couples the loop integrations. A graph representation has L parallel massless edges between two vertices and a chain of R massive edges joining the same vertices. It has $L + R$ edges and $R + 1$ vertices, giving L independent loops. All massive denominators depend on Q^2, which gives algebraic relations between them. This definition is a parent family for partial fractions with fixed massless powers; it does not specify a complete independent denominator and numerator basis for a generic L-loop scattering integral.

For profile number p, we choose

$$R = 2 + ((p-1) \bmod 3), \qquad \rho_1 = 1, \qquad \rho_{j+2} = 2 + 3p + j(p+2), \quad j = 0, \ldots, R-2. \tag{53}$$

The ratios are distinct within each profile, and $\rho_1 = 1$ fixes a common rescaling. The initial profiles use $p = 1, \ldots, 100$ at $L = 2, \ldots, 12$. The later groups use $p = 101, \ldots, 350$ and $p = 351, \ldots, 1350$ at $L = 1, \ldots, 12$. The latest group has 333 profiles with two massive denominators, 333 with three and 334 with four at each loop order. A twelve-loop family with four massive denominators has sixteen indices, which fits the existing model representation.

6.13 Exact masters and source equations for the added families

The dependence on the common momentum makes the master basis explicit. Write $\boldsymbol{e}_j$ for the unit vector in massive index position j and choose

$$M_j^{(L)} = V_{\boldsymbol{e}_j}^{(L)}, \qquad j = 1, \ldots, R. \tag{54}$$

To see why these integrals span the generated domain, first use $D_i - D_j = \mu_i - \mu_j$, where $D_j = Q^2 + \mu_j$. If $b_i, b_j > 0$, this identity gives

$$(\mu_i - \mu_j) V_{\boldsymbol{b}}^{(L)} - V_{\boldsymbol{b}-\boldsymbol{e}_i}^{(L)} + V_{\boldsymbol{b}-\boldsymbol{e}_j}^{(L)} = 0. \tag{55}$$

Repeated use removes products of distinct massive denominators. This equation is an algebraic mass relation and carries no IBP seed.

The remaining powers follow from the sum of the L IBP operators $\partial_{\ell_i^\nu} \ell_i^\nu$. Applying this sum to the integrand of Eq. (52) gives

$$0 = [Ld - 2(L + B)] V_{\boldsymbol{b}}^{(L)} + 2 \sum_{j=1}^{R} b_j \mu_j V_{\boldsymbol{b}+\boldsymbol{e}_j}^{(L)}. \tag{56}$$

Each such equation retains its actual seed $(1, \ldots, 1, \boldsymbol{b})$ and the contributing derivative operators. For one massive denominator this becomes

$$V_{k\boldsymbol{e}_j}^{(L)} = \left[\prod_{h=1}^{k-1} \frac{L + h - Ld/2}{h\mu_j} \right] M_j^{(L)}, \qquad k \geq 1. \tag{57}$$

The empty product at $k = 1$ equals one. Equations (55) and (57) therefore reduce every generated integral to the stated masters with coefficients rational in d and μ.

An analytic expression also checks the basis. Set $D = d/2$, $\alpha = L(D-1)$ and $c = L - (L-1)D$. Repeated integration of the massless subgraph at fixed Q gives

$$C_L(D)(Q^2)^{-c}, \qquad C_L(D) = \frac{\Gamma(c)\Gamma(D-1)^L}{\Gamma(\alpha)}. \tag{58}$$

For $L = 1$, $Q = \ell_1$, $c = 1$ and $C_1(D) = 1$, so this expression reduces directly to the single massless propagator. The remaining radial integration uses $\int d^dQ/\pi^D = \Gamma(D)^{-1} \int_0^\infty dx\, x^{D-1}$ for a rotationally invariant integrand, where $x = Q^2$. It gives

$$M_j^{(L)} = \mu_j^{\alpha-1} \frac{\Gamma(c)\Gamma(D-1)^L \Gamma(1-\alpha)}{\Gamma(D)}. \tag{59}$$

Here Γ is the Euler gamma function. These manipulations use the standard analytic continuation of dimensionally regularized integrals [3]. At generic d, distinct ρ_j give distinct exponential functions of d in $\rho_j^{\alpha-1}$. Their linear independence over rational functions of d establishes independence of the R masters in this restricted domain. After the common factors cancel, a proposed relation would combine exponentials with distinct growth rates. At fixed generic μ, rational coefficients grow or decay only as powers of d, so they cannot cancel the largest exponential unless its coefficient vanishes. Repeating this argument removes each coefficient. The latest addition supplies 3,001 master entries per loop order and 36,012 entries in total. The preceding extensions supplied 3,289 and 9,000 entries. We count the basis within each family, so the master with $\rho_1 = 1$ recurs across profiles at the same loop order.

We also obtain the master coefficients without following the selected row operations. With $z = Q^2/\mu$, the rational identity

$$\prod_{i=1}^{R}(z+\rho_i)^{-b_i} = \sum_{j:b_j>0}\sum_{k=1}^{b_j} A_{jk}(z+\rho_j)^{-k} \tag{60}$$

defines rational numbers A_{jk}. Expanding the other factors around $z = -\rho_j$ gives them directly:

$$A_{jk} = [w^{b_j-k}]\prod_{i\neq j}(\rho_i - \rho_j + w)^{-b_i}. \tag{61}$$

Here $w = z + \rho_j$ is the expansion variable, and $[w^n]$ means the coefficient of w^n in this Taylor series. Combining these coefficients with Eq. (57) yields the independent reference

$$V_{\boldsymbol{b}}^{(L)} = \mu^{1-B}\sum_{j:b_j>0}\sum_{k=1}^{b_j} A_{jk}\left[\prod_{h=1}^{k-1}\frac{L+h-Ld/2}{h\rho_j}\right] M_j^{(L)}. \tag{62}$$

The factor μ^{1-B} ensures that both sides have mass dimension $Ld - 2L - 2B$. This reference allows us to check the reduction path through a separate calculation.

6.14 Reverse target construction and the expanded data division

We supply every Euler seed with $1 \le B \le 5$ and retain the summed Euler identity in Eq. (56) as its source equation together with all mass partial-fraction identities through $B = 6$. Scalar target roots have $2 \le B \le 5$. The source pools contain 35, 160 or 545 equations for $R = 2, 3, 4$, respectively. These counts stay fixed as L changes because the massless powers and massive-index bounds stay fixed. The additional loop orders therefore test this controlled family structure, while large generic IBP systems require wider denominator and numerator domains. After obtaining a checked scalar relation, we reverse its steps to construct the corresponding integral from its master combination. For expressions containing several roots, we first combine their known master expansions and then apply the recorded inverse steps. Each inverse step adds a stated multiple of an exact zero identity, so the master coefficients remain known throughout the construction.

For roots with total powers B_ν, we use target weights of the form

$$\mathcal{T} = \sum_\nu \kappa_\nu d^{\eta_\nu}\mu^{B_\nu - B_*}V_{\boldsymbol{b}_\nu}^{(L)}, \qquad B_* = \max_\nu B_\nu, \qquad \eta_\nu \in \{0,1\}. \tag{63}$$

We choose $\kappa_\nu = n_\nu/q_\nu$, where $n_\nu \in \{-3,-2,-1,1,2,3\}$ and $q_\nu \in \{1,2,3\}$. The powers of μ give every term the same mass dimension. Each family contains eighteen individual target integrals for $R = 2$ and thirty-two for $R = 3$ or $R = 4$, with full massive sectors included before lower sectors. The remaining targets are weighted expressions with one, two or three distinct roots. Each such expression includes a root with every massive denominator present and at least one additional denominator power. We retain 128 distinct targets per family. The latest addition supplies 1,536,000 targets, following the earlier additions of 140,800 and 384,000 targets.

For each target we retain its master coefficients, exact reverse construction, source equations, sufficient IBP seed set and a forward elimination trace. The trace records all allowed source rows, the chosen pivot, the coefficients before and after the operation, and the arithmetic operation counts. Mass partial-fraction steps remain separately identified, so a seed label always refers to an actual IBP seed. The reference path minimizes the next symbolic row's number of nonmaster terms, then its total support and immediate operation count, with a fixed final tie rule. This supplies a reproducible valid path without establishing a globally minimal sequence.

We verify every source equation against Eq. (62) and separately check the partial-fraction formula and the Euler derivative before integration. We replay the symbolic constructions and forward reductions, then independently replay all saved records at three finite-field samples. A candidate target is rejected if a reference pivot vanishes at one of those samples. The checks also verify connected graphs and their loop counts. At $d = 2 + 1/L$ the radial master integral converges; numerical quadrature then checks Eq. (59) at both ends of the loop-order range.

The complete collection now contains 17,100 family instances, 57,979 family-local master entries and 2,301,272 targets. It contains 2,247 one-loop families, 1,353 two-loop families and 1,350 families at each order from three through twelve. We preserve every earlier grouped model assignment and assign 700 of the latest profiles to training and 300 to testing. A profile has the same assignment at every loop order, giving 1,610,890 training targets in 11,963 families and 690,382 test targets in 5,137 families. Shared lower-sector integrals can occur across different profiles. These tests measure transfer between the specified mass profiles with every loop order represented during training; they do not establish transfer to unseen loop orders or general high-loop topologies. The accompanying records contain the complete definitions and checks [15].

7 Machine Learning Model for Seed Selection

To train and test the seed-selection model, we use the target expressions and sufficient seed sets constructed in Section 6. We have implemented a small neural network that assigns a binary decision and its estimated confidence to each supplied candidate seed. We trained the model on the fixed training division and then evaluated it on the held-out targets. This section defines the inputs, probabilities and training procedure, and Section 8 gives the resulting seed predictions.

7.1 Model inputs and seed probabilities

A seed specifies an integral index vector at which the solver generates IBP identities. The resulting equations can contain other integrals whose elimination uses identities from additional seed vectors [2]. A target can therefore use several seeds in a sufficient reduction set. Some targets admit a reduction from one seed, while a target already expressed in the chosen masters needs no IBP seed. We formulate the prediction as a separate binary decision for every candidate, so the model can select several seeds for the same target.

For a fixed family and chosen master basis, write a target as

$$\mathcal{T} = \sum_{j=1}^{n_T} c_j(\boldsymbol{x}, d) I_{\boldsymbol{a}^{(j)}}, \qquad \mathcal{C} = \{\boldsymbol{\sigma}_1, \ldots, \boldsymbol{\sigma}_{n_C}\}. \tag{64}$$

Here n_T is the number of terms in the target, c_j is the rational coefficient of its jth term, $\boldsymbol{x}$ is the set of kinematic parameters and d is the space-time dimension. The vector $\boldsymbol{a}^{(j)}$ gives the integral indices of that term. The set $\mathcal{C}$ contains n_C candidate seed vectors within the stored family bounds. It supplies all candidates that this implementation scores for the target. Extending the seed bounds would require supplying an enlarged candidate set.

The model receives every term in the stored order, including its indices and symbolic coefficient. Each integral contributes its index values, the locations of positive indices and the quantities r, s, t and n_{dot} defined in Eq. (14). We transform an index value a to $\mathrm{sgn}(a)\log(1 + |a|)$ and transform the nonnegative counts with $\log(1 + v)$, where v denotes the count. The input also records the number of family indices and which positions are present. The current representation has capacity for 16 indices; the expanded collection uses up to sixteen.

To include the coefficient functions, we separate their written expressions into variable names, integers and mathematical symbols. We encode counts of individual symbols and consecutive groups of two and three symbols in a fixed vector with 64 components. A fixed signed assignment maps each group to a component, and division by the square root of the total number of groups sets the scale. Every coefficient symbol enters this representation, although different groups can share a component. The resulting vector is a compressed description of the coefficient expression. It does not perform symbolic simplification or establish equality between different expressions. In particular, modular residues do not enter as ordinary real coefficient values.

For each term we append its relative position in the stored list and the inverse term count. Denote the resulting term vector by $\boldsymbol{f}_j$. A shared learned map ϕ_θ forms

$$\boldsymbol{h}_j = \phi_\theta(\boldsymbol{f}_j), \qquad \boldsymbol{h}_j \in \mathbb{R}^{32}. \tag{65}$$

Here θ denotes the adjustable parameters of the model and $\boldsymbol{h}_j$ is a vector describing the term. The same map acts on each term, so its number of parameters does not depend on n_T. Combining shared term maps through averages follows the set-based construction of Ref. [16], with additional position information to retain the stored order.

We also include every ordered pair of distinct terms. If $n_T > 1$, their combined description is

$$\boldsymbol{h}_{\text{pair}} = \frac{1}{n_T(n_T-1)} \sum_{\substack{j,k=1 \\ j\neq k}}^{n_T} \psi_\theta([\boldsymbol{h}_j, \boldsymbol{h}_k, \boldsymbol{h}_j - \boldsymbol{h}_k, \boldsymbol{h}_j \odot \boldsymbol{h}_k])\,. \tag{66}$$

Square brackets join vectors, $\odot$ multiplies corresponding components, and ψ_θ maps each joined vector to 32 components. For an individual-integral target, we set $\boldsymbol{h}_{\text{pair}} = \boldsymbol{0}$. This representation includes both orders of each pair and the positions of the associated terms. The present model uses pairs as its specified form of term combinations; it does not enumerate longer subsets or all permutations of the full expression. The fixed stored term order must also be used when preparing later inputs.

The family information includes the propagator expressions, masses, external scalar products, invariant names, and the numbers of loops, propagators, external momenta and invariant parameters. We encode its symbolic structure with the same fixed 64-component rule and map it to a 32-component family vector $\boldsymbol{h}_F$. A further shared map acts on the index vectors of the chosen masters, and their mean gives the 32-component basis vector $\boldsymbol{h}_M$. This input identifies the basis available for reduction. The target's master coefficients, construction history and recorded seed choices remain outside the input.

Each candidate seed contributes its index and complexity information through a shared map that gives a vector $\boldsymbol{u}_b \in \mathbb{R}^{32}$ for candidate b. To associate a seed with particular target terms, we form

$$\begin{aligned} \omega_{bj} &= \frac{\exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_j/\sqrt{32})}{\sum_{k=1}^{n_T} \exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_k/\sqrt{32})}, \\ \boldsymbol{v}_b &= \sum_{j=1}^{n_T} \omega_{bj} \boldsymbol{h}_j. \end{aligned} \tag{67}$$

The weight ω_{bj} determines how much term j contributes to candidate b's description, and $\boldsymbol{v}_b$ is their weighted mean. This uses the scaled dot-product weighting of Ref. [17]. The weights sum to one over target terms for each seed.

The model then joins the mean and componentwise maximum of the term vectors with the pair, family, basis, seed and weighted-term vectors:

$$\begin{aligned} \boldsymbol{z}_b &= [\text{mean}_j\, \boldsymbol{h}_j,\ \max_j \boldsymbol{h}_j,\ \boldsymbol{h}_{\text{pair}}, \boldsymbol{h}_F, \boldsymbol{h}_M,\ \boldsymbol{u}_b, \boldsymbol{v}_b], \\ g_b &= F_\theta(\boldsymbol{z}_b), \qquad p_b = \frac{1}{1+\exp(-g_b)}. \end{aligned} \tag{68}$$

Here F_θ has a hidden layer with 64 components and one scalar output g_b. The other learned maps each use one layer with 32 components. All hidden layers use the hyperbolic tangent function. The implementation has 28,097 adjustable parameters and shares them across candidates, so reordering the candidate list only reorders the returned scores.

The quantity p_b represents the model's probability that $\boldsymbol{\sigma}_b$ belongs to the recorded sufficient seed set. Several seeds can contribute to the same reduction, so these probabilities are separate and need not sum to one. A high score ranks a seed for selection. The current data establish a valid reference set for each target; they do not establish the probability that a seed belongs to a globally minimal reduction. Exact reduction remains the criterion for deciding whether the selected seeds supply enough equations.

For each candidate we report probabilities for both possible labels:

$$\begin{aligned} P_\theta(y_b = 1 \mid \mathcal{T}, F, \mathcal{M}, \boldsymbol{\sigma}_b) &= p_b, \\ P_\theta(y_b = 0 \mid \mathcal{T}, F, \mathcal{M}, \boldsymbol{\sigma}_b) &= 1 - p_b. \end{aligned} \tag{69}$$

Here y_b is the reference label, F is the family definition and $\mathcal{M}$ is the chosen master basis. Label 1 means membership in the recorded sufficient seed set. Label 0 means absence from that set. The two class probabilities for one seed sum to one, while probabilities for different seeds remain separate.

The binary prediction uses a fixed threshold of one half:

$$\widehat{y}_b = \begin{cases} 1, & p_b \geq \frac{1}{2}, \\ 0, & p_b < \frac{1}{2}. \end{cases} \qquad q_b = \widehat{y}_b p_b + (1 - \widehat{y}_b)(1 - p_b). \tag{70}$$

The value $\widehat{y}_b$ is the returned label, and q_b is the model's estimated probability that this label agrees with the reference label. For example, $p_b = 0.90$ gives label 1 with confidence $q_b = 0.90$, while $p_b = 0.08$ gives label 0 with confidence $q_b = 0.92$. These values illustrate the output rule; they are not trained results. Candidate selection uses p_b, while q_b describes confidence in the returned binary decision.

The predicted seed set is

$$\widehat{\mathcal{S}}(\mathcal{T}) = \{\boldsymbol{\sigma}_b \in \mathcal{C} : \widehat{y}_b = 1\}. \tag{71}$$

Its size can vary between targets. Exact reduction must establish whether the identities from this set suffice to reduce the target. The probability q_b concerns one reference-membership decision; it is not a probability that the complete selected set gives a valid or minimum-cost reduction. Section 8 compares these confidence values with their observed agreement with the reference labels.

7.2 Training labels and the grouped data division

For each target, let $\mathcal{S}_\mathcal{T} \subseteq \mathcal{C}$ be the sufficient seed set used by its recorded reduction. The reference label for candidate b is

$$y_b(\mathcal{T}) = \begin{cases} 1, & \boldsymbol{\sigma}_b \in \mathcal{S}_\mathcal{T}, \\ 0, & \boldsymbol{\sigma}_b \notin \mathcal{S}_\mathcal{T}. \end{cases} \tag{72}$$

A zero label records absence from this particular set; another valid reduction can use that seed. Algebraic mass partial fractions carry no seed label. For example, a target whose reference uses the first and third of five candidates has

$$\boldsymbol{y}(\mathcal{T}) = (1, 0, 1, 0, 0). \tag{73}$$

The target coefficients and public family information enter the prediction, while this label vector enters the training loss.

We keep each family together and preserve the grouping of related mass and kinematic profiles described in Section 6.12. The expanded division contains 1,610,890 training targets in 11,963 families and 690,382 test targets in 5,137 families. Every loop order from one through twelve occurs in both sets. Each added mass profile stays in one set across all loop orders. The ratio applies to complete target examples, each with its full candidate list. Shared lower-sector integrals can occur across different profiles, and previously reserved families were also evaluated in earlier project stages. The latest mass profiles were assigned before the new parameter updates.

The training objective gives each target equal weight:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}_{\mathrm{train}}|} \sum_{\mathcal{T} \in \mathcal{D}_{\mathrm{train}}} \frac{1}{n_C(\mathcal{T})} \sum_{b=1}^{n_C(\mathcal{T})} \left[y_b \log p_b + (1 - y_b) \log(1 - p_b) \right]. \tag{74}$$

Here $\mathcal{D}_{\mathrm{train}}$ is the training set and $n_C(\mathcal{T})$ is the candidate count for target $\mathcal{T}$. All positive and zero labels enter this binary cross-entropy without additional class weighting. The threshold of one half determines reported decisions after the model supplies its probabilities.

7.3 Completed training and implementation checks

We continued the saved model and its AdamW optimizer on the expanded training set for twenty complete passes. The starting checkpoint had completed 20 passes over the preceding expanded collection. It also retains the earlier training lineage. The latest phase uses the enlarged collection and its own twenty-pass counter. It contains 32,217,800 target presentations and 1,012,000 parameter updates.

The architecture still has 28,097 parameters. AdamW uses a learning rate of 10^{-3}, weight decay of 10^{-4} and a maximum gradient norm of one [18]. Each update uses up to 32 targets from a training family. We change family and target order between passes and use the actual batch size for the final group. Masked batch calculations retain every target term, every ordered pair of distinct terms and every supplied seed. Comparisons with the individual-target calculation verified matching scores, losses and parameter derivatives across individual-integral and multi-term examples. Training uses four CPU threads, while inference uses one. A comparison on training examples verified matching scores and parameter derivatives across the selected thread counts.

The mean training loss changed from 0.0516102 in the first expanded-data pass to 0.008548661 in the twentieth. Figure 1 shows the complete history. These means include parameter changes within each pass and describe fitting on the training examples.

The checks verify target-label correspondence, membership of every recorded seed in the candidate pool, complete coverage during each pass, and equality of the published weights with the resumable checkpoint. The model retains the separate class probabilities and confidence rule in Eq. (70). Held-out examples supplied no gradient updates or threshold fitting.

The representation accepts up to sixteen indices, 512 target terms and 10,000 supplied candidates. The added twelve-loop examples reach the index limit. This capacity describes the input representation; the training evidence concerns the actual family domains defined above. Section 8 evaluates the frozen parameters on their reserved mass profiles.

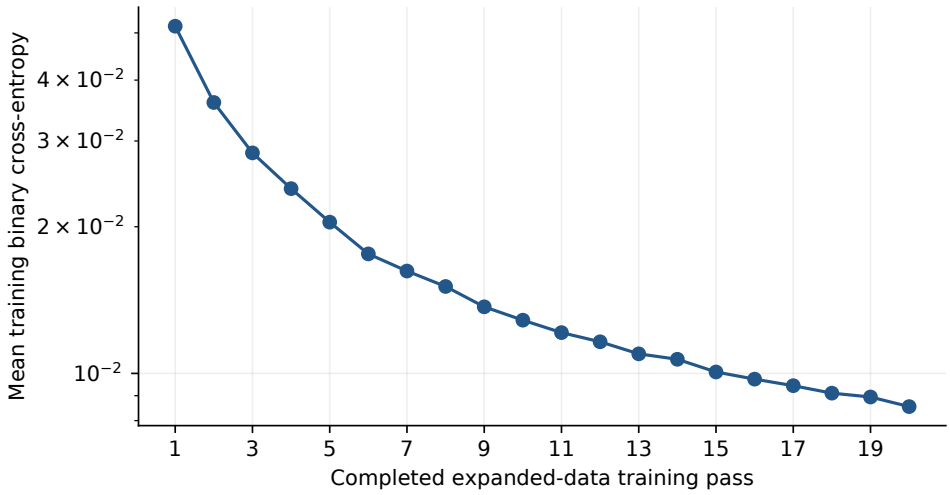


Figure 1: Mean training binary cross-entropy during the twenty expanded-data passes. Each target has equal weight within the reported mean. The logarithmic vertical axis shows the training history; held-out examples do not enter this curve.

8 Testing the Seed-Selection Model

To measure seed selection after the expanded training, we froze the weights at the end of the twenty new passes and evaluated all 690,382 targets from 5,137 reserved families. The test contains 148,494 individual integrals and 541,888 integral expressions, with loop orders from one through twelve. It evaluates mass profiles excluded from parameter updates within the family domains of Section 6. The fixed threshold remains $p_b = 1/2$, and no test result changes the weights or the confidence map [19].

8.1 Reference agreement and seed ranking

The test supplies 76,536,115 candidate labels, of which 2,346,128 belong to the recorded sufficient sets. A true positive is a selected candidate with reference label one, while a false positive is a selected candidate absent from that set. A false negative is a recorded seed that the model omits. With these counts,

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad F_1 = \frac{2\,\text{precision}\,\text{recall}}{\text{precision} + \text{recall}}. \tag{75}$$

The pooled precision is 95.750%, recall is 96.892% and F_1 is 96.318%. The fraction of targets whose complete predicted set equals its recorded set is 84.332%. Candidate-label accuracy is 99.773%; the all-zero rule gives 96.935% because zero labels are frequent. Precision, recall and complete-set agreement therefore describe seed recovery more directly than accuracy alone [20].

To examine the scores without changing the threshold, we rank candidates by p_b and group candidates with equal scores. For a target T, let P_g and R_g be the precision and recall after including score group g in descending order. With $R_0 = 0$, the target average precision is [21]

$$\text{AP}_T = \sum_g (R_g - R_{g-1}) P_g. \tag{76}$$

Each increase in recovered reference seeds therefore weights the precision at that point. Averaging over targets with at least one recorded seed gives mean target average precision 99.090%. The first ten candidates recover, on average, 99.787% of recorded seeds and contain every recorded

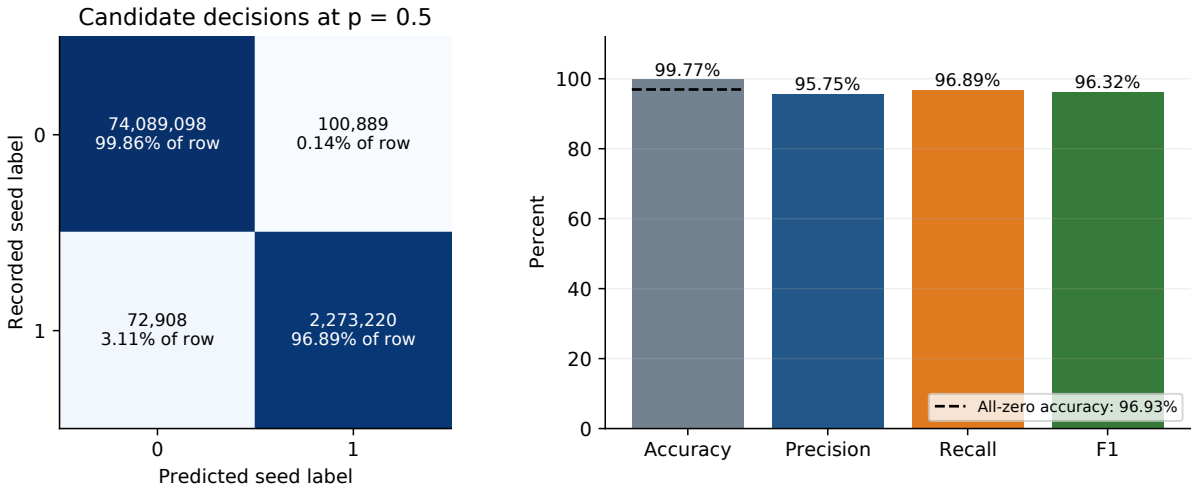


Figure 2: Candidate decisions at the fixed threshold, with pooled precision, recall and F_1. Each confusion-matrix percentage is normalized within its reference-label row.

seed for 98.947% of targets in the corresponding nonempty-reference comparison. The score-curve calculation uses 4,096 fixed probability intervals; it supplies a descriptive curve and does not select a new operating threshold.

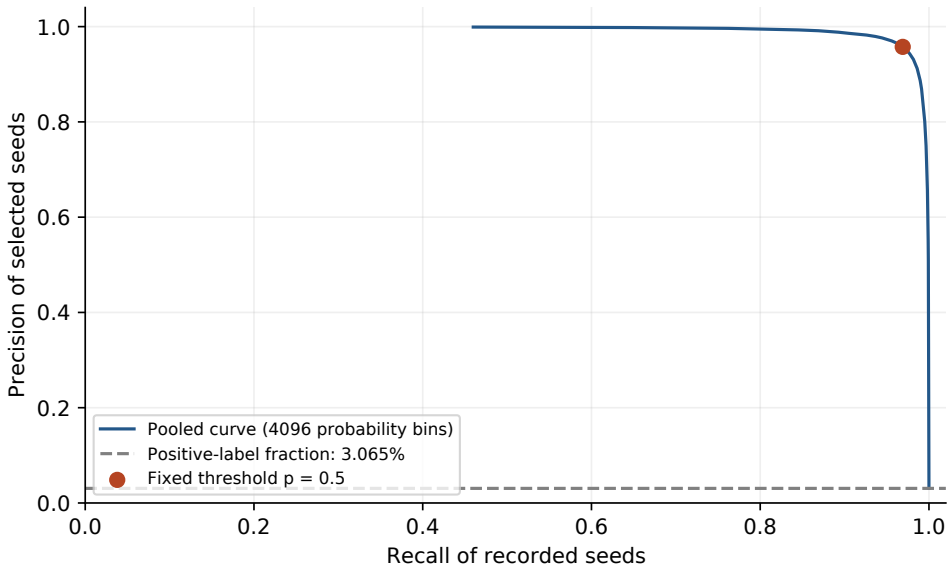


Figure 3: Pooled precision and recall as the score threshold varies. The marked point retains the fixed threshold of one half. The horizontal reference gives the positive-label fraction.

8.2 Confidence and dependence on the family

For a returned binary label, the model reports q_b from Eq. (70). We group decisions by this value and compare their mean confidence with the fraction matching the reference. Equality would place a group on the diagonal of Figure 5. These empirical comparisons assess the saved scores without fitting a calibration correction.

Figure 6 separates performance by family and loop order. The one-loop group has 681 families, the two-loop group has 406, and each remaining order has 405 held-out mass profiles. This

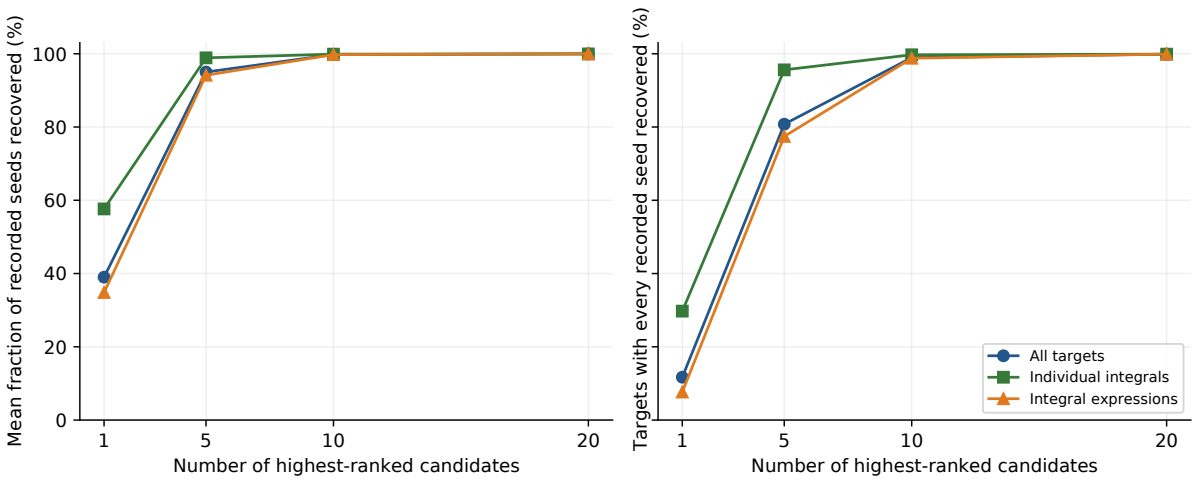


Figure 4: Recorded-seed recovery as the number of supplied highest-ranked candidates increases. The right panel requires every recorded seed to appear in the selected prefix. Individual integrals and integral expressions are also shown separately.

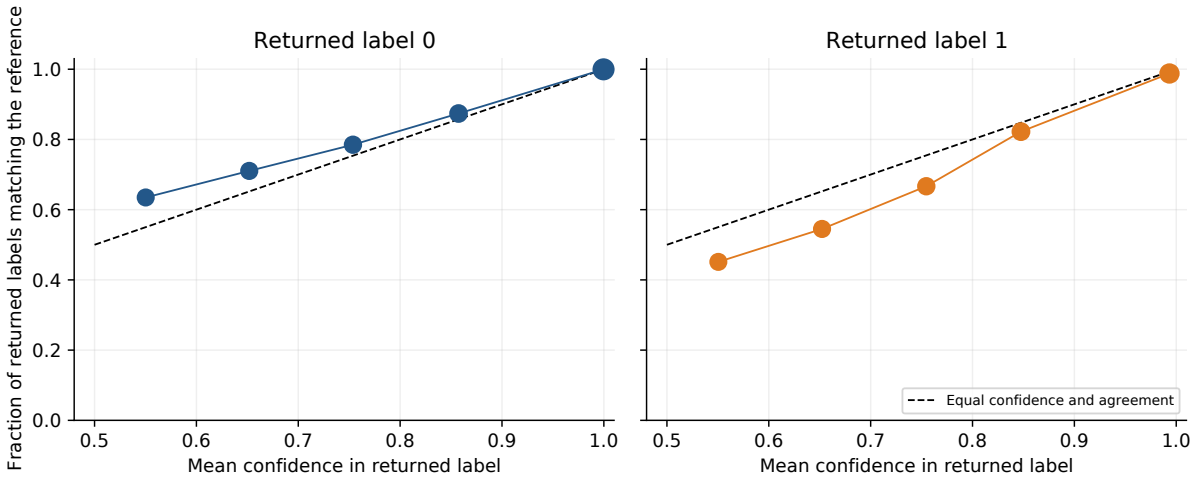


Figure 5: Reported confidence and observed label agreement, shown separately for returned labels zero and one. Marker area increases with the number of decisions in a group.

separation matters because the added vacuum families use different identities and candidate bounds from the earlier one-loop collection. Loop-order comparisons also reflect these differences in the mathematical domain. At one loop the pooled recall is 93.790% and F_1 is 92.181%; at twelve loops they are 97.334% and 97.300%, respectively. Across the groups from two through twelve loops, the lowest recall is 96.184% at $L = 6$. These missed reference seeds show why a proposed set still needs an exact sufficiency check before use in a complete reduction.

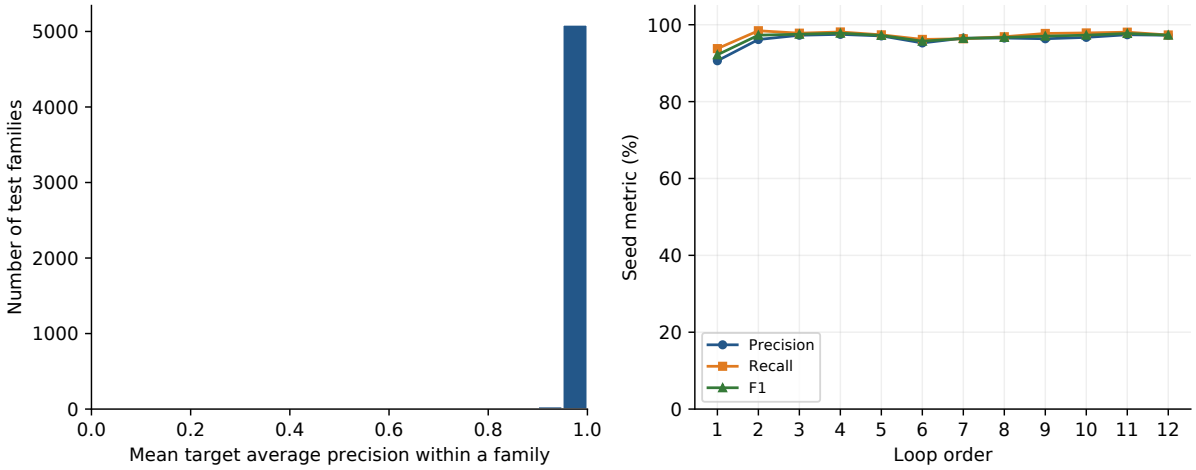


Figure 6: Distribution of mean target average precision across held-out families, and pooled seed precision, recall and F_1 at each loop order. The higher-loop groups cover the restricted vacuum mass profiles defined in the text.

8.3 Checks and interpretation

Independent checks reproduce the saved confusion counts and target counts, verify the prediction hashes and confirm that the weights remain fixed. All stored probabilities are finite and lie between zero and one. The mean time for input preparation and model inference is 1.6881 ms per target on the test allocation; this excludes a subsequent exact reduction.

The labels identify one recorded sufficient set. Other seed sets can also reduce the target, so disagreement with these labels does not by itself establish failure of the proposed reduction. The classification measurements above describe reference-set recovery and confidence. To determine whether the selected equations complete the target, we next use the saved predictions in Kira.

8.4 Sufficiency of the predicted seeds inside Kira

To determine whether the predicted seeds supply enough equations, we use Kira 3.1 to test whether each target belongs to the span of its specified masters modulo the available identities. This means that the target can be written as a linear combination of those masters after using the selected equations. We retain the frozen model, reserved families and threshold of one half from Eq. (71).

Let $\mathcal{E}_{\boldsymbol{\sigma}}$ contain the stored IBP identities from seed $\boldsymbol{\sigma}$, and let $\mathcal{B}$ contain the background identities, including partial fractions and symmetries. A supplied seed set $\mathcal{S}$ provides the equations

$$\mathcal{A}(\mathcal{S}) = \mathcal{B} \cup \bigcup_{\boldsymbol{\sigma} \in \mathcal{S}} \mathcal{E}_{\boldsymbol{\sigma}}. \tag{77}$$

Every selected seed contributes all its stored IBP identities. The reference seed labels, construction paths and master coefficients supply no equations to this test.

The sufficiency condition follows from linear algebra over the field $\mathbb{K} = \mathbb{Q}(\boldsymbol{x}, d)$ of rational functions in the kinematic parameters $\boldsymbol{x}$ and dimension d. Let V be the vector space with one formal basis vector for each integral in the supplied system, let $v_{\mathcal{T}}$ be the coefficient vector of the target, and let $\mathcal{R}(\mathcal{S})$ be the row space of the equations in $\mathcal{A}(\mathcal{S})$. Also let V_M be the span of the basis vectors for the specified masters. Then

$$\begin{aligned} \mathcal{T} = \sum_m r_{\mathcal{T}m} M_m \quad \text{using } \mathcal{A}(\mathcal{S}) \quad &\Longleftrightarrow \quad v_{\mathcal{T}} \in \mathcal{R}(\mathcal{S}) + V_M, \\ &\Longleftrightarrow \quad [v_{\mathcal{T}}] = 0 \quad \text{in } V/(\mathcal{R}(\mathcal{S}) + V_M). \end{aligned} \tag{78}$$

Here M_m is a specified master, $r_{\mathcal{T}m} \in \mathbb{K}$ is its reduction coefficient, and square brackets denote the equivalence class after removing combinations of available equations and master vectors. This condition tests whether such coefficients exist without requiring their full symbolic reconstruction. Sufficiency applies to the supplied target expression and permits exact cancellation among its terms.

We implement the condition through Kira's user-defined-equation interface, which accepts homogeneous linear systems with rational coefficients and uses Fermat for symbolic operations [8, 9]. Each target has an auxiliary variable $Z_{\mathcal{T}}$ and the input equation

$$Z_{\mathcal{T}} - \sum_{j=1}^{n_T} c_j(\boldsymbol{x}, d) I_{\boldsymbol{a}^{(j)}} = 0. \tag{79}$$

All terms in this equation come from the public target expression in Eq. (64). We add one unit row for each specified master vector. These rows impose $M_m = 0$ in the formal quotient used for the test, where every possible master combination represents zero. They do not change the physical values of the master integrals.

Kira reduces $Z_{\mathcal{T}}$ exactly in this system. A zero remainder establishes the condition in Eq. (78), and a nonzero remainder shows that the supplied equations leave an independent contribution outside the specified master span. The conclusion applies to rational functions at generic parameter values where their denominators are defined. A successful program exit alone does not establish sufficiency.

Different seed sets use separate copies of every integral variable so that they cannot share equations. Targets with the same selected set can share a system within one family. We retain every equation in a component connected to the target terms and remove only disconnected components. For each target we test its predicted set, the full candidate set $\mathcal{C}$, and the empty seed set. The full set checks the available fallback, while the empty set measures reduction through the target definition, master basis and background identities alone.

We independently reconstruct the selected row spaces from the original identities at each of the three stored finite-field samples. At each sample, sparse elimination checks membership after removing master columns and must agree with Kira's symbolic decision. Each nonzero symbolic remainder also has a regular sample with a nonzero value, which provides an exact witness that its coefficient does not vanish identically. The sample coordinates enter these checks after the symbolic calculation; reference master coefficients remain outside the full sufficiency test. For two pilot families, 765 target-and-seed-set comparisons also agree with reductions that retain the masters and compare their explicit rational coefficients.

8.5 Sufficiency results and the role of additional seeds

The test includes all 690,382 reserved targets from 5,137 families. The predicted sets supply enough equations for 652,517 targets (94.515%), while 37,865 (5.485%) have a nonzero remainder

outside the master span. The full candidate pool is sufficient for 690,382 targets (100.000%), and background identities alone suffice for 26,915 (3.899%). Within the coupled-vacuum subset, the predicted sets suffice for 595,919 of 618,240 targets (96.390%). Within the one-loop box subset, they suffice for 56,294 of 71,184 targets (79.082%). For the other one-loop samples, the predicted sets suffice for 89 of 380 bubble targets (23.421%), 152 of 431 triangle targets (35.267%), 60 of 144 propagator targets (41.667%), and 3 of 3 tadpole targets (100.000%). The overall fraction therefore depends on the mixture of tested family domains. All symbolic membership decisions agree with the independent row-space checks at the three stored samples. All requested targets have a recorded outcome, and the final evaluation has no unresolved solver or export errors [22].

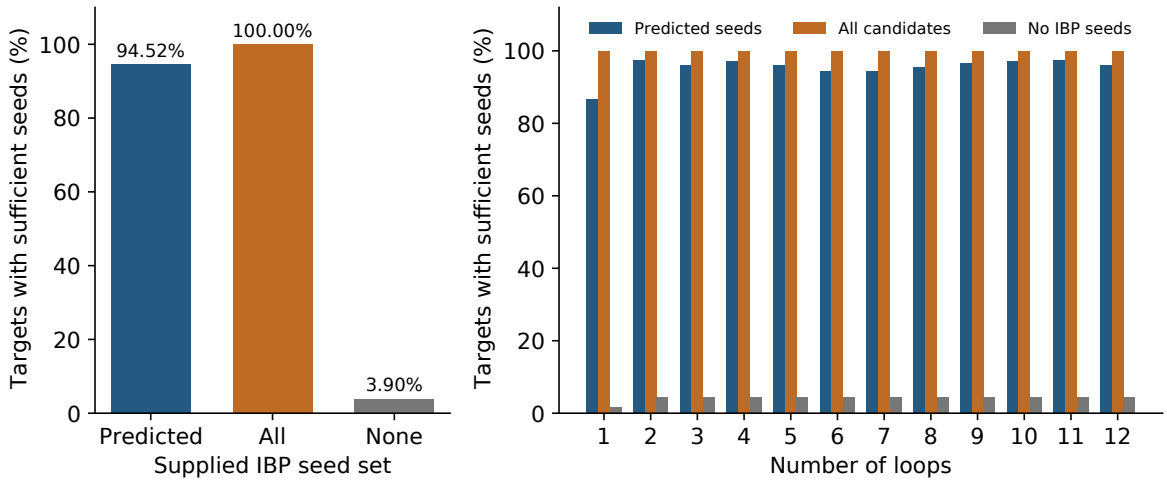


Figure 7: Targets that belong to the specified master span after using the supplied identities, shown for predicted seeds, all candidate seeds and no IBP seeds. Each loop-order fraction uses every reserved target at that order. All three policies retain the same background identities.

Of the targets with sufficient predicted sets, 70,302 use a seed set different from the recorded reference set. This comparison shows that a model can supply enough equations through an alternative selection, so reference-set agreement and algebraic sufficiency measure different properties. The model selects 3.439 seeds per target on average. Across all target–candidate pairs, it selects 3.102% of the supplied candidates. This fraction describes selection within the bounded pools; it does not measure a runtime improvement over Kira's usual strategies.

The all-candidate control supplies a direct check of the fallback within the stored equation domain. A predicted set that leaves extra integrals requires further equations before we accept its reduction. We solve the full-pool control independently, which checks whether adding all candidates would supply a sufficient fallback. This test does not identify the smallest necessary addition or measure reuse of a preceding elimination. Its conclusions concern rational functions at generic parameter values, where the denominators in the reduction are defined.

These results establish algebraic sufficiency rates inside Kira for the stored identities. The user-defined interface is needed for the present family representation, including the dependent massive denominators of the higher-loop vacuum families. An insufficient outcome refers to the stored identities for the selected seeds. Additional valid IBP operators at the same seeds can enlarge that pool. Section 11 separately compares native Kira seeding through learned sector bounds with the stated native baselines. The grouped execution also shares work among targets with identical selected sets, so its elapsed times do not provide an end-to-end solver speedup measurement. The next section addresses the separate learned choice of an allowed elimination step.

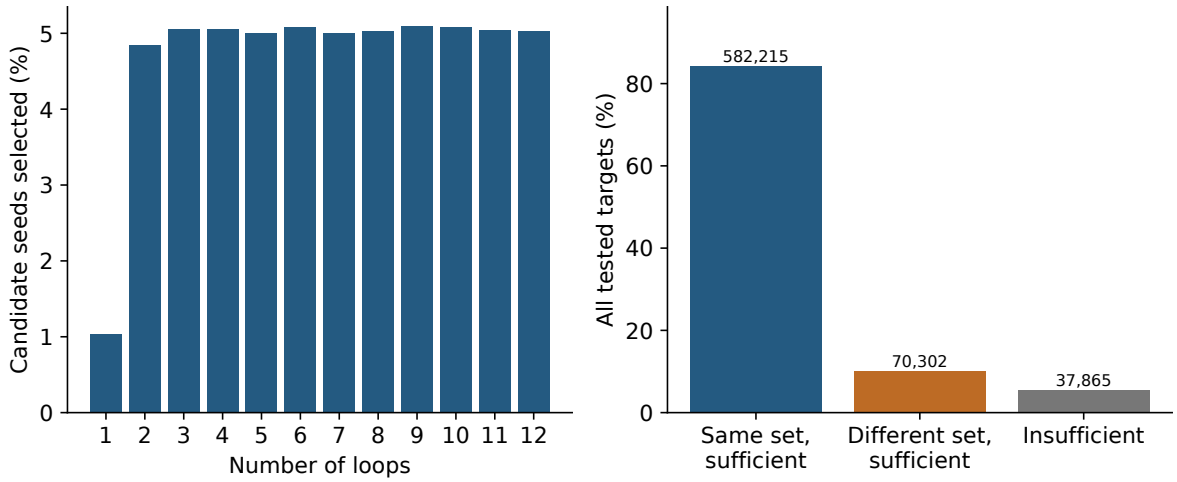


Figure 8: The selected fraction of candidate seeds at each loop order, and the relation between reference-set agreement and seed sufficiency in Kira. Numbers above the bars are target counts. Different sufficient sets account for the difference between the two tests.

9 Machine Learning Model for Reduction Propagation

The seed model selects identities from which a reduction can proceed. Once these equations are available, the reduction model selects the next row operation from a supplied list of allowed steps. We use the current coefficient matrix and its integral labels to describe each choice, and the model returns a probability for every listed step. The step with the largest probability gives the proposed next operation. Exact arithmetic then updates the matrix, which supplies the input for the next choice. This section describes the model and its training on the reductions constructed in Section 6.

9.1 Current equations and allowed reduction steps

Let $A^{(n)}$ be the coefficient matrix after n reduction steps. A column j corresponds to an integral $I_{\boldsymbol{a}^{(j)}}$, and a row contains its coefficients in an identity or in the target expression currently being reduced. The chosen master basis specifies the columns protected from elimination. An action $a = (q, r, k)$ uses source row q to eliminate column k from destination row r:

$$A_{rj}^{(n+1)} = A_{rj}^{(n)} - \frac{A_{rk}^{(n)}}{A_{qk}^{(n)}} A_{qj}^{(n)}, \qquad q \neq r, \quad A_{qk}^{(n)} \neq 0, \quad A_{rk}^{(n)} \neq 0. \tag{80}$$

All other rows remain unchanged during this elementary step. If both rows are identities, this is an elementary row operation. If row r represents a target, row q must be a zero identity so that the target retains its value. The solver supplies the allowed action set $\mathcal{A}_n$, enforcing its integral ordering, master basis and row roles. The model ranks this finite list; completeness of the list remains the responsibility of the solver.

The stored examples describe a particular instance of this matrix. They contain one changing target row and the source identities available for eliminating its largest remaining nonmaster integral under Eq. (36). Each candidate specifies one of these identities, the target row and that integral. After a choice, the source identities remain fixed and the target row changes. The prediction interface also accepts several changing rows, while the present training data cover the stored target-row procedure. Section 12 tests the frozen policy on Kira's evolving selected

IBP matrix using admissible actions that retain its integral ordering [9]. This test changes the inference states while retaining the target-row training described here.

9.2 Information supplied to the model

For each allowed action, we construct a vector $\boldsymbol{f}_a$ from the current matrix, the pivot integral and the family definition. The pivot description includes its indices, sector membership, and the values of $r, s, t, n_{\mathrm{dot}}$ in Eq. (14). Descriptions of the source and destination rows give their nonzero counts, master content and the mean and largest integral complexities. A matrix summary gives its number of rows, columns and nonzero entries. The public family definition supplies the numbers of loops and kinematic parameters together with the propagators and scalar-product relations. We encode their written form through the same fixed signed token counts described in Section 7.1, using 32 components here. This encoding admits collisions between different groups of symbols. We transform nonnegative counts with $\log(1+v)$, where v is the count, and each signed index a_i with $\operatorname{sgn}(a_i)\log(1+|a_i|)$. The representation allows up to 16 indices and records which positions belong to the family. These definitions give 132 features for each candidate.

The current records supply coefficients in $\mathbb{F}_P$, the field of integers modulo a prime P. We use these coefficients to calculate the overlap of each source row with its destination, the number of terms that the proposed operation would cancel or introduce, and its immediate arithmetic count. For a source with n_q nonzero entries, this count is $1 + 2n_q$: one division and n_q multiplications and subtractions, using the convention in Section 6. These quantities require one sparse trial row update per action. They use only the current matrix and do not inspect later reduction steps or reference master coefficients. A symbolic coefficient can vanish at the chosen sample, so the model uses counts recomputed from the sampled rows. The data audit retains the distinction from the original symbolic support counts. Structural counts and integral descriptions carry this information into the model. The numerical size of a modular residue does not represent the magnitude of a real coefficient.

The representation compresses the supplied matrix into these features, so different states can have the same description. It includes local cancellation information and a summary of the candidate set. These quantities describe the current choices without enumerating complete reduction paths. Forming these features and evaluating the model contribute to the eventual runtime and must be included when measuring a reduction speedup.

9.3 Action probabilities and confidence

A shared map ϕ_θ converts each candidate vector into a 32-component vector,

$$\boldsymbol{h}_a = \tanh(W_1 \boldsymbol{f}_a + \boldsymbol{b}_1), \qquad \boldsymbol{u} = \frac{1}{|\mathcal{A}_n|} \sum_{a \in \mathcal{A}_n} \boldsymbol{h}_a, \qquad \boldsymbol{v} = \max_{a \in \mathcal{A}_n} \boldsymbol{h}_a. \tag{81}$$

Here θ denotes the adjustable parameters, W_1 is a learned matrix, $\boldsymbol{b}_1$ is a learned offset, and the maximum acts on each component separately. Sharing the map and combining candidates through averages and maxima makes the scores independent of their position in the supplied list [16]. The model joins $\boldsymbol{h}_a, \boldsymbol{u}, \boldsymbol{v}$ and applies a second map with 64 hidden components to obtain a scalar score $g_\theta(a)$. For a fixed family, master basis and finite-field sample, it returns

$$\begin{aligned} p_\theta(a \mid A^{(n)}, \mathcal{A}_n) &= \frac{\exp g_\theta(a)}{\sum_{b \in \mathcal{A}_n} \exp g_\theta(b)}, \\ \widehat{a}_n = \arg\max_{a \in \mathcal{A}_n} p_\theta(a), &\qquad q_n = p_\theta(\widehat{a}_n). \end{aligned} \tag{82}$$

Thus the probabilities sum to one over the supplied alternatives, and q_n is the reported confidence for the selected step. With one allowed step the probability is one by construction, which provides no evidence of learned discrimination. An empty candidate list returns no proposed operation; the solver must determine whether the reduction is complete or further identities are required.

Training gives these probabilities an interpretation as agreement with the recorded next choice, conditional on the supplied state and actions. Several choices can give valid reductions, and an alternative choice may even lead to a shorter sequence. The reference paths establish valid reductions and their recorded costs; they do not establish global minima. Section 10 compares this confidence with agreement on reserved choices. Confidence does not certify exactness, minimal cost or the success of a complete reduction. Equation (80) and the solver's checks retain responsibility for the algebra.

9.4 Training examples and separation from testing

We retain the same grouped family division used for the seed model: 11,963 families with 1,610,890 targets supply training records, while 5,137 families with 690,382 targets remain reserved for testing. Every state from one target stays with that target's family. We read the row before each recorded step and its complete stored list of admissible source equations. The chosen equation supplies the reference action $a_n^\star$. The row after the step and the stored operation counts check the data preparation but remain outside the predictive inputs. The target's construction history, final master coefficients and future choices also remain outside those inputs.

For the set $\mathcal{D}_{\text{choice}}$ of training states with at least two allowed actions, the objective is

$$\mathcal{L}_{\text{red}}(\theta) = -\frac{1}{|\mathcal{D}_{\text{choice}}|} \sum_{(A,\mathcal{A},a^\star)\in\mathcal{D}_{\text{choice}}} \log p_\theta(a^\star \mid A, \mathcal{A}). \tag{83}$$

Each such state has equal weight. States with one admissible action have zero loss and remain in the data audit. The recorded reference choice is the training label; absence of a label on another action does not establish that its reduction is invalid.

We use AdamW [18] with a learning rate of 10^{-3}, weight decay 10^{-4}, batches of 256 states and a gradient norm limit of one. We continued the saved expanded-data model and optimizer for twenty complete passes over the enlarged training states. The new phase retains the architecture and optimizer settings while adding 1,250 mass profiles at every represented loop order. Preparation replays every selected row operation with exact modular arithmetic and verifies its final recorded master row. Training shuffles states within bounded groups of families and retains complete resume states. The reserved families remain outside parameter updates and confidence fitting. Earlier project stages evaluated the preceding held-out families; the current experiment also reserves the newly assigned mass profiles at every added loop order.

9.5 Completed training and present scope

The expanded training set contains 14,684,059 recorded states from 1,610,890 targets in 11,963 families. Of these states, 5,840,122 supply at least two allowed actions and 8,843,937 supply one action. The records contain 26,775,999 candidate appearances. The unchanged model uses 132 features per candidate and has 10,529 adjustable parameters.

Preparation replays every reference row operation, checks continuity between steps and verifies the final master support. Symbolic coefficients can vanish at the sampled kinematic point, so the model uses the support of the actual sampled matrix. The audit records these differences without changing the exact symbolic references.

We continued the preceding expanded-data model and its AdamW optimizer for twenty complete passes over the enlarged data. Each new pass uses every training state with an action choice; a forced step has probability one and supplies no ranking decision. The new phase contains 116,802,440 choice-state presentations and 463,763 parameter updates. The preserved lineage contains thirty passes over the original smaller collection and twenty passes over the preceding 2,100-family collection. The latest twenty passes use the enlarged collection across all represented loop orders.

Mean training cross-entropy changed from 0.3823373 to 0.3804633 during this phase, while agreement with the recorded action changed from 99.841% to 99.895%. These values describe losses and choices encountered during parameter updates. Section 10 measures the frozen parameters on reserved families.

The implementation retains the original architecture, learning rate, gradient bound and state-batch limits. Training uses four CPU threads, while inference uses one. Parallel preparation reproduces the saved training features and labels exactly. Checks verify equal weights in the inference and resumable checkpoints, complete state coverage and the preserved optimizer lineage. The published implementation retains the inputs, learned parameters and exact reference records [23].

The training domain still consists of one changing target row and a fixed source-equation pool. The new records extend this domain through twelve loops for the coupled vacuum families specified above. They provide evidence about these allowed reduction steps; general sparse elimination inside Kira requires a separate integration and benchmark.

10 Testing the Reduction-Propagation Model

To test the model after twenty expanded-data passes, we froze its parameters and evaluated 690,382 targets in 5,137 held-out families. Every loop order from one through twelve occurs in this set. We first compared the predicted next step with the recorded choice, then allowed the model to choose a complete path from the initial target row. Both tests retain exact finite-field row operations [24].

10.1 Allowed steps and the comparison rule

At a given state, the candidates are the source identities whose leading nonmaster integral matches the target row's leading nonmaster integral. Both the learned policy and the comparison rule receive this same list and respect the recorded integral order. The source rows remain fixed while elimination changes the target row. This restriction makes the available actions and complete reference answers explicit.

For each allowed source row E, let u_E be the target row after the corresponding exact elimination. The local comparison rule selects the smallest tuple

$$(n_{\mathrm{nm}}(u_E),\ \mathrm{nnz}(u_E),\ 1 + 2\,\mathrm{nnz}(E),\ \mathrm{id}(E))\,. \tag{84}$$

Here n_{nm} is the number of nonmaster columns with nonzero coefficients, nnz is the total number of nonzero coefficients and $\mathrm{id}(E)$ is a fixed equation identifier used only to resolve ties. The third entry counts one division and one multiplication and subtraction for each source entry. The rule compares the immediate next rows; it does not search all complete reduction paths.

10.2 Next-step agreement and confidence

The reserved traces contain 6,364,695 states, including 2,558,472 with an action choice and 3,806,223 with one allowed action. On states with a choice, the learned model selects the recorded action in 95.845% of cases and places it among its first three candidates in 99.980%. Its selected action gives the same next sampled row as the reference in 95.846% of cases. The local rule selects the recorded action in 99.473% of these states.

When n_{tie} actions share the largest saved probability, the program selects one through its fixed identifier order. Such ties occur in 37.747% of choice states. We also assign credit $1/n_{\text{tie}}$ when the recorded action belongs to the highest-scoring set, and zero otherwise. Averaging this credit gives 74.506% reference agreement. This diagnostic describes the scores while retaining the stated deterministic choice rule for the actual reductions.

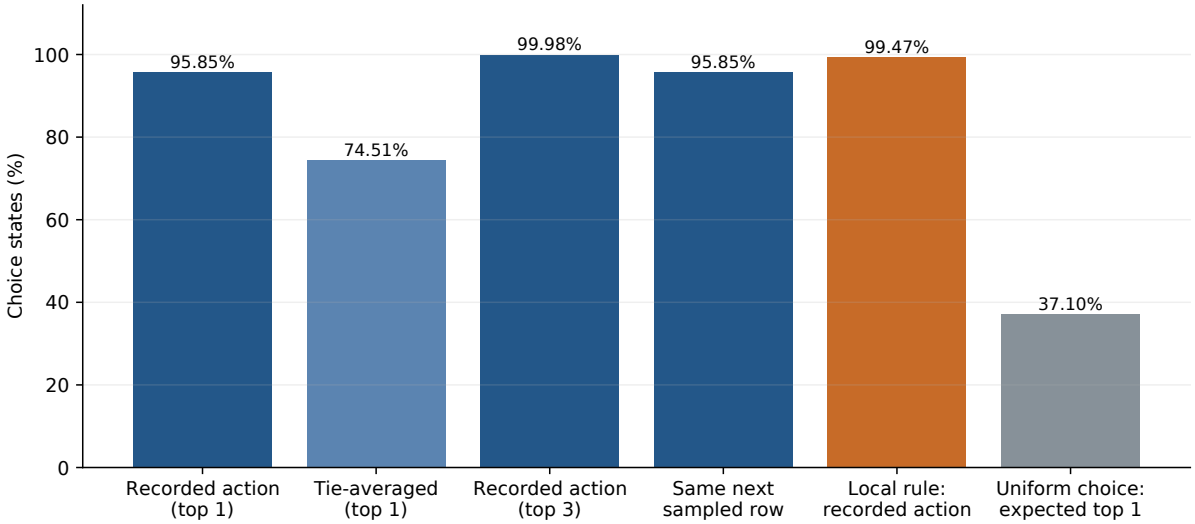


Figure 9: Next-step comparisons on states with at least two allowed actions. The uniform reference assigns equal probability to every available action. The tie-averaged result shares credit equally among highest-scoring tied actions. The same-row comparison permits different identities that produce the same sampled next row.

The model assigns a probability to every supplied action, with probabilities summing to one. We compare the selected action's reported probability with observed agreement after grouping predictions into ten fixed confidence intervals. Let $\mathcal{D}^{\text{test}}_{\text{choice}}$ be the set of held-out states with at least two allowed actions, and let $a^{\star}_{n}$ be the recorded action at state n. Applying the negative logarithmic loss in Eq. (83) to this set gives 0.400146. The calculation replaces probabilities below 10^{-38} by 10^{-38} when taking their logarithms.

To compare the full probability distribution with the recorded choice, we also calculate the multiclass Brier score [25],

$$B_{\text{test}} = \frac{1}{|\mathcal{D}^{\text{test}}_{\text{choice}}|} \sum_{n \in \mathcal{D}^{\text{test}}_{\text{choice}}} \sum_{a \in \mathcal{A}_n} \left[p_\theta(a \mid A^{(n)}, \mathcal{A}_n) - \mathbf{1}_{a=a^{\star}_{n}} \right]^2 . \tag{85}$$

Here $\mathcal{A}_n$ is the supplied action set and $\mathbf{1}_{a=a^{\star}_{n}}$ equals one for the recorded action and zero for every other action. We sum over all actions without dividing by their number, then give each state equal weight. The resulting score is 0.250632. The confidence comparison in Figure 10 uses the frozen scores without fitting a correction. A forced action has probability one, so these ranking and confidence comparisons exclude forced states.

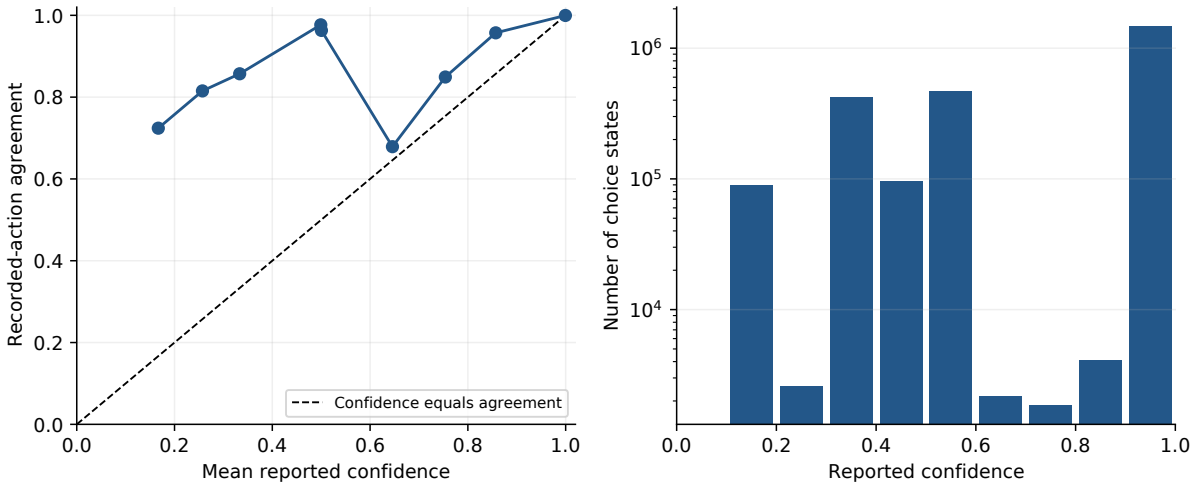


Figure 10: Reported confidence and recorded-action agreement, with the number of choice states in each confidence interval. The diagonal represents equality between confidence and agreement.

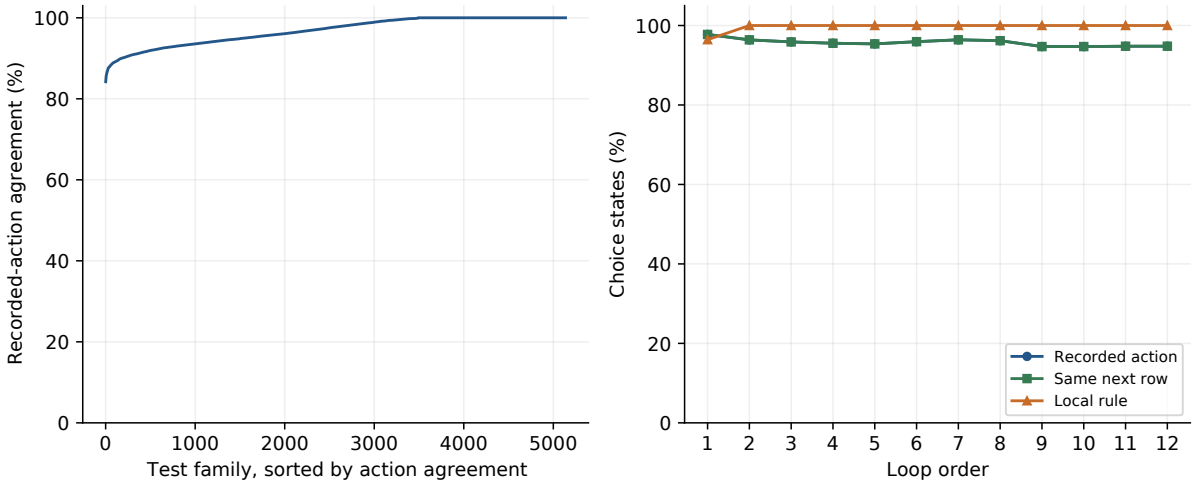


Figure 11: Agreement across individual families and loop orders. Every higher-loop group uses held-out mass profiles of the restricted vacuum families. These comparisons include changes in the mathematical family domain as well as loop order.

10.3 Complete reductions and arithmetic work

For complete-path testing, the policy receives the initial target row and repeatedly selects an allowed identity until the row contains only protected masters or a stated failure condition occurs. Decisions use the first finite-field sample. We then replay the selected sequence at both additional samples and compare the resulting rows with the exact reference master coefficients evaluated at those points.

The learned policy matches all three sampled master expansions for 690,302 of 690,382 targets (99.988%). The local rule matches all three for 689,700 targets (99.901%). The respective numbers without this full three-sample agreement are 80 and 682. For the learned policy, 77 targets stop when the supplied pool has no allowed identity for the remaining leading integral, and 3 complete at the decision sample but fail an additional-sample check. The local rule gives 664 and 18 in these two categories. Agreement at the samples is the stated test criterion; the test does not reconstruct a new symbolic rational result from the learned path.

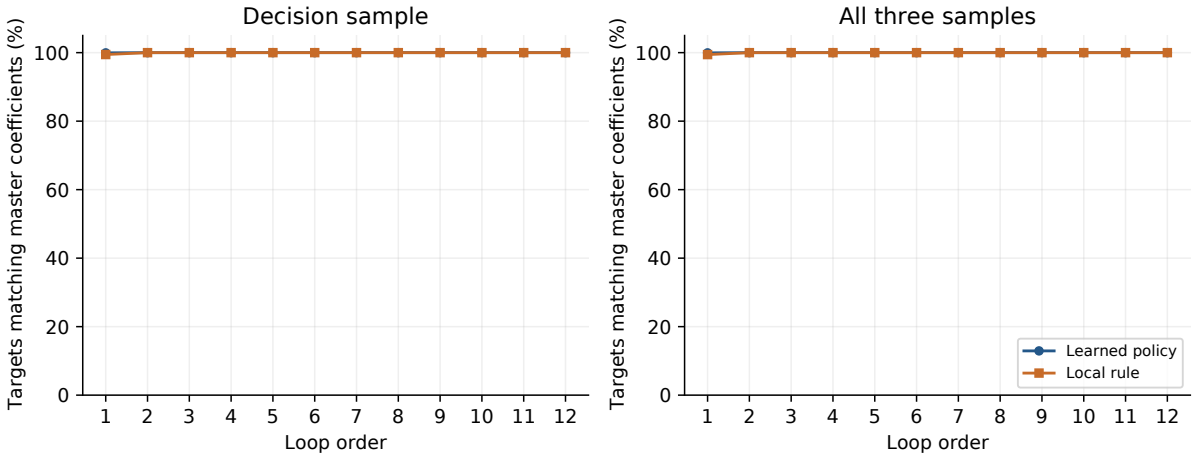


Figure 12: Complete target reductions at each loop order. The left panel requires agreement at the decision sample; the right panel requires agreement at all three samples.

We compare arithmetic counts on the 689,684 targets for which both policies pass all three samples and the reference count is nonzero. The learned policy uses fewer committed row operations than the local rule for 6,846 targets, the same number for 610,196 and more for 72,642. A committed operation changes the selected target row. Candidate trials used to construct model features or evaluate the local rule also perform arithmetic, so Figure 13 shows both accounting choices. The recorded path supplies a valid reference count without a proof that this count is minimal.

10.4 Timing, checks and scope

The mean attempted reduction time is 12.3438 ms per target for the learned policy and 0.6991 ms for the local rule. The learned policy therefore takes 17.657 times as long on this allocation. These times include action selection and committed elimination within the target-row experiment. We alternate policy execution order between targets and keep verification and disk output outside the timed interval. Four CPU workers on the same compute node process disjoint families, with one computation thread per worker. Both policies run in the same worker for each target. Figure 14 shows the timing distributions on the test allocation.

Independent checks reproduce probability summaries and replay the committed finite-field operations from the saved paths at the decision sample. They also verify the source coefficients,

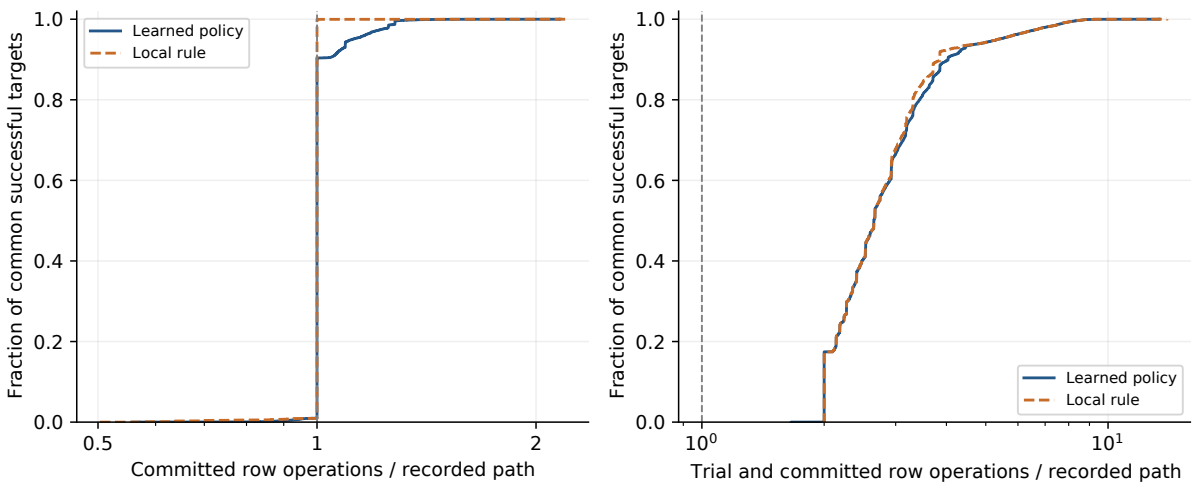


Figure 13: Cumulative distributions of arithmetic counts relative to the recorded path, restricted to targets successful for both policies. The right panel also includes trial row operations used when choosing an action. Both horizontal axes use logarithmic scales.

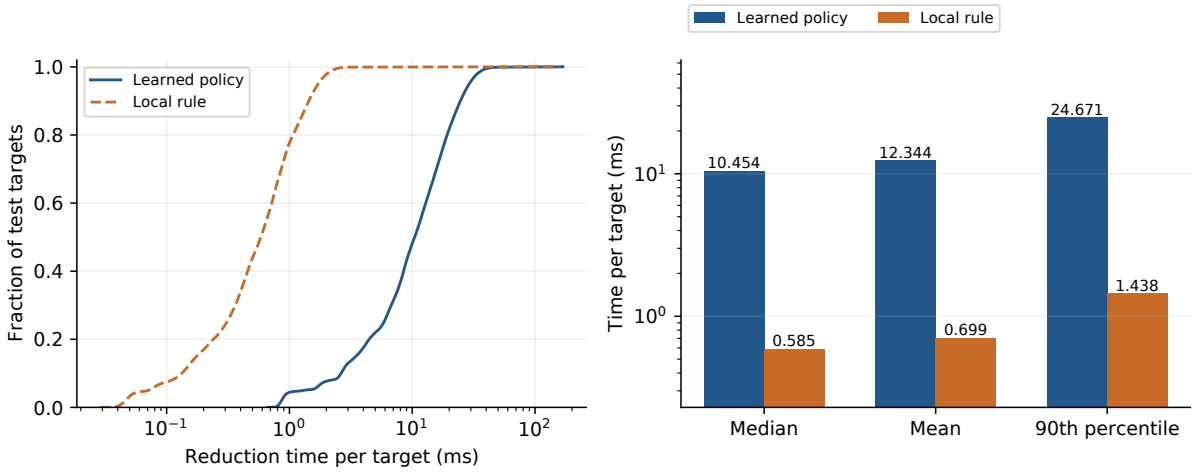


Figure 14: Attempted target-row reduction times for the learned policy and local rule. Both receive identical initial rows, source pools and integral ordering.

data split and unchanged model hash. The tests distinguish agreement with a recorded next action from completion of an allowed alternative path.

The results concern the supplied equation pools and family domains. Every tested loop order also occurs in training, and related lower-sector integrals can occur in different mass profiles. Consequently, the experiment measures held-out profile performance within these domains. It does not establish globally optimal seed sets, globally minimal elimination paths or an end-to-end Kira speedup. Section 11 then measures complete symbolic reductions with Kira, including model and fallback costs.

11 Comparison with Kira

A useful learned choice must reduce the cost of obtaining the final master coefficients. The sufficiency test in Section 8.4 establishes which stored seed sets permit a reduction, while the preceding target-row test measures individual reduction paths. We now compare complete symbolic reductions with Kira's existing equation selection and seeding. The comparisons retain both trained models, the held-out families and the seed threshold of one half. The candidate seed lists and prescribed master bases are inputs; their earlier construction is outside the measured reduction costs.

11.1 Targets, baselines and exact checks

We use 19 held-out family instances: one restricted vacuum profile at each loop order from one through twelve, three box profiles, and one bubble, triangle, tadpole and two-loop propagator family. The three box profiles represent the smallest, middle and largest candidate-pool sizes in that domain. A fixed hash order selects the other profiles and the target expressions without consulting previous model outcomes. Nested batches contain 1, 4 and 16 targets, with batches of 1 and 3 for the three-target tadpole family. This gives 56 batch cases and 291 distinct targets. The restricted vacuum profiles retain the dependent denominators described in Section 6.12; their loop orders do not represent the complexity of general multiloop scattering.

For the supplied-equation comparison, Kira receives the connected part of the complete stored source pool and applies its own equation selection, forward elimination and back substitution. The learned seed method takes the union of the predicted seed sets for the requested batch and supplies the corresponding stored IBP identities together with the same background identities. If any target retains an integral outside the prescribed master basis, the method reruns the batch with the complete pool. Both attempts contribute to its cost. This comparison isolates selection from a common source library; generating that library is outside these timings.

For the row comparison, the learned model chooses source identities using a fixed finite-field sample, while Eq. (80) updates the target coefficients symbolically in $\mathbb{K} = \mathbb{Q}(\boldsymbol{x}, d)$. Kira then reduces any remaining terms. A second version applies the local rule from Section 10 through the same symbolic procedure. This control identifies the cost of the learned choice relative to the local choice. Both methods retain fixed source identities and a changing target row, which is the procedure covered by the training data.

We compare each final coefficient with its constructed rational reference and independently evaluate the coefficients and saved row operations at three finite-field points. The physical master coefficients remain present throughout this experiment. A reported completion therefore includes the full symbolic target expansion and its checks. Each batch-method pair runs three times on the same compute node, with the execution order rotated. Kira and the model each use one computation thread. At most two trials run concurrently on the cluster.

11.2 Learned seed bounds in native Kira

The seven families with independent propagator bases also permit native Kira equation generation. Kira reduces the component integrals, and its exact algebra combines their results into the requested target expressions. We record generation and reduction separately from this final combination. Native equation counts refer to the component reduction; the later combination has separate counts and contributes to every reported total time. For the default baseline, the seeding bounds are one above the largest values of r and s among the target components and prescribed masters. The restricted baseline also bounds n_{dot} one above its largest required value and uses Kira's subsector truncation with $l = N - s_{\max}$, where N is the number of propagators. A third control sets r, s and n_{dot} directly to the largest target/master values, uses the same subsector truncation, and falls back to the default bounds if needed. Both this control and the learned method reserve 60 seconds for the first native attempt and 100 seconds for its fallback, within the complete 180-second limit. These fixed settings retain Kira's own symmetry handling and equation selection [9]; they do not represent an exhaustive search over its settings.

To apply the seed model before native equation generation, let $\widehat{\mathcal{S}}_S$ be the predicted seeds in sector S. For every nonempty set, we pass the bounds

$$\begin{aligned} r_S &= \max_{\boldsymbol{\sigma}\in\widehat{\mathcal{S}}_S} r(\boldsymbol{\sigma}), \qquad s_S = \max_{\boldsymbol{\sigma}\in\widehat{\mathcal{S}}_S} s(\boldsymbol{\sigma}), \\ n_{\mathrm{dot},S} &= \max_{\boldsymbol{\sigma}\in\widehat{\mathcal{S}}_S} n_{\mathrm{dot}}(\boldsymbol{\sigma}). \end{aligned} \tag{86}$$

to Kira. These bounds enclose the selected seeds, and Kira can generate additional seeds within the bounds and inherited subsectors. Its own generated and selected equation counts therefore measure the resulting work. An incomplete first attempt triggers the default native bounds. This direct comparison repeats all four native methods on the largest fixed target batch in each of the seven families, giving 99 distinct targets and 84 trials. The learned method uses the same model and threshold as the supplied-pool test.

11.3 Measured cost and results

We record the complete elapsed time from process start to a checked result. It includes input loading, model loading, feature preparation, inference, exact algebra, any fallback, coefficient checks and output. We also record the time with the family already loaded and the model ready for use. This second quantity represents repeated use of prepared inputs; the complete process time retains the cost of starting the model. For a matched case, we compare the median of the three repeats and define

$$\rho_T = \frac{\operatorname{median} T_{\mathrm{learned}}}{\operatorname{median} T_{\mathrm{baseline}}}. \tag{87}$$

Thus $\rho_T < 1$ indicates a lower measured time for that case. Aggregate ratios use the geometric mean of the paired case ratios. Trials that reach their time or memory limit remain in the completion counts and are excluded from ratios that require two completed results.

We distinguish supplied equations, equations selected by Kira, logged forward insertions and coefficient-field operations. For a committed target-row step with n_q nonzero source entries, the last count is $1 + 2n_q$, as in Section 9. Feature construction also performs trial updates of candidate rows. These counts do not measure the internal polynomial work needed to simplify a rational coefficient. Runtime includes that work. Memory is the largest sampled sum of resident process memory for the driver, Kira and its algebra process; the sampling interval is 50 ms, and shared pages can contribute to more than one process's count. Every trial has a 180-second limit, a 1.5 GiB address-space limit per process and a 32 MiB scratch limit. The

comparisons use the existing symbolic Kira 3.1 build with Fermat. They do not benchmark finite-field reconstruction.

The main comparison contains 792 trials. The full stored-pool baseline completed 168/168, learned seeds completed 168/168, learned rows completed 168/168, and local-rule rows completed 168/168. On the 56 completed matched batch cases, learned seed selection gave complete-process and prepared-input time ratios of 1.720 and 1.116, respectively. It used the full-pool fallback in 24 trials. Across those batches, its supplied source-equation count, including fallback, was 0.251 times the baseline count, while the number of equations selected by Kira was 1.027 times the baseline. The distinction shows how Kira's existing selection changes the work left after an external seed choice. The learned method had lower complete-process median time in 0 of 56 batches and lower prepared-input median time in 15 of 56 batches.

The learned row procedure gave time ratios of 1.719 and 1.205 against the full stored-pool baseline over 56 matched cases. Its ratios against the local row rule were 1.520 and 1.043. Before the final Kira call, the learned row procedure completed every target in 54 of the 56 batches, while the local rule did so in 55. Kira completed the remaining terms in the other batches. For the 53 batch cases in which both row procedures themselves finished all targets before the final Kira call, their summed committed coefficient-operation counts were 19,401 and 18,881, giving a ratio of 1.028. This comparison excludes paths that left part of the reduction to Kira. The elapsed-time measurements retain all of that later work, as well as candidate feature preparation and exact coefficient simplification [26].

Kira's own equation selection removes much of the redundant input in the full-pool baseline. The learned seed filter reduces the supplied pool, and its measured complete cost includes the added model work and every fallback. Both learned methods have aggregate complete-process and prepared-input ratios above one in this comparison. The row-operation ratio also exceeds one relative to the local rule on the matched complete preprocessing paths. These results identify added cost in the current implementations and give no aggregate reduction in the stated time or committed-operation measures.

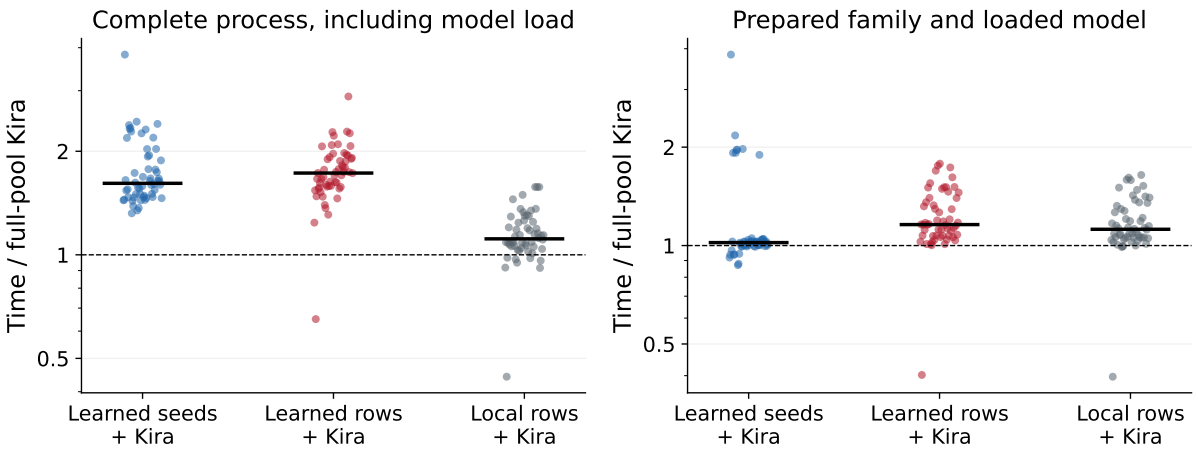


Figure 15: Matched time ratios relative to Kira with the full stored source pool. Each point uses a batch's three-repeat median. The left panel includes process and model loading; the right panel uses prepared family inputs and a loaded model. Short solid lines show the median case ratio; the dashed line denotes equal time.

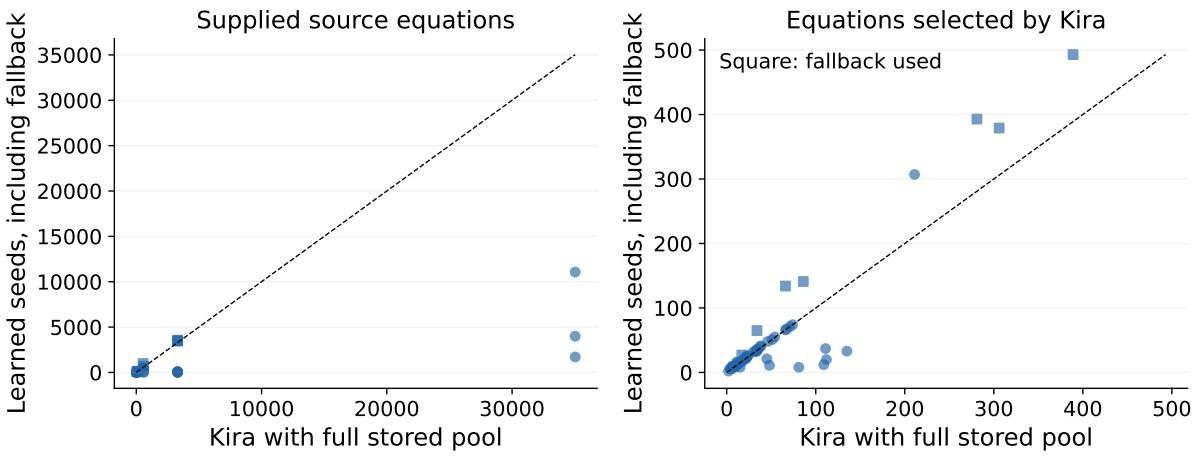


Figure 16: Supplied source equations and equations selected by Kira for identical batches. Learned-seed counts include any full-pool fallback. Squares identify batches with fallback, and the diagonal denotes equal counts.

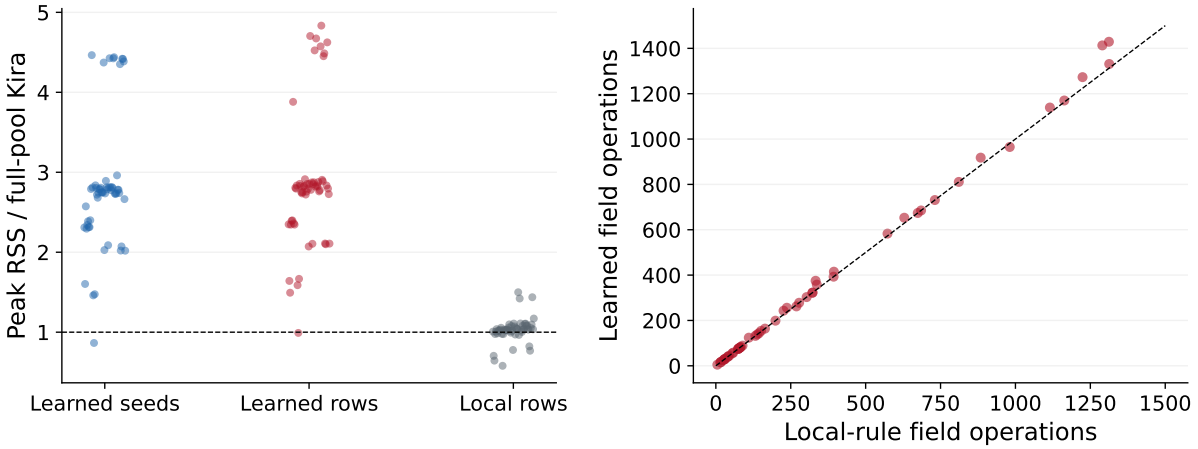


Figure 17: Sampled summed resident-memory ratios and committed coefficient-operation counts. Memory ratios use the largest recorded peak within each three-repeat group. The operation panel compares only batches that both row procedures completely reduced before the final Kira call. These counts exclude polynomial simplification and candidate-scoring work.

11.4 Native seeding results

The direct native-seeding comparison completed 21/21 learned trials, 21/21 default-baseline trials, 21/21 restricted-baseline trials and 21/21 target-index-bound control trials. Learned bounds required a default-bound fallback in 9 trials. For the 7 paired completed family batches, its complete-process and prepared-input time ratios against default native Kira were 1.596 and 1.344. Against restricted native Kira, the corresponding ratios were 1.671 and 1.373 over 7 paired batches. Against the control that sets bounds directly from target indices, the ratios were 1.425 and 1.141 over 7 paired batches. These measurements include native equation generation, exact reduction, target combination, coefficient checks and any repeated attempt.

The learned native method had lower complete-process median time in 0 of 7 matched batches against the default baseline, 0 of 7 against the restricted baseline and 1 of 7 against the target-index control. Across the completed paired batches, the ratios of summed initial equation counts were 0.624, 1.144 and 0.881, respectively. These counts include repeated attempts. The learned bounds generate fewer equations than the stated default bounds, and the restricted deterministic baseline generates still fewer equations on these batches. The learned method has aggregate complete-process and prepared-input time ratios above one against all three native controls. Thus the current native integration adds total cost under the tested settings.

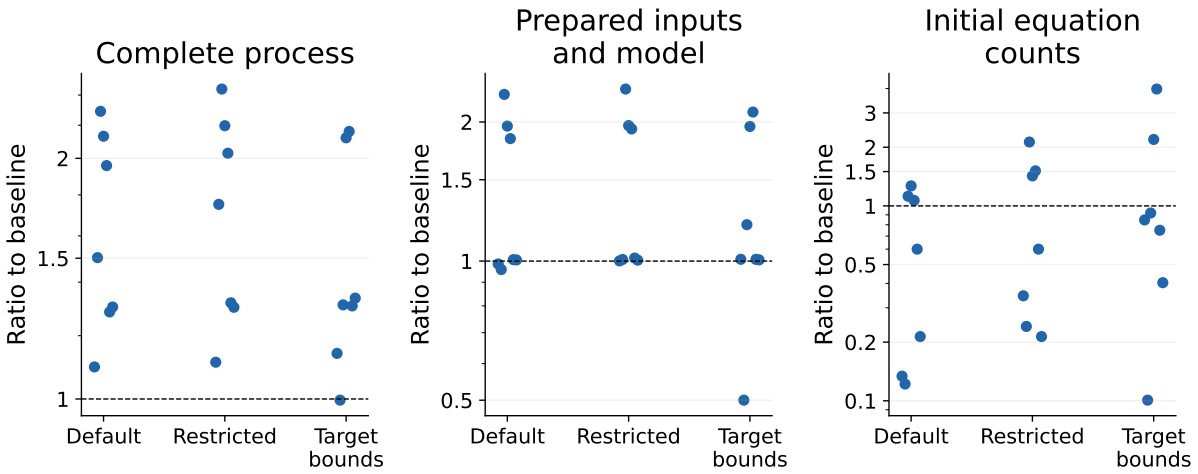


Figure 18: Direct native-seeding comparison on the largest frozen batches in seven independent-basis families. Ratios compare learned sector bounds with the three stated native baselines. Equation counts sum the initial system sizes that Kira reports for each attempt; internal regeneration can perform additional work.

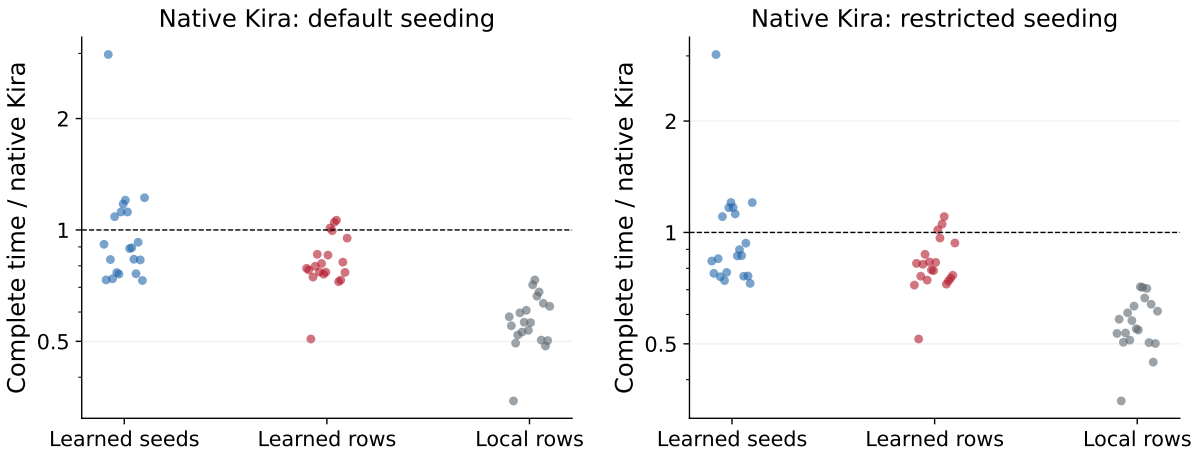


Figure 19: The methods using stored identities compared with native component reduction at the same targets and master basis. Native timings include equation generation and exact combination of the component results. This comparison changes the available source representation as well as the external method, so it does not isolate the benefit of learning.

Across both comparisons, all 876 trials produced a checked complete result within the stated bounds. The paired timing ratios above use three completed repeats for both methods. The recorded limits and three-repeat design bound the interpretation of small timing differences on the shared cluster.

These results describe the implemented policies and the stated symbolic Kira settings. The stored-pool tests reuse previously generated identities, and the native tests measure learned sector bounds that can include seeds beyond the model's selected set. The row comparison in this section acts on fixed source identities and target rows. Section 12 extends the test to changing identity rows inside Kira. General high-loop scattering families and finite-field reconstruction require further tests. Reduced input counts alone do not establish a speed improvement because equation selection, model execution, exact simplification and fallback all contribute to the total cost.

12 Learned Elimination Inside Kira's Evolving Matrix

The row comparison in Section 11 used fixed source identities to simplify target expressions before the final Kira call. To test the intended use inside the solver, we now apply the frozen row model during Kira's symbolic forward elimination. The input is the current selected IBP matrix, and every exact row update changes the state used for the next prediction. Kira continues to generate and select equations, preserve its integral ordering, normalize the triangular system and perform back substitution [9].

12.1 Choices in the current IBP matrix

We retain the matrix notation of Eq. (80). At step n, $A^{(n)}_{rj}$ is the coefficient of integral j in identity row r. Let $w(j)$ be Kira's ordering weight for integral column j, with larger weights denoting integrals to eliminate earlier. The leading column of a nonzero row r and the rows with a common leading column are

$$h_n(r) = \arg\max_{j:A^{(n)}_{rj}\neq 0} w(j), \qquad R^{(n)}_k = \{r : h_n(r) = k\}. \tag{88}$$

At each step we take the largest-weight column that leads at least two rows. If this column is k_n, the allowed choices are

$$\mathcal{A}_n = \{(q, r, k_n) : q, r \in R_{k_n}^{(n)},\ q \neq r\}. \tag{89}$$

Here q is the source row and r is the destination row. The prescribed master columns remain protected. A proposed elimination of a protected master, or a zero pivot at the public sampling point, causes the learned trial to fail its interface check.

The model assigns probabilities to every pair in Eq. (89) and selects the largest score. Kira then uses its existing symbolic update to calculate

$$A_{rj}^{(n+1)} = A_{rj}^{(n)} - \frac{A_{rk_n}^{(n)}}{A_{qk_n}^{(n)}} A_{qj}^{(n)}. \tag{90}$$

All these rows represent zero identities, so this operation preserves their solution space. The new destination row has a lower leading integral, or becomes zero and leaves the matrix. It can supply a source row in a later step. The next prediction therefore uses the updated identities throughout the matrix.

We use the saved 132-feature representation and model weights without retraining. The family definition, master labels and current rows supply the features; construction records and reference master coefficients remain outside the predictions. Rational coefficients at the fixed public finite-field point supply cancellation features. Kira and Fermat perform the actual row updates with symbolic rational coefficients. The process holding the model remains available during one forward solve, and it receives the initial matrix followed by each changed row. A system with no allowed row updates does not load the model.

12.2 Matched controls and measurements

We use the same seven families with independent master bases and 99 targets as the native comparison in Section 11. They include three one-loop box profiles, a one-loop bubble, triangle and tadpole, and a two-loop propagator. All methods use the same restricted native seed bounds, target components, preferred masters and symbolic output checks.

Four methods distinguish the learned choice from changes needed to measure it. Stock Kira uses the installed solver. Instrumented Kira retains that solver's forward algorithm and records its exact updates. The deterministic control uses the largest repeated leading integral defined above and selects a source by Kira's row-length and integral-order rule. A stable row identifier selects its destination. The learned method uses the same serial schedule and selects both rows from the allowed pairs. This control separates the model's decisions from the effect of the new schedule. Both methods return the triangular system to Kira for normalization and back substitution.

Each method runs three times per family on the same node, with rotated execution order and one computation thread. Complete elapsed time includes preparation, inference, exact algebra, target combination and coefficient verification. We also time the forward-elimination phase directly inside Kira and record model loading separately. The diagnostic quantity

$$T_{\text{fwd,load excluded}} = T_{\text{fwd}} - T_{\text{model load}} \tag{91}$$

subtracts that recorded loading interval; it retains feature construction, scoring, communication and exact row updates. It is a timing decomposition of these runs; a separate run with a preloaded model has not been measured. Ratios use three-repeat medians and the paired geometric mean of Eq. (87). Forward-stage ratios include only cases with actual row updates

in both methods. The tadpole batch needs no forward row updates and does not contribute to those phase ratios.

For each committed update, we retain the convention $1 + 2n_q$, where n_q is the source-row nonzero count. This counts coefficient-field operations and excludes the internal polynomial work in rational simplification. Elapsed time includes that work and the candidate trial updates used to prepare features. The tests keep the action capacity at 10,000, the matrix capacity at 10,000 rows and 250,000 nonzeros, and use bounded time and memory. These limits bound the systems and action sets tested here.

12.3 Exact checks and measured results

All 84 final trials produced the prescribed symbolic master coefficients. Independent evaluation of the archived native exports passed 3,564 checks of target coefficients at independent sample points, and the saved row updates passed 12,708 independent finite-field checks. The model made 1,413 choices across its repeated runs. In 465 choices, its source row had already changed in an earlier elimination. These records establish that the model acted on an evolving identity matrix [27].

The complete elapsed-time ratios for the learned method were 1.305 against stock Kira, 1.286 against instrumented Kira, and 1.274 against the deterministic control. Within forward elimination itself, the ratio against instrumented Kira was 276.149 over 6 active family batches. After subtracting the recorded model-loading interval, the ratio was 62.270. For those active batches, the native forward-stage medians ranged from 0.003267 to 0.076143 seconds, while the learned medians ranged from 3.061 to 6.909 seconds.

Summing one median operation count per family gives 5,771 coefficient operations for the learned method, 5,070 for instrumented Kira and 5,097 for the deterministic control. The corresponding learned and native row-update totals were 471 and 470. These sums count each family once, while the independent checks above cover all repeated runs. Figures 20 and 21 show the family-level measurements.

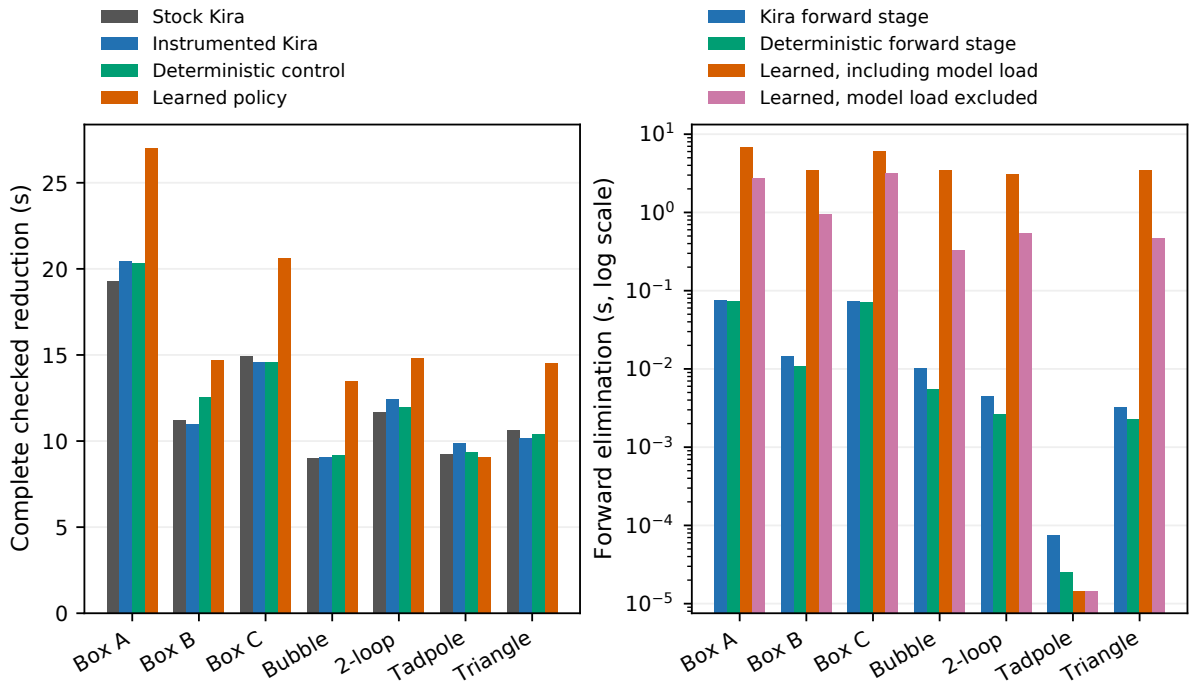


Figure 20: Complete checked reduction time and the forward-elimination time measured inside Kira. Bars show three-repeat medians. The forward panel uses a logarithmic time axis and separately shows the learned time after subtracting the recorded model-loading interval. The tadpole case has no row updates, so its forward time measures phase overhead. Box A, B and C denote profiles 0173, 0633 and 0156, respectively.

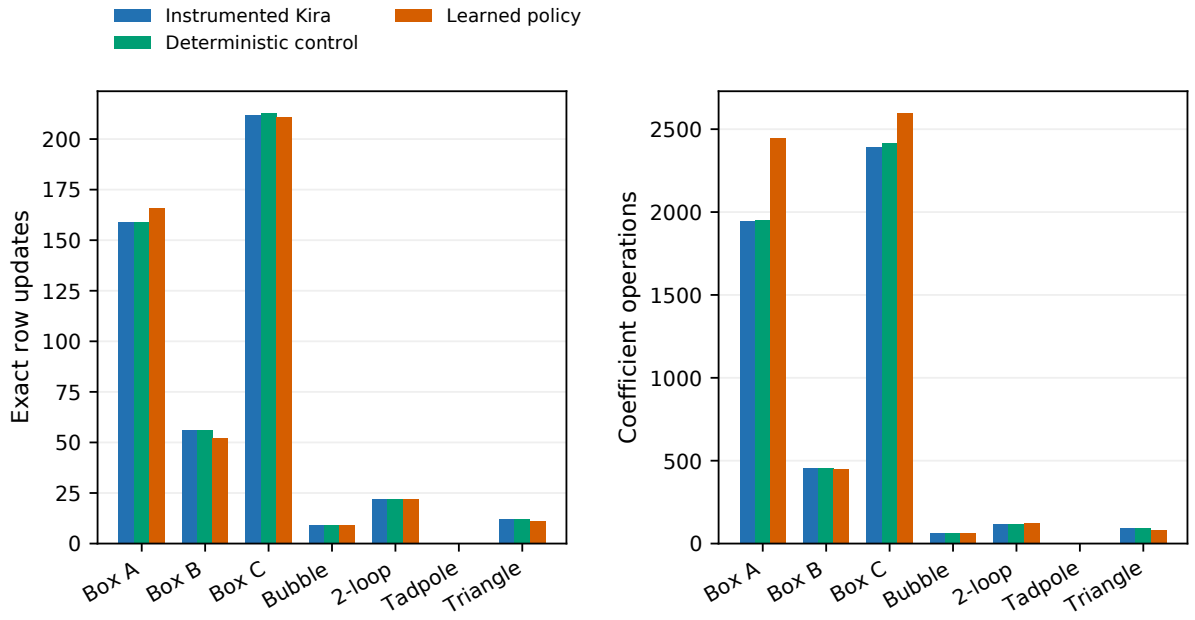


Figure 21: Committed row updates and coefficient-field operations for the same evolving matrices. Each update contributes $1 + 2n_q$ operations. The count excludes candidate scoring and polynomial simplification, whose costs remain in the time measurements.

12.4 Implications for the proposed reduction policy

This experiment tests learned choices inside Kira's changing selected IBP matrix. The measured cost includes the consequences of those choices for later rows and for the final exact reduction. The current model increases aggregate runtime, and its summed coefficient-operation count also exceeds the native count. Excluding model loading does not produce a forward-elimination speedup in this comparison.

Kira's native forward phase is already short for these selected systems. The added feature construction and scoring therefore have a small available time budget. The model was trained on target-row reductions with fixed source identities, so its application to changing identity rows also tests transfer beyond those training states. The results apply to the stated one- and two-loop families and a forward policy that preserves Kira's integral ordering. Larger independent multiloop systems, other admissible pivot-column choices and learned back-substitution order remain outside the present experiment.

13 Direct Seed Filtering and Lower-Cost Model Inference

The preceding comparisons show that a learned choice must save more algebraic work than its prediction costs. We therefore changed the interfaces between the frozen models and Kira, and measured the complete reductions again. The seed interface can now select individual seeds before Kira constructs their equations. We also tested a smaller set of row choices that follows the form of the reduction model's training examples. The original checkpoints remain unchanged.

13.1 Relating the predicted seeds to Kira's targets

A stored target can contain several integrals,

$$\mathcal{T}_\nu = \sum_{\boldsymbol{a}\in\mathcal{J}_\nu} c_{\nu\boldsymbol{a}}(\boldsymbol{x}, d) I_{\boldsymbol{a}}. \tag{92}$$

Here ν identifies a target, $\mathcal{J}_\nu$ is its set of component index vectors, and $c_{\nu\boldsymbol{a}}$ is the known coefficient of each component. The native interface asks Kira to reduce these components and then combines their reductions with exact algebra. This procedure provides two possible inputs for the seed model: the complete expression $\mathcal{T}_\nu$, or each component $I_{\boldsymbol{a}}$ with coefficient one.

Let $\mathcal{C}$ be the stored candidate seed pool and let $p_\theta(\boldsymbol{b} \mid \mathcal{T}_\nu)$ denote the frozen model's score for seed $\boldsymbol{b}$, with the family and chosen masters also supplied. For a threshold τ, the expression-based seed set is

$$\widehat{\mathcal{S}}_{\mathrm{expr}}(\tau) = \bigcup_{\nu} \{\boldsymbol{b} \in \mathcal{C} : p_\theta(\boldsymbol{b} \mid \mathcal{T}_\nu) \geq \tau\}. \tag{93}$$

The component-based set instead uses

$$\widehat{\mathcal{S}}_{\mathrm{comp}}(\tau) = \bigcup_{\boldsymbol{a}\in\bigcup_\nu \mathcal{J}_\nu} \{\boldsymbol{b} \in \mathcal{C} : p_\theta(\boldsymbol{b} \mid I_{\boldsymbol{a}}) \geq \tau\}. \tag{94}$$

Components already in the prescribed master basis need no prediction. A seed set sufficient for the combined expression can differ from a set sufficient for all components separately, since terms can cancel in Eq. (92). Both interfaces therefore require a check of the final expression in the prescribed master basis.

We also test a local enlargement of a predicted set,

$$\mathcal{N}_1(\mathcal{S}) = \left\{ \boldsymbol{b} \in \mathcal{C} : \min_{\boldsymbol{a} \in \mathcal{S}} \sum_{i=1}^{N} |b_i - a_i| \leq 1 \right\}. \tag{95}$$

Here N is the number of family indices. This rule adds candidate seeds that differ by one unit in one index. It supplies nearby IBP relations without using the reference reduction. It gives no general proof that the set suffices.

13.2 Filtering before equation construction

The earlier native adapter represented predictions by sector bounds. Those bounds include intermediate seeds even when the model omitted them. The new interface still supplies bounds that contain the chosen seeds, but it also checks each seed vector before Kira constructs its IBP equations. A vector outside the selected set contributes no IBP equations through this call. Kira retains its symmetry handling, equation selection, exact elimination and back substitution [9]. The audit records both the allowed seed vectors and the vectors that the generator actually used.

The model scores and the reference master coefficients serve separate purposes. Predictions use only public family, target, master-label and candidate information. After reduction, we check whether the target contains any integral outside the prescribed master basis. An incomplete learned attempt triggers a standard restricted native calculation within the remaining time allowance. The final symbolic coefficients are then compared with the recorded exact answer. Prediction, preparation, unsuccessful attempts, fallback, target combination and exact checks all contribute to the measured complete time.

13.3 Reducing prediction costs and choosing a source row

We exported the trained weights to a NumPy implementation of the same layers. This removes the need to import the training framework for each reduction, and it reuses the family, master and candidate embeddings within a target batch. On 64 development targets, the seed probabilities differed from the original implementation by at most 4.18×10^{-6}, and every decision at threshold 0.5 agreed. Separate bounded feature checks verified the exported row network. These checks concern numerical inference; exact reduction still comes from Kira and the final coefficient checks.

For the row model, we first retained the full changing-matrix interface of Section 12 with the exported network. We then restricted each decision to a source row for one current destination. Using the leading-column group $R^{(n)}_{k_n}$ defined there, the largest row identifier selects the destination r_n, and the model scores

$$\mathcal{A}^{\mathrm{source}}_n = \{(q, r_n, k_n) : q \in R^{(n)}_{k_n},\ q \neq r_n\}. \tag{96}$$

The feature input contains the current rows in this group, with the destination assigned the target role. Thus the model sees one changing destination and its allowed sources, as in its training examples. Kira applies the selected exact update from Eq. (90). The source can itself have changed in an earlier step. The candidate count falls from $g(g-1)$ to $g-1$ for a group of g rows, while Kira's ordering still fixes the pivot column. If the group contains only two rows, the supplied action set has one element, so we apply that forced action without evaluating the network.

We also implemented the source-row network inside Kira's compiled reduction code. Each choice reads the current rows directly, samples their rational coefficients at the fixed finite-field

point and forms the same input features. The trained weights and features retain their single-precision values, while the compiled layer arithmetic uses double precision. An independent implementation with the same arithmetic precision checks the probabilities, and development tests also compare selected sources with the original single-precision network. This numerical prediction only selects a valid source; Kira continues to perform the rational row update exactly.

The compiled interface removes per-choice communication with a Python process. It retains an audit of chosen rows and their probabilities, and the measured time includes this audit. A combined method applies both the learned seed filter and the compiled row policy during the same reduction. An incomplete seed attempt uses the same exact fallback rule as the seed-only method.

This restriction changes the model's supplied state and action set. Its purpose is to reduce prediction work and test transfer to a closer training configuration. It does not establish an optimal elimination order, and its benefit must be measured against Kira's native choices.

13.4 Checking and expanding an incomplete seed set

In separate development trials, we tested whether Kira's initial finite-field master check could reject an incomplete seed set before symbolic elimination [9]. This check compares the integrals left by the generated system with the prescribed master basis. A rejected set can then be enlarged before Kira starts the more expensive symbolic calculation. The final coefficients still require exact checks, since a numerical rank check alone does not prove the full symbolic reduction. We retained the frozen policies in the reserved comparison while testing these extensions.

For the first expansion rule, we rank each candidate seed by its largest model score across the supplied expressions,

$$Q_\theta(\boldsymbol{b}) = \max_\nu p_\theta(\boldsymbol{b} \mid \mathcal{T}_\nu). \tag{97}$$

We begin with the first $k_0 = 32$ seeds and, after a rejected master check, increase the budget according to

$$k_{j+1} = \min(2k_j, |\mathcal{C}|). \tag{98}$$

Here j is the number of completed expansion steps and $\mathcal{C}$ is the candidate pool defined above. A control ranks seeds by their smallest index distance to a component of a target expression, and uses the same budget rule. The full candidate pool and restricted native seeding provide two further controls.

A second rule begins with 64 seeds and uses the additional integrals returned by the master check as new model inputs. Each such input contains one integral with coefficient one. We add its four highest-ranked unused candidates and repeat the check, using the full pool after five rejected checks. A distance-based control applies the same expansion to the same additional integrals. These checks require every component to reduce in the prescribed basis, which can be stricter than reducing the complete expression when terms cancel.

The 26 development trials gave 23 exact completions and three bounded incomplete calculations. The latter used 64-target batches and affected the native control and both learned variants in the initial check experiment. All completed outputs passed 1104 independent finite-field target checks. Across four complete 16-target comparisons, the doubling rule with learned ranking took 1.022–1.422 times the native control's complete time. In the subsequent residual-driven trial, learned ranking reached a complete basis with 71 seeds, while the distance control used 121. Its complete-time ratio against native seeding was 0.996. These are single development measurements; they do not establish a repeatable speedup for either expansion strategy [28].

13.5 Development, validation and reserved comparisons

We selected family instances before the new timing comparisons. Development and adapter-validation instances use separate kinematic groups from the original model-training partition. The reserved cost-test instances come from the original test partition, and their kinematic groups exclude every box group in the earlier cost comparison. The network weights remain fixed throughout this selection. The adapter-validation families belong to the original training partition, so this stage selects an interface configuration and does not measure generalization of the network itself. We performed development comparisons on three of the reserved development instances, selected the seed configuration on six validation instances and then froze the policy before testing twelve instances from six reserved kinematic groups.

All new native cases use symbolic reduction of one-loop box families with independent propagator bases. Each case uses the same target expressions and master basis for every method. Controls include restricted native seed bounds, bounds taken directly from target indices, and the full stored candidate pool with the individual-seed filter. These controls separate the learned selection from changes to seed bounds or equation generation.

Every trial uses one computation thread with explicit time, memory and scratch limits. We increased the array allowance from two simultaneous jobs to four and then eight after checking the available node memory and project storage. This changed the scheduling of independent trials while preserving their individual resource limits and frozen policies. Methods run in rotated order on the same node for each family and target-batch size. The retained-evidence limit interrupted some batches before a new trial could start; after lossless archive recompression, we resumed the missing trials on their original nodes and retained every completed observation. Every reserved method receives the same initial 160-second native calculation allowance within a 180-second complete-process limit. The memory and scratch limits are also the same. We retain failed trials and report their completion status. The validation rule assigns the 180-second total cap to an incomplete trial when choosing the learned seed configuration. This capped score is a selection rule; it is not a measured time for a completed reduction. The reserved comparison uses the frozen configuration, five repetitions and batches of 16 and 64 stored target expressions. After an unsuccessful attempt, the remaining identical repetitions for that method and batch are marked as skipped. Skipped repetitions receive no measured time. Timing ratios require five successful repeats for both methods, while completion and failure counts retain every attempted case.

13.6 Reserved results and exact checks

The frozen comparison contains 720 planned records across twelve family instances, two target-batch sizes, six methods and five repetitions. We executed 616 trials and obtained 590 exact completions; the remaining 26 executed trials exceeded a resource limit or failed to complete. The early-stop rule left 104 repetitions unexecuted. These skipped records contain no timing measurement. The retained inputs, policy choices, successful reductions and failure records accompany the data release [28].

Validation selected the complete target expression with threshold $\tau = 0.01$. This configuration and both trained checkpoints remained fixed during the reserved tests. Each method used the same master basis and targets in a batch.

For the three controls, restricted native Kira produced 105 exact completions, 3 failed trials and 12 skipped repetitions, the target-bound control produced 100 exact completions, 4 failed trials and 16 skipped repetitions, and the full candidate-pool control produced 105 exact completions, 3 failed trials and 12 skipped repetitions. These counts describe the bounded experiment and

do not measure whether an unlimited calculation would complete.

Successful restricted-native trials took 10.54–112.46 seconds for the complete checked calculation. This range includes different family instances, target-batch sizes and CPU models. The paired ratios compare methods on the same node within each batch.

For the learned methods, the learned seed filter produced 100 exact completions, 4 failed trials and 16 skipped repetitions, the compiled row policy produced 105 exact completions, 3 failed trials and 12 skipped repetitions, and the combined method produced 75 exact completions, 9 failed trials and 36 skipped repetitions. Completed seed and combined trials used the standard fallback 30 and 50 times, respectively.

Every completed final expression matched its recorded rational master coefficients. The independent audit performed 67,920 additional master-coefficient sample checks and 65,355 exact-row sample checks. It checked 11,405 compiled model choices and 10,380 forced source actions. Trace checks cover completed native attempts; 55 failed attempts had incomplete traces. The final coefficient checks still apply to any successful fallback. Input and checkpoint hashes agree with the values frozen before testing.

13.7 Complete reduction time and algebraic work

For each family and batch size, we take the median of five successful repetitions. We then form the ratio of learned time to control time and report the geometric mean across complete paired batches. A ratio below one means a lower measured time. We resample complete kinematic groups 10,000 times to obtain descriptive 95% intervals, keeping related profiles and batch sizes together. These intervals describe the completed subset of this small collection; failed and skipped trials remain in the completion counts above.

Restricted native Kira completed all five repetitions in 1 batches where the learned seed filter did not, while the learned seed filter completed all five repetitions in 0 batches where that control did not.

The learned seed filter has a complete-time ratio of 1.295 (descriptive 95% interval 1.156–1.542; 20 batches) against restricted native Kira. Its native-calculation subtotal, including seed prediction when used, has ratio 1.400 (descriptive 95% interval 1.201–1.816; 20 batches). The subtotal excludes input preparation, final target combination and verification, which remain included in the complete time.

Restricted native Kira completed all five repetitions in 0 batches where the compiled row policy did not, while the compiled row policy completed all five repetitions in 0 batches where that control did not.

The compiled row policy has a complete-time ratio of 1.024 (descriptive 95% interval 0.998–1.053; 21 batches) against restricted native Kira. Its native-calculation subtotal, including seed prediction when used, has ratio 1.025 (descriptive 95% interval 1.001–1.078; 17 batches). The subtotal excludes input preparation, final target combination and verification, which remain included in the complete time.

Restricted native Kira completed all five repetitions in 6 batches where the combined method did not, while the combined method completed all five repetitions in 0 batches where that control did not.

The combined method has a complete-time ratio of 1.983 (descriptive 95% interval 1.575–2.476; 15 batches) against restricted native Kira. Its native-calculation subtotal, including seed prediction when used, has ratio 2.289 (descriptive 95% interval 1.247–3.673; 8 batches). The subtotal excludes input preparation, final target combination and verification, which remain included in

the complete time.

Against the target-bound control, complete-time ratios are 0.902 (descriptive 95% interval 0.711–1.163; 20 batches), 0.712 (descriptive 95% interval 0.597–0.864; 20 batches), 1.415 (descriptive 95% interval 0.902–2.160; 14 batches), in seed, row and combined order. Against the full candidate-pool control, the corresponding ratios are 1.317 (descriptive 95% interval 1.167–1.569; 20 batches), 1.040 (descriptive 95% interval 1.016–1.061; 21 batches), 2.028 (descriptive 95% interval 1.565–2.616; 15 batches). Each ratio uses its own complete paired cohort, so comparisons with different failure patterns can contain different batches.

The learned seed filter has a generated-equation ratio of 0.168 (descriptive 95% interval 0.097–0.294; 20 batches), a selected-equation ratio of 0.982 (descriptive 95% interval 0.797–1.205; 20 batches) and a forward-update ratio of 3.575 (descriptive 95% interval 2.647–5.211; 20 batches) against restricted native Kira. These are counters from the native reduction passes, including completed fallback passes.

The compiled row policy has a generated-equation ratio of 1.141 (descriptive 95% interval 1.032–1.275; 21 batches), a selected-equation ratio of 1.141 (descriptive 95% interval 1.032–1.275; 21 batches) and a forward-update ratio of 0.905 (descriptive 95% interval 0.840–0.972; 17 batches) against restricted native Kira. These are counters from the native reduction passes, including completed fallback passes.

The combined method has a generated-equation ratio of 0.462 (descriptive 95% interval 0.254–0.745; 15 batches), a selected-equation ratio of 1.341 (descriptive 95% interval 1.080–1.582; 15 batches) and a forward-update ratio of 2.363 (descriptive 95% interval 1.041–4.534; 8 batches) against restricted native Kira. These are counters from the native reduction passes, including completed fallback passes.

Generation counters count equations reported in successive generation passes and can include repeated equations. A forward update is one committed elimination step; this count does not measure every polynomial or rational-function operation. Failed attempts can have truncated work counters, so the complete time and completion status remain the primary measures.

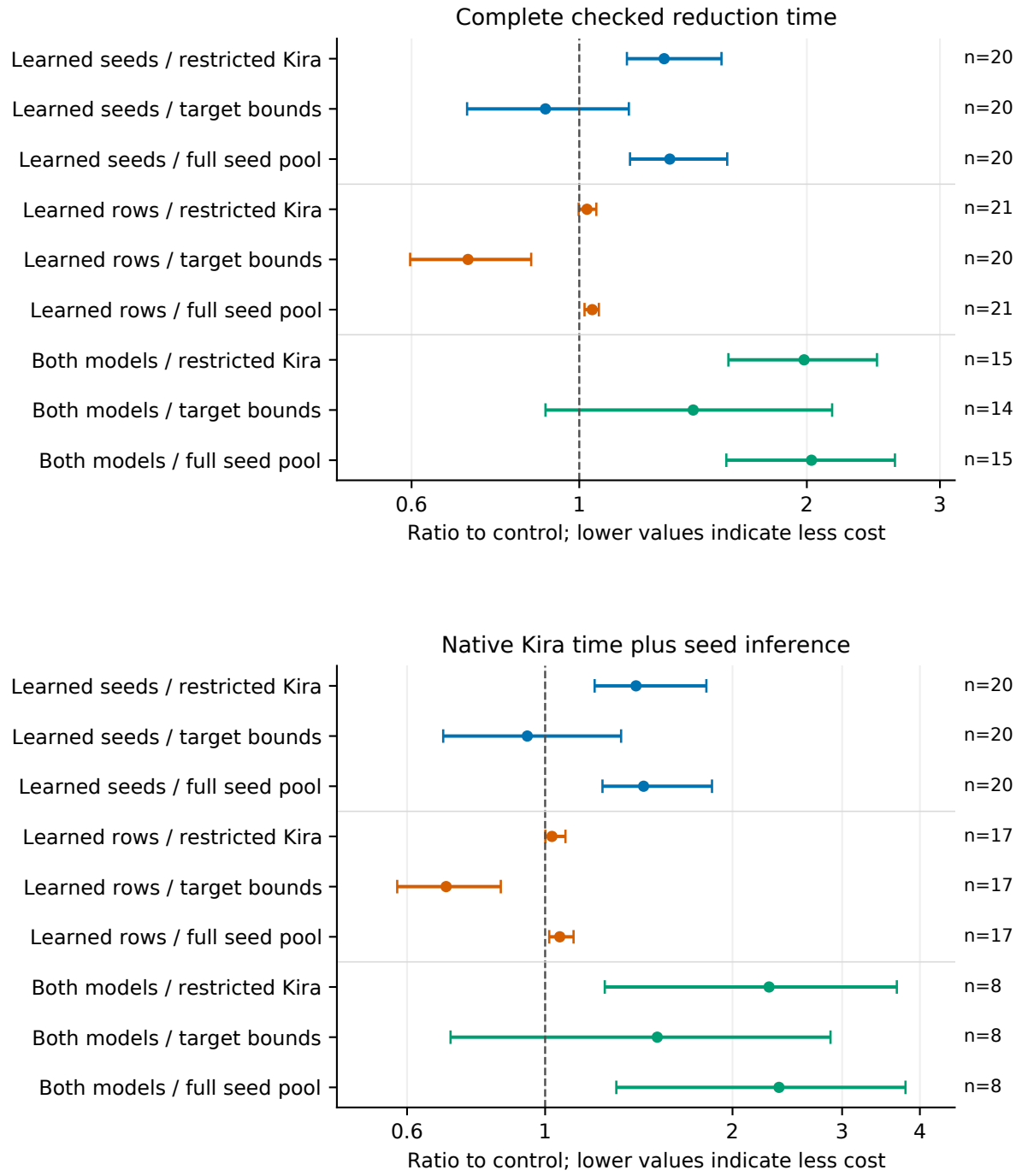
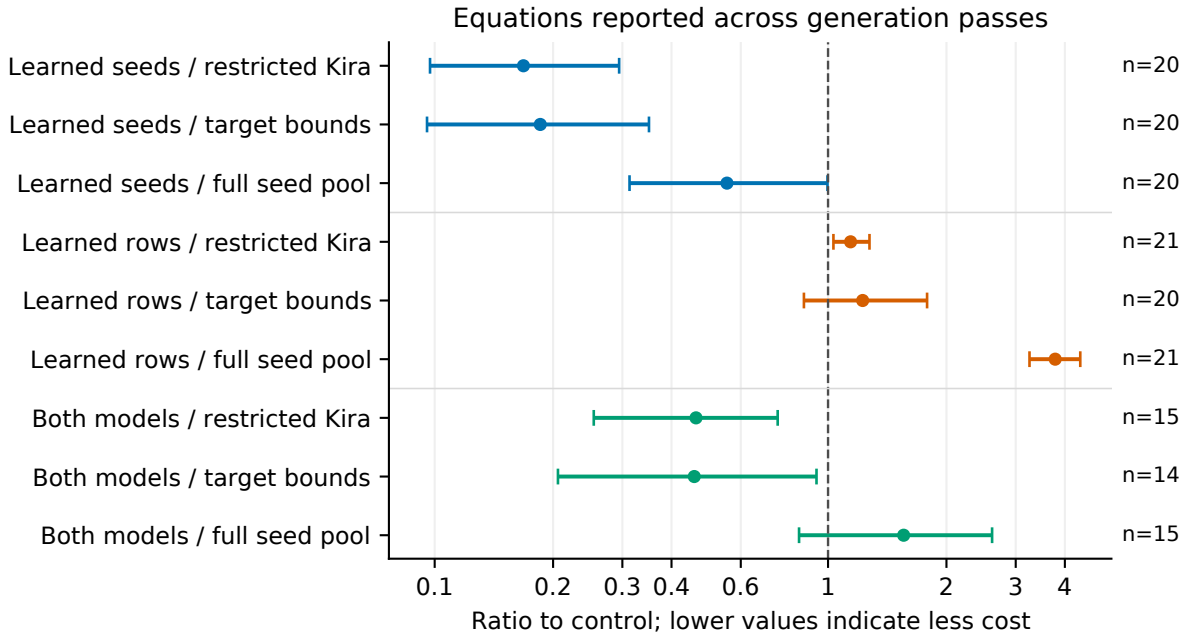


Figure 22: Learned-to-control time ratios in the reserved native comparison. The upper panel includes the complete checked reduction, and the lower panel shows the native subprocess subtotal plus seed prediction. Points show geometric mean ratios and lines show descriptive intervals obtained by resampling kinematic groups. The label n gives the number of paired batches with five successful repeats for both methods.



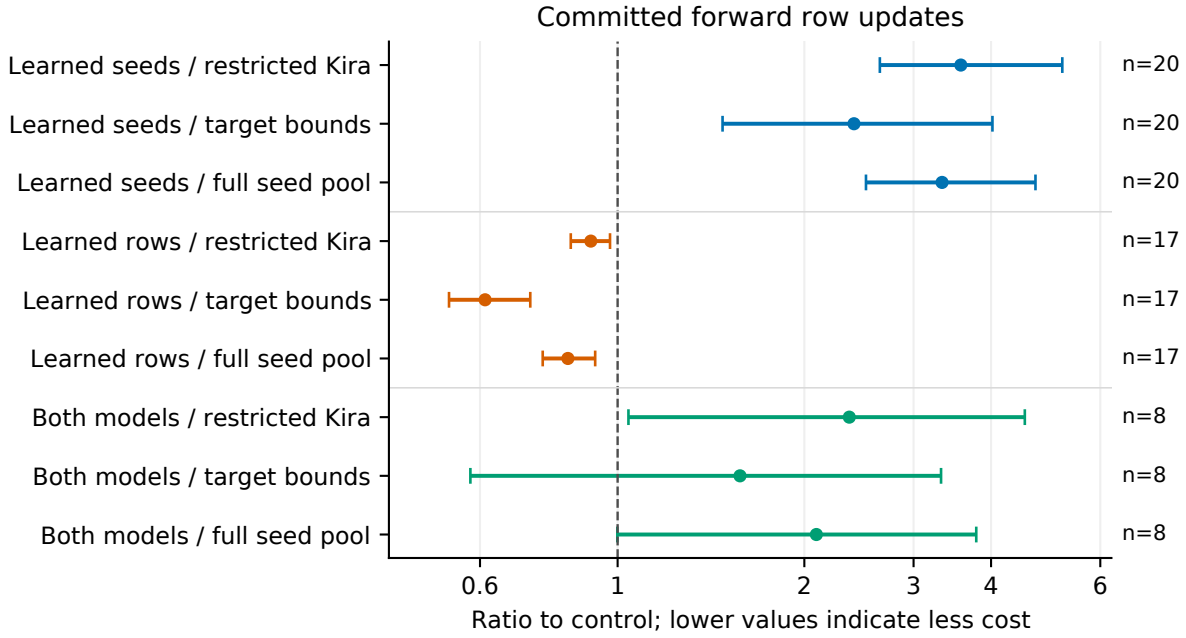


Figure 23: Generated-equation and forward-update ratios on complete paired batches. The controls and cohort labels follow Fig. 22. These counters measure reported generation work and committed updates, while symbolic coefficient complexity can change the cost of each update.

13.8 Numerical inference and scope of the result

The compiled-model development audit checked 256 action feature vectors and 2,247 sampled coefficients. The maximum feature difference was 0, and the largest probability difference from the independent double-precision calculation was 1.41×10^{-13}. All selected sources agreed with the original single-precision model in these development trials. The separate maximum probability difference from that original arithmetic was 1.6×10^{-05}.

The largest sampled process-tree memory was 0.506 GB, and the largest trial scratch directory was 49.29 MB. The bounded runs kept their scientific records and model files on Hoffman; the report contains the derived plots and results.

The complete paired comparisons do not show a lower aggregate runtime than restricted native Kira for these learned methods. Lower inference costs alone do not ensure that the chosen seeds and rows save enough algebraic work. The new native tests cover one-loop boxes and do not establish performance on independent high-loop scattering families. The existing weights were trained to recover recorded seed sets and row choices, and Section 14 reports a separate cost-trained policy on native Kira trajectories.

13.9 Transfer to a two-loop propagator family

We also applied the unchanged interfaces to the massless two-loop propagator family used in the earlier cost comparisons. Its five independent family indices fit the compiled row interface. We fixed two batches of 16 and 64 stored target expressions and all policies before this transfer calculation. Each method uses five rotated repetitions, the same master basis and the same time and memory limits. Storage-interrupted trials resume on their original node, with earlier observations preserved. Because this family appeared in earlier comparisons, this experiment measures transfer of the revised implementation and does not establish generalization to a newly unseen family.

Alongside native seeding and the full candidate pool, we include a deterministic source-row control using the same compiled reduction structure. This control selects a source by row length and integral ordering within the same changing leading-column group. Comparing the learned row policy with this control separates the source choice from changes to the compiled reduction structure. Every completed reduction receives the same exact coefficient check and an independent finite-field check.

The transfer experiment contains 60 measured trials among 60 planned records, with 60 exact completions. The independent audit performed 7,200 target-coefficient checks and 2,355 row-update checks.

For 16 targets, the native control's median complete time was 12.003 seconds. The learned seed filter gave a time ratio of 0.992 with descriptive interval $[0.962, 1.030]$; the learned source-row policy gave a time ratio of 0.991 with descriptive interval $[0.984, 1.022]$; the combined policy gave a time ratio of 0.976 with descriptive interval $[0.961, 0.996]$. In the matched controls, the learned seed filter relative to the full pool had ratio 1.051 with interval $[1.024, 1.072]$ and the learned source choice relative to the deterministic compiled control had ratio 0.987 with interval $[0.979, 1.034]$.

For 64 targets, the native control's median complete time was 12.770 seconds. The learned seed filter gave a time ratio of 1.758 with descriptive interval $[1.744, 1.777]$; the learned source-row policy gave a time ratio of 0.976 with descriptive interval $[0.964, 0.992]$; the combined policy gave a time ratio of 1.755 with descriptive interval $[1.740, 1.777]$. In the matched controls, the learned seed filter relative to the full pool had ratio 1.907 with interval $[1.880, 1.920]$ and the

learned source choice relative to the deterministic compiled control had ratio 0.987 with interval $[0.952, 1.012]$.

The compiled learned-row pipeline reduced the median complete time for 64 targets from 12.770 to 12.460 seconds, a measured reduction of 2.4%. Its ratio against the deterministic compiled control has an interval that includes one, so this experiment does not resolve a benefit caused by the learned source choice itself. The full-pool control was faster, at 11.773 seconds. For 16 targets, the combined pipeline reduced the native median by 2.4%, but the full-pool control again completed sooner. For 64 targets, every learned-seed and combined trial required fallback, whose cost is included in the reported time. These observations establish limited gains relative to the stated native baseline and also identify stronger deterministic controls.

Figure 24 shows all complete paired comparisons. A ratio below one denotes a lower median complete time. The intervals come from 10,000 paired resamples of the five repetitions and describe repeat-to-repeat variation within this one family. They do not measure uncertainty across different integral families. Incomplete pairs remain in the completion counts and receive no timing ratio [28].

13.10 Applying the seed model to complete target expressions

The native interface reduces each component of Eq. (92) before combining the result. We also applied the fast seed model directly to that complete expression, using Kira's supplied-equation mode. This tests the form of the target used to train the seed model and allows cancellations between its components during the reduction.

Let $\mathcal{E}_{\boldsymbol{b}}$ be the stored IBP identities generated from candidate seed $\boldsymbol{b}$, and let $\mathcal{E}_{\mathrm{bg}}$ contain the stored symmetry and other background identities. The selected equation set is

$$\mathcal{E}(\mathcal{S}) = \mathcal{E}_{\mathrm{bg}} \cup \bigcup_{\boldsymbol{b}\in\mathcal{S}} \mathcal{E}_{\boldsymbol{b}}. \tag{99}$$

For each requested expression, we add its defining relation

$$\mathcal{T}_\nu - \sum_{\boldsymbol{a}\in\mathcal{J}_\nu} c_{\nu\boldsymbol{a}} I_{\boldsymbol{a}} = 0. \tag{100}$$

Kira eliminates the supplied system with the target expressions ordered above their component integrals and the prescribed masters preferred at the end of the ordering. It therefore obtains the master expansion of $\mathcal{T}_\nu$ in one exact solve. The same public identity pool, target definitions and master basis enter every compared method.

The model takes the union of candidates whose score reaches $\tau = 0.01$, using the unchanged weights and the fast inference implementation. We tested this set both with and without an early finite-field master check. An incomplete selected system triggers a full-pool calculation within the remaining time allowance. The early check compares the remaining integrals with the prescribed master basis; exact final coefficient checks still determine whether the requested reductions agree with their recorded answers. A global master check can require more relations than a particular target combination needs.

These calculations begin with stored IBP identities, whose prior generation cost is excluded for every method. The measured complete time includes loading those inputs, prediction, equation selection and export, Kira's exact calculation, admission checks when used, failed attempts, fallback and final verification. Thus the result concerns repeated reduction from a common identity pool. It does not measure native generation of a new family's equations.

The separate development and adapter-validation collection contains 80 trials, with 74 exact completions and 6 incomplete calculations. Independent checks evaluated 2,013 target coeffi-

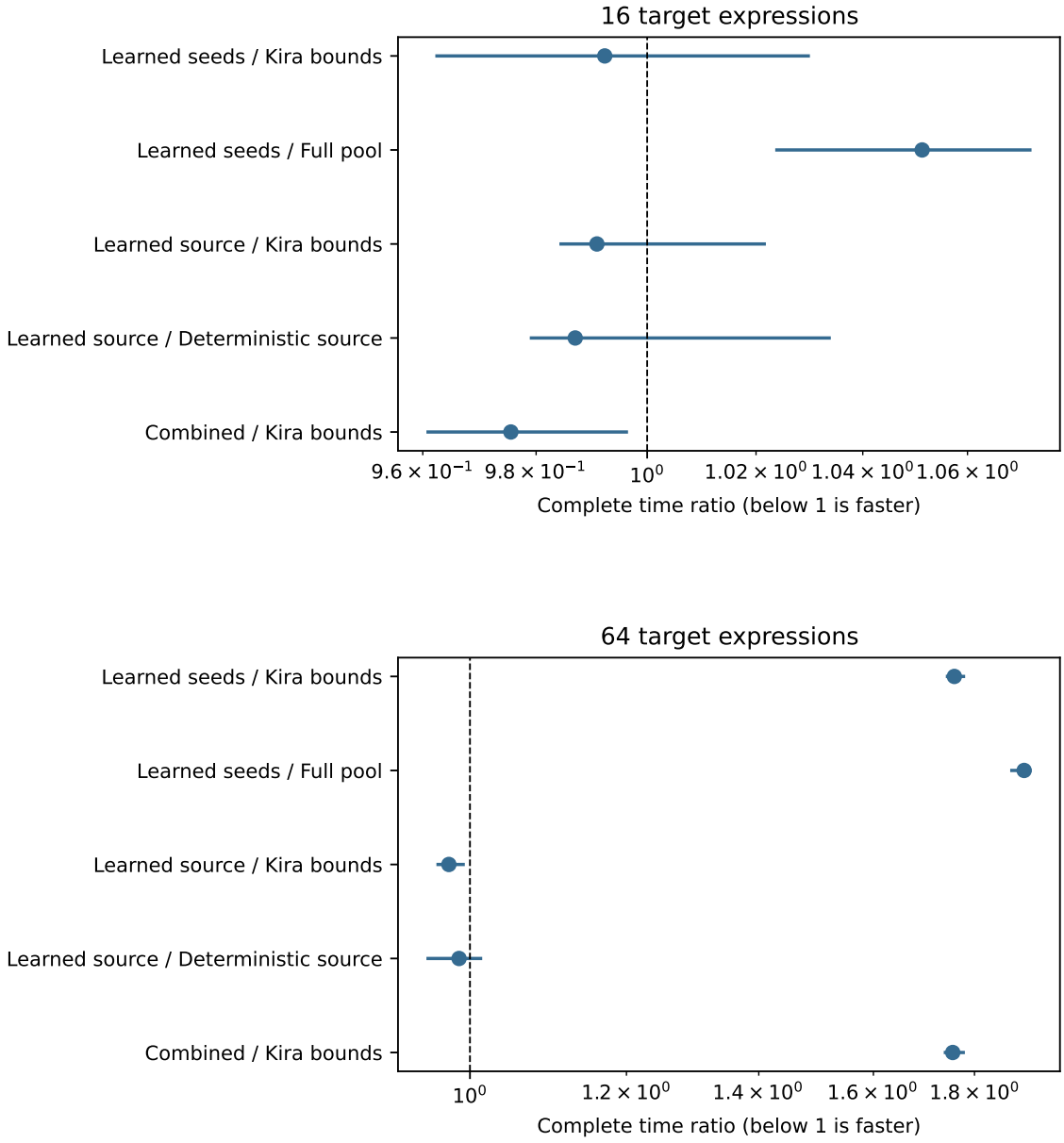


Figure 24: Complete-time ratios in the two-loop transfer comparison for 16 targets (upper panel) and 64 targets (lower panel). Points show ratios of five-repeat medians, and bars show descriptive paired-repeat intervals. All times include prediction, target preparation, exact reduction, combination, checks and any required fallback.

cients at stored finite-field points. The six adapter-validation family instances belong to the original training partition and use two batch sizes, one and sixteen targets. We retained every measured cost and selected a learned interface only when its geometric mean complete-time ratio to the full-pool control was below one and it preserved every control completion.

The validation ratio without the early master check was 1.113 across 11 complete paired batches. The full-pool control completed 0 additional batches that this learned interface did not complete. These single validation measurements select the interface and do not establish a repeatable timing gain.

The validation ratio with the early master check was 0.930 across 11 complete paired batches. The full-pool control completed 0 additional batches that this learned interface did not complete. These single validation measurements select the interface and do not establish a repeatable timing gain.

13.11 Reserved tests of complete-expression reduction

Validation selected the early master check. We froze this choice before applying it to the twelve original-held-out box instances used in the native comparison. The network weights and threshold remain fixed. Each instance supplies batches of one and sixteen target expressions, with five rotated repetitions per method on the same node. These are reused physical families and constructed targets; the new comparison tests the revised interface on original-held-out model data.

The controls use the full stored identity pool and a fixed set of 64 seeds ranked by their smallest index distance to any target component. The distance control uses the same exact reduction and full-pool fallback as the learned method. This first distance control proceeds directly to symbolic elimination, while the selected learned method uses the early master check. Their time ratio therefore includes both seed selection and the difference in admission checks.

The frozen plan contains 360 records. We executed 348 trials and obtained 345 exact completions; 3 measured trials failed to complete within the bounds, and 12 remaining repeats were skipped after an earlier failure. Every completed master expansion matched its exact reference, and the independent audit checked 8,460 target coefficients directly against the exported Kira reductions. Skipped repetitions have no assigned runtime.

The learned interface produced 115 exact completions, 1 failed trial and 4 skipped repeats. Completed trials used fallback 25 times.

The full-pool control produced 115 exact completions, 1 failed trial and 4 skipped repeats. Completed trials used fallback 0 times.

The distance control produced 115 exact completions, 1 failed trial and 4 skipped repeats. Completed trials used fallback 55 times.

We form a median from five successful repetitions for each method and batch, and take geometric mean ratios over complete paired batches. Descriptive 95% intervals resample the six kinematic groups 10,000 times, preserving the relationship between profiles and batch sizes. Completion losses remain separate from these ratios.

The learned interface has a complete-time ratio of 0.944 against the full-pool control, with descriptive interval $[0.863, 1.042]$ across 23 paired batches. The control completed all repeats in 0 additional batches, and the learned method completed all repeats in 0 additional batches.

The learned interface has a complete-time ratio of 0.604 against the distance control, with descriptive interval $[0.534, 0.678]$ across 23 paired batches. The control completed all repeats in

0 additional batches, and the learned method completed all repeats in 0 additional batches.

The observed geometric mean complete time is 5.6% lower than the full-pool Kira control on batches where both methods completed all five repetitions. Because the descriptive interval includes one, the aggregate timing gain across both batch sizes remains uncertain. The result applies to these constructed target expressions and their existing identity pools; it does not establish a speedup for general multiloop amplitude reductions [28].

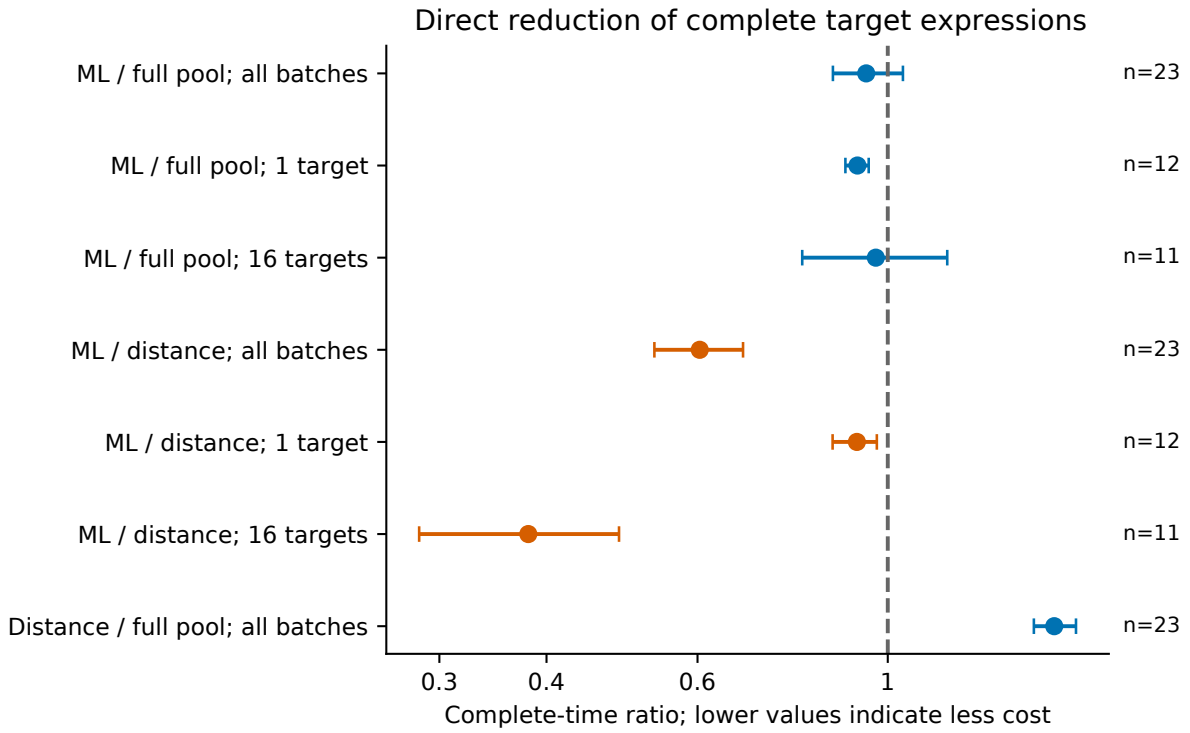


Figure 25: Complete-time ratios for direct reduction of complete target expressions. Points show geometric means of five-repeat median ratios, and intervals resample kinematic groups. The label n gives the number of complete paired batches. Failed and skipped trials remain in the completion counts.

13.12 Using the same master check for both seed selectors

The preceding distance control enters symbolic elimination directly, while the learned method first checks the selected master basis. To compare seed choices under the same admission procedure, we fixed a second confirmation in which both selected sets receive the same early check and full-pool fallback. The learned scores, threshold, distance rule, twelve family instances and target batches stay unchanged. The full-pool Kira control again uses its standard supplied-equation reduction. We retained every measurement in the first comparison.

Each method again uses five rotated repetitions on the same node within a family and batch. This new comparison contains independent timing repetitions of the same physical cases, so it checks the effect of matching the admission procedure and does not introduce new integral families. Complete times and descriptive intervals follow the definitions in the preceding subsection.

The comparison with the same early check contains 360 records, of which 348 are measured trials. It produced 345 exact completions, 3 incomplete measured trials and 12 skipped repetitions. Independent checks evaluated 8,460 target coefficients against the raw Kira exports. All completed reductions matched their exact reference coefficients.

The learned method completed 115 trials exactly, with 1 failed trial and 4 skipped repetitions. Its completed trials required fallback 25 times.

The full-pool control completed 115 trials exactly, with 1 failed trial and 4 skipped repetitions. Its completed trials required fallback 0 times.

The distance control with the early check completed 115 trials exactly, with 1 failed trial and 4 skipped repetitions. Its completed trials required fallback 55 times.

Across both batch sizes, the learned complete-time ratio to the full-pool control is 0.932, with descriptive interval $[0.845, 1.031]$ across 23 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For batches of 1 target, the learned complete-time ratio to the full-pool control is 0.914, with descriptive interval $[0.871, 0.958]$ across 12 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For batches of 16 targets, the learned complete-time ratio to the full-pool control is 0.952, with descriptive interval $[0.772, 1.148]$ across 11 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

Across both batch sizes, the learned complete-time ratio to the distance control with the same early check is 0.925, with descriptive interval $[0.848, 1.024]$ across 23 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For batches of 1 target, the learned complete-time ratio to the distance control with the same early check is 0.913, with descriptive interval $[0.855, 0.972]$ across 12 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For batches of 16 targets, the learned complete-time ratio to the distance control with the same early check is 0.939, with descriptive interval $[0.764, 1.136]$ across 11 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For the predefined single-target comparison, the learned method reduced geometric mean complete time by 8.6% relative to the full pool and by 8.7% relative to the distance rule with the same early check. Both descriptive intervals lie below one, and every method completed all five repeats for all twelve instances. The first confirmation gave a 7.8% reduction relative to the full pool for this batch size. These results support a repeated gain for one constructed target expression at a time in this stored-identity setting. The sixteen-target intervals include one, so that batch size does not establish a consistent gain. The geometric means of the twelve per-family median complete times were 5.478 seconds for the full pool and 5.005 seconds for the learned method.

The learned workflow has an observed 6.8% lower geometric mean complete time than the full-pool Kira control on batches where both methods completed all five repetitions. Because the descriptive interval includes one, the aggregate timing gain across both batch sizes remains uncertain. These conclusions concern reduction from the supplied identity pools and the tested constructed target expressions. Native equation generation, arbitrary scattering amplitudes and larger independent multiloop families remain outside this result [28].

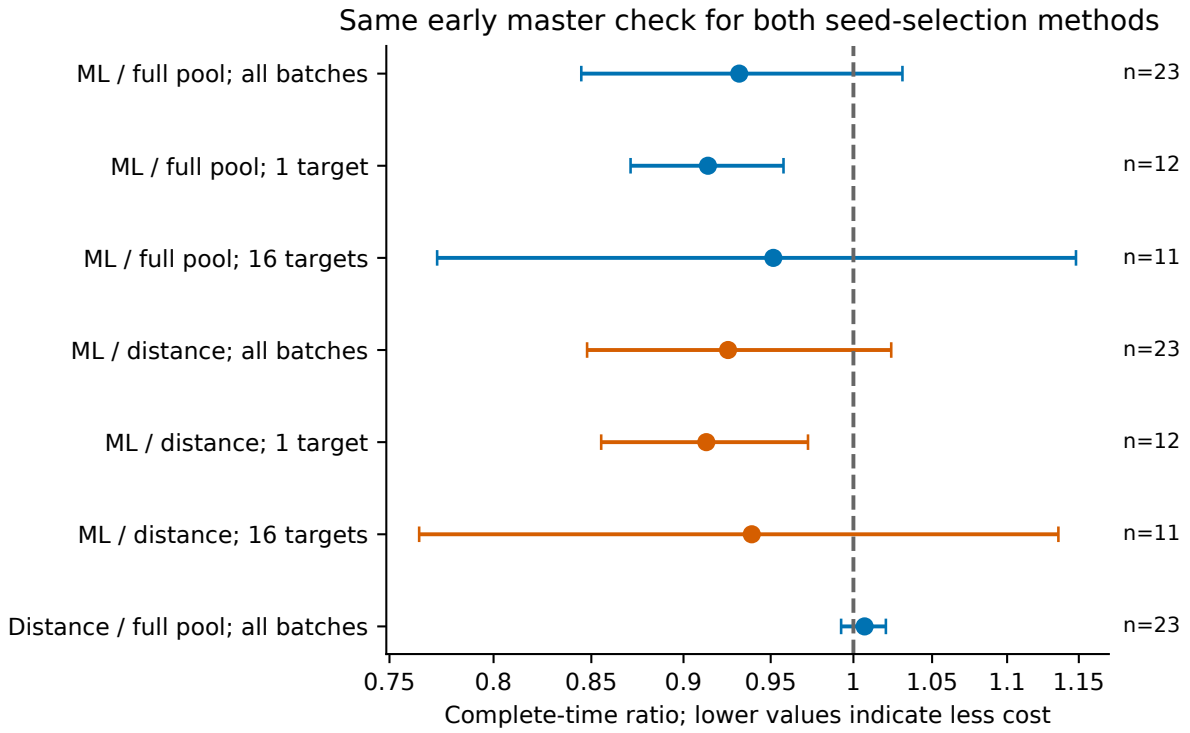


Figure 26: Complete-time ratios with the same early master check for learned and distance-selected seeds. Points show geometric means of five-repeat median ratios, intervals resample kinematic groups, and n gives the number of complete paired batches. The full-pool control uses standard supplied-equation Kira reduction.

13.13 Reducing fixed waits in the exact reduction workflow

The preceding comparisons show that the cost of using the seed model depends on the full reduction workflow. Inspection of the Kira version used here found a fixed one-second wait each time the program closed a Fermat process. Fermat performs exact rational-function arithmetic, and this wait occurs after the interface sends its exit command and closes its pipes. The program then checks whether the child has exited and forces termination if it is still running.

We replaced this fixed wait with repeated checks of the same child process at two-millisecond intervals. The modified interface returns as soon as it observes the exit and retains a one-second grace period before forced termination. It also handles interrupted checks and already reaped children. Tests checked that the arithmetic process exits normally or reaches the existing termination limit. We built this change separately from the original Kira executable. The symbolic elimination code, rational coefficients, model weights and seed threshold remain unchanged [29].

This change acts on a distinct part of the complete cost,

$$T_{\mathrm{total}} = T_{\mathrm{input}} + T_{\mathrm{model}} + T_{\mathrm{algebra}} + T_{\mathrm{exit}} + T_{\mathrm{check}}. \tag{101}$$

Here T_{input} includes loading and equation preparation, T_{model} includes loading the seed model and computing its scores, T_{algebra} is the time spent in exact reduction, T_{exit} is the time needed to close its arithmetic processes, and T_{check} is the final coefficient-verification time. Failed attempts and fallback contribute to the corresponding terms. Removing the fixed part of T_{exit} can improve a short reduction without changing its algebraic operation count.

13.13.1 Separating shutdown costs from learned seed selection

We compare four complete workflows: the original Kira with all supplied equations, the original Kira with learned seeds, the modified Kira with all supplied equations, and the modified Kira with learned seeds. The learned workflows use the existing model, the fixed threshold 0.01, the early master check and the same full-pool fallback. Thus, comparing the two learned workflows isolates the shutdown change, while comparing learned selection with the full pool within the modified Kira measures the remaining benefit of that seed-selection workflow.

Development uses three instances from the original training partition. The reserved comparison uses twelve other one-loop box instances in six original-test kinematic groups that were absent from all earlier box cost-test groups. Each instance has batches of one and sixteen constructed targets, and each workflow receives five rotated repetitions on the same node for that instance and batch size. We fixed the cases before timing the change and froze the executable, code, model and scientific inputs before the reserved runs.

Every measured time includes a fresh process start, input preparation, model inference where applicable, exact Kira reduction, failed attempts, fallback and exact coefficient checks. The prior generation of the shared IBP identity pools is outside all four methods. Each trial uses one thread, a 180-second complete-process cap, a 1.7 GB process-tree memory limit and 32 MiB of temporary storage. The array began with two simultaneous trials and increased to three and then six after memory and storage checks. The individual limits, targets, models and calculation code stayed fixed. After a failed measured attempt, further identical repetitions are marked as skipped and receive no measured time.

The timing comparison uses the median of five successful repetitions for each method and batch. We form the geometric mean of their paired time ratios and obtain descriptive 95% intervals by resampling entire kinematic groups 10,000 times. These intervals describe the completed subset, and the separate completion counts retain every planned case.

Development produced 10 exact completions from 12 measured trials. The reserved comparison contains 480 planned records, 416 measured trials and 400 exact completions. Independent checks compared 8,850 sampled master-coefficient vectors with the raw Kira exports. Every completed reduction matched its exact reference coefficients.

Each workflow completed 100 trials exactly and recorded 4 failed attempts and 16 skipped repetitions. Each learned workflow used full-pool fallback 10 times among its completed trials. The paired comparisons below identify the successful batch sets.

13.13.2 Measured complete-time changes

For one target at a time, the modified learned workflow has a complete-time ratio of 0.385, with a descriptive 95% interval $[0.342, 0.453]$ from 12 complete paired batches relative to the original learned workflow. The corresponding geometric mean times are 5.528 and 2.126 seconds, a 61.5% reduction. For sixteen targets, the ratio to the original learned workflow is 0.686, with a descriptive 95% interval $[0.540, 0.844]$ from 8 complete paired batches.

With the responsive shutdown active in both methods, learned seed selection has a one-target complete-time ratio of 0.690, with a descriptive 95% interval $[0.424, 0.931]$ from 11 complete paired batches relative to the full pool. The corresponding sixteen-target ratio is 0.983, with a descriptive 95% interval $[0.915, 1.063]$ from 8 complete paired batches. The one-target comparison therefore retains a measured 31.0% reduction from the learned seed workflow after the shutdown improvement. The sixteen-target interval includes one, so a consistent additional benefit remains unresolved for that batch size.

The modified learned workflow has a one-target complete-time ratio of 0.282, with a descriptive 95% interval $[0.199, 0.349]$ from 11 complete paired batches relative to the original full-pool workflow. The full-pool workflow alone has a modified-to-original ratio of 0.408, with a descriptive 95% interval $[0.355, 0.476]$ from 11 complete paired batches. These controls show that the shutdown improvement applies to the exact solver independently of machine learning. The unchanged seed model supplies a separate choice of equations, whose incremental effect is measured against the modified full-pool control.

Across 200 completed original/modified pairs, equation inputs and reported elimination counters agreed. The final raw coefficient exports were identical in 200 pairs, and 104 paired prediction records agreed exactly. The change therefore improves process completion time while preserving the tested mathematical calculations. It does not establish fewer algebraic operations, a better trained model or a general multiloop speedup. The measured domain remains the specified supplied-identity reductions of constructed one-loop targets.

The two learned workflows both completed all five repetitions in 12 of the twelve single-target batches and 8 of the twelve sixteen-target batches. Their successful batch sets agreed at each batch size. With the modified shutdown, learned selection alone completed 1 batch of 1 target. With the modified shutdown, the full pool alone completed 1 batch of 16 targets. The quoted ratios use only the stated complete paired subsets; all other outcomes remain in the completion counts.

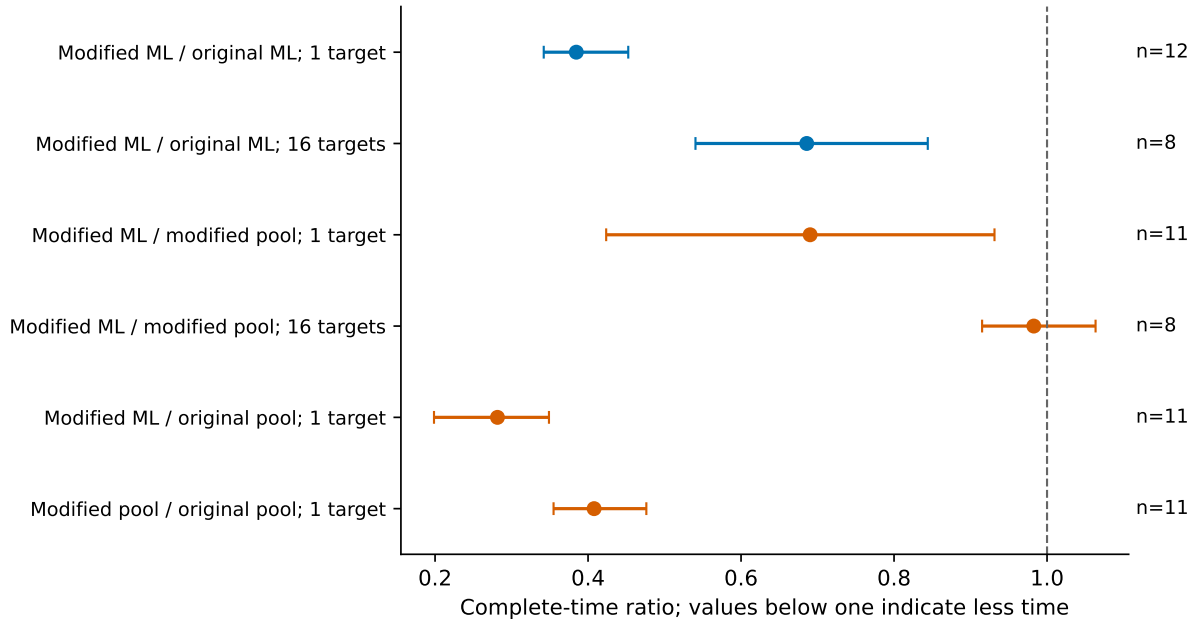


Figure 27: Complete-time ratios for responsive Fermat shutdown and learned seed selection. Points show geometric means of paired five-repeat medians; intervals resample kinematic groups. Each row compares the same physical targets, and values below one indicate less complete time. Here n is the number of complete paired batches.

14 Complete native multiloop reductions

The preceding supplied-equation tests begin with an existing IBP pool. To include the work needed to form that pool, we now give Kira physical denominators and indexed target integrals and repeat equation generation in every measured calculation. We also train a separate row model from the algebraic work caused by choices inside Kira. This connects the training objective

to the intended reduction cost and permits a direct comparison with Kira's own source-row choices.

14.1 Physical families and independent targets

The integral notation remains $I_{\boldsymbol{a}}$, with $D_i = q_i^2 - m_i^2 + \mathrm{i}0$ as defined earlier. We use the following ordered momentum lists:

$$\boldsymbol{q}_{\text{sunrise}} = (\ell_1, \ell_2, p - \ell_1 - \ell_2, p - \ell_1, p - \ell_2), \tag{102}$$

$$\boldsymbol{q}_{\text{box}} = (\ell_1, \ell_1 - p_1, \ell_1 - p_1 - p_2, \ell_2, \ell_2 - p_1 - p_2, \ell_2 - p_1 - p_2 - p_3, \ell_1 - \ell_2, \ell_1 - p_3, \ell_2 - p_1), \tag{103}$$

$$\boldsymbol{q}_{\text{vacuum}} = (\ell_1, \ell_2, \ell_3, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3), \tag{104}$$

$$\boldsymbol{q}_{\text{ladder}} = (\ell_1, \ell_1 - p, \ell_2, \ell_2 - p, \ell_3, \ell_3 - p, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3). \tag{105}$$

The first three sunrise entries, seven box entries and eight ladder entries are physical propagators. Their remaining entries complete the scalar-product basis and can appear with nonpositive indices. All six vacuum entries are physical propagators. The first two lists contain two loop momenta, and the last two contain three. In particular, the three-loop vacuum and ladder systems couple the loop momenta through shared propagators.

For each list, we expand the squared momenta in $\ell_i \cdot \ell_j$ and $\ell_i \cdot p_j$ and check that the resulting coefficient matrix has rank

$$N_{\text{sp}} = \frac{L(L+1)}{2} + LE.$$

Here E is the number of independent external momenta, and N_{sp} is the number of independent scalar products involving loop momenta. All fifteen family definitions pass this check, and their nonempty external Gram matrices have nonzero determinants. This prevents a dependent denominator list from entering the comparison.

All squared masses and external scalar products have fixed rational values in a common squared-momentum unit, while the dimension d remains symbolic. The reduction coefficients therefore belong to $\mathbb{Q}(d)$. For the box, the external momenta satisfy

$$p_i^2 = 0, \qquad 2p_1 \cdot p_2 = s, \qquad 2p_1 \cdot p_3 = t, \qquad 2p_2 \cdot p_3 = -s - t.$$

The six training cases consist of two sunrise mass profiles $(1,1,1)$ and $(1,2,3)$ with $p^2 = -1, -2$ respectively, two massless boxes with $(s,t) = (-1,-2), (-2,-3)$, and two vacuum profiles $(1,1,1,1,1,1)$ and $(1,1,1,2,2,2)$. These profiles specify squared masses in the displayed propagator order. Validation uses the sunrise profile $(1,1,2)$ at $p^2 = -3$, a box at $(s,t) = (-3,-4)$, and the vacuum profile $(1,1,1,1,1,2)$.

The six reserved cases use boxes at $(s,t) = (-5,-7), (-7,-11)$, vacuum profiles $(1,1,2,1,2,2)$ and $(1,2,2,1,2,2)$, and two ladder instances. One ladder is massless with $p^2 = -1$; the other has $m_1^2 = 1$, all other squared masses zero, and $p^2 = -2$. The ladder topology appears only in the reserved set. The other reserved cases test different kinematic or mass conditions within training topologies. These tests concern rational kinematic points with symbolic d and do not establish performance for general symbolic multiscale coefficients.

Targets are individual indexed integrals. We begin with the full-sector corner, then increase each positive index by one and insert each available auxiliary numerator once. We repeat this enumeration after removing one, two and three physical denominators, retain at least two physical denominators, and remove duplicate index vectors. The reserved batches contain the first 32 or 128 targets in this fixed order. This construction selects targets without using learned scores or reverse construction paths.

14.2 Seed regions, learned subsets and complete fallback

For a batch $\mathcal{T}$, define

$$r_{\mathcal{T}} = \max_{\boldsymbol{a}\in\mathcal{T}} \sum_i \max(a_i, 0), \qquad s_{\mathcal{T}} = \max_{\boldsymbol{a}\in\mathcal{T}} \sum_i \max(-a_i, 0), \qquad b_{\mathcal{T}} = \max_{\boldsymbol{a}\in\mathcal{T}} \sum_i \max(a_i - 1, 0).$$

The last quantity is the largest total number of additional propagator powers, also called dots. We specify Kira seed bounds $(r_{\mathcal{T}} + \delta, s_{\mathcal{T}} + \delta, b_{\mathcal{T}} + \delta)$, where $\delta = 1$ defines the conservative reference region and $\delta = 0$ defines the smaller region. Kira still applies its native symmetry, equation-generation and equation-selection procedures within these bounds [9]. The bounds are explicit input choices; their difference is not a learned improvement to Kira.

The saved seed model scores the finite candidate region using the ordered sum of the batch targets, with coefficient one for each target. This sum supplies model context, while Kira still reduces every target separately. Every sector corner, target and prescribed master that belongs to the candidate region remains available. An additional deterministic rule can retain all seeds with at most one dot or one numerator power in total. The learned policy can add seeds above its probability threshold, and a separate control applies the same deterministic rule without model inference.

Kira first checks whether the selected system requires masters outside the prescribed reference basis. A rejected subset triggers the documented fallback sequence, and its time remains part of the measurement. A final successful calculation must produce every requested target and match its reference coefficient in $\mathbb{Q}(d)$. Other failed attempts also move through the prescribed fallback sequence. A coefficient mismatch makes the complete workflow unsuccessful. Splitting a batch into groups of 32 targets bounds each calculation, but it also repeats equation generation and process startup; the timing comparison includes both effects.

14.3 Training row choices on their algebraic consequences

The new row model chooses a source among equations with the same leading integral. It preserves Kira's native pivot and group order, and Kira carries out every resulting row operation exactly. At state S, the allowed action q selects one such source row, which removes the leading integral from the other rows in that group. The model therefore acts on the changing native matrix through a restricted, explicitly defined action set.

The first training objective measures the immediate symbolic insertion time for each source. We clone the destination rows, apply the exact native update, and average two rotated measurement orders while excluding clone time. This measures local arithmetic cost, but it does not include the equations created by later eliminations. We therefore also measure a continuation cost,

$$C(S, q) = \sum_{j\in\mathcal{U}(S,q)} (1 + 2n_j). \tag{106}$$

Here $\mathcal{U}(S, q)$ is the sequence of committed forward insertions after choosing q and then following Kira's native choices, and n_j is the number of nonzero coefficients in the source of insertion j. The count assigns one division and two coefficient operations per source entry. It is an algebraic operation estimate tied to measured native insertions; it does not count Fermat's internal polynomial operations or back substitution.

We choose up to four evenly spaced eligible states from the saved local-cost sample in each development case, which contains at most 100 measured groups. We prefer groups with at least three source choices and use smaller groups when no such group is available. Each alternative receives a separate complete Kira continuation. The recorded pivot and all candidate features

must match the original state, and the final master coefficients must match the reference reduction before its cost can enter training. The resulting labels measure a choice followed by native continuation. They do not prove a globally minimum reduction path.

Each candidate has eighteen features describing row length, coefficient text length and operations, overlap with destination columns, estimated new entries, and its position relative to Kira's preferred source. A linear fit to log continuation cost retained the native source in all nine validation states. We therefore also fit a small regression tree [30], which can represent conditions involving several features.

For this tree, the target is the normalized change in measured continuation cost,

$$y_q = \frac{C(S,q) - C(S,0)}{\max\{1, \max_{q'} C(S,q') - \min_{q'} C(S,q')\}}, \qquad \widehat{y}_q = h(\boldsymbol{x}_q - \boldsymbol{x}_0).$$

The index 0 denotes Kira's preferred source, $\boldsymbol{x}_q$ is the candidate feature vector, and h is the fitted tree. Lower targets correspond to fewer recorded operations. We fix $\widehat{y}_0 = 0$ and fit the remaining candidates with a weighted least-squares loss that gives each training family equal weight. Validation selects tree depth, minimum leaf population, target normalization and the required predicted gain.

The selected tree has 21 nodes, depth four and at least two training examples in each leaf. Its training set contains 68 candidate continuations in 24 states from six families, while validation contains 30 candidates in nine states from three families. All 98 continuations match their exact reference reductions. The tree changes nine training choices and three validation choices, with family-weighted continuation count ratios of 0.998602 and 0.999967 to the native choices. These ratios measure individual choices followed by native continuation; the next test must determine the effect of using the model repeatedly.

The model chooses the lowest predicted score and reports

$$P(q \mid S) = \frac{\exp(-\widehat{y}_q)}{\sum_{q'} \exp(-\widehat{y}_{q'})}.$$

This quantity expresses a relative model score, with no calibration claim. Kira evaluates each score directly in compiled code. Groups with two rows or more than 32 rows keep the native source, which also avoids feature construction for those groups. We preserve the original neural checkpoint and both earlier linear fits as separate artifacts, and hold the final tree fixed during reserved testing.

The seed implementation also reads its unchanged weight archive once per process and retains at most two sets of family and candidate embeddings within a workflow. A new target batch still receives its own target features and scores. Independent checks compare every probability with the original implementation and compare native tree scores and choices with the Python model. These changes reduce repeated loading and feature preparation while preserving the tested seed predictions.

14.4 Complete timing and reference checks

Every final trial starts a new process and includes input preparation, model loading and inference, native IBP generation and selection, symbolic elimination, back substitution, exact coefficient comparison, and all failed attempts and fallback reductions. Evidence compression follows the stop time. These checks establish reduction to the prescribed reference basis; they do not prove global minimality of the master set. We report the initial reference calculation separately as common benchmark preparation. Each policy receives the same prescribed master basis and exact comparison result within a case.

All controls use the responsive Fermat shutdown from the preceding section. We compare both the unchanged reduction executable with that shutdown fix and an instrumented executable that keeps native row choices. This separates the cost of measurement and integration from the learned action. We also compare smaller deterministic regions and matched deterministic seed subsets, since either could explain a gain over conservative bounds.

Each trial uses one calculation thread, a bounded scratch directory and limits on elapsed time and process-tree memory. Production trials have 128 MiB of scratch space, a 2,400 MiB solver address-space limit, and a 2,100 MiB solver process-tree resident-memory limit. The enclosing worker has a 2,800 MiB address-space limit and a 2,600 MiB process-tree resident-memory limit. A single common-reference calculation can use 256 MiB while retaining the same symbolic bounds, and the storage ledger includes additional monitoring headroom. Final comparisons use paired cases and report completion counts alongside time ratios. Repeated runs reduce timing noise within a case, while different kinematic instances remain distinct from different topologies. The reserved results do not select the policy or its thresholds.

14.5 Reserved results and complete-calculation cost

The frozen comparison contains six reserved cases, two target-batch sizes, nine policies and three repetitions of each policy. Each trial begins with physical denominators and includes native IBP equation generation. The results and exact evidence are available with the comparison data [31].

We measured 324 of 324 planned trials, and 324 completed every target with exact reference coefficients. The records retain 0 unsuccessful measured trials and 0 skipped trials. The independent audit checked 550 successful native reductions, including the training continuations and reserved attempts, and made 114,965 coefficient comparisons by exact polynomial cross multiplication. All 98 recorded training and validation continuations passed. The separate larger-region check completed 6 of 6 cases. These comparisons verify reduction to the prescribed basis; a finite region check does not prove that the basis is globally minimal.

The measured workflows contain 585 native reduction attempts. Kira rejected 66 attempted subsets because they required masters outside the prescribed basis. The remaining unsuccessful attempts comprise 84 other Kira errors, 3 elapsed-time limits. These attempts remain in complete-workflow times, and the fallback sequence determines whether every target can still be reduced within the budget.

The common reference preparation completed 12 of 12 case–batch calculations. The successful reference times ranged from 5.967 to 28.083 seconds and are outside the paired production timings. The two 128-target box cases use 8 master integrals each. Both contain 8 zero targets and 3 targets already in the master basis, leaving 117 nonzero targets that require reduction in each case. The two 128-target vacuum cases use 11 and 17 master integrals, respectively. The first contains 4 zero targets and 8 targets already in the master basis, leaving 116 nonzero targets that require reduction. The second contains 4 zero targets and 13 targets already in the master basis, leaving 111 nonzero targets that require reduction. The two 128-target ladder cases use 5 and 21 master integrals, respectively. The first contains 14 zero targets and 0 targets already in the master basis, leaving 114 nonzero targets that require reduction. The second contains 14 zero targets and 1 targets already in the master basis, leaving 113 nonzero targets that require reduction.

The following ratios use only case–batch pairs for which both policies completed all three repetitions. We take the median within each case and batch, average log ratios within a case, and then give cases equal weight. Ratios of operation counts also require positive counts in both methods. The intervals use 5,000 resamples of the fixed physical cases, so they describe this sample rather than a general distribution of multiloop families.

14.5.1 Seed selection and deterministic controls

Validation selected the smaller-region learned seed policy at probability threshold 0.01. Its complete-time ratio to instrumented native Kira was 1.118 on the validation sample; this selection therefore did not establish an aggregate validation gain. On reserved tests, its ratio to the responsive stock control is 0.742 with descriptive interval $[0.560, 1.009]$, and its ratio to the instrumented native control is 0.755 with descriptive interval $[0.573, 1.050]$. Against the smaller deterministic region with fallback, the ratio is 0.916 with descriptive interval $[0.723, 1.152]$. Against the same low-complexity coverage rule without ML, the ratio is 0.833 with descriptive interval $[0.679, 1.010]$. These controls separate the effects of region size, deterministic coverage, model inference and the learned additions.

For 32 targets, the learned seed ratio to responsive stock Kira is 0.713 with descriptive interval $[0.543, 0.967]$, using 6 complete paired batches. The corresponding geometric mean median times are 10.490 and 14.720 seconds. For 128 targets, the learned seed ratio to responsive stock Kira is 0.772 with descriptive interval $[0.558, 1.082]$, using 6 complete paired batches. The corresponding geometric mean median times are 13.723 and 17.764 seconds.

For the two-loop boxes, the seed-to-stock time ratio is 0.563 with descriptive interval $[0.531, 0.598]$. Its ratio to the smaller deterministic region is 0.642 with descriptive interval $[0.627, 0.658]$. Its ratio to the matched coverage rule is 0.606 with descriptive interval $[0.567, 0.648]$. For the three-loop vacuum integrals, the seed-to-stock time ratio is 1.266 with descriptive interval $[1.180, 1.357]$. Its ratio to the smaller deterministic region is 1.291 with descriptive interval $[1.245, 1.338]$. Its ratio to the matched coverage rule is 1.093 with descriptive interval $[1.016, 1.175]$. For the three-loop ladders, the seed-to-stock time ratio is 0.573 with descriptive interval $[0.554, 0.592]$. Its ratio to the smaller deterministic region is 0.928 with descriptive interval $[0.852, 1.010]$. Its ratio to the matched coverage rule is 0.871 with descriptive interval $[0.833, 0.910]$. These topology breakdowns retain every case in the stated class. The ladder class tests a topology absent from the new row-training and validation data.

The learned seed ratio for the initially generated equations summed across attempts is 0.086 with descriptive interval $[0.049, 0.152]$ relative to native Kira. The learned seed ratio for the committed forward coefficient-operation estimate is 0.798 with descriptive interval $[0.655, 0.992]$ relative to native Kira. The first generation count in each native attempt defines the equation measure; later selection counts are not added to it. The operation estimate follows Eq. (106), and all attempted stages contribute when a workflow requires fallback.

14.5.2 Repeated row decisions in the changing matrix

The cost-trained row tree has a complete-time ratio of 1.008 with descriptive interval $[0.937, 1.078]$ to the instrumented native policy. Its forward coefficient-operation ratio is 1.003 with descriptive interval $[0.983, 1.021]$, while the measured forward-stage time ratio is 0.943 with descriptive interval $[0.828, 1.033]$. The paired geometric mean forward times are 0.091 seconds for native choices and 0.085 seconds for the tree, while their complete times are 15.887 and 16.013 seconds. Most measured time therefore lies outside this recorded forward stage. The sampled continuation objective measures one altered source choice followed by native choices. Repeated tree choices change the later matrices and can therefore change the cost in a different direction. The full-workflow results determine whether the trained model saves work in this use.

The combined seed and row policy has a complete-time ratio of 0.759 with descriptive interval $[0.613, 0.962]$ to responsive stock Kira. Its paired geometric mean median time is 12.280 seconds, compared with 11.998 seconds for seeds alone. The row addition therefore does not lower this

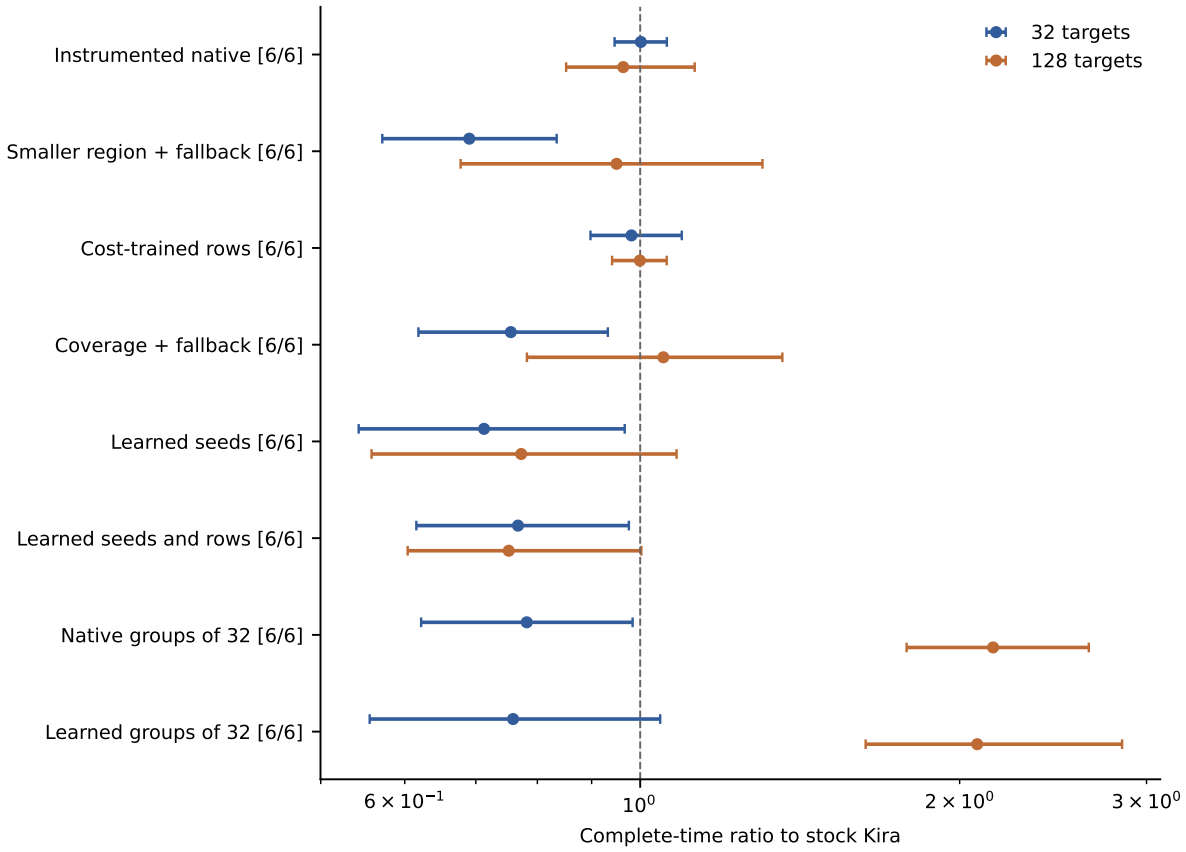


Figure 28: Complete-time ratios to responsive stock Kira for 32 and 128 indexed targets. Points use medians of three exact repetitions and equal case weights. Bars show descriptive case-resampling intervals; the labels give complete paired batch counts. A ratio below one means lower measured time in the stated paired cohort.

point estimate. Combining two decisions requires a separate measurement because a different seed set changes the equations on which the row policy acts.

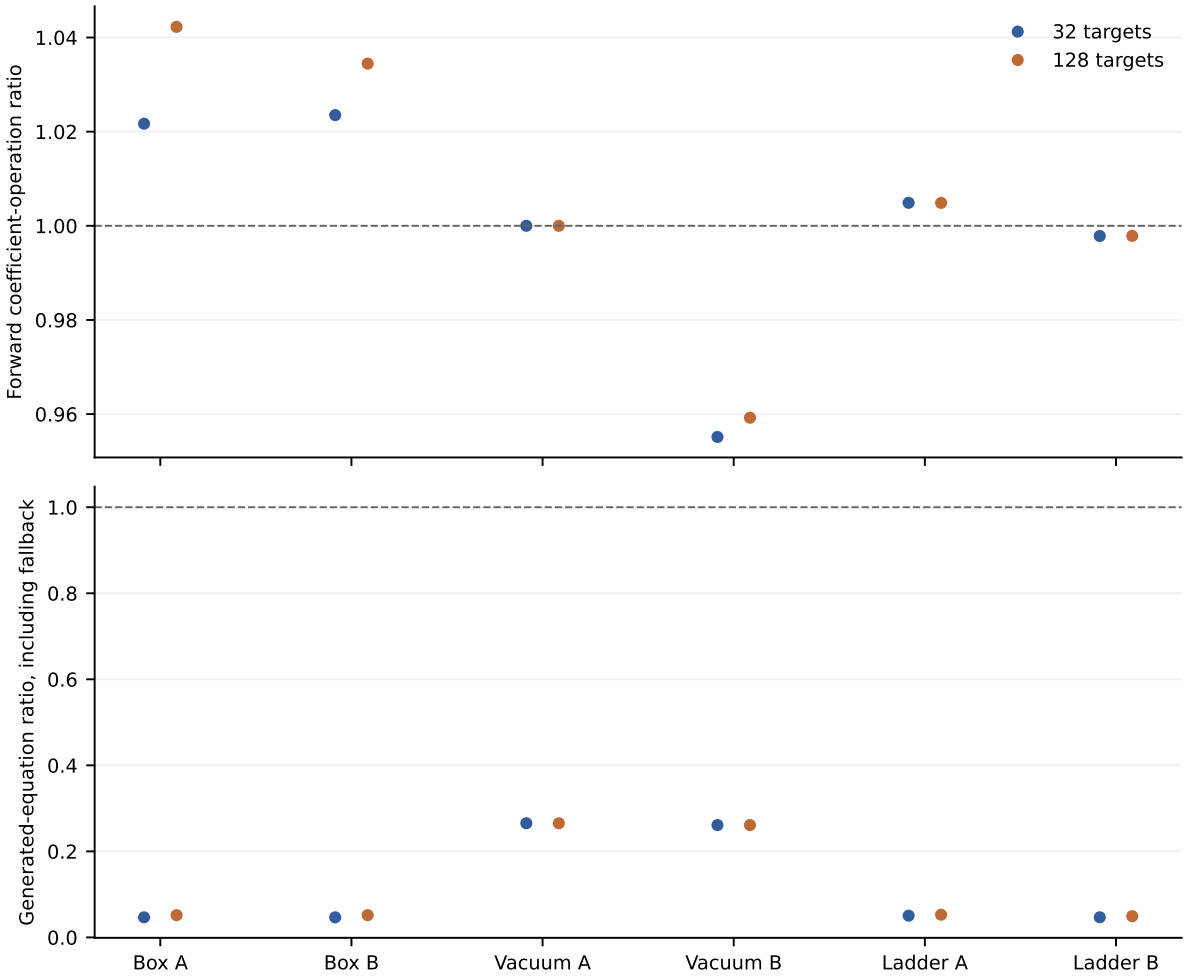


Figure 29: Per-case median work ratios in the reserved comparison. The row panel measures committed forward coefficient operations relative to the same instrumented native policy. The seed panel measures initial generated equations, summed over attempts, relative to native Kira. Cases A and B follow the parameter order given in the text. Counts concern different stages and do not measure Fermat's internal polynomial work.

14.5.3 Completion of larger target batches

With 32 targets, responsive stock Kira completed 18 of 18 trials. For the learned seed workflow, 18 of 18 completed, including 6 rescued by fallback. For the combined workflow, 18 of 18 completed, including 6 rescued by fallback. For native groups of 32 targets, 18 of 18 completed, including 0 rescued by fallback. For combined groups of 32 targets, 18 of 18 completed, including 6 rescued by fallback.

With 128 targets, responsive stock Kira completed 18 of 18 trials. For the learned seed workflow, 18 of 18 completed, including 6 rescued by fallback. For the combined workflow, 18 of 18 completed, including 6 rescued by fallback. For native groups of 32 targets, 18 of 18 completed, including 18 rescued by fallback. For combined groups of 32 targets, 18 of 18 completed, including 18 rescued by fallback.

For 128 targets, splitting the native workflow gives a time ratio of 2.151 with descriptive interval $[1.783, 2.647]$ to stock Kira. Splitting the combined workflow gives 2.078 with descriptive interval $[1.631, 2.845]$. The separate fallback counts show whether completion improves; the paired times include repeated generation and process startup.

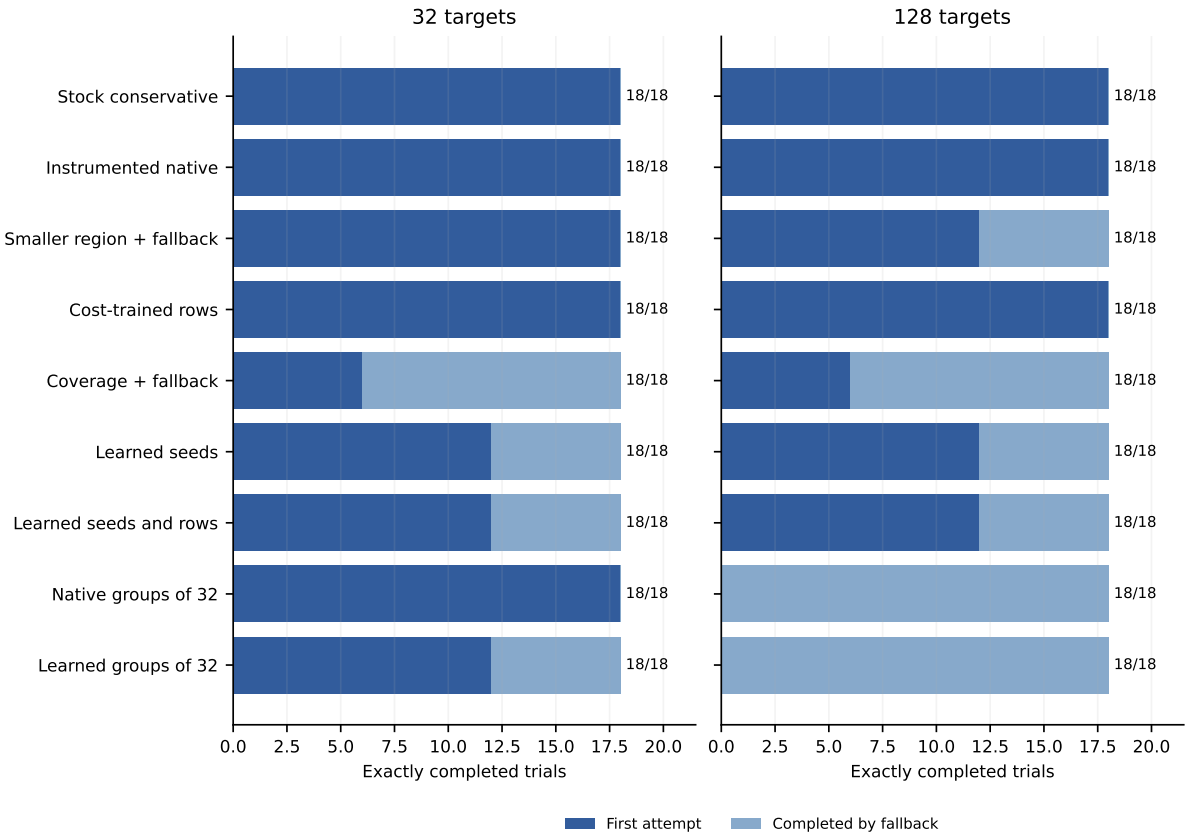


Figure 30: Completion of all requested targets under the fixed time and storage limits. The filled components distinguish successful initial reductions from reductions completed through fallback. Unfilled trials include failed or explicitly skipped outcomes. Each policy has eighteen planned trials at each batch size.

14.5.4 Scope and resource outcomes

The largest recorded production process-tree memory use was 106.8 MiB, and the largest monitored production scratch allocation was 1.0 MiB. Among unsuccessful attempts, 7 logs report an allocation failure and 10 report recursive termination; these diagnostic counts can overlap. The fixed process limits constrain these outcomes, and the logs alone do not establish exhaustion of physical RAM. The workflow retains their costs and follows the same fallback rule. A separate post-comparison check repeated the smaller seed region for the 128-target validation box using both the responsive stock executable and the instrumented native executable. Both reported `std::bad_alloc` under the same solver limits, so this failure also occurs without the row-policy hook. This check concerns the failure mechanism and supplies no additional timing comparison; it leaves all frozen reserved settings unchanged.

One scheduler interruption stopped a test worker when the sum of its processes' virtual address spaces exceeded the job limit. We preserved its completed records and resumed only missing work with a larger scheduler address-space allowance, while keeping the measured physical-memory, scratch and elapsed-time limits fixed. This interruption is recorded separately from completed

workflow timings. With the resumed case excluded, the learned seed to stock complete-time ratio is 0.676 with descriptive interval $[0.549, 0.963]$. With the resumed case excluded, the learned row to native complete-time ratio is 1.039 with descriptive interval $[0.991, 1.102]$. The tests include two- and three-loop systems at fixed rational kinematics, a finite prescribed seed region and a restricted source-row action. They do not establish a speedup for arbitrary symbolic kinematics, every multiloop topology, or the earlier restricted collection through twelve loops.

15 Integrated seed and row policies with bounded execution

The complete reductions in Section 14 show that a learned choice can save algebraic work while other parts of the calculation determine most of the elapsed time. We therefore examine both the cost of applying the models and the native operations that surround them. Every comparison in this section starts from physical denominators, generates IBP equations and checks the final master coefficients. We retain the earlier checkpoints and results, and use separate training, validation and reserved cases for the new policies.

15.1 Removing costs shared by both decision methods

Back substitution replaces the remaining nonmaster integrals in each triangular equation with their reductions. If the equation for $I_{\boldsymbol{a}}$ contains $I_{\boldsymbol{b}}$ on its right-hand side, the latter reduction must be available before the former equation can finish. We represent this dependency by the directed edge

$$I_{\boldsymbol{b}} \longrightarrow I_{\boldsymbol{a}}.$$

A topological order is a sequence in which every dependency appears before the equation that uses it. The modified single-worker schedule constructs this order from the loaded equations, preserves the native arithmetic and rejects a cycle or an invalid dependency. This permits one bounded group of back-substitution workers. The inspected native implementation created helper workers for each sector range, even when the calculation used one arithmetic worker.

An initial merge that retained the loaded row order reached a dependency wait and exceeded its time limit. The corrected order completed all six subsequent bounded pilot reductions and reproduced their reference coefficients. The corresponding smaller-region native control reported an allocation failure. These are development checks of the scheduling change. They do not attribute a gain to either learned model or establish a general memory-exhaustion mechanism.

The calculation also creates temporary databases. We compare the same native reductions with those databases on project storage and on the compute node's temporary storage. The eight exact pilot runs use the same resource limits and two repetitions of each region and storage condition. Their geometric mean time ratio for node storage relative to project storage is 0.307. We use node storage for all later learned and deterministic controls. Both controls therefore receive the same scheduling and storage improvements.

15.2 Applying the seed model with unchanged probabilities

The seed policy retains the trained neural weights and the finite candidate region described earlier. For each batch, it constructs the family features, the ordered target features and the candidate index features before predicting a separate probability for every seed. We construct the index features in arrays and reuse shared family information so that the model performs less repeated preparation.

A parity check compares 5,000 generated index vectors and all candidate predictions for the nine development profiles. The feature values, embeddings and final probabilities agree exactly with the preceding implementation. In the development box checks with 3,510 candidates, shared feature construction and prediction decrease from approximately 0.08 to 0.01 seconds. These component measurements motivate the implementation change; complete-workflow comparisons must also include model loading, equation generation, exact algebra and any fallback.

The selected seeds always retain the prescribed mandatory integrals and the stated deterministic coverage rule. Kira checks whether the resulting system introduces masters outside the reference basis. An insufficient set triggers the fixed fallback sequence, and the elapsed time includes the unsuccessful attempt. Final coefficients must agree with the common reference in $\mathbb{Q}(d)$.

15.3 Estimating cancellations before a row choice

The row model chooses among source equations with the same leading integral, as in the preceding native experiment. To include cancellations in its description of each choice, consider source row $\boldsymbol{u}_q$ and destination row $\boldsymbol{v}_j$, with common leading column p and nonzero source coefficient u_{qp}. Exact elimination forms

$$v'_{jk} = v_{jk} - \frac{v_{jp}}{u_{qp}} u_{qk}.$$

A resulting coefficient vanishes when

$$v_{jk} u_{qp} - v_{jp} u_{qk} = 0.$$

We estimate this condition by evaluating the rational coefficients at $d = 17$ and $d = 19$ modulo the prime $2\,147\,483\,647$. Two zero evaluations supply a feature estimate of cancellation; they do not certify a symbolic identity. Kira still performs the exact update, and the final reduction still undergoes an exact coefficient check. Unsupported expressions and poles cause the policy to retain the native source.

Let $\widehat{n}_{jq}$ be the estimated number of surviving entries in destination j after using source q. Six additional features describe the total $\sum_{j \neq q} \widehat{n}_{jq}$, the cancellation fraction, the largest resulting row, the number of zero rows, the sum $\sum_{j \neq q} \widehat{n}_{jq}^2$, and whether the modular estimate is valid. They extend the previous eighteen features to twenty-four. The implementation bounds the group size, coefficient text and modular cache, and retains native behavior outside these limits.

Independent checks compare the evaluator with exact rational arithmetic for 210 expressions and three row-choice fixtures. A separate in-process comparison checks shared feature construction against the original formula in 4,041 changing row groups. Python and the compiled policy also agree on all 3,247 recorded decisions in that check, and all three full reductions match their reference coefficients. Shared construction calculates each row's length and coefficient statistics once per group and each pairwise overlap once.

15.4 Learning from local cost and complete trajectories

Each of the six training profiles and three validation profiles has an ordinary batch and a deeper batch. The deeper batch includes integrals with two extra propagator powers and mixed numerator and propagator powers. Both batches from a physical profile remain in the same partition. Collection gives 2,520 eligible training groups and 1,372 validation groups, with exact reference agreement required before any group can enter training.

We first fit small regression trees to immediate insertion time, resulting coefficient text length, and a combination of these quantities. The time-selected trees keep every native choice in their

recorded validation groups. The coefficient-length tree changes 371 choices and gives a family-weighted immediate coefficient-length ratio of 0.920, accompanied by an insertion-time ratio of 1.097. These labels describe a single group. Repeated deployment can change later matrices, so the full reduction supplies the required cost comparison.

The next search therefore evaluates a policy throughout the native forward reduction. A candidate assigns source q the score

$$s_{\boldsymbol{w}}(S,q) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}(S,q), \qquad q^*(S) = \arg\min_q s_{\boldsymbol{w}}(S,q),$$

where S is the current row group, $\boldsymbol{x}(S,q)$ is its feature vector and $\boldsymbol{w}$ is the policy's coefficient vector. Ties retain the native source. The bounded candidate set varies the emphasis on source length, coefficient length, predicted surviving entries, squared resulting row lengths and preference for the native source.

We evaluate eighteen coefficient vectors on all twelve training batches. For each complete trajectory, the forward-work estimate remains $\sum_j (1 + 2n_j)$ from Eq. (106). This count now includes every committed update caused by repeated policy choices. We rank candidates by the geometric mean ratio to native work, first averaging the ordinary and deeper batches within each physical profile and then giving profiles equal weight. Thus later algebraic consequences enter the training objective directly. The measured count excludes Fermat's internal polynomial operations and back substitution, which remain covered by the complete-time measurement.

The two nontrivial linear policies retained for the final comparison use

$$s_A(S,q) = 2\log(1+n_q) + \log\left(1 + \sum_{j\neq q} \widehat{n}_{jq}\right), \tag{107}$$

$$s_B(S,q) = \tfrac{1}{2}\log(1+n_q) + \log\left(1 + \sum_{j\neq q} \widehat{n}_{jq}\right) + 0.3\log\left(1 + \sum_{j\neq q} \widehat{n}_{jq}^{\,2}\right) - 0.01\,\delta_{q0}. \tag{108}$$

Here n_q is the number of entries in source row q, and δ_{q0} is one for the native source and zero for another source. The lower score selects the source. These coefficient vectors come from the stated finite search; validation then selects which vector to use for each topology. The small native-source preference in s_B reduces changes when its other features give nearly equal scores.

Separate validation compares the selected policies with native choices and a deterministic rule that minimizes the predicted number of surviving entries. It also measures the seed policy alone and the combined seed and row policy. A model reports the normalized score

$$P(q \mid S) = \frac{\exp[-s_{\boldsymbol{w}}(S,q)]}{\sum_{q'} \exp[-s_{\boldsymbol{w}}(S,q')]}.$$

This gives a relative preference among the supplied actions, with no calibration or global-optimality claim. Reserved cases do not select weights, thresholds or deployment rules.

15.5 Timing a complete integrated workflow

All methods use fresh processes, the same temporary storage and the same bounded back-substitution schedule. Production timing includes preparation, prediction, equation generation and selection, elimination, back substitution, exact comparison and every required fallback. A child-exit notification ends the wait promptly and avoids adding the monitor's polling interval to short calculations. Detailed training traces remain disabled during production timing, while aggregate operation and time counters remain enabled.

The reference reduction and prescribed master basis are common benchmark preparation and remain outside the paired production time. Successful trials must reduce every supplied target to that basis with exact coefficients. At most two production reductions run concurrently, each with 128 MiB of temporary storage and the preceding process-memory limits. The project ledger reserves storage for temporary files, retained records, publication copies and the local report while remaining below the user's combined 10 GB ceiling.

15.6 Reducing the cost of a single model call

The vector implementation accelerates repeated inference, but a short reduction also pays for loading its numerical library. We therefore export the same trained weights to a compiled evaluator that reads the same family, target and candidate features. The evaluator uses single-precision arithmetic and preserves the network's matrix operations, nonlinear functions and attention calculation. Different accumulation orders can change the last digits of a probability, so we check both probabilities and the selected sets.

Across thirty-six development contexts, the largest absolute probability difference is 9.48×10^{-6}. All selection masks agree at thresholds 0.003, 0.01, 0.03, 0.1, 0.3 and 0.5. A further forty-eight fresh-process checks use eight validation contexts, two implementations and three repetitions. Their selected sets also agree. The compiled implementation takes between 0.264 and 0.804 times the numerical-library implementation's median preparation and inference time across these contexts. Its arithmetic alone is slower in the warm tests; the fresh-process benefit comes from the lower startup cost. Production comparisons therefore retain startup in the timed workflow.

We also reduce the row policy's feature cost. Once training selects a sparse coefficient vector $\boldsymbol{w}$, only features with $w_k \neq 0$ affect its score. The production evaluator computes those features, while the diagnostic path retains the full vector. Thirty-six complete comparisons check three policies on six validation batches with this optimization enabled and disabled. The selected sources, committed operations and exact exports agree. The deterministic cancellation rule receives the same feature optimization, and its coverage control loads no neural model. These controls prevent model startup and unused feature work from creating an artificial learned advantage.

15.7 Selecting thresholds from complete reduction cost

The seed probabilities describe membership in the recorded construction data. Their classification threshold need not minimize the cost of a new native reduction. We therefore select an execution threshold using only the twelve native training batches. For each threshold τ in the six-value set above, the policy selects every candidate whose probability is at least τ, together with the mandatory coverage seeds. The search retains all time spent on unsuccessful attempts and the fixed native fallback.

Let $T_{i,\tau}$ be this complete time for training batch i, and let $T_{i,\mathrm{native}}$ be the time for the smaller native region with its fallback. Batches share a threshold when they have the same topology, largest number of additional propagator powers and largest total numerator power. For each such class $\mathcal{C}$, training selects

$$\tau_{\mathcal{C}} = \arg\min_{\tau} \exp\left[\frac{1}{|\mathcal{C}|}\sum_{i\in\mathcal{C}} \log\frac{T_{i,\tau}}{T_{i,\mathrm{native}}}\right].$$

Here $|\mathcal{C}|$ is the number of training batches in that class. The scan contains ninety-six complete workflows, including the smaller native and deterministic coverage controls. This threshold

search changes execution settings and preserves the earlier classification tests at their original threshold.

One development subset exposed an additional admission requirement. Its export still contained an integral outside the prescribed master basis, although the native master check had accepted the initial system. The exact reference comparison rejected the result. We now also inspect every integral on the right-hand side of the exported reduction. An extra integral marks an incomplete reduction and triggers the fixed expansion of the seed region. A disagreement between coefficients in the admitted basis stops the calculation. The original rejected result remains in the evidence, and its replacement workflow includes the unsuccessful subset's elapsed time.

15.8 Controls and the scope of the optimization

The smaller native control starts with the target's own bounds on propagator and numerator powers and expands them if needed. The conservative control starts one step beyond these bounds. The learned and deterministic coverage methods first try a subset of the smaller region, then the whole smaller region, then the conservative region. Row policies make each source choice inside the current native matrix throughout these attempts. This comparison separates the effect of a learned subset from the effect of starting with fewer native equations.

An additional experiment changes the database settings only for fresh, disposable trial directories. Three paired exact reductions give a median time ratio of approximately 1.011 to the unchanged settings, so the final configuration retains the usual database settings. A separate system-call diagnostic finds negligible time in database flushes for the examined box calculation. Disabling synchronization or moving a journal to memory changes recovery guarantees [32]; the retained configuration receives no performance claim from this rejected experiment.

These searches compare a finite set of models and execution choices under the stated memory and storage limits. They identify configurations with lower measured cost on their supported inputs. They do not prove a globally minimal number of algebraic operations or a globally shortest calculation time.

15.9 Initial independent tests of the integrated calculation

The initial comparison uses eight physical profiles that did not select the weights, thresholds or execution rules. Two two-loop sunrise profiles have squared masses $(1, 2, 2)$ and $(1, 2, 4)$, with external invariant $p^2 = -5$ and $p^2 = -7$, respectively. Two massless double-box profiles have $(s, t) = (-11, -13)$ and $(-13, -17)$. The three-loop vacuum profiles have squared masses $(1, 1, 1, 1, 1, 3)$ and $(1, 1, 1, 3, 3, 3)$. The three-loop ladder profiles have one squared mass equal to one, all other physical propagator masses equal to zero, and $p^2 = -3$ or -5. All values use the common arbitrary mass unit and denominator conventions of Section 14. The dimension d remains symbolic. The ladder topology remains absent from the new row-policy training and validation data.

Each sunrise profile supplies its ordinary twenty-one-target batch and a deeper thirty-two-target batch. Each other profile supplies batches of thirty-two and 128 targets. We compare seven methods on these sixteen batches: the conservative native region, the smaller native region, the deterministic cancellation rule, deterministic seed coverage, the seed model, the row model and both models together. Five repetitions per method give 560 planned workflows. We rotate and reverse method order within each physical profile. All methods use the same prescribed master basis, exact coefficient checks, temporary storage and single-worker schedule [33].

The validation-selected row score emphasizes source length for the boxes and uses the mixed

score for the sunrise and vacuum cases. The transfer test also uses the mixed score for the ladder topology. Seed thresholds come from the training classes; an unseen class uses the predefined threshold 0.01. The complete comparison executes the model ablations even where validation recommends native execution, so an unfavorable model cost remains visible in the results.

The recorded time begins with per-workflow preparation and ends after the final exact check and cleanup. It includes model import, weight loading, feature construction, prediction, every native generation and elimination attempt, and every fallback. Python interpreter launch, the common reference calculation, shared integrity checks and result compression remain outside this interval. Every candidate model and control uses the same timing boundary.

For each batch and method, we take the median of five repetitions. We then form paired ratios, average their logarithms within each physical profile and give the eight profiles equal weight. The descriptive intervals resample whole physical profiles while retaining their two batches. A topology-specific interval therefore contains only two independent profiles. Repeated timings measure execution variation; they do not create additional independent integral families.

The comparison records 560 workflows, of which 560 complete every target with the prescribed coefficients. It retains all 670 native attempts, including unsuccessful seed subsets. The independent audit reparses native exports and checks rational coefficients by exact polynomial cross multiplication in $\mathbb{Q}(d)$. Across the full development and test record, it reparses 2,386 successful native reductions and checks 507,630 coefficient comparisons. The separately retained development basis rejection remains a failed attempt and supplies no accepted reduction.

15.10 Measured model effects against matched native choices

Figure 31 compares each model with the smaller native control. This control receives the same runtime improvements and starts from the same target bounds. The comparison therefore tests whether the learned decisions save enough work to cover their own cost.

The seed policy has a complete-time ratio of 1.149, with descriptive interval $[1.021, 1.311]$, relative to the smaller native control. Its point estimate corresponds to a 14.9% increase in the profile-weighted complete time.

The row policy has a complete-time ratio of 1.004, with descriptive interval $[0.991, 1.016]$, relative to the smaller native control. Its point estimate corresponds to a 0.4% increase in the profile-weighted complete time.

The combined policy has a complete-time ratio of 1.165, with descriptive interval $[1.036, 1.323]$, relative to the smaller native control. Its point estimate corresponds to a 16.5% increase in the profile-weighted complete time.

These aggregate values combine inputs with different equation-generation and elimination costs. The separate physical classes show where a model contributes a measured saving and where its additional attempt or decision cost dominates.

For the two-loop sunrise profiles, the seed, row and combined complete-time ratios are 1.388, 0.990 and 1.424, respectively. Their descriptive intervals are $[1.386, 1.390]$, $[0.975, 1.005]$ and $[1.384, 1.464]$. These values average the two tested batches within each of the two physical profiles.

For the two-loop double-box profiles, the seed, row and combined complete-time ratios are 0.953, 1.003 and 0.961, respectively. Their descriptive intervals are $[0.945, 0.960]$, $[0.983, 1.024]$ and $[0.953, 0.968]$. These values average the two tested batches within each of the two physical profiles.

For the three-loop vacuum profiles, the seed, row and combined complete-time ratios are 1.309, 1.010 and 1.287, respectively. Their descriptive intervals are $[1.110, 1.543]$, $[0.997, 1.022]$ and $[1.104, 1.500]$. These values average the two tested batches within each of the two physical profiles.

For the three-loop ladder profiles, the seed, row and combined complete-time ratios are 1.007, 1.013 and 1.047, respectively. Their descriptive intervals are $[1.003, 1.011]$, $[1.003, 1.023]$ and $[1.002, 1.094]$. These values average the two tested batches within each of the two physical profiles.

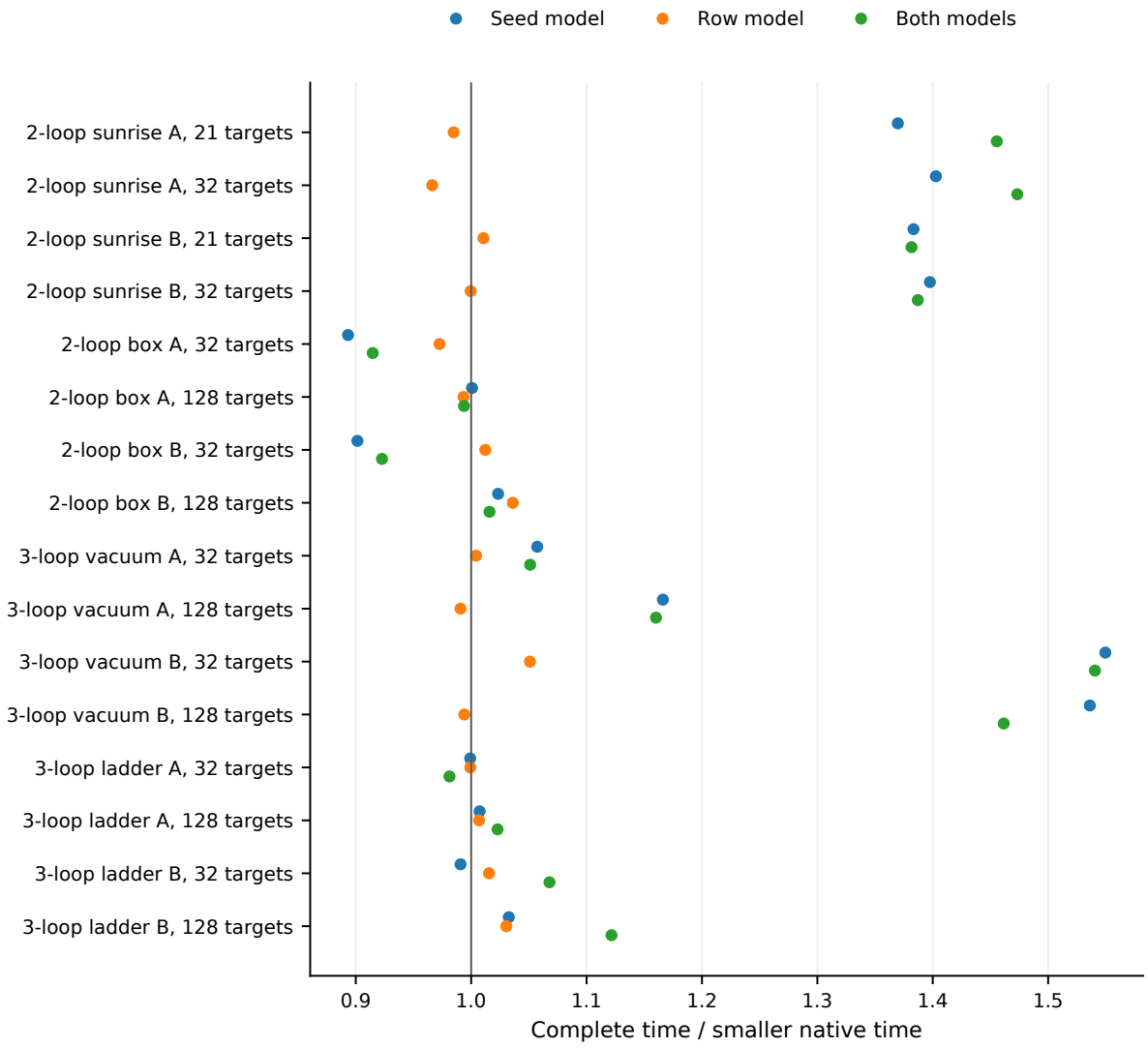


Figure 31: Median complete time for the frozen seed, row and combined policies, divided by the smaller native control's median time on the same target batch. Every point uses five repeated workflows with fixed physical input and master basis. Values below one indicate lower complete time. The preceding text states the timing boundary and profile-weighted intervals.

15.11 Elimination work and the contribution of each model

A lower total runtime can come from seed generation, elimination or shared execution changes. We therefore also inspect committed forward operations and compare the combined method directly with the seed-only method.

The row policy relative to native choices gives an operation ratio of 0.985, a forward-stage time ratio of 1.030, and a complete-time ratio of 1.004. The operation count measures the stated coefficient-update proxy. It excludes Fermat's internal polynomial operations, while the two timing measures include the corresponding arithmetic within their stage boundaries.

The combined policy relative to the seed-only method gives an operation ratio of 0.988, a forward-stage time ratio of 1.018, and a complete-time ratio of 1.014. The operation count measures the stated coefficient-update proxy. It excludes Fermat's internal polynomial operations, while the two timing measures include the corresponding arithmetic within their stage boundaries.

The seed model's complete-time ratio to deterministic coverage is 0.978, with interval $[0.872, 1.071]$. The row model's ratio to the deterministic cancellation rule is 0.985, with interval $[0.975, 0.996]$. Both deterministic controls receive the same feature and runtime optimizations.

Starting from the smaller native region gives a complete-time ratio of 0.502 to the conservative native region. The combined model gives a ratio of 0.585 to that conservative control. The former comparison changes the initial native region without a learned decision. It therefore identifies a shared source of improvement that must be separated from the models' contributions.

15.12 Interpretation of the initial comparison

The integrated entry point applies the settings fixed from validation. It can run either model separately, run both models, or select the validation-recommended method from the topology and target powers. Automatic selection requires at least a two-percent validation improvement over the smaller native control; otherwise it retains that control. Unknown input classes also retain native execution. This rule avoids paying for models on inputs where development did not support their use. The reserved ablations above evaluate the fixed choices; reserved timings do not change their weights or the deployment rule.

The basis check and exact coefficient comparison remain active under every mode. Incomplete seed sets expand through the fixed fallback, and a coefficient disagreement in the admitted basis stops the workflow. The test uses exact rational functions of d at fixed rational masses and kinematics, with the master basis supplied in advance. Its measured times exclude discovering and certifying a new generic symbolic basis.

The new row policies learn from complete native trajectories and act inside Kira's changing matrix. Their allowed action is the source equation among rows with the same leading integral. They do not choose arbitrary columns, sectors or all mathematically possible reductions. The finite policy search, limited physical profiles and descriptive intervals bound the conclusion: the report establishes the measured changes on the stated inputs, while a general gain for both models and a globally fastest reduction require evidence beyond these comparisons.

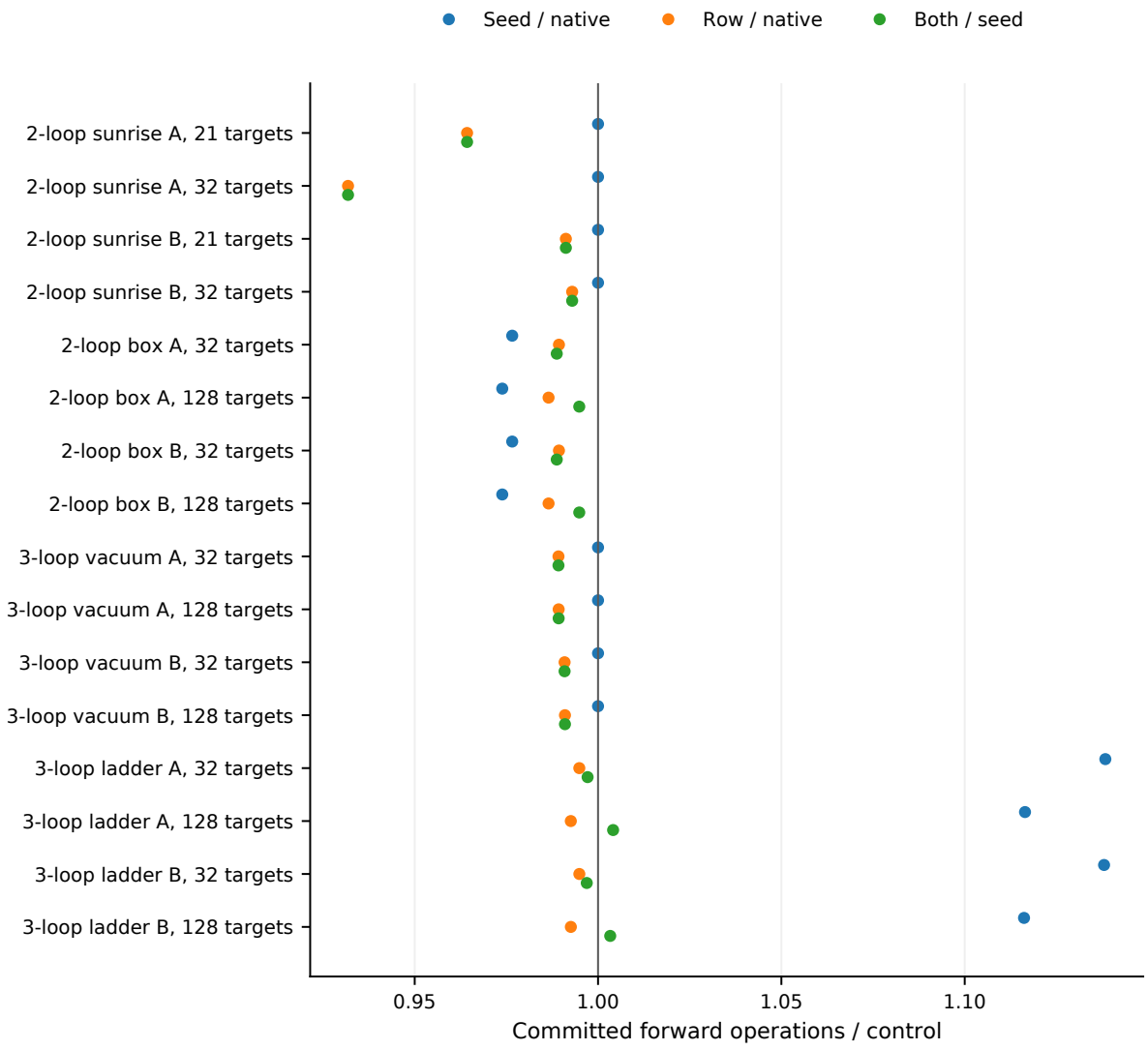


Figure 32: Committed forward coefficient-operation ratios for the seed model against native choices, the row model against native choices, and the combined policy against seed selection alone. The last comparison isolates the additional row policy within the selected seed workflow. These counts describe the recorded update proxy and do not count internal polynomial work.

15.13 Removing redundant features in two-row decisions

The preceding comparison lowers the row-operation count while leaving the cost of evaluating row choices visible in the forward stage. The recorded training data contain 1,923 groups with two source candidates among 2,520 eligible groups. Their algebra supplies an additional way to reduce this cost.

Let $\boldsymbol{u}$ and $\boldsymbol{v}$ be two rows with the same leading integral, and let u_p and v_p be their nonzero leading coefficients. The two possible elimination updates satisfy

$$\boldsymbol{v}' = \boldsymbol{v} - \frac{v_p}{u_p}\boldsymbol{u}, \qquad \boldsymbol{u}' = \boldsymbol{u} - \frac{u_p}{v_p}\boldsymbol{v} = -\frac{u_p}{v_p}\boldsymbol{v}'.$$

Their nonzero positions therefore agree. The estimated number of surviving entries and its square also agree at the finite-field points where both leading coefficients remain nonzero. For either retained linear score, these shared terms cancel when we compare the two candidates. The remaining score difference is

$$s(S,1) - s(S,0) = \lambda \log \frac{1+n_1}{1+n_0} - \eta,$$

where S is the current row group, $s(S,i)$ is the score of candidate i, n_i is its number of nonzero entries, and candidate zero is the native source. The quantity λ is the coefficient of source length and η is the coefficient of the native-source indicator. The two retained policies use $(\lambda,\eta) = (2,0)$ and $(1/2,-0.01)$.

When this comparison selects source zero, the implementation retains that source without constructing cancellation features. Unsupported modular expressions already retain the native source, so the same decision also holds in that case. Other decisions follow the full feature calculation. The shortcut applies only to the stated sparse linear scores, and detailed diagnostic traces retain the full calculation.

We compare the shortcut enabled and disabled in 144 complete validation workflows. All seventy-two pairs agree on source-choice digests, operation counts, attempt outcomes and exact exports. The enabled path bypasses cancellation-feature construction in 42,702 row groups. The model-weighted ratios of decision time, forward-stage time and complete time are 0.345, 0.955 and 0.997, respectively. These ratios compare the same learned policy with and without the shortcut; they describe a reduction in inference cost without changing its algebraic choices.

A separate 4,096-entry cache for modular inverses also preserves all 144 validation reductions. Its forward-stage ratio is 1.002, its decision-time ratio is 1.024, and its complete-time ratio is 0.984. It fails the predefined forward-stage improvement criterion, so the retained implementation leaves this additional cache disabled. The original bounded cache of complete coefficient evaluations remains active.

15.14 Confirming the reduced inference cost

We freeze the two-row shortcut after validation and run a second comparison on the same eight previously reserved profiles. This confirmation preserves the model weights, seed thresholds, physical inputs, reference bases and routing rule. It does not supply eight additional independent families. The smaller native control, deterministic cancellation control, seed model, row model and combined policy each receive five repetitions on all sixteen batches, giving 400 workflows.

Every model and control uses the same new executable and temporary-storage settings. All 400 workflows complete and match their exact reference coefficients. They use 460 attempts, including sixty failed subset attempts whose time contributes to the completed workflows. Their

selected seed masks, committed operations, row-group counts and native exports also agree with the corresponding initial trials. Figure 33 separates complete time from the forward stage in this confirmation [33].

The seed model has a complete-time ratio of 1.155 to the smaller native control, with interval $[1.024, 1.316]$. The descriptive interval lies above one. Its forward-stage ratio is 1.083, with interval $[1.001, 1.186]$.

The row model has a complete-time ratio of 0.999 to the smaller native control, with interval $[0.992, 1.007]$. The descriptive interval includes one. Its forward-stage ratio is 1.041, with interval $[1.017, 1.065]$.

The combined policy has a complete-time ratio of 1.159 to the smaller native control, with interval $[1.030, 1.319]$. The descriptive interval lies above one. Its forward-stage ratio is 1.097, with interval $[1.010, 1.212]$.

For the two-loop sunrise profiles, the seed, row and combined complete-time ratios are 1.397, 0.992 and 1.400. The complete-time intervals are $[1.392, 1.402]$, $[0.985, 0.998]$ and $[1.388, 1.412]$, respectively. The row policy's forward-stage ratio is 1.044, with interval $[1.003, 1.085]$.

For the two-loop box profiles, the seed, row and combined complete-time ratios are 0.964, 1.012 and 0.969. The complete-time intervals are $[0.949, 0.979]$, $[1.003, 1.021]$ and $[0.959, 0.979]$, respectively. The row policy's forward-stage ratio is 1.028, with interval $[0.992, 1.065]$.

For the three-loop vacuum profiles, the seed, row and combined complete-time ratios are 1.319, 1.003 and 1.317. The complete-time intervals are $[1.129, 1.542]$, $[0.996, 1.010]$ and $[1.129, 1.537]$, respectively. The row policy's forward-stage ratio is 1.041, with interval $[1.007, 1.078]$.

For the three-loop ladder profiles, the seed, row and combined complete-time ratios are 1.001, 0.990 and 1.010. The complete-time intervals are $[0.999, 1.003]$, $[0.986, 0.994]$ and $[1.003, 1.017]$, respectively. The row policy's forward-stage ratio is 1.052, with interval $[1.047, 1.057]$.

Applying the row policy within the learned-seed workflow gives a complete-time ratio of 1.004 to seed selection alone, with interval $[1.000, 1.008]$. Its forward-stage ratio in that comparison is 1.013, with interval $[0.997, 1.031]$. These measurements include the model cost and the algebraic consequences of its choices.

The row model's complete-time ratio to the deterministic cancellation rule is 0.993, with interval $[0.983, 1.002]$. The deterministic rule receives the same two-row shortcut, which keeps this comparison matched.

The native integration audit verifies that every used seed belongs to the selected mask and that each learned row attempt loads the frozen policy for its physical class. It records 404 learned-seed attempts with 147,556 used selected seeds, and 324 learned-row attempts with 363,759 eligible groups and 5,795 changes from the native source. These counters show that the models affect native equation generation and the evolving elimination.

The aggregate row-operation reduction therefore does not establish a reduction in forward-stage time. The complete-time interval for the row policy includes one, while always enabling seed selection or both models increases aggregate complete time. The box cases support a conditional seed-model gain. The automatic rule remains the rule fixed on validation; the confirmation results do not retune it.

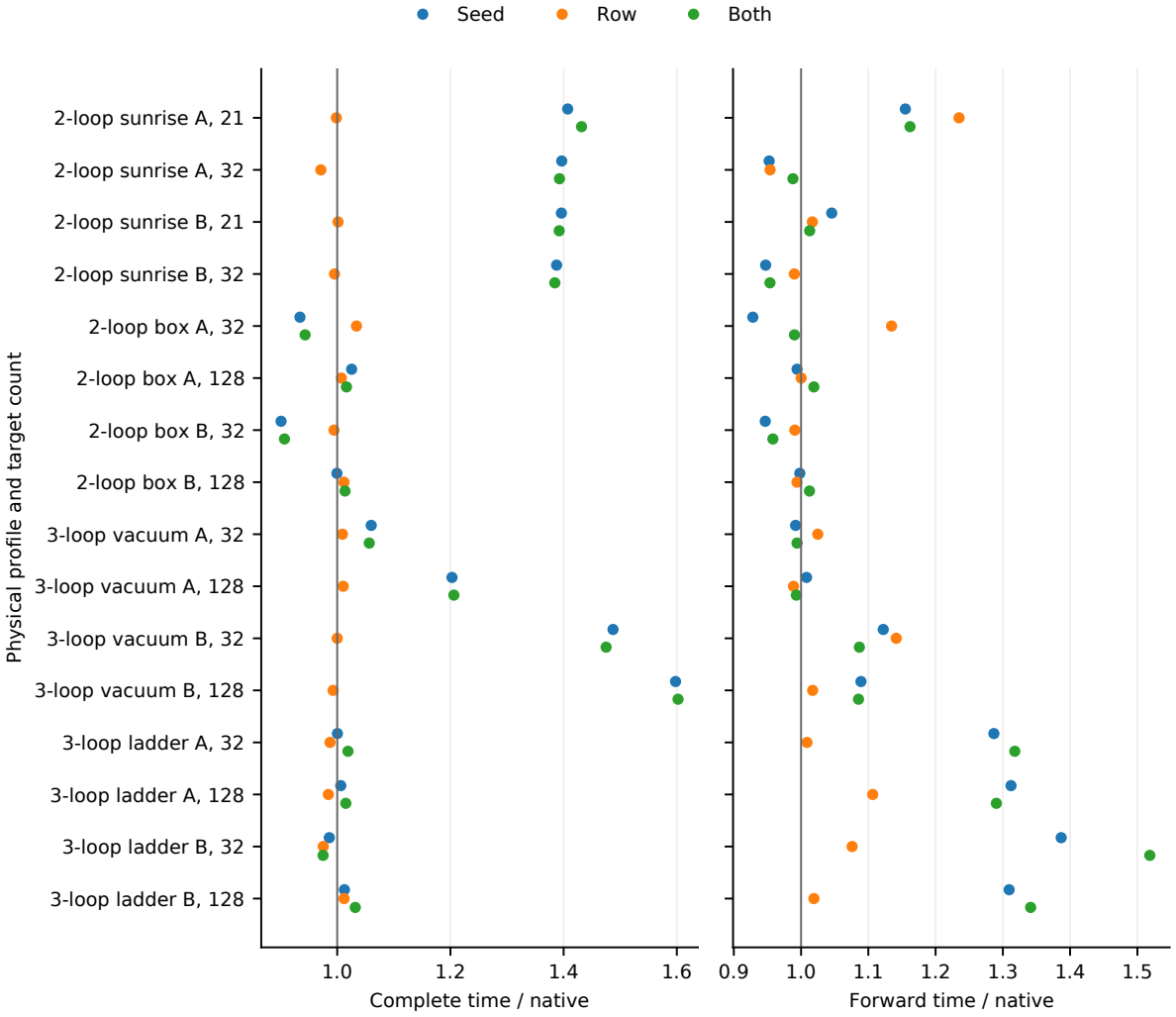


Figure 33: The implementation confirmation compares the same frozen models with the smaller native control after reducing two-row inference cost. The left panel shows complete-time ratios and the right panel shows forward-stage ratios. Each point uses five repeated complete reductions of the indicated batch. The physical profiles repeat those in the initial comparison; all algebraic choices and exact exports remain unchanged.

15.15 Reducing new targets from a known master basis

The reference answers above provide an independent benchmark check. The production interface also accepts only the physical family, target indices and prescribed master indices. It then determines the target coefficients through native IBP generation and exact elimination.

To state this distinction mathematically, let $\boldsymbol{I}$ be the vector of integrals in the generated system and let $A(d)$ be its IBP coefficient matrix. The identities give $A(d)\boldsymbol{I} = 0$. Native exact row operations replace this system by $U(d)A(d)\boldsymbol{I} = 0$, where $U(d)$ represents the valid elementary row operations. Once elimination expresses all requested targets through the prescribed masters, it gives

$$\boldsymbol{I}_{\mathcal{T}} = R_{\mathcal{T}}(d)\boldsymbol{M}.$$

Here $\boldsymbol{I}_{\mathcal{T}}$ contains the requested targets, $\boldsymbol{M}$ contains the supplied master integrals, and $R_{\mathcal{T}}(d)$ is the rational coefficient matrix calculated by Kira. The learned model chooses seeds and admissible source rows; the exact identities and row operations determine this relation.

The controller checks that every target has a reduction and that every integral on its right-hand side belongs to the supplied basis. An additional integral triggers the fixed seed expansion. The interface reports incompleteness if the allowed attempts fail. These checks establish completion relative to the supplied basis under native exact IBP algebra. They do not establish that an arbitrary supplied basis is globally minimal. If reference coefficients are available, the interface can additionally compare them exactly, and its records distinguish these two verification modes.

Eight application checks exercise native, seed, row, combined and automatic execution on sunrise and box inputs. The reducer receives master indices without target coefficients. After each run, a separate caller compares the result with the retained exact reference, and all eight checks pass. The application record includes input and routing time as well as the complete reduction time. It excludes lossless evidence compression and optional common release-integrity checks.

Automatic execution retains the routing rule fixed from validation. It enables a learned method only in the tested feature classes where validation found the required advantage, and uses native execution elsewhere. The individual model modes remain available for controlled comparisons. The reported model effects concern fixed rational kinematics, a known basis and symbolic dimension d. Generic multiscale reconstruction, unrestricted higher-loop families and global minimality remain outside this measured domain.

16 Reusing model calculations without changing reductions

The preceding comparison shows that model evaluation can offset savings in algebraic work. We therefore reduce repeated calculations inside the same models, while retaining their weights, seed thresholds, row choices and automatic routing rule. This section measures how the change affects both model evaluation and the complete reduction.

16.1 Sharing calculations in the seed network

The seed network uses the same family and target context for every candidate. Let $\boldsymbol{c} \in \mathbb{R}^{5h}$ be that context, $\boldsymbol{z}_j \in \mathbb{R}^h$ the embedding of candidate seed j, and $\boldsymbol{a}_j \in \mathbb{R}^h$ its attention-weighted target embedding. The embedding dimension is $h = 32$. The first score layer forms

$$\boldsymbol{h}_j = \tanh(W_c\boldsymbol{c} + W_z\boldsymbol{z}_j + W_a\boldsymbol{a}_j + \boldsymbol{b}),$$

where the matrices are the corresponding blocks of the existing weight matrix, $\boldsymbol{b}$ is its bias vector, and $\boldsymbol{h}_j$ is the hidden score vector. The term $W_c\boldsymbol{c}$ is common to all candidate seeds. We calculate its floating-point accumulator once and continue from that accumulator for each seed. The continuation preserves the original order of multiplications and additions.

The target-pair layer admits similar reuse. If $\boldsymbol{e}_i$ and $\boldsymbol{e}_k$ are two target embeddings, its pair embedding is

$$\boldsymbol{p}_{ik} = \tanh(U_1\boldsymbol{e}_i + U_2\boldsymbol{e}_k + U_3(\boldsymbol{e}_i - \boldsymbol{e}_k) + U_4(\boldsymbol{e}_i \odot \boldsymbol{e}_k) + \boldsymbol{b}_p).$$

Here $U_1, \ldots, U_4$ are blocks of the existing pair-layer matrix, $\boldsymbol{b}_p$ is its bias, and $\odot$ denotes componentwise multiplication. The first partial accumulator depends only on i, so we reuse it for each allowed k. The remaining operations and ordered-pair aggregation keep their original order. This removes repeated arithmetic without refitting the network.

All thirty-six development contexts give bitwise identical probability vectors and identical masks at the six tested thresholds. The largest probability difference is zero. The geometric mean ratio of new to preceding preparation and inference time is 0.527. These component measurements use repeated calls with the models already loaded; the complete-workflow comparisons below also include loading and fallback [34].

16.2 Sharing cancellation estimates between source choices

The two-row identity in the preceding section also applies to each pair within a larger group of equations. Let $\boldsymbol{R}_j$ and $\boldsymbol{R}_q$ be two coefficient rows whose first nonzero entries a_j and a_q multiply the same integral. If both entries are nonzero, the two possible updates give

$$\boldsymbol{R}'_j = \boldsymbol{R}_j - \frac{a_j}{a_q}\boldsymbol{R}_q, \qquad \boldsymbol{R}'_q = \boldsymbol{R}_q - \frac{a_q}{a_j}\boldsymbol{R}_j = -\frac{a_q}{a_j}\boldsymbol{R}'_j.$$

The updated rows differ by a nonzero scalar and therefore contain the same nonzero positions. Their predicted surviving-entry counts satisfy

$$\widehat{n}_{jq} = \widehat{n}_{qj}$$

at the valid finite-field points used by the policy. Here $\widehat{n}_{jq}$ is the predicted number of entries after source q updates row j. The union size, cancellation count and zero-row indicator also agree in the two directions.

We evaluate each unordered pair once and add its statistics to both candidate sources. The previous implementation evaluated both directions separately. The coefficient parser, finite-field

points, cache limits and invalid-input fallback remain the same. A further change reads fixed policy settings once per forward-reduction stage and evaluates only nonzero score coefficients. The policy calculates its normalized confidence when a requested decision record uses that value. The production path retains the selected source without calculating an unused confidence.

One thousand sparse-row checks compare all six cancellation features with the previous implementation. All checks agree, including 210 cases with invalid expressions or finite-field poles. Full validation then compares the old and new implementations on six batches from the three validation profiles. Its 180 workflows preserve probability vectors, masks, row-choice digests, operation counts, attempt outcomes and exact exports. The validation conditions require lower seed complete time, lower row decision time and a row complete-time ratio no larger than 1.02. These conditions are fixed before validation.

16.3 Complete calculations with the unchanged decisions

After validation, we freeze the implementation and compare five methods: native choices, the seed model, the row model, both models, and automatic routing. Each method runs with both implementations on the same sixteen batches from the eight physical profiles in the preceding section. Five repetitions give 800 complete workflows. We increase the confirmation from three to five repetitions before its freeze because validation shows execution-time variation between runs. This confirmation reuses the earlier physical profiles and provides no additional independent-family evidence.

The time includes preparation, model loading, inference, fresh IBP generation, exact elimination, back substitution, reference checks and every fallback. The common reference calculation, interpreter launch, top-level policy lookup and evidence compression remain outside the interval. Old and new runs alternate within each method, and method order rotates across repetitions. We take a median for each batch and use the same profile-weighted geometric ratios and descriptive bootstrap intervals as before.

All 800 workflows finish with exact reference agreement, using 920 attempts. The retained failures comprise 120 unsuccessful attempts and contribute to the completed workflow times. All 400 old–new pairs preserve the recorded algebraic choices and outputs. Across validation and confirmation, the independent parser verifies 269,820 coefficient equalities through exact polynomial cross multiplication in $\mathbb{Q}(d)$.

The native control has a new-to-preceding complete-time ratio of 0.997, with descriptive interval $[0.960, 1.030]$.

The seed model has a new-to-preceding complete-time ratio of 0.982, with descriptive interval $[0.970, 0.993]$.

The row model has a new-to-preceding complete-time ratio of 1.006, with descriptive interval $[0.978, 1.046]$. Its decision-time ratio is 0.669, and its forward-stage ratio is 0.977.

The combined method has a new-to-preceding complete-time ratio of 0.997, with descriptive interval $[0.976, 1.015]$. Its decision-time ratio is 0.647, and its forward-stage ratio is 0.966.

The automatic method has a new-to-preceding complete-time ratio of 0.982, with descriptive interval $[0.951, 1.009]$.

These comparisons isolate implementation cost while retaining each method's decisions. Comparing the new model paths with the new native control addresses the separate question of whether their choices repay the remaining model cost.

Relative to the new native control, the seed model has a complete-time ratio of 1.162, with interval $[1.019, 1.348]$, and a forward-stage ratio of 1.114.

Relative to the new native control, the row model has a complete-time ratio of 1.023, with interval $[1.002, 1.047]$, and a forward-stage ratio of 0.993.

Relative to the new native control, the combined method has a complete-time ratio of 1.145, with interval $[0.998, 1.334]$, and a forward-stage ratio of 1.065.

Relative to the new native control, the automatic method has a complete-time ratio of 0.998, with interval $[0.948, 1.049]$, and a forward-stage ratio of 0.987.

The seed implementation reduces the complete-time estimate by about 1.8% relative to its preceding implementation, and its descriptive interval remains below one. The row decision calculation takes about 33% less time, while the interval for complete row-model time includes one. The row comparison therefore resolves a reduction in decision cost but does not establish a reduction in complete time. The seed and row methods both have larger complete-time estimates than the new native control on this set of inputs. Automatic routing gives a ratio close to one, with an interval that spans both a gain and a loss. These measurements support the stated implementation savings within the tested inputs; they do not establish a general complete-reduction advantage for the learned policies.

The production interface uses the shared calculations with the unchanged validation-selected routing rule. Eight additional application checks supply only the family, target indices and master indices to the reducer, then compare its outputs independently with retained reference coefficients. All eight checks pass. The original model checkpoints, preceding implementations and negative comparisons remain available.

The operation counts are unchanged between implementations because the models choose the same seeds and source rows. The saving comes from calculating these choices with less repeated work. The comparison establishes measured costs for the stated two- and three-loop inputs with fixed rational kinematics and symbolic dimension d. A general gain across unrestricted higher-loop families and a globally fastest reduction remain outside this evidence.

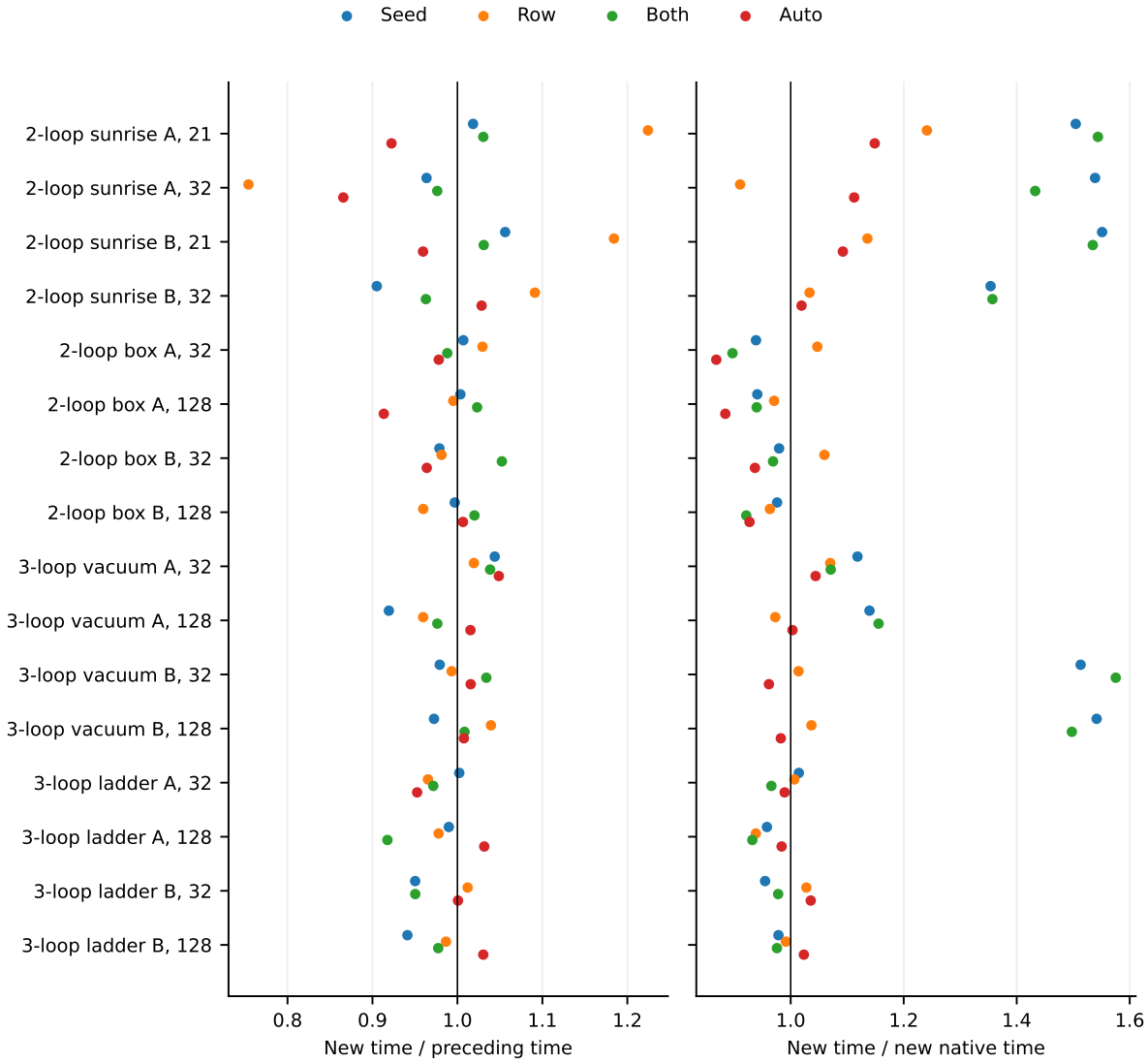


Figure 34: Complete-time ratios after sharing model calculations. The left panel compares each method with its preceding implementation, and the right panel compares it with the new native control. Every point uses the median of five repetitions for one target batch. The tests keep model weights and algebraic choices fixed, include unsuccessful attempts, and reuse the preceding eight physical profiles.

17 Parallel seed evaluation and exact modular inverses

The shared calculations in the preceding section reduce repeated work, but the remaining model evaluation still contributes to the reduction time. We now evaluate independent outputs of the seed network together and use a different exact algorithm for modular division in the row features. We also reduce repeated coefficient parsing and storage allocation. The weights, thresholds, seed rules and source-row policy remain fixed, so this comparison measures the cost of calculating the same decisions.

17.1 Evaluating independent neural outputs together

Let $W \in \mathbb{R}^{n\times m}$ be a layer's weight matrix, $\boldsymbol{x} \in \mathbb{R}^m$ its input and $\boldsymbol{b} \in \mathbb{R}^n$ its bias. Here m is the number of input components and n is the number of output components. Each output uses the partial sums

$$y_j^{(0)} = 0, \qquad y_j^{(k+1)} = \mathrm{fl}\Big[y_j^{(k)} + \mathrm{fl}(W_{jk}x_k)\Big], \qquad k = 0, \ldots, m-1,$$

where j identifies an output component and fl denotes rounding to the existing 32-bit floating-point format. The layer then adds b_j and applies its activation when required. This expression states the order of arithmetic used by the preceding implementation.

The previous loop finishes one output before starting the next. The new loop advances several outputs at each input component k. These outputs are independent, so the compiler can evaluate four of them with one vector instruction while each output retains its original addition order. We store the weights with adjacent output entries and include this rearrangement in the model-loading cost. The shared target-context and target-pair partial sums from the preceding section remain in use. We retain separate multiplication and addition, the same activation functions, and the original target aggregation order.

All thirty-six development contexts give bitwise identical probability vectors. The selected masks also agree at all six tested thresholds. The geometric mean ratio of new to preceding preparation and inference time is 0.423. This component comparison uses five alternating repetitions with loaded models. The complete reductions below include loading and the weight rearrangement.

17.2 Computing modular inverses with Euclid's algorithm

The row policy evaluates rational coefficients at the same two finite-field points as before. Division by a nonzero field element a requires its inverse modulo the prime $p = 2^{31}-1$. The preceding implementation uses $a^{-1} = a^{p-2} \bmod p$. The extended Euclidean algorithm gives integers x and y such that

$$ax + py = 1, \qquad a^{-1} = x \bmod p.$$

Here x supplies the inverse because the greatest common divisor of a and p is one. The algorithm repeatedly divides the current integer remainders and updates the coefficients of a and p until it reaches that greatest common divisor.

We use this inverse in the same coefficient parser, including negative integer powers. The prime, evaluation points, cache bounds and unordered-pair cancellation estimates remain unchanged. A zero denominator still rejects that estimate and retains the existing fallback. These finite-field calculations guide the row choice; native Kira performs the symbolic row operations.

Integer coefficients have the same residue at both dimension points. For a coefficient $c \in \mathbb{Z}$, we calculate its residue r once,

$$r = c \bmod p, \qquad (c(17), c(19)) = (r, r) \quad \text{in } \mathbb{F}_p^2.$$

Here the two entries are the coefficient values used in the cancellation test, and $\mathbb{F}_p$ is the field of integers modulo p. For short integer expressions, this removes two general expression evaluations and the cache lookup. Other expressions retain the bounded parser and cache. We also allocate the evaluated coefficients for a row group in one contiguous array, which removes separate allocations for each row. These changes preserve every coefficient value and the order of the cancellation calculations.

Checks on 100,000 nonzero field elements agree with the preceding inverse and satisfy $aa^{-1} = 1 \bmod p$. A further 1,000 sparse-row checks agree on all six features, including the 210 cases with invalid expressions or finite-field poles. Five alternating repetitions give a ratio of 0.803 for new to preceding row-feature calculation time with cleared coefficient caches. The complete reductions also exercise the ordinary cache reuse and source-selection code.

17.3 Complete reduction costs and exact checks

We first tested the Euclidean inverse alone with the parallel seed calculation. Its 144 validation workflows preserved all decisions and exact exports, but its complete row-time ratio was 1.027 and its row-decision ratio was 1.017. Both missed the conditions stated below, so we retain this candidate as a rejected development result. The revised candidate adds the integer evaluation and contiguous storage described above. These changes use development and validation inputs; confirmation results do not select them.

The next 144 validation workflows compare native choices, seed selection, row selection and both learned choices on six batches from three physical profiles. We fix the adoption conditions before validation and require identical decisions and exports, lower seed complete time, lower row decision time and a row complete-time ratio at most 1.02. Both component comparisons must also improve.

After validation, we freeze the implementation and run the same four methods on sixteen batches from the eight physical profiles used in the preceding comparison. Five repetitions with each implementation give 640 workflows. These inputs provide a repeated implementation comparison and add no new independent-family evidence. Model weights and thresholds remain fixed.

Timing includes preparation, loading, prediction, fresh IBP generation, exact elimination, back substitution, reference checks and every fallback. Reference preparation, interpreter launch, release-hash verification, policy lookup and evidence compression remain outside the timed interval. We alternate the implementations within each method and rotate method order. We take the median for each batch and report the same profile-weighted geometric ratios and descriptive 95% bootstrap intervals as before. A ratio below one means less time. The internal decision timer covers completed policy paths. The complete workflow timer also includes feature evaluations that return an invalid estimate and retain the native source.

All 640 workflows pass the exact reference check, using 760 attempts, including 120 unsuccessful attempts. All 320 old–new pairs preserve probability vectors, selected seeds, row-choice digests, committed operations and exact exports. The independent audit verifies 215,856 coefficient equalities over $\mathbb{Q}(d)$ across the 784 validation and confirmation workflows [35].

The native control has a new-to-preceding complete-time ratio of 1.005, with descriptive interval $[0.976, 1.035]$.

The seed model has a new-to-preceding complete-time ratio of 0.990, with descriptive interval $[0.977, 1.003]$.

The row model has a new-to-preceding complete-time ratio of 0.990, with descriptive interval $[0.971, 1.010]$. Its decision-time ratio is 0.791, and its forward-stage ratio is 0.998.

The combined method has a new-to-preceding complete-time ratio of 0.984, with descriptive interval $[0.965, 1.007]$. Its decision-time ratio is 0.816, and its forward-stage ratio is 1.039.

The new native control provides a separate comparison for the cost and benefit of the learned decisions. It uses the same execution settings and exact checks.

Relative to that control, the seed model has a complete-time ratio of 1.096, with interval $[0.963, 1.275]$.

Relative to that control, the row model has a complete-time ratio of 0.982, with interval $[0.959, 1.003]$.

Relative to that control, the combined method has a complete-time ratio of 1.100, with interval $[0.966, 1.279]$.

All three new-to-preceding complete-time intervals include one. The component timings show lower evaluation cost, while these complete-workflow measurements leave the size of the overall change uncertain. Relative to native choices, the aggregate seed and combined ratios exceed one, and the row-only interval also includes one. The results therefore support lower model-evaluation cost within the measured scope; a general complete-time advantage over native Kira remains unestablished.

For the seed method, the complete-time ratios relative to native choices are 0.911 for the double boxes, 0.935 for the three-loop ladders, 1.380 for the sunrise integrals and 1.226 for the three-loop vacuum integrals.

For the row method, the complete-time ratios relative to native choices are 1.011 for the double boxes, 0.958 for the three-loop ladders, 0.962 for the sunrise integrals and 0.999 for the three-loop vacuum integrals.

For the combined method, the complete-time ratios relative to native choices are 0.902 for the double boxes, 0.945 for the three-loop ladders, 1.394 for the sunrise integrals and 1.232 for the three-loop vacuum integrals.

Eight further application checks supply family definitions, targets and master indices without supplying target coefficients. Independent comparisons after execution pass in all eight cases. The production interface uses the validated implementation with the unchanged routing rule.

The algebraic operation counts remain fixed between the two implementations. The time change comes from evaluating the same model decisions with different arithmetic. This distinguishes a reduction in model cost from a reduction in the algebra needed by Kira. The comparisons concern two- and three-loop inputs with fixed rational kinematics and symbolic dimension d; they do not establish a general speed advantage across unrestricted higher-loop or multiscale reductions.

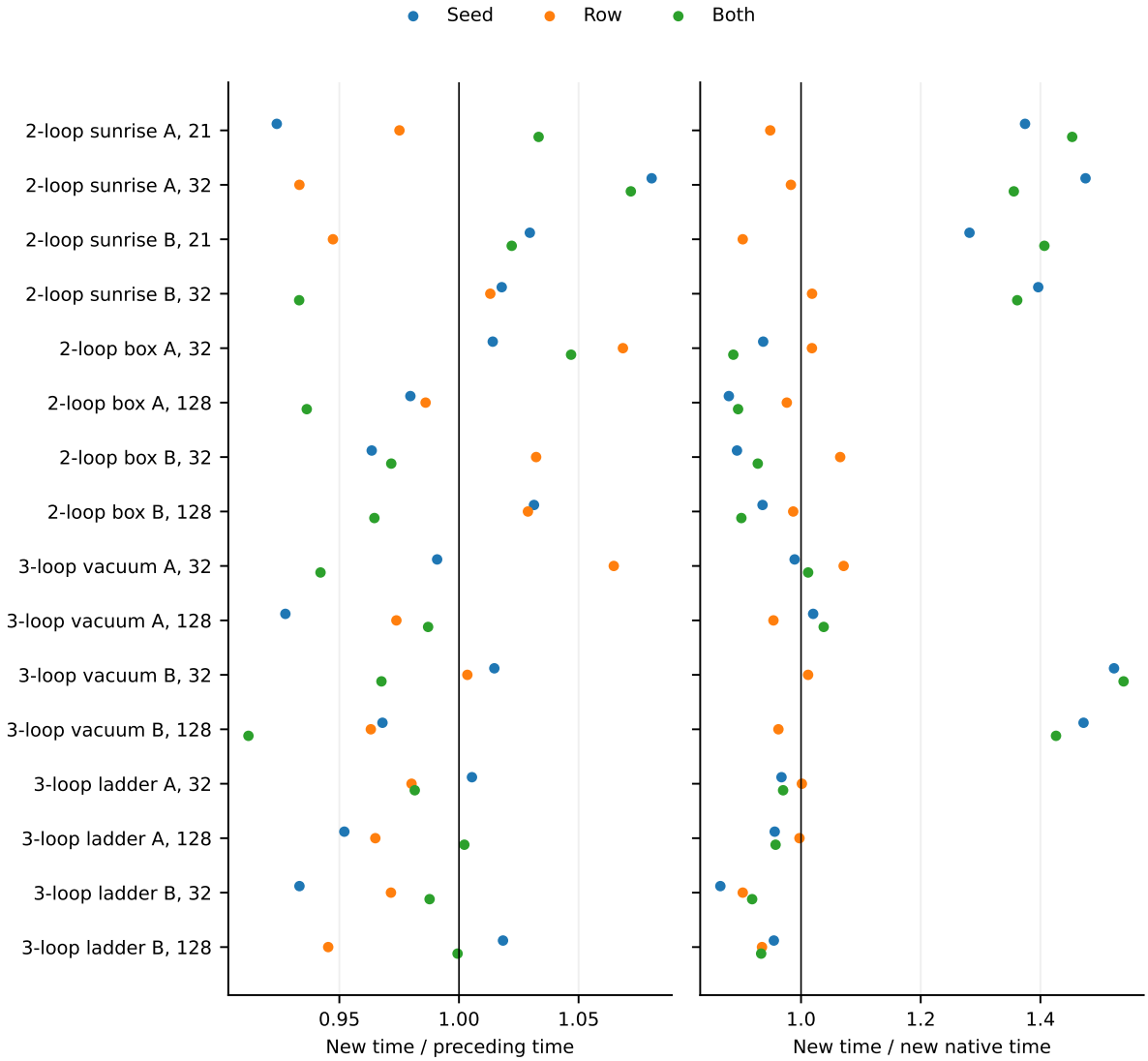


Figure 35: Complete reduction times for the unchanged learned policies. The left panel compares the new implementation with the preceding shared calculations. The right panel compares each new method with the new native control. Each point is the median of five repetitions for one target batch, including failed attempts and fallback. A and B identify the two physical profiles within each topology, and the numbers give the target counts.

18 Target-dependent equation selection

The preceding policies choose seeds and source rows, but each retained seed can produce several identities. A third model selects identities for the requested targets before Kira performs modular independence elimination. This choice can reduce the system that Kira processes, provided that the saved algebraic work exceeds the cost of making the choice. We therefore measure equation preparation and the complete reduction with the model included [9, 36].

18.1 The equations and their training labels

Let $\mathcal{T}$ be the set of requested integral indices. An identity generated from seed $\boldsymbol{\sigma}$ and derivative or Lorentz-invariance operator o has the form

$$E_{\boldsymbol{\sigma},o}: \qquad \sum_{\boldsymbol{a}\in A_{\boldsymbol{\sigma},o}} c_{\boldsymbol{\sigma},o,\boldsymbol{a}}(\boldsymbol{x},d) I_{\boldsymbol{a}} = 0. \tag{109}$$

Here $A_{\boldsymbol{\sigma},o}$ is the set of integral indices that occur in the identity and $c_{\boldsymbol{\sigma},o,\boldsymbol{a}}$ is the coefficient of $I_{\boldsymbol{a}}$. The variables $\boldsymbol{x}$ and d retain their definitions as the kinematic parameters and integration dimension. The operator index distinguishes the several identities from one seed, so the third policy can retain only part of a seed's output.

We first run native Kira reductions on training families and record the integral supports of the equations retained in the final regenerated system. An integral support is the set of indices with nonzero coefficients after combining repeated terms at the finite-field selection point. If $\mathcal{S}_{\mathrm{native}}$ is the collection of these supports, the training label is

$$y_{\boldsymbol{\sigma},o} = \mathbf{1}[\mathrm{supp}(E_{\boldsymbol{\sigma},o}) \in \mathcal{S}_{\mathrm{native}}]\,. \tag{110}$$

The indicator $\mathbf{1}$ equals one when the condition holds and zero otherwise. Several equations can have the same support and different coefficients, so this label describes support membership in the recorded selection. It supplies a conservative training proxy and does not establish a necessary or smallest equation set.

The training collection contains 74,692 candidate equations, with 5,978 positive labels, from ten target batches in six physical profiles. These profiles comprise two-loop sunrise and double-box integrals and three-loop vacuum integrals. We keep three additional profiles for validation and eight for testing. The held-out tests also include a three-loop ladder topology absent from the training and validation collections. Each profile fixes rational masses and kinematic invariants, while the reductions retain symbolic dependence on d.

18.2 Features and the learned probability

The model receives 23 numerical features that describe the family, the target set, the seed and the generated equation. These features include the numbers of loops and external momenta, the number of integral indices, the target count, positive propagator powers, numerator powers, sector membership, the operator index and the equation's integral terms. For example, the distance from a seed to the targets is

$$\delta(\boldsymbol{\sigma},\mathcal{T}) = \min_{\boldsymbol{a}\in\mathcal{T}} \sum_{i=1}^{N} |\sigma_i - a_i|, \tag{111}$$

where N is the number of expressions in the integral family. The model also uses differences between the seed's dot or numerator counts and the largest corresponding counts in the target

set and in the same sector. These quantities describe which equations lie near the requested part of the integral-index lattice.

We fit a classification tree using weighted impurity reduction, with equal total weight for each training profile [30]. A sequence of feature comparisons leads to a leaf, which stores

$$p_{\boldsymbol{\sigma},o} = \frac{n_+ + 1/2}{n_{\mathrm{leaf}} + 1}, \qquad E_{\boldsymbol{\sigma},o} \text{ is retained if } p_{\boldsymbol{\sigma},o} \geq \tau. \tag{112}$$

Here n_+ is the number of positive training rows in that leaf, n_{leaf} is its total number of training rows and τ is the retention threshold. The fitted tree has 233 nodes and a maximum depth of eight. Each accepted split leaves at least 24 rows in each child. The score is a smoothed empirical frequency of the recorded label, and its numerical value has no separate calibration guarantee.

Training the tree takes 2.75 seconds after the native examples have been collected. This time excludes the generation of training reductions. We retain one additional diagnostic field for coefficient-text length, but exclude it from the decisions because equivalent coefficient expressions can have different printed lengths. The saved model and its compiled evaluator give identical probabilities for 18,754 candidates in the implementation checks.

18.3 Native integration and the cost of a decision

Kira constructs a candidate identity before the model first scores it. The retained identities then enter native modular equation processing. Symmetry identities and preferred-basis relations continue through their existing paths. The model scores the IBP and Lorentz-invariance identities at the generation interface; it does not replace Kira's later algebraic independence checks.

Several operators share one seed, so we calculate their common seed-to-target features once. We also store each seed–operator decision. When Kira later requests the same identity, a stored omission allows the generation code to skip rebuilding its coefficients. A retained identity is regenerated through the native code. The tree also calculates an equation feature only when a comparison on its chosen path needs that feature. Propagator and numerator counts come directly from the native integral ordering. These saved decisions avoid repeated predictions and remove part of the repeated equation generation. Exact comparisons check that the implementation changes preserve the decision features, probabilities, masks and reduction results.

After reduction, we require every target to lie in the prescribed master basis. An incomplete filtered attempt triggers an unfiltered native attempt, followed by an enlarged native seed range when necessary. The benchmark additionally compares every exported rational coefficient with an independently stored native reference. The application interface requires the master basis and can operate without reference target coefficients; we check these application outputs separately after the reducer returns.

18.4 Validation and held-out comparisons

We select thresholds using complete reduction times on the three validation profiles. A topology uses the learned filter in the automatic route only when its validation time ratio is below 0.98; other topologies retain native equation choices. The test also includes a forced learned filter and a deterministic distance filter using Eq. (111). Validation selects each filter's threshold before the test runs, and the saved hashes fix the model, integration and test protocol.

The test contains fourteen target batches from eight physical profiles. Each sunrise batch contains 21 targets; the other profiles use batches of 32 and 128 targets. Five alternating repetitions

compare native choices, the forced learned filter, the distance filter and the automatic route, giving 280 complete workflows. All four arms use the same compiled binary, with native choices switching off the three learned policies. Each time includes preparation, loading, predictions, equation generation, exact algebra, output checks and all failed attempts before fallback. The experiment driver is already running when timing begins; Kira process startup and model loading are included. Creation of the independent references and compression of the evidence files lie outside this timer.

Equation preparation includes generation with modular processing, equation selection, modular solving and equation regeneration. It therefore measures the full preparation affected by the filter. We report median times within each batch and geometric ratios with equal weight for each physical profile. Descriptive intervals resample these eight profiles 10,000 times. These intervals describe this finite set of profiles and do not establish performance for arbitrary multiloop families.

18.5 Results and scope

The validation route is double box: off, sunrise: learned at $\tau = 0.001$, vacuum3: learned at $\tau = 0.0001$. Other topologies use native equation choices. The forced learned filter uses $\tau = 0.001$, while the distance filter retains seeds with $\delta \leq 4$. These choices remain fixed throughout the held-out tests.

All 280 complete workflows reproduce their native reference coefficients. The independent audit checks 91,240 coefficient pairs, including 1,931 distinct expression pairs, by exact rational simplification in d. The workflows contain 40 failed filtered attempts before successful fallback, and their times include these attempts. Native median complete times range from 0.785 to 5.880 seconds across the fourteen batches.

The forced learned filter has a complete-time ratio of 1.080, with descriptive interval $[0.976, 1.220]$. Its equation-preparation ratio is 1.053, with descriptive interval $[0.904, 1.224]$. The generated-equation ratio is 0.769, and the selected-equation ratio is 0.985. All ratios use the matched native control, so a value below one indicates a lower measured cost.

The distance filter has a complete-time ratio of 1.016, with descriptive interval $[0.994, 1.040]$. Its equation-preparation ratio is 1.089, with descriptive interval $[1.035, 1.144]$. The generated-equation ratio is 1.000, and the selected-equation ratio is 1.000. All ratios use the matched native control, so a value below one indicates a lower measured cost.

The automatic route has a complete-time ratio of 1.098, with descriptive interval $[0.981, 1.259]$. Its equation-preparation ratio is 1.062, with descriptive interval $[0.896, 1.261]$. The generated-equation ratio is 0.849, and the selected-equation ratio is 1.000. All ratios use the matched native control, so a value below one indicates a lower measured cost.

The automatic route does not lower the aggregate complete time in this test. Equation-count changes therefore do not establish an overall runtime gain.

After freezing the model, we also collect 77,246 held-out candidate labels, with 6,932 positive labels. At the forced learned threshold, precision is 11.33% and recall is 95.63%, with mean squared probability error 0.0621. These quantities measure agreement with the native support-membership proxy. Exact coefficient comparisons measure the validity of the resulting reductions.

A further 48 complete workflows test the equation route together with the existing row policy and with both existing policies on four profiles. Adding the equation route gives a complete-time ratio of 1.165 to the row-only configuration and 1.184 to the configuration with both earlier

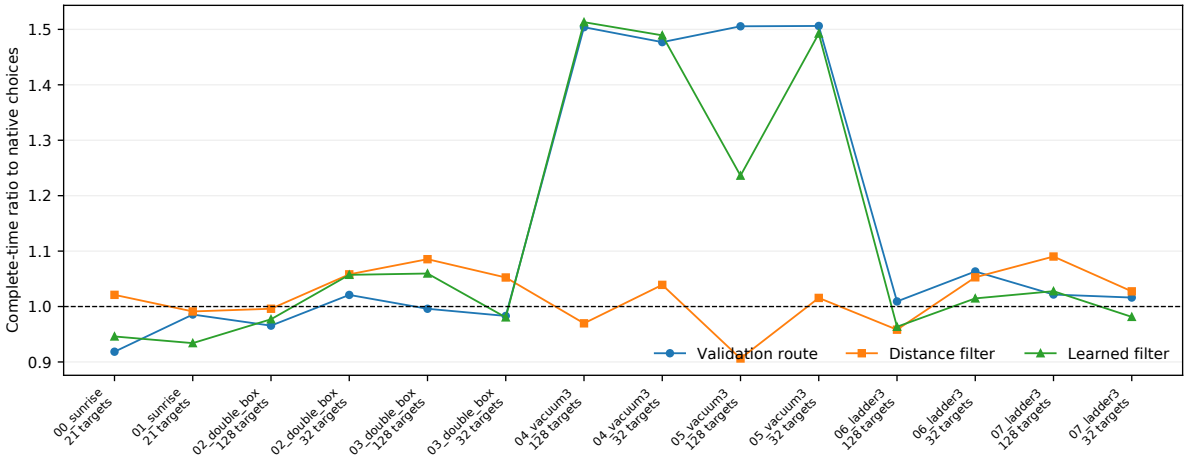


Figure 36: Median complete-time ratios for the fourteen held-out batches. Each point uses five repetitions and includes checks and fallback. The dashed line is the native control.

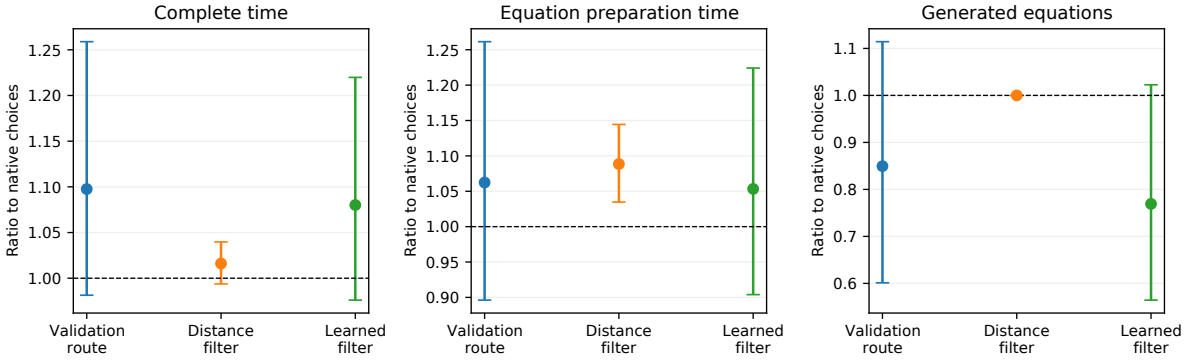


Figure 37: Profile-weighted geometric ratios of complete time, equation preparation time and generated-equation count. Bars show descriptive intervals from resampling the eight physical profiles.

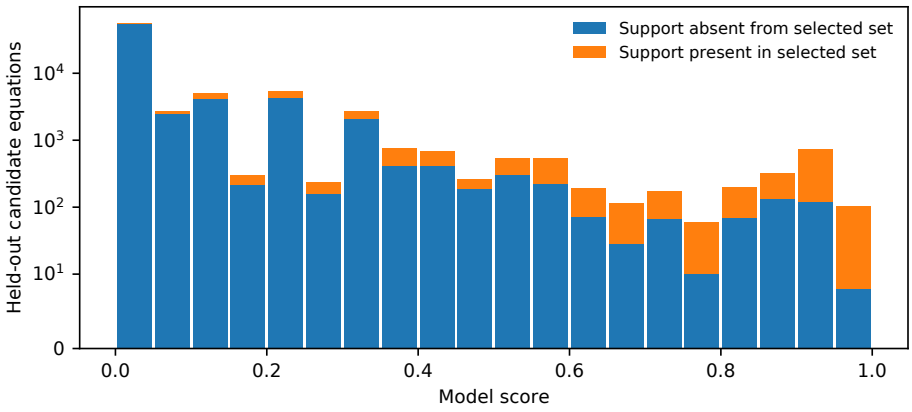


Figure 38: Held-out model scores grouped by the native support-membership label. The vertical scale is linear near zero and logarithmic above ten equations. The labels describe selected supports rather than necessary equations.

models. These comparisons use three repetitions and retain all fallback costs. All pass the reference checks. Eight application runs also pass exact coefficient comparisons after execution with reference coefficients withheld from the reducer.

The early full-feature implementation and subsequent feature-sharing implementations remain in the evidence collection, including their slower results. The final comparison uses the frozen implementation selected before held-out testing. Its performance depends on target size, topology and the cost of generating and reducing the retained equations. The experiment covers physical two- and three-loop reductions at fixed rational kinematics with symbolic d; it does not establish gains on arbitrary higher-loop or fully multiscale problems [36].

18.6 The domain selected before testing

Validation enables the equation model for the sunrise and three-loop vacuum topologies. This identifies four of the eight test profiles before any test timing is available. On these profiles, the automatic route has a complete-time ratio of 1.194, with descriptive interval $[0.951, 1.498]$. Its equation-preparation ratio is 1.078, with descriptive interval $[0.765, 1.519]$. The corresponding generated-equation ratio is 0.722.

For sunrise profiles, the automatic complete-time ratio is 0.951 and the preparation ratio is 0.765. For vacuum3 profiles, the automatic complete-time ratio is 1.498 and the preparation ratio is 1.519. For double box profiles, the automatic complete-time ratio is 0.991 and the preparation ratio is 0.988. For ladder3 profiles, the automatic complete-time ratio is 1.027 and the preparation ratio is 1.110. The double-box and ladder routes retain native equation choices, so differences between their automatic and native timings mainly reflect repeated-run variation and routing overhead. The all-profile comparison remains the primary result; these fixed-domain results state where the learned filter was active.

18.7 Restricted deployment and fresh confirmation

The broad test exposes incomplete equation filters in the new vacuum profiles. Their native fallback retains exact results and adds cost. We therefore fix a restricted deployment using the same trained tree. It enables the equation filter at $\tau = 0.001$ for the recorded 21-target sunrise batches and retains native equation choices for other topologies and target sets. The broader test supplies development evidence for this deployment revision.

Each supported batch contains the top-sector scalar integral and its one-dot and one-numerator variants, together with the corresponding integrals in the three two-propagator sectors. This gives 21 targets whose index vectors are fixed in the deployment definition. We reserve six additional mass and momentum profiles before running their reductions. None of these profiles occurs in the preceding training, validation or broad-test sets. Their squared masses and external invariant are fixed rational numbers, while d remains symbolic. The tree, threshold, target domain and new profile definitions are fixed before this confirmation begins.

Nine alternating repetitions compare the learned filter, native choices and the previously selected distance control in each profile. These 162 complete workflows use the same timing and exact-check rules as the preceding test, including every failed attempt.

The learned filter has a complete-time ratio of 0.994, with descriptive interval $[0.991, 0.997]$ and an equation-preparation ratio of 0.785, with descriptive interval $[0.776, 0.794]$. Its generated-equation ratio is 0.414, and its selected-equation ratio is 1.000. The distance control has a complete-time ratio of 1.003, with descriptive interval $[1.000, 1.006]$.

The complete-time interval remains below one for these six profiles, supporting a measured gain within the restricted domain.

A further 36 workflows compare the existing seed and row policies with the configuration that also includes the equation filter. Adding the third model gives a complete-time ratio of 1.317, with descriptive interval $[1.296, 1.337]$ and a preparation ratio of 1.151, with descriptive interval $[1.136, 1.168]$ to the two-model configuration. This comparison uses three repetitions in each of the six profiles.

All 162 main workflows, 36 composition workflows and six basis-only applications pass the independent exact audit, which checks 7,582 coefficient pairs. The 204 workflows include 54 failed attempts. The six applications receive the prescribed masters without reference target coefficients, and their outputs pass coefficient comparisons after execution.

Native median complete times range from 0.692 to 0.709 seconds in these six profiles. The geometric mean preparation fraction of native complete time is 3.29%. This small fraction explains why the reduction in preparation time produces a smaller change in complete time. The main confirmation has zero failed attempts; all 54 failed attempts in its extended audit arise in the composition comparisons.

The application interface enables this equation policy when it runs with native seed and row choices. Requests for either earlier learned policy retain native equation choices. This guard avoids the slower measured three-model combination. The broader initial route and its slower vacuum results remain available as experimental evidence. The measured domain contains two-loop sunrise integrals with the stated target set, and extension to other target sets or topologies requires further independent evidence [36].

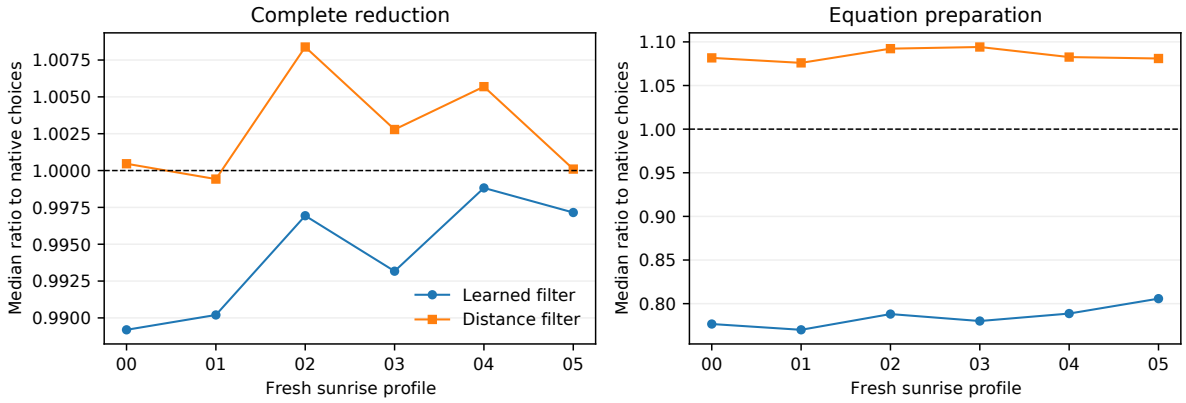


Figure 39: Fresh confirmation of the restricted sunrise deployment. Each profile uses nine repetitions for each method. Complete and preparation times include prediction and fallback costs.

18.8 Predicting equations before constructing their coefficients

The preceding equation model first inspects a constructed identity, so its initial prediction still requires that identity's coefficients. To reduce this cost, we train a new version of the same model that uses only the seed, the operator and the requested targets. This moves the decision to the point where Kira can avoid constructing an omitted identity.

A diagnostic study motivates this change. In twelve exact native runs, equation preparation forms the largest recorded block in the 128-target three-loop ladder reductions. The two ladder profiles have median preparation intervals of 1.41 and 1.23 seconds, within complete times of 4.23 and 4.36 seconds. Initial family preparation and equation preparation together account for about 54–55% of those times. These diagnostic intervals come from logged stage boundaries and include wrapper costs; the final comparison below uses the internal timers defined for this study.

For the identity in Eq. (109), the new decision has the form

$$\widehat{p}_{\boldsymbol{\sigma},o} = f_\theta(\phi(\boldsymbol{\sigma}, o, \mathcal{T}))\,, \qquad \widehat{p}_{\boldsymbol{\sigma},o} \geq \tau \implies E_{\boldsymbol{\sigma},o} \text{ is constructed.} \tag{113}$$

Here ϕ is the vector of features available before construction, f_θ is the classification tree with trained parameters θ, and τ is the retention threshold. The 17 features describe the numbers of loops, external momenta and indices, the target count, the seed's sector, dots and numerator rank, their differences from target limits, the distance in Eq. (111), whether the seed is itself a target, and the operator index. The tree therefore has no need to inspect the equation's coefficients or integral terms.

Kira supplies the ordered IBP operators, and the integration retains their positions when it omits an equation. Several operators share the same seed features, so we calculate those features once and store the decision for each seed–operator pair. Kira constructs accepted equations through its native algebra and reuses the stored decisions during later regeneration. Symmetry identities and preferred-basis relations retain their existing treatment. This placement reduces both repeated model work and the initial construction of identities that the model rejects.

We retain the support-membership labels in Eq. (110) and add four training batches from two new ladder profiles. The combined collection contains 153,124 candidate equations, including 12,270 positive labels, in fourteen batches from eight physical profiles. These profiles contain the two-loop sunrise and double-box and the three-loop vacuum and ladder structures. We fit trees with maximum depths eight and twelve, using equal total weight per profile and the same smoothed leaf frequency as before [30]. The trees contain 295 and 1,335 nodes and take 3.39 and 5.85 seconds to fit after collection of the native training reductions. Their splits leave at least 24 and eight rows in each child, respectively. A separate implementation check compares 918,744 decisions and finds agreement between the stored Python features and the C++ evaluator's recomputed seed features.

The training label still describes membership in a recorded native equation selection. It supplies neither a smallest sufficient equation set nor a calibrated completeness probability. Kira therefore checks whether the result expresses each target in the prescribed master basis. If the filtered equations are insufficient, the application retries native equations and then a larger native seed region. Exact algebra determines all coefficients, and the benchmark additionally compares them with an independent native reduction.

18.9 Selecting the complete calculation on validation profiles

We compare both trees at thresholds 0.001, 0.003 and 0.01 with native choices and a distance filter. Repeated validation requires at least 1% lower complete time, at least 5% lower equation-

preparation time and no fallback before activating an early filter. We then measure combinations with the existing seed and row models. An additional model must lower complete time by at least 1% relative to early filtering alone and require no fallback in validation.

The validation record contains 77 initial candidate and control workflows, 84 repeated candidate workflows, 72 row-composition workflows and 96 seed-composition workflows. A second vacuum profile checks another mass pattern in the composition stage. Adding the row model gives less than 1% incremental saving in the ladder and vacuum comparisons and increases double-box time. Seed combinations require extra attempts for some larger double-box and vacuum batches. These costs remain in the comparisons and prevent those combinations from meeting the activation rule.

Validation therefore selects the depth-twelve tree for double-box batches and the depth-eight tree for vacuum and ladder batches, with $\tau = 0.01$. The final route retains native seed and row choices. It keeps the preceding equation policy for the recorded 21-target sunrise batch, where early filtering fails the validation activation rule. Other structures and ordered target lists use the preceding guarded route. This selection minimizes measured cost among the eligible validation candidates; it does not establish a globally fastest reduction.

We preserve an excluded preliminary collection whose proposed additional profiles overlapped earlier reserved definitions. We replace those profiles before the final collection and exclude every provisional row, model and outcome from the results above. The final training, validation and test profiles are distinct. They can share topologies, target indices and lower-sector integral structure.

18.10 Independent complete-reduction comparison

We fix the models, integration, thresholds and target domains before testing eight additional physical profiles. Two profiles belong to each of the sunrise, double-box, vacuum and ladder structures. Sunrise uses 21 targets, and each remaining profile uses batches of 32 and 128 targets. Five alternating repetitions compare native choices, the preceding release, the deployed early policy and the validation-selected distance filter. This gives 280 complete comparison workflows. The same new binary runs the native, early and distance arms; the preceding release uses its saved binary and policy. The native control includes the existing project runtime improvements and switches off learned choices.

The profiles fix rational squared masses and external invariants, while d remains symbolic. The sunrise squared masses are $(2, 7, 11)$ and $(3, 7, 13)$, with $p^2 = -101$ and -103, respectively. The double-box pairs are $(s, t) = (-101, -103)$ and $(-107, -109)$. The vacuum squared-mass vectors are $(1, 1, 1, 1, 1, 11)$ and $(1, 1, 1, 13, 13, 13)$, and the ladder squared-mass vector is $(1, 0, 0, 0, 0, 0, 0, 0)$ with $p^2 = -101$ and -103. Here p is the external momentum of each two-point family, and s and t are the independent four-point invariants of the double-box family. These values use the common mass scale of the family definitions. The coefficient comparisons therefore take place in $\mathbb{Q}(d)$, the field of rational functions of d with rational coefficients. The graph structures and ordered target lists are known from development; the new profiles test transfer across masses and invariants within that domain.

Complete time includes route selection, input preparation, model loading and evaluation, Kira startup, IBP generation, exact algebra, basis and coefficient checks, cleanup and every fallback. The comparison driver is already running. Independent reference and master-basis preparation, the later independent audit and compression of retained results lie outside the timer. Equation-preparation time includes modular generation, selection and solving, together with symbolic equation regeneration. These internal intervals differ from the broader diagnostic intervals used to locate the initial bottleneck.

We take the median of five repetitions within each batch, average logarithmic time ratios over sizes within a physical profile, and give each profile equal weight. Descriptive 95% intervals resample whole profiles 10,000 times. For methods A and B, the reported ratio is

$$R_{A/B} = \exp\left[\frac{1}{G}\sum_{g=1}^{G}\frac{1}{|B_g|}\sum_{b\in B_g}\log\frac{\widetilde{T}_{A,b}}{\widetilde{T}_{B,b}}\right], \tag{114}$$

where G is the number of physical profiles, B_g is the set of batches for profile g, and $\widetilde{T}_{A,b}$ is the median complete time for method A and batch b. The same expression applies to preparation time. A ratio below one means that method A uses less time.

The deployed policy has a complete-time ratio of 0.943 to native choices, with descriptive interval $[0.915, 0.970]$, and a preparation ratio of 0.638, with interval $[0.546, 0.734]$. These correspond to 5.7% lower complete time and 36.2% lower preparation time in the profile-weighted comparison. Its complete-time ratio to the preceding release is 0.943, with interval $[0.912, 0.974]$. Its ratio to the distance control is 0.934, with interval $[0.904, 0.965]$.

For the two-loop sunrise profiles, the complete-time ratio is 0.986 and the preparation ratio is 0.819. For the two-loop double box profiles, the complete-time ratio is 0.884 and the preparation ratio is 0.655. For the three-loop vacuum profiles, the complete-time ratio is 0.958 and the preparation ratio is 0.453. For the three-loop ladder profiles, the complete-time ratio is 0.947 and the preparation ratio is 0.681.

Across fourteen batches, native median complete times range from 0.593 to 2.709 seconds, and deployed medians range from 0.589 to 2.579 seconds. Summing one median per batch gives 21.83 seconds for native choices and 20.28 seconds for the deployed route. This workload sum uses a different weighting from Eq. (114).

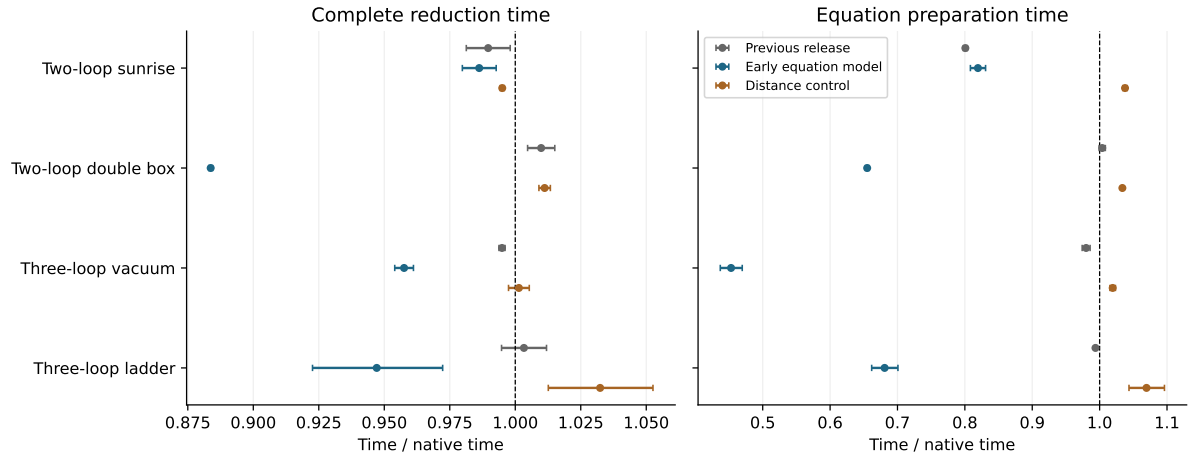


Figure 40: Complete and equation-preparation time relative to native choices. Points use physical-profile weighting and intervals resample profiles. Each topology has two new physical profiles.

All 280 comparison workflows and 8 additional applications pass the independent exact audit. The applications receive only the prescribed basis, and their coefficients agree with the reference after execution. The audit checks 94,970 coefficient pairs, including 1,915 distinct pairs. Every method completes on its first attempt. Maximum sampled memory for Kira and its child processes is 53.46 MiB, and maximum temporary solver storage is 1.39 MiB.

These results test a trained decision before native equation construction, with complete costs and exact output checks. They establish the measured domain at fixed rational kinematics.

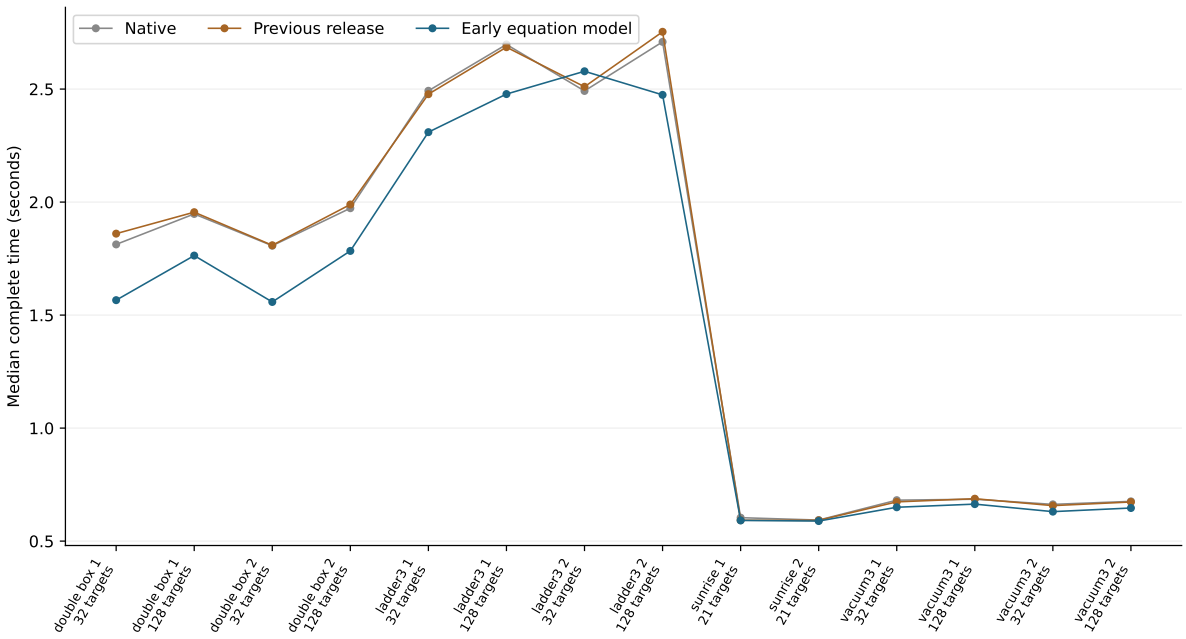


Figure 41: Median complete time for each fixed target batch, including inference, exact checks and all fallback attempts. Each point is the median of five alternating repetitions.

General symbolic multiscale coefficients, new graph structures, broader target lists and high-loop reductions require separate evidence. Section 19 supplies subsequent tests on new scattering structures with independent symbolic invariants. The earlier negative comparisons remain part of the study, and the selected route activates only the combinations supported by validation [37].

18.11 Equation selection and shared reductions at higher loop orders

The preceding native comparisons cover specific structures with two or three loops. To study the higher orders already present in the integral collection, we now use the coupled vacuum family in Eq. (52). This family has one power of each massless denominator and two to four distinct massive denominators that depend on the same momentum Q. Its Euler identities and mass partial fractions remain valid for any positive integer number of loops. This structure allows us to test every order from one through twelve with an exact reference, while keeping the family restriction explicit.

The dependence on the loop order suggests a further reduction in repeated work. Let $B = \sum_j b_j$, retain the squared-mass scale μ, and define

$$z = L(d-2), \qquad W_{\boldsymbol{b}}^{(L)} = \mu^{B-1} V_{\boldsymbol{b}}^{(L)}. \tag{115}$$

The rescaling gives every $W_{\boldsymbol{b}}^{(L)}$ the same mass dimension as the masters $M_j^{(L)}$. Equations (56) and (55) then become

$$0 = (z - 2B) W_{\boldsymbol{b}}^{(L)} + 2 \sum_j b_j \rho_j W_{\boldsymbol{b}+\boldsymbol{e}_j}^{(L)}, \tag{116}$$

$$0 = (\rho_i - \rho_j) W_{\boldsymbol{b}}^{(L)} - W_{\boldsymbol{b}-\boldsymbol{e}_i}^{(L)} + W_{\boldsymbol{b}-\boldsymbol{e}_j}^{(L)}. \tag{117}$$

Here ρ_j is the positive squared-mass ratio defined earlier, and the second identity requires $b_i, b_j > 0$. The coefficient equations depend on L only through z. We can therefore solve one

system over $\mathbb{Q}(z)$ at fixed rational mass ratios and obtain

$$V_{\boldsymbol{b}}^{(L)} = \mu^{1-B} \sum_{j=1}^{R} C_{\boldsymbol{b}j}\big(L(d-2);\boldsymbol{\rho}\big) M_j^{(L)}. \tag{118}$$

Each $C_{\boldsymbol{b}j}$ is a polynomial in z with rational coefficients in the present domain. The master integrals retain their physical dependence on L. The shared calculation supplies their reduction coefficients.

We next reduce the number of identities passed to Kira. For a mixed integral, the model scores each allowed pair (i, j) in Eq. (117). The selected identity replaces that integral with two integrals of total massive power $B-1$. We repeat the choice for the resulting integrals and merge repeated dependencies across the target batch. An integral with only one massive denominator uses Eq. (116) at the seed $(k-1)\boldsymbol{e}_j$ to reduce power k. At $B = 1$, the integral belongs to the prescribed master basis. This decreasing total power proves termination and sufficiency for every allowed model choice. Mass partial fractions carry no IBP seed; the saved dependency choices also determine the actual Euler seeds with one massive denominator.

The training reference counts the distinct nonmaster integrals in recursively chosen dependency sets. If $\mathcal{D}(\boldsymbol{b})$ denotes such a set, the candidate cost is

$$c(\boldsymbol{b}; i, j) = \left|\{\boldsymbol{b}\} \cup \mathcal{D}(\boldsymbol{b} - \boldsymbol{e}_i) \cup \mathcal{D}(\boldsymbol{b} - \boldsymbol{e}_j)\right|. \tag{119}$$

Every pair tied for the smallest candidate cost receives label one. The recursion fixes a valid local reference and does not establish the minimum cost for an entire target batch. We train small classification trees using the ordered massive powers, pair positions, active-denominator count and simple power summaries. There are 18 features, with no loop order feature. The absence of L follows from Eqs. (116)–(117). A leaf score is an empirical training frequency. The runtime selects the highest score and uses exact identities for the algebra. For two active massive denominators, the single available pair is selected directly.

The training set contains 2,016 candidate actions in 546 exponent shapes, with total power through nine. We fit depth limits four, eight and twelve using a minimum child size of four examples. Validation selects the depth-4 candidate, which contains 15 nodes. On 277 distinct development coefficients, the reader agrees with symbolic algebra and takes 0.159 seconds, compared with 4.149 seconds for the symbolic reader. This reading improvement applies to every comparison arm and is separate from the learned decision.

Kira solves each selected system through its interface for supplied systems [11, 9]. We generate the Euler and mass identities within the measured workflow and use the same native elimination policy for every comparison arm. This experiment measures supplied-identity reductions. It does not measure Kira's native generation of a general IBP system at higher loop orders. The earlier native benchmarks retain that separate role.

The integration reads the exported coefficients with bounded rational polynomial arithmetic and checks that polynomial division leaves zero remainder. Development comparisons against symbolic algebra check this reader independently. The final check uses the Taylor partial fraction formula in Eq. (62), which does not follow the selected dependency path. The workflow rejects unresolved nonmaster integrals and retains a fallback to the full equation pool. Timings include input and model loading, selection, equation construction, Kira, coefficient reading, substitution, verification and any fallback. We also report external process times, which include Python startup and evidence writing. Queue waiting time and offline training are excluded.

The refined validation contains 144 workflows on six batches of mass profiles. It compares the three trees with first-pair, smallest-pair, largest-pair and extreme-pair rules and the full pool. Each candidate has three repetitions. The extreme-pair rule chooses an active denominator with

the smallest power and one with the largest power. Validation selects the extreme route; the selected learned candidate costs 1.7% more than the selected rule. We freeze sources, weights, profiles and target lists before 90 reserved workflows. The reserved set has two profiles for each of $R = 2, 3, 4$, with maximum powers eight and twelve, and runs every $L = 1, \ldots, 12$. Some exponent shapes with lower powers also occur in training; the cases with maximum power twelve extend its power bound. The tests vary mass profiles within the same topology and do not test new graph classes. We use three repetitions and report geometric means of the six ratios of batch medians. Descriptive intervals resample those six batches.

For each mass profile and target batch, we compare separate reductions with the full equation pool at each L, separate learned reductions, a shared reduction with the full equation pool in z, a shared reduction with a simple rule, and a shared learned reduction. The full pool contains Euler seeds through the largest target power and all mass-pair identities through one additional power. The control with a simple rule uses the rule selected on validation. All arms use the same project Kira binary with the earlier runtime improvements and native row choices. These controls separate the benefit of exact loop reuse from the benefit of selecting a smaller equation set.

For separate reductions at each loop order, learned selection gives a complete-time ratio of 0.911 to the full pool, with descriptive interval $[0.815, 0.986]$. With exact loop reuse in both arms, learned selection gives a ratio of 0.925 to the shared full pool, with interval $[0.889, 0.958]$. The learned/shared ratio to the rule/shared control is 0.968, with interval $[0.927, 1.003]$. These comparisons keep the incremental learned effect separate from the exact reuse of coefficient equations.

The deployed shared route gives a complete-time ratio of 0.155 to separate reductions with the full equation pool, with interval $[0.132, 0.184]$. This corresponds to 6.47 times faster execution of the twelve-order workload under the stated timing definition. Figures 42 and 43 show the batch costs and equation counts. Validation fixes the deployed rule before these tests; we retain the learned results as a measured alternative.

We separate calculation time from the cost of starting Python and archiving the test evidence. Before those two outer costs, the separate learned route has time ratio 0.943, with descriptive interval $[0.873, 0.995]$, to the full pool. The time spent inside Kira has ratio 0.965, with interval $[0.925, 0.992]$. Thus the calculation saves 5.7% and Kira itself saves 3.5% in this comparison. Archiving fewer source equations contributes to the larger saving in external process time.

With shared coefficients, the ratio of calculation times of the learned route to the full pool is 0.966, with interval $[0.941, 0.982]$. Its ratio to the control with a simple rule is 0.997, with interval $[0.984, 1.011]$. This comparison does not establish an added advantage over the simple rule. The deployed shared calculation has ratio 0.128 to separate calculations with the full equation pool, with interval $[0.102, 0.164]$. The exact change of variable supplies the main reduction in repeated work.

All 90 reserved workflows and six additional applications pass exact checks. The extra applications use loop orders 13, 16, 24 and 32. The audit checks 65,504 coefficient positions, including 5,696 distinct positions, across 492 Kira calls. It finds 0 failed attempts and 0 fallback workflows. The largest sampled memory of Kira and its child processes is 23.05 MiB, and the largest temporary solver storage is 0.58 MiB. The analytic reference also supplies a direct reduction for this special family. The reported comparisons measure routes that use Kira.

Equation (118) holds for any positive L in this scalar family. The finite tests cover the stated orders. Generic scattering families contain additional scalar products, numerator powers and kinematic variables, so their native IBP reduction needs separate tests. These results apply to the stated coupled vacuum family and supplied identities [38].

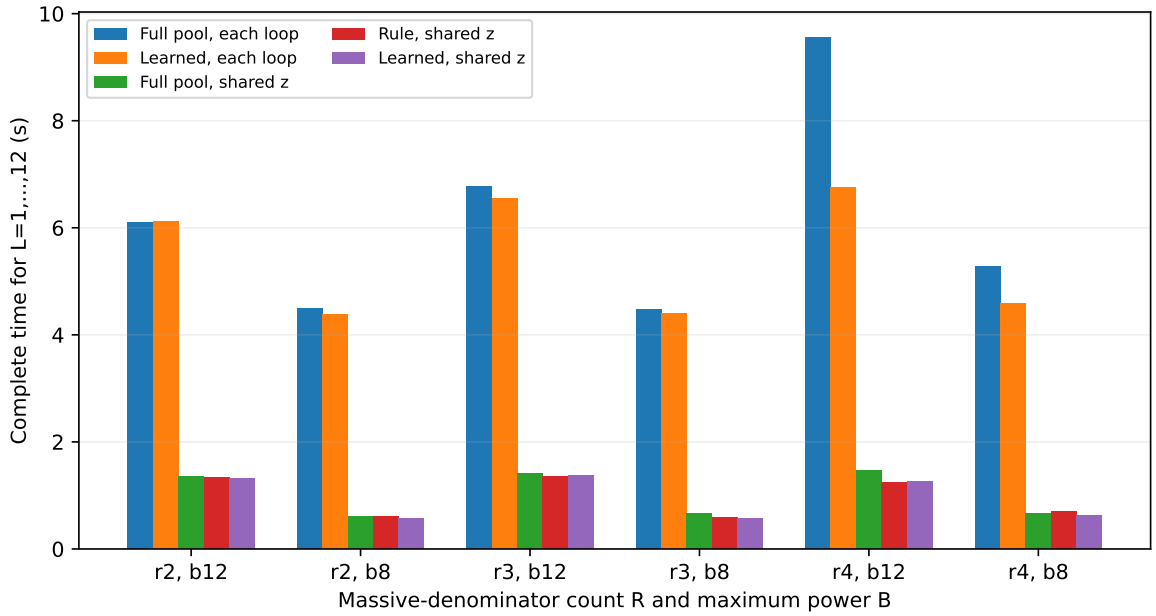


Figure 42: Median external complete times for the twelve loop orders in each reserved batch. Each bar includes the full cost of its stated workflow. The shared methods solve once in z and check the substituted coefficients separately at every requested loop order.

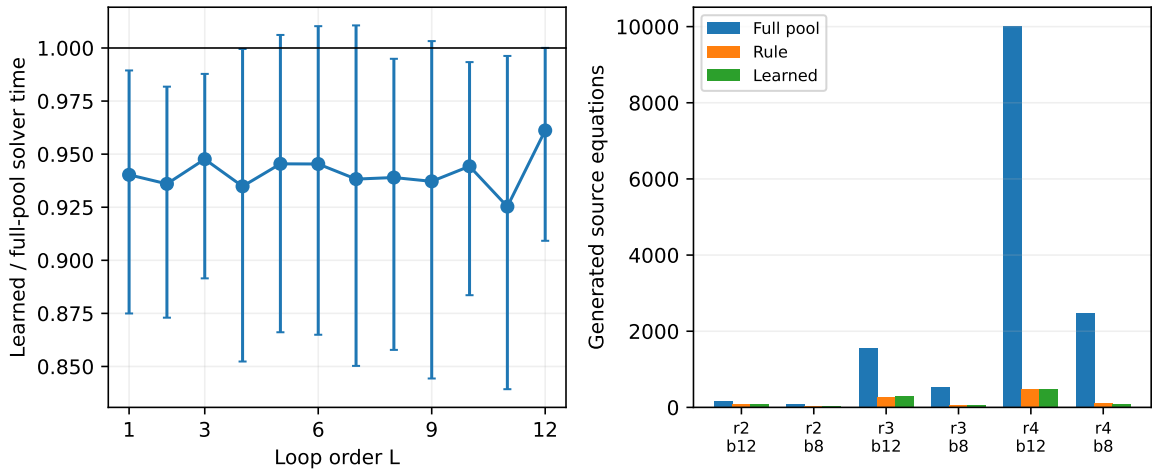


Figure 43: Separate-loop solver-time ratios and generated equation counts. The left panel includes construction, prediction, Kira and coefficient reading for each loop order; the complete workflow also includes the checks shown in Fig. 42. The intervals describe variation across the six reserved batches.

18.12 Learning equation choices for groups of targets

The shared variable in Eq. (115) removes repeated reductions at different loop orders. We now reduce repeated work within each target set and across independent mass profiles of the same restricted vacuum family. The calculation still uses one power of every massless denominator and $R = 2, 3, 4$ distinct massive denominators. Kira performs exact elimination on the supplied Euler and partial-fraction identities, and the retained native IBP-generation interfaces continue to serve their previously tested two- and three-loop domains.

For a target set $\mathcal{T}$, let π specify an ordering of the R massive denominators. At each mixed integral, we use the first two active denominators in this ordering in Eq. (117). Every resulting integral has a smaller total power B, and repeated integrals share the same subsequent reduction. Let $\mathcal{D}_\pi(\mathcal{T})$ be the resulting set of distinct nonmaster integrals. To train a model that favors small shared systems, we assign the cost

$$S_\pi(\mathcal{T}) = \sum_{\boldsymbol{b}\in\mathcal{D}_\pi(\mathcal{T})} \left(3\sum_{j=1}^{R} b_j - 2 \right). \tag{120}$$

The weight increases with the number of polynomial coefficients that can enter a substitution. It provides a training measure of algebraic work; actual Kira operation counts and the cost of large rational numbers also depend on its elimination and simplification choices. The smallest S_π among the $R!$ mass orderings supplies a restricted reference for the quality of a choice. More general reduction paths can have different costs.

We train ensembles of four and sixteen extremely randomized regression trees [39] to predict $\log S_\pi$, then select the ordering with the smallest predicted value. The 32 inputs describe the target count, number of massive denominators, total powers, the mean and mean squared power at each position of π, the largest power, the fraction of targets with that denominator present, and pairwise mean minima and products of powers. These inputs describe sharing across the entire target set. Each prediction estimates the reduction cost, while exact identities and Kira determine the reduction coefficients. Two massive denominators admit a single pair, so those requests bypass inference.

The training set contains 420 target batches and 6,300 candidate orderings, with total powers up to fourteen and at most 128 targets per batch. Both ensembles have maximum depth twelve and at least three training examples per leaf. We compare their complete calculation times with four simple ordering rules, the earlier learned pair selector, and enumeration of all fixed orderings. The first validation uses 324 calculations on twelve new mass profiles and selects the four-tree ensemble as the cheaper learned candidate. Its time ratio to the fastest simple rule is 1.000, so single-profile deployment retains that rule. A further 48 validation applications combine three profiles at a time and give a learned-to-simple time ratio of 0.925, which selects learned ordering for that route.

The selected trees require 7634 nodes in total. We also store their unchanged thresholds and leaf values in compiled inference routines to reduce loading and prediction cost. All 12,600 compared candidate scores agree exactly with the original tree evaluation. For the selected ensemble, compiled evaluation takes 0.921 times the Python decision time over the training batches. Fresh complete-calculation validation then chooses the inference implementation used for production calculations; this comparison includes model loading and inference and measures the time of the complete calculation.

18.12.1 Sharing exact checks and Kira calls

The original shared reduction still expanded and checked the same coefficient separately at each requested loop order. We instead compare the Kira coefficient $C_{\boldsymbol{bj}}(z)$ with an independent Taylor partial-fraction construction directly in $\mathbb{Q}[z]$. Exact equality in this polynomial ring implies equality after the substitution $z = L(d-2)$ for each requested L. Every returned coefficient therefore keeps an exact check while each symbolic comparison takes place once.

We also use integer polynomial arithmetic when reading the coefficients and applying the substitution. If $C(z) = D^{-1}\sum_{n=0}^{q} a_n z^n$, where the a_n are integers and D is a positive common denominator, then the coefficient of d^k after substitution is

$$[d^k]C(L(d-2)) = \frac{1}{D}\sum_{n=k}^{q} a_n L^n \binom{n}{k}(-2)^{n-k}. \tag{121}$$

Here q is the polynomial degree and $[d^k]$ denotes extraction of the coefficient of d^k. This form keeps the inner sums in integer arithmetic and reduces the number of repeated fraction normalizations. These savings follow from exact algebra, separately from the learned choice of equations.

Independent mass profiles can also share a Kira invocation. We give each profile its own integral identifiers and master basis, then form

$$A = \operatorname{diag}(A_1, \ldots, A_P), \qquad A \begin{pmatrix} \boldsymbol{I}_1 \\ \vdots \\ \boldsymbol{I}_P \end{pmatrix} = 0. \tag{122}$$

Here P is the number of profiles, A_p is the coefficient matrix of the selected identities for profile p, and $\boldsymbol{I}_p$ contains its integrals. Kira reduces this combined system in one call. The output checks require each target to contain only masters of its own profile, after which the same independent symbolic checks apply. The learned model chooses the equations for each profile before assembly. The current interface admits up to eight profiles, 512 targets and 12,000 selected equations per call. The timing tests use three profiles per call. A failed combined reduction retries the profiles separately, and a failed selected system uses the preserved full supplied system.

18.12.2 Independent timing and correctness results

The first reserved comparison uses twelve new mass profiles, powers up to fourteen, up to 96 requested targets per profile and every loop order from one through twelve. Five repetitions of five methods give 300 complete calculations. The learned route with shared checks has time ratio 0.394 to the preceding release, with descriptive interval $[0.314, 0.487]$. Its ratio to the updated simple rule is 0.986, with interval $[0.955, 1.019]$. These measurements separate the common arithmetic improvements from the additional equation-choice model. The mean weighted-cost ratio to the best enumerated mass ordering is 1.035. This reference concerns Eq. (120) within the finite set of mass orderings.

We then compare separate and combined reductions on four groups of three of these profiles. Eighty complete applications give a combined-to-separate learned time ratio of 0.516, with interval $[0.485, 0.563]$. Within combined calculations, the learned-to-simple ratio is 1.025, with interval $[0.941, 1.121]$. This comparison counts the model cost on both sides where the model is used.

A second reserved set contains twelve further mass profiles, powers up to thirteen and up to 80 requested targets per profile. Eighty applications compare the preceding separate workflow, the updated combined simple rule, and both learned inference implementations on the

same four groups. The compiled learned route has complete-time ratio 0.218 to the preceding workflow, with interval $[0.169, 0.276]$, and ratio 0.923 to the combined simple rule, with interval $[0.885, 0.962]$. Its ratio to Python inference in the otherwise identical learned workflow is 0.990, with interval $[0.885, 1.117]$. The two learned implementations produce identical selected equation systems. Complete-time validation retains the Python learned inference implementation for combined requests and the simple rule for a single profile. The compiled inference implementation remains available with its measured comparison.

All 460 reserved applications pass exact checks without fallback. Separate integration checks cover loop orders 1, 12, 16, 24 and 32 and include one deliberately failed check that successfully invokes the full system fallback. The independent audits check 54,594 saved coefficient occurrences in expanded loop-dependent expressions, and the final audit also compares 21,675 raw Kira coefficient occurrences with the earlier rational-expression reader. The peak sampled Kira process-tree memory is 21.27 MiB and peak native scratch use is 0.75 MiB in these tests.

Complete calculation time includes input reading, model loading, equation selection and construction, Kira execution, exact coefficient reading, loop substitution, verification and any fallback. Separate external-process measurements also include Python startup and evidence archiving. Reported ratios are geometric means of batch median ratios; the descriptive 95% intervals resample the twelve single batches or four combined groups. The small number of groups limits conclusions about other workloads. Figure 44 shows the single-profile comparison, and Fig. 45 shows the second combined comparison [40].

These results support acceleration within the stated vacuum family and fixed mass profiles. The Taylor construction also supplies a direct analytic solution for this family; these comparisons concern the reductions performed through Kira here. General scattering families, native generation of high-loop IBP systems and a globally fastest reduction remain outside the evidence.

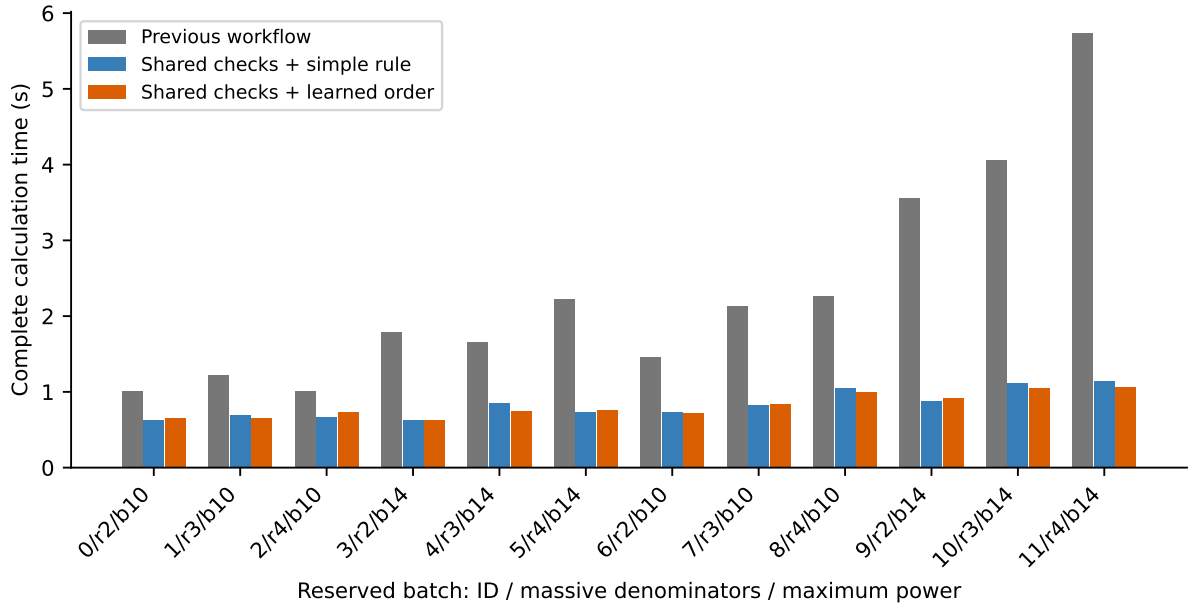


Figure 44: Median complete calculation time for the twelve reserved single-profile batches. All methods return the same exact reductions at loop orders one through twelve. The updated simple and learned routes share the improved exact checks and coefficient arithmetic.

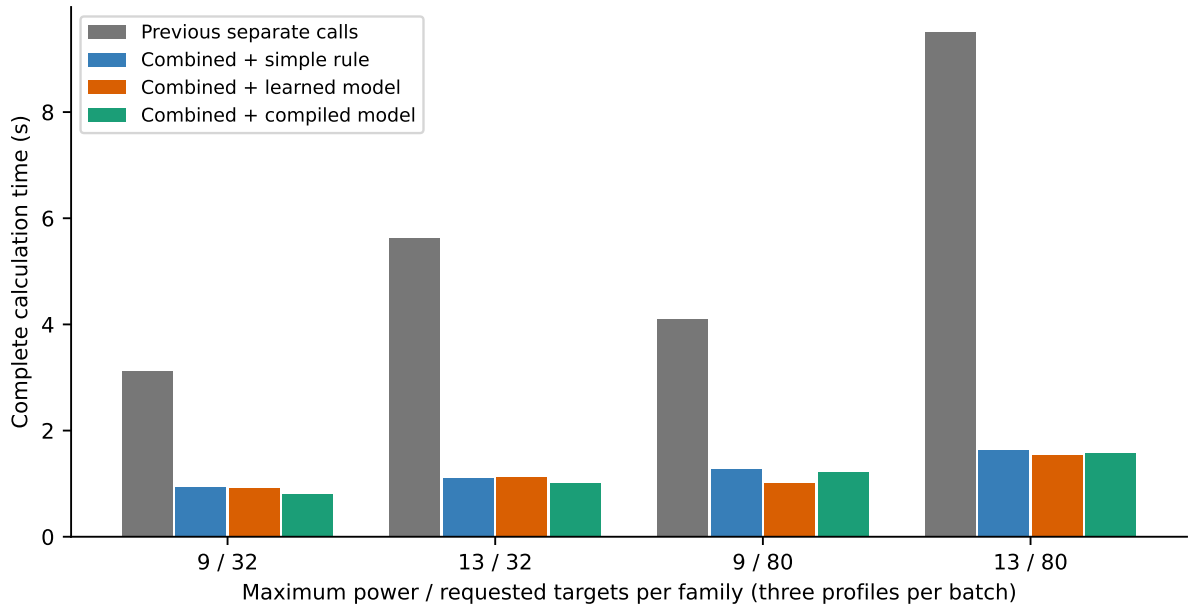


Figure 45: Median complete time for the second reserved comparison. Each group contains three independent mass profiles. The previous route makes separate Kira calls; the other routes combine the selected systems. The target count on the axis is the requested count per profile and is capped by the available index vectors.

18.13 Exact pre-reduction and a checked direct path

The combined workflow in Section 18.12 still spends much of its time in Kira after the model selects sufficient identities. For the restricted vacuum family, every selected identity lowers the total massive-denominator power. We can therefore evaluate the substitutions in increasing order of that power and return the completed result after an independent exact check. Kira remains available for comparison and as a fallback when this direct path fails its checks.

Let $\boldsymbol{C}_{\boldsymbol{b}}(z)$ be the vector of coefficients of the prescribed masters in $W_{\boldsymbol{b}}^{(L)}$, with $z = L(d-2)$ as defined earlier. For the master with index $\boldsymbol{e}_j$, this vector is the unit vector $\boldsymbol{C}_{\boldsymbol{e}_j} = \boldsymbol{e}_j$. The selected mass identity gives

$$\boldsymbol{C}_{\boldsymbol{b}}(z) = \frac{\boldsymbol{C}_{\boldsymbol{b}-\boldsymbol{e}_i}(z) - \boldsymbol{C}_{\boldsymbol{b}-\boldsymbol{e}_j}(z)}{\rho_i - \rho_j}, \qquad b_i, b_j > 0. \tag{123}$$

Here ρ_i and ρ_j are the distinct positive mass ratios specified for the family. When only denominator j is present, the Euler identity gives

$$\boldsymbol{C}_{k\boldsymbol{e}_j}(z) = \frac{2(k-1) - z}{2(k-1)\rho_j} \boldsymbol{C}_{(k-1)\boldsymbol{e}_j}(z), \qquad k \geq 2. \tag{124}$$

Both formulas reduce $B = \sum_j b_j$, so they terminate at the specified masters. We evaluate each distinct intermediate integral once.

The existing four-tree model selects the mass ordering before these substitutions, with unchanged weights and training examples. The learned direct route uses the previously checked compiled evaluator. We store each polynomial coefficient as integers over a positive common denominator and cancel their common divisor after each addition or scaling. This keeps the arithmetic exact and reduces repeated fraction normalization. The model selects the intermediate integrals that we visit, while Eqs. (123) and (124) determine their coefficients.

An independent Taylor partial-fraction construction checks the result. For an auxiliary dimensionless variable x, define

$$\prod_{i:b_i>0}(x+\rho_i)^{-b_i}=\sum_{j:b_j>0}\sum_{k=1}^{b_j}\frac{a_{jk}}{(x+\rho_j)^k},\qquad a_{jk}=[t^{\,b_j-k}]\prod_{\substack{i\neq j\\ b_i>0}}(\rho_i-\rho_j+t)^{-b_i}. \tag{125}$$

Here $[t^n]$ means the coefficient of t^n in the Taylor series. The corresponding master coefficient is

$$C_{\boldsymbol{b}j}(z)=\sum_{k=1}^{b_j}a_{jk}\prod_{h=1}^{k-1}\frac{2h-z}{2h\rho_j}. \tag{126}$$

An empty product equals one. This construction uses the same exact single-denominator relation and obtains the mixed-denominator coefficients independently of the selected pairs. We require equality of every polynomial coefficient before substituting $z=L(d-2)$. A failed check retries the original request through the preceding Kira route and its existing full-system fallback.

We compare five routes on identical targets. The preceding deployed route sends model-selected equations to Kira. A second route first evaluates the recurrence, then sends the completed target-to-master relations to Kira. Three direct routes return checked coefficients: one uses learned ordering, one uses simple extreme-power ordering, and one constructs the answer with Eq. (126) and checks it with the simple recurrence. The last route supplies an analytic control that includes an independent exact check.

Six new three-profile batches provide 90 validation calculations. Their mass ratios lie outside the ranges used in earlier training and tests. Validation selects the simple direct route before evaluating eight further three-profile batches with masses in a separate range. Each profile has two, three or four massive denominators, the largest total power is fourteen, and every request includes loop orders one through twelve. Five repetitions of five routes give 200 reserved calculations, all of which pass exact checks without fallback. The independent audit checks 30924 expanded master-coefficient positions and compares 25770 raw Kira coefficient expressions with the earlier rational reader. Five further applications include loop orders 1, 12, 16, 24 and 32 and a deliberately failed direct check that recovers through Kira.

The selected route has calculation-time ratio 0.306 to the preceding combined learned workflow, with descriptive interval $[0.246, 0.373]$. Its full worker-time ratio is 0.430, with interval $[0.414, 0.446]$. Calculation time includes request reading, model loading and inference where used, preparation, exact algebra, checks, loop substitution and fallback. Full worker time also includes interpreter startup and saving the output. We take the median of five repetitions per batch and then the geometric mean of the eight batch ratios. The intervals resample those eight batches and describe this finite comparison.

The learned direct route has calculation-time ratio 0.982 to the simple direct route, with interval $[0.971, 0.997]$, and ratio 0.991 to the analytic control, with interval $[0.973, 1.013]$. The comparison does not establish a further complete-time advantage from learned ordering over both direct controls. Pre-reduction followed by Kira has time ratio 0.927 to the preceding route. Figure 46 shows the batch times. The principal saving in the direct routes comes from finishing the exact triangular recurrence before a Kira invocation is needed. These measurements cover an outer reduction path for the stated vacuum family. Native IBP-generation policies retain their previously tested domains, and general high-loop families require separate comparisons. The implementation, fixed choices, raw outputs and checks accompany the measurements [41].

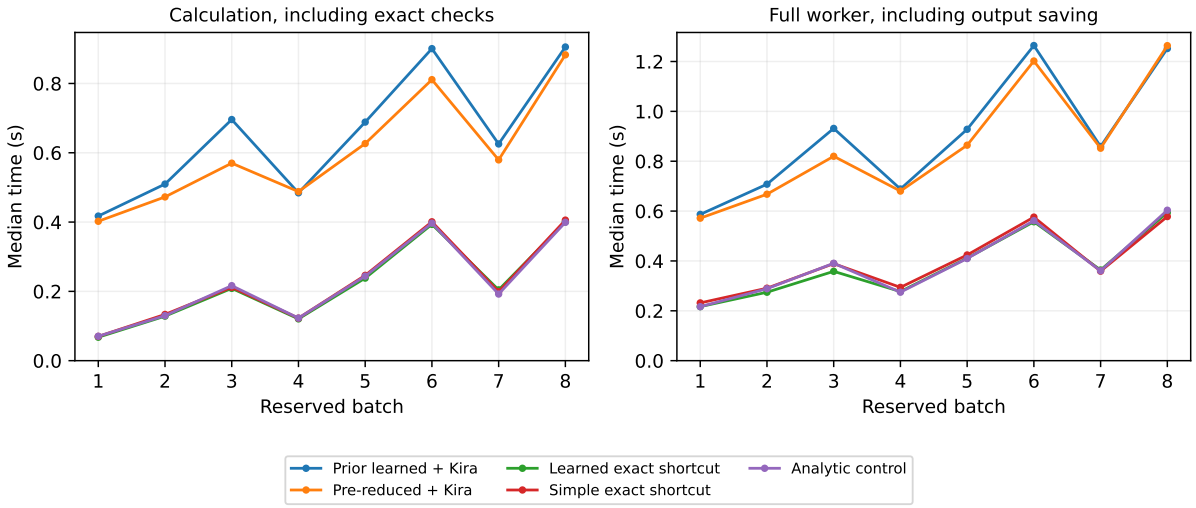


Figure 46: Reserved batch times for the preceding Kira route, pre-reduction followed by Kira, and three independently checked direct routes. Each point is the median of five complete calculations. The right panel includes startup and saving the output.

18.14 Sharing exact polynomial work across targets and loop orders

The direct reduction in Section 18.13 removes the Kira call when exact recurrence relations complete the requested reduction. Its remaining cost includes construction of the independent check and expansion of each coefficient at every requested loop order. We now reuse the algebraic terms that these operations have in common and keep the calculation process available for a bounded sequence of requests. The master basis, selected identities and exact checks remain the same.

Write a coefficient as $C(z) = D^{-1}\sum_{n=0}^{q} a_n z^n$, where D is a positive integer, the a_n are integers, and q is its degree. For each loop order L, define the integer transformation

$$T_{kn}^{(L)} = \begin{cases} \binom{n}{k} L^n (-2)^{n-k}, & n \geq k, \\ 0, & n < k. \end{cases} \tag{127}$$

$$C(L(d-2)) = \frac{1}{D}\sum_{k=0}^{q}\left(\sum_{n=k}^{q} T_{kn}^{(L)} a_n\right) d^k.$$

Here k is the power of d and n is the original power of z. We form this transformation once for each requested L and reuse it for all target coefficients. The present power limit gives $q \leq 13$. We also obtain the common denominator once per symbolic coefficient, then apply the integer transformations and cancel common factors. This removes repeated binomial coefficients, powers and denominator construction while retaining the full expanded result.

The independent check in Eq. (125) contains repeated Taylor terms. For a positive denominator power β, a nonnegative series index n, and a nonzero mass difference $\Delta = \rho_i - \rho_j$, these terms are

$$[t^n](\Delta + t)^{-\beta} = (-1)^n \binom{\beta + n - 1}{n} \Delta^{-(\beta+n)}. \tag{128}$$

We retain each required truncated factor within its mass profile and multiply the factors with integer polynomials over common denominators. We also form each single-denominator polynomial $\prod_{h=1}^{k-1}(2h-z)/(2h\rho_j)$ once for that profile and reuse it in Eq. (126). Every target still receives

its own independent master expansion and exact comparison with the selected recurrence. This reuse changes the arithmetic work needed to construct the check.

For a sequence of requests, one running process retains the transformation matrices and, when requested, the unchanged trained model. Each new mass profile receives fresh mass-dependent Taylor terms, so a repeated integral identifier cannot transfer coefficients between different physical inputs. The current interface bounds the retained transforms and the number of requests per process. A failed exact comparison retries the preserved Kira route. The previous ordering policy remains the default, and this extension introduces no new model training.

We compare the preceding arithmetic, shared transformations alone, shared Taylor terms alone, and both changes together. All routes use the same output writer, which stores each exact coefficient block once. Four new three-profile batches provide 48 validation calculations and select both arithmetic changes before the reserved test. Eight further batches provide 160 calculations: four variants with five repetitions each. The tests use two, three or four massive denominators, total massive-denominator power $B \leq 14$, up to 96 requested targets per profile, and every loop order from one through twelve. Validation and reserved mass ranges are disjoint from each other and from earlier training and comparisons.

The two changes together have calculation-time ratio 0.372 to the preceding arithmetic, with descriptive 95% interval $[0.343, 0.402]$. The ratio including process startup and output saving is 0.682, with interval $[0.592, 0.797]$. Shared transformations alone have calculation-time ratio 0.701, and shared Taylor terms alone have ratio 0.693. The combined route reduces expansion time to 0.414 of its previous value and construction time for the independent check to 0.158. These ratios compare batch medians and then take the geometric mean over eight batches. The descriptive intervals resample those eight batches. Figure 47 separates the component changes.

We then compare separate processes with one persistent process for streams of four, four and eight batches. The eight-batch stream contains the inputs from both four-batch streams. Three repetitions of four configurations give 36 stream runs and 192 additional batch calculations. All stream times include initial process startup, input reading, calculation, exact checks and output saving. With the updated arithmetic, process reuse has time ratio 0.524 to separate updated processes. Combining arithmetic and process reuse has ratio 0.365 to separate processes with the preceding arithmetic. The three specified streams overlap, so we report these descriptive ratios without a statistical interval. Figure 48 gives the individual stream times.

All 352 reserved batch calculations return identical exact coefficients across the applicable routes. The independent audit checks 28860 expanded coefficient positions with the older formula at each loop order. Additional interface checks cover changed masses and loop orders under the same integral identifier, continued operation after an invalid request, the request limit, retained model evaluation, and recovery through Kira after a deliberately failed check. These savings come from shared exact arithmetic and reduced process startup. They apply to the restricted vacuum family and its existing reduction interface; the earlier native IBP-generation scope remains unchanged. The frozen implementation, measurements and checks are available with the data [42].

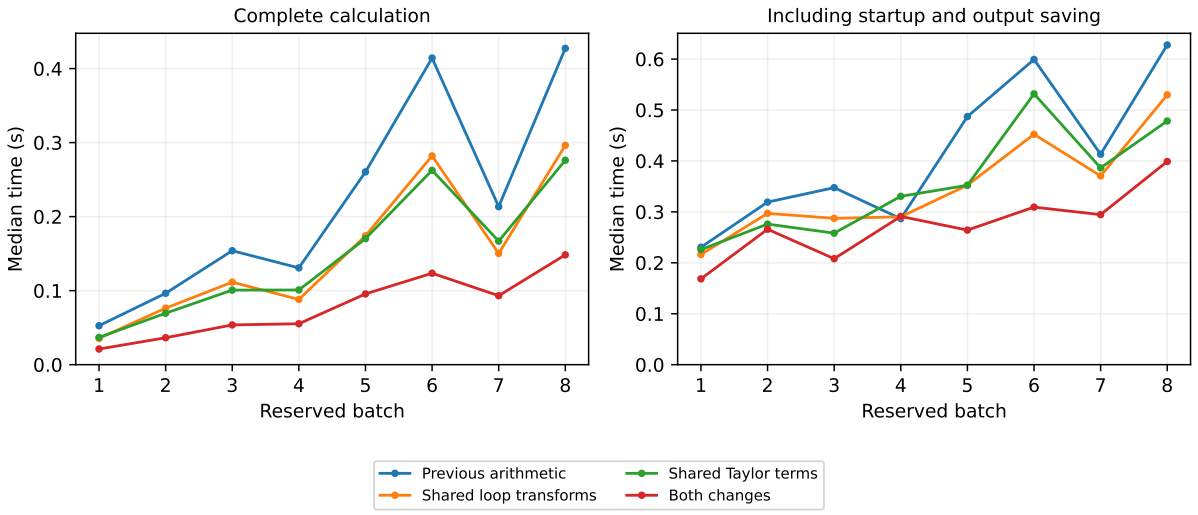


Figure 47: Reserved batch medians for the preceding arithmetic, each reuse change separately, and both changes together. Each point uses five complete calculations with identical target outputs and checks.

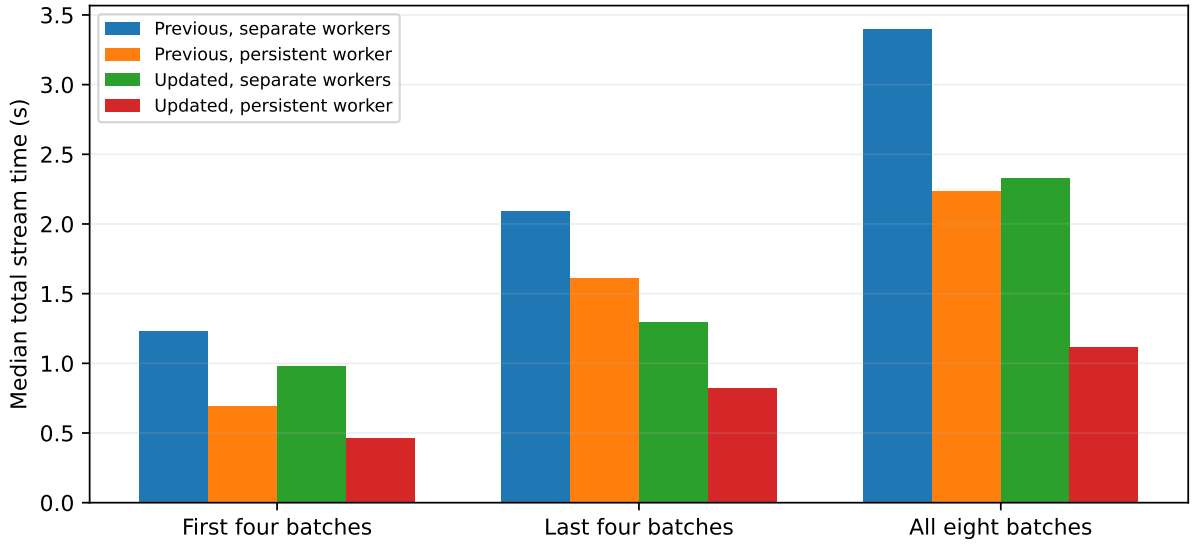


Figure 48: Total time for the specified request streams, including startup, all exact checks and saving the results. Each bar gives the median of three runs. The eight-batch stream uses all inputs from the two four-batch streams.

18.15 Exact integer arithmetic and learned batch strategies

Section 18.14 reuses the transformations for $z = L(d-2)$, but its implementation still converts coefficients between integer polynomials and separate rational numbers. It also writes each integer as decimal text. We now retain the common-denominator representation, compile the loop-order expansion, and test a compact exact output format. We then train a small model to choose a recurrence strategy for a whole target batch and compare its cost with fixed strategies.

For a polynomial $C(z) = D^{-1}\sum_{n=0}^{q} u_n z^n$, we store the integer vector $(u_0, \ldots, u_q)$ and positive denominator D, with $\gcd(D, u_0, \ldots, u_q) = 1$. Both the selected recurrence and the independent Taylor construction return this form, so their exact comparison needs no conversion to separate fractions. The compiled expansion uses Eq. (127) to obtain

$$v_k = \sum_{n=k}^{q} T_{kn}^{(L)} u_n, \qquad g_k = \gcd(|v_k|, D), \qquad [d^k]C(L(d-2)) = \frac{v_k/g_k}{D/g_k}. \tag{129}$$

Here v_k is the integer numerator before cancellation, g_k is its greatest common divisor with D, and $[d^k]$ extracts the coefficient of d^k. The implementation uses integer operations from the GNU Multiple Precision Arithmetic Library (GMP); the installed library version is 6.0.0 [43]. Every coefficient retains its exact reduced numerator and denominator.

The compact format records the sign, byte length and base-256 digits of each integer, together with the target, master and loop-order indices. It therefore preserves the full symbolic and expanded coefficients. A supplied reader recovers the same coefficient arrays as the decimal format. We measure the cost of reading these arrays as well as the cost of producing and saving them, so the comparison includes the conversion needed by a consumer of the result. The same master basis, independent Taylor check and Kira fallback apply throughout.

The learned strategy selects one of five rules for each batch. The first rule is the preceding extreme-power rule. Two further rules order denominator slots by their total powers across the targets, in increasing or decreasing order. Another rule selects the active mass pair with the largest absolute mass difference, and the last uses a fixed slot order. Each rule generates a terminating set of exact recurrence relations. The model can change which intermediate integrals occur and the sizes of the rational coefficients during substitution.

For batch $\mathcal{B}$ and rule s, let $T_{\mathcal{B}sr}$ be the measured time in repetition r of an already running worker. This time includes request reading, recurrence selection, exact arithmetic, the independent check and output writing. The training target and the rule selected by the model are

$$y_{\mathcal{B}s} = \log(\mathrm{median}_{r=1,2,3}\, T_{\mathcal{B}sr}), \qquad \widehat{s}(\mathcal{B}) = \arg\min_{s} f_s(x_{\mathcal{B}}), \tag{130}$$

where $x_{\mathcal{B}}$ contains 49 input features and f_s is the tree prediction of the log time for rule s. The features describe target powers, numbers of active denominators, loop orders, masses, mass differences, and shared integrals obtained by lowering one positive target power. We combine profile means and maxima into a fixed-length batch description. A multi-output regression tree with maximum depth four and at least six training batches per leaf learns these costs; the fitted tree has 19 nodes. The scores estimate runtime and carry no calibrated probability claim.

Training uses 96 batches and 1,440 timed calculations. The first twelve validation batches provide 324 calculations that compare the preceding arithmetic, retained integer polynomials, compiled expansion with decimal output, compiled expansion with binary output, and the learned rule. The validation also compares every fixed recurrence rule with the compiled binary route. Each batch contains three profiles with two, three and four massive denominators. Total massive powers reach fourteen, requested target counts reach 96 per profile, and every profile includes loop orders one through twelve. The training, validation and reserved test stages use distinct targets and mass profiles.

The first validation selects retained integer polynomials with decimal output. Its reserved comparison contains 504 batch calculations and passes every exact check. Its full-time ratio, including output reading, is 0.998 with descriptive interval $[0.867, 1.162]$, so this comparison does not establish a complete-time improvement. The binary variants reduce production time, but their Python reader adds enough cost to remove the gain in the full comparison.

We therefore compile the expanded-coefficient reader while retaining the same binary format, exact arithmetic and trained tree. This change follows the reader cost observed during validation. The original frozen comparison remains intact. The compiled reader passes 1,000 loop-output comparisons with the Python reader and rejects 250 truncated records. A new set of twelve validation batches supplies 324 further calculations, and a separate set of twelve test batches supplies the final reserved comparison. An incomplete reader comparison is excluded because its selection step referenced the earlier validation set. We preserve those 360 measurements, correct the stage references, and freeze the prescribed choice from the new validation set before generating results for fresh reserved targets.

The second validation selects compiled expansion and binary output with a fixed recurrence rule using total process time plus output reading. A learned route must improve on the best fixed route by at least two percent to replace it. We freeze the implementation, tree, deployment choice and test inputs before the reserved comparison. Twelve new batches then provide 300 separate-process calculations, with five repetitions per method. The implementation comparison uses the same request interface and output writer for matched routes.

The selected route has calculation-time ratio 0.522 to the preceding arithmetic. Its full process-time ratio is 0.850, with descriptive 95% interval $[0.785, 0.909]$, and the ratio including output reading is 0.858, with interval $[0.800, 0.914]$. Retaining integer polynomials alone gives a ratio of 0.981 when output reading is included. Compiled expansion with decimal output gives 0.933, and compiled expansion with binary output and the preceding rule gives 0.858. These comparisons separate arithmetic changes from the output format. The ratios use geometric means of within-batch median ratios; the descriptive intervals resample the twelve batches. Figure 49 shows their measured times.

The learned route has total-time ratio 0.873 to the preceding arithmetic and ratio 1.017 to the fixed rule selected by validation, both including output reading. The latter interval is $[0.969, 1.070]$. This interval includes one, so the reserved comparison does not establish a further time reduction from learned choices. The deployed default uses the fixed validation choice. The model and its measurements remain available with the fixed-rule controls, including any extra inference cost.

We also compare persistent workers on three disjoint streams of four reserved batches. Four recorded route labels and three repetitions give 36 stream runs containing 144 additional batch calculations. The selected route has stream-time ratio 0.702 to the preceding arithmetic, or 0.725 when reading every output is included. Figure 50 separates these two costs. The compiled fixed-rule and deployed labels use the same settings and provide separately timed repetitions. These stream ratios describe the three specified sequences. A separate comparison of identical binary blocks gives compiled-to-Python reader-time ratio 0.147, using five repetitions per reader and batch.

All 444 final reserved batch calculations pass exact comparisons, and the two comparisons together contain 948 such calculations. The independent audit checks 35472 expanded master-coefficient positions against the older loop-dependent formula and verifies equality between the two output formats. The compiled arithmetic also passes 3,277 earlier exact checks. Additional interface checks cover changed physical inputs under a repeated identifier, invalid requests, the worker request limit, truncated output, and recovery through Kira after a deliberately failed independent check. These results apply to the restricted coupled vacuum family. The earlier

native IBP-generation results retain their stated domains. The implementation, new tree, frozen inputs and measurements accompany the report [44].

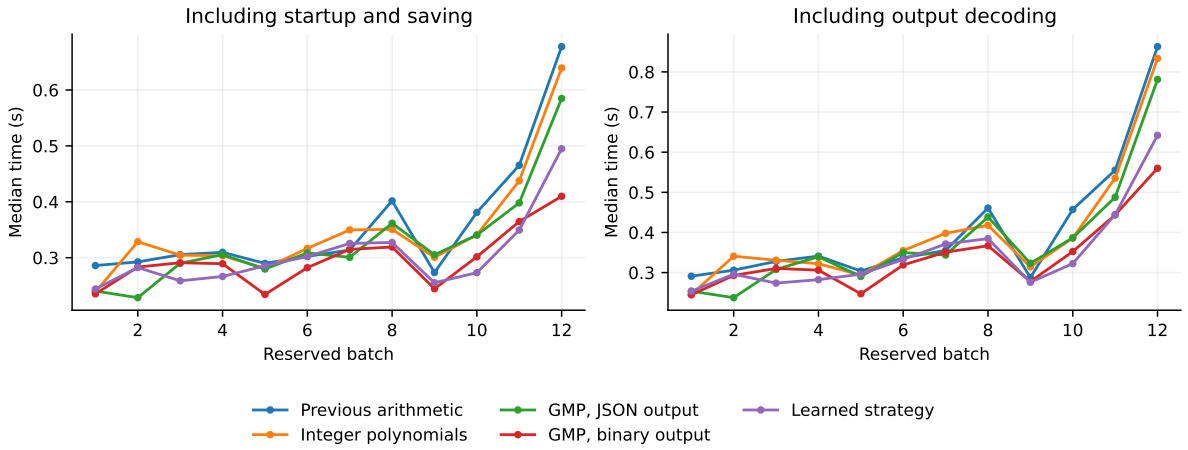


Figure 49: Reserved batch medians for the arithmetic and strategy variants. Each point uses five repetitions. The right panel includes reading and decoding the complete exact coefficient output.

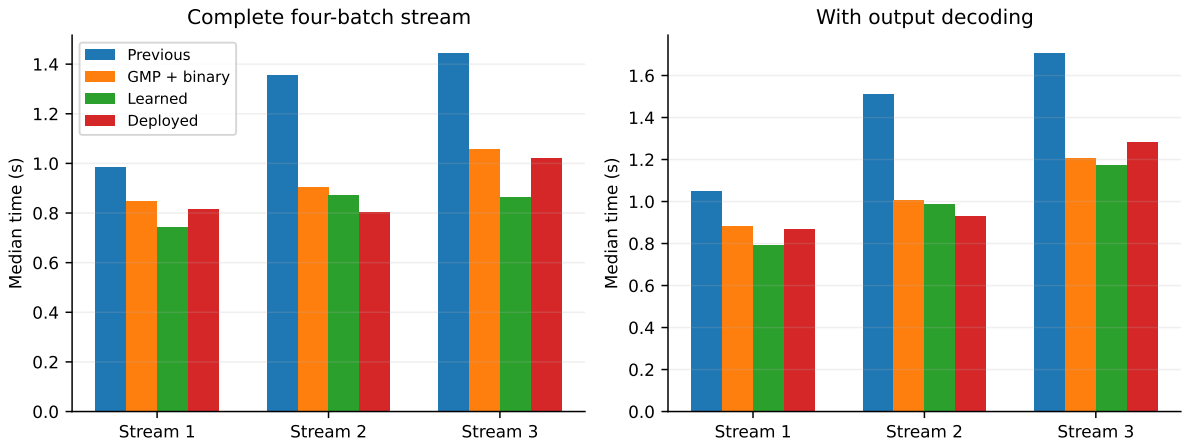


Figure 50: Complete times for three disjoint four-batch streams. Each bar is the median of three repetitions. The right panel adds reading and decoding every output. The deployed route follows the validation choice.

18.16 Direct measurement of the cumulative change

The preceding sections compare successive changes on different target sets. To measure the cumulative change directly, we now run the original baseline and final deployed calculation on identical integrals. We retain the coupled vacuum family of Eq. (52), with unit massless powers, two to four distinct massive denominators and fixed positive integer mass ratios. We set the common squared-mass scale to $\mu = 1$, as in the preceding comparisons. Each target has the same prescribed master basis in every comparison.

The first control uses the saved original Kira 3.1 executable, whose file hash agrees with the first frozen cost comparison. It generates the full Euler and mass partial-fraction equation pool and reduces each mass profile separately at every requested loop order. The Euler seeds extend through the largest target power, and the mass-pair identities extend through one additional power. A second control uses exactly the same requests, equation generation, reader and verification, with the later project executable used in Section 18.11. Its learned policies are disabled. Comparing these controls separates the executable change from the subsequent removal of repeated reductions.

The final route remains the validation-selected implementation from Section 18.15. It evaluates the selected recurrences directly and calls Kira only if the exact check fails. It uses the fixed extreme-power recurrence, shares coefficients in $z = L(d-2)$, checks them through the independent Taylor construction, expands them with compiled exact arithmetic and saves lossless binary output. It retains the existing Kira fallback. No model is retrained and no strategy is selected from the cumulative test results. These calculations use explicitly supplied identities; the test does not invoke native generation of a general multiloop IBP system.

We declare eight new groups of three mass profiles, one each with $R = 2, 3, 4$, and request every loop order $L = 1, \ldots, 12$. Two groups use each maximum total massive power $8, 10, 12, 14$. The requested target counts per profile are $16, 32, 48, 64$, respectively, capped by the available index vectors. The mass profiles are new, while index shapes can overlap earlier data within this restricted family. A separate upper-bound pilot checks executable compatibility and resource use. We then freeze the implementations, executable hashes, final deployment choice and all reserved requests.

Five repetitions of each method give 120 complete workflows. Both controls use the same scheduler-provided job-local scratch. Four allocated CPU slots process independent groups on one scheduled node. Each worker binds to a distinct available physical core, and every matched comparison remains bound to that core, with a shuffled sequential order within each three-method block. Each trial starts a fresh worker process and uses ordinary filesystem caching. Content-addressed storage can reuse an identical saved output on a repeated trial in any arm, while each arm recomputes its reduction. The primary time includes process startup, input loading, equation construction, exact algebra, verification, output saving, and reading and decoding the full output. It also includes the diagnostic evidence written inside each worker. Queue waiting, offline training and the later independent audit lie outside this time. We separately retain calculation times before final output saving and reading.

Let T_{abr} denote the primary time for method a, batch b and repetition r. We compare the medians within each batch and combine the eight ratios through

$$\widetilde{T}_{ab} = \mathrm{median}_{r=1,\ldots,5}\, T_{abr}, \qquad R_T = \exp\left[\frac{1}{8}\sum_{b=1}^{8} \log \frac{\widetilde{T}_{\mathrm{final},b}}{\widetilde{T}_{\mathrm{original},b}}\right], \qquad S = \frac{1}{R_T}. \tag{131}$$

Here R_T is the final-to-original time ratio and S is the cumulative speedup. The percentage of time saved is $100(1 - R_T)$. A descriptive 95% interval resamples the eight complete batches 10,000 times, keeping all repetitions and methods within each batch. This direct comparison

uses neither products of earlier ratios nor different input sets for the two methods.

The measured time ratio is 0.001839, with descriptive interval $[0.001653, 0.002049]$. It corresponds to 543.815 times faster execution, with interval $[488.134, 605.064]$, and 99.82% less time. The ratio to the later full-pool control is 0.012, corresponding to 81.170 times faster execution. The calculation subtotal excludes outer worker startup and final output saving and reading, while retaining every Kira startup within the baseline reduction. Its final-to-original ratio is 0.000294, or 3402.846 times faster. Summing one primary-time median per batch gives 1027.781 seconds for the original control and 1.922 seconds for the final route, a workload speedup of 534.881. This sum weights large batches by their elapsed time, whereas Eq. (131) gives equal weight to each batch. Figure 51 shows the absolute times and individual speedups.

Within each batch, every method and repetition returns the same exact expanded coefficients. All 120 reserved workflows complete with no failure or fallback. Each baseline arm makes 1,440 Kira calls; the final arm makes none on these successful requests. The independent audit checks 27960 nonzero target–master coefficient polynomials after substituting each loop order, using the earlier loop-dependent Taylor formula. It also regenerates all 288 distinct baseline systems, checks their input hashes and uses SymPy to read 1464 coefficient positions directly from saved Kira exports. The two baseline executables receive identical systems at each mass profile and loop order. The maximum sampled worker process-tree memory is 78.488 MiB and temporary solver storage is 1.238 MiB.

The cumulative saving includes shared symbolic reductions, direct checked recurrences, compiled expansion and reduced output costs. The fixed-rule deployment makes this a measurement of the complete mathematical and implementation change. It supplies no separate ML-only speedup estimate. General scattering families and native generation of broader high-loop IBP systems remain outside this comparison. The frozen inputs, implementations, individual timings, raw exports and audits accompany the report [45].

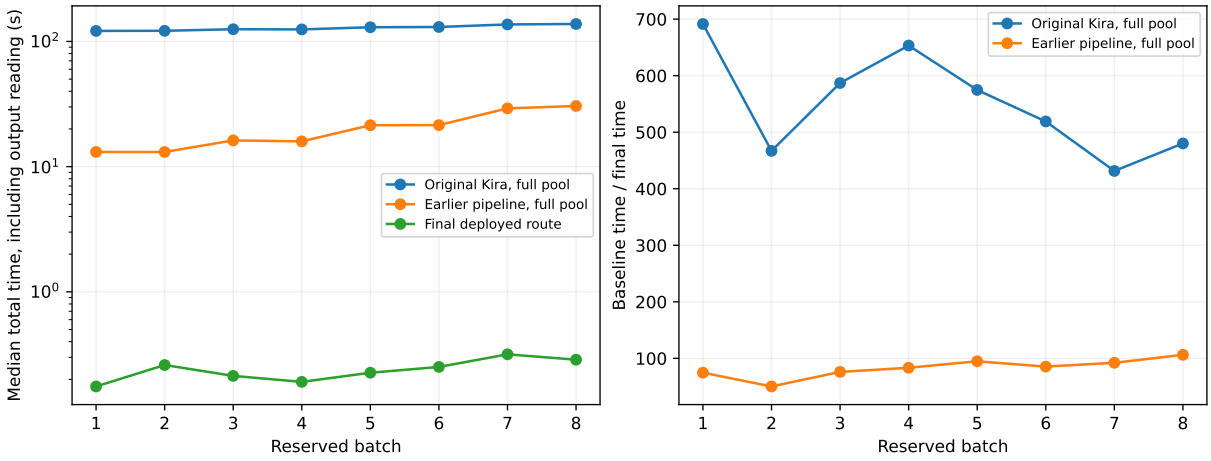


Figure 51: Direct cumulative comparison on identical reserved vacuum batches. The left panel shows median total times, including output reading, on a logarithmic scale. The right panel divides each baseline median by the final-route median. Each point uses five repetitions, three mass profiles and loop orders one through twelve.

19 Scattering families, symbolic kinematics and certified reduction paths

The preceding comparisons establish costs for specified native families and for a restricted vacuum construction. To extend the physical scope, we now reduce new four-point and five-point families and retain independent kinematic invariants as symbols. We also construct small IBP systems for which every allowed elimination path can be examined. These two extensions test different questions: whether a frozen learned filter transfers to new physical inputs, and whether a recorded reduction cost can be proved minimal within an explicit set of operations.

19.1 New physical families and independent invariants

We continue to use $I_{\boldsymbol{a}}$ for an indexed integral and $D_i = q_i^2 + \mathrm{i}0$ for each massless denominator. The new two-loop family is the massless limit of the nonplanar four-point family `topo4` in Ref. [2]. Its ordered denominator momenta are

$$\begin{aligned}\boldsymbol{q}_{\mathrm{NP}} = (&\ell_1, \ell_2, p_1 - \ell_1, p_2 - \ell_2, p_1 + p_2 - p_3 - \ell_1,\\ &p_1 - \ell_1 + \ell_2, p_1 - p_3 - \ell_1 + \ell_2, \ell_1 - p_3, \ell_2 - p_1 - p_2).\end{aligned} \tag{132}$$

The first seven entries are physical propagators, and the last two complete the scalar-product basis and can appear as numerators. The momenta satisfy $p_1^2 = p_2^2 = p_3^2 = 0$ and $q = p_1 + p_2 - p_3$ with $q^2 = 0$. We define

$$s = (p_1 + p_2)^2, \qquad t = (p_1 - p_3)^2, \qquad 2p_1 \cdot p_2 = s, \quad 2p_1 \cdot p_3 = -t, \quad 2p_2 \cdot p_3 = s + t. \tag{133}$$

The symbolic reduction therefore takes place in $\mathbb{Q}(d, s, t)$.

The new one-loop pentagon has five massless external momenta satisfying $\sum_{i=1}^{5} p_i = 0$. Its denominator momenta are

$$\boldsymbol{q}_{\mathrm{P}} = (\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_1 - p_2 - p_3, \ell - p_1 - p_2 - p_3 - p_4). \tag{134}$$

All five entries are physical propagators. We retain the five adjacent invariants

$$s_{12} = (p_1 + p_2)^2, \quad s_{23} = (p_2 + p_3)^2, \quad s_{34} = (p_3 + p_4)^2, \quad s_{45} = (p_4 + p_5)^2, \quad s_{51} = (p_5 + p_1)^2.$$

Momentum conservation fixes the other scalar products. For example, $2p_1 \cdot p_3 = s_{45} - s_{12} - s_{23}$ and $2p_1 \cdot p_4 = s_{23} - s_{45} - s_{51}$. The coefficient field is $\mathbb{Q}(d, s_{12}, s_{23}, s_{34}, s_{45}, s_{51})$.

A one-loop box supplies an additional test with an external virtuality. Its denominator momenta are $(\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_3)$. Here p_1, p_2, p_3 are massless, while $v = q^2 = (p_1 + p_2 - p_3)^2$ is an independent symbol. The definitions of s and t remain the same, and $2p_2 \cdot p_3 = s + t - v$. Thus its coefficients belong to $\mathbb{Q}(d, s, t, v)$. This family has massless internal lines and one external line with nonzero virtuality.

The denominator coefficient matrices have ranks nine, five and four for the nonplanar box, pentagon and one-loop box, respectively. Each rank equals $L(L + 1)/2 + LE$, where E is the number of independent external momenta. The external Gram determinants are nonzero as rational functions and at the specified numerical points. The calculations use generic kinematics away from singular denominators. They retain every listed independent invariant, with no replacement of a scale by one. Internal masses remain zero in these scattering tests.

We first test two rational profiles of the nonplanar box and two of the pentagon, keeping d symbolic. The box profiles are $(s, t) = (-149, -147)$ and $(-167, -157)$ in a common squared-momentum unit. The pentagon profiles are $(-51, -53, -57, -59, -63)$ and

$(-71, -73, -77, -79, -83)$ in the displayed invariant order. We then test one fully symbolic instance of each of the three families. Targets start with the full-sector corner and each single additional propagator power, followed by auxiliary numerators and singly pinched sectors in a fixed index order. The four rational-profile batches contain sixteen targets each. The symbolic nonplanar box, pentagon and one-loop box contain ten, six and eight targets. The seven batches therefore contain 88 targets. Their definitions and target lists precede the timed comparisons.

19.2 Frozen model transfer and exact comparison

The experiment retains the preceding depth-twelve equation-selection tree, its weights and its retention threshold 0.01. The tree sees seed, operator and target features before Kira constructs an identity. These scattering topologies were absent from its training collection. No new fit or threshold selection uses the new results. Small feasibility pilots establish that the symbolic inputs run within the resource limits. Some symbolic pilot targets also occur in the final batches; the pilots do not select a learned policy.

Both methods use the same instrumented Kira 3.1 executable and the existing project runtime changes. The native control disables learned equation selection and retains native seed and row choices.

Kira first discovers a reference basis with seed limits enlarged by one relative to the target's maximum denominator sum, numerator rank and number of additional propagator powers. The native control and experimental filter then start with the target limits and the same prescribed basis. An incomplete filtered attempt retries native equations, and an incomplete native attempt uses the enlarged region. The complete workflow includes each failed attempt, model evaluation, IBP generation, exact algebra, coefficient comparison and cleanup. Reference discovery occurs outside this timer. The comparison driver is already running.

Five alternating repetitions of the two methods give seventy complete workflows. For each batch, we compare the median complete times. We also compare the coefficients as rational functions, repeat native reduction with seed limits enlarged by two, and substitute fresh rational kinematics into each symbolic result before comparing it with a separately generated native reduction. These specialization checks support the symbolic comparisons; they do not replace them. Kira's exact symbolic elimination determines the coefficients [9].

All seventy workflows reproduce the exact reference reductions. The independent audit checks 5,600 coefficient pairs, including 560 distinct pairs. All seven enlarged-seed checks and all three independent kinematic specializations agree. The frozen filter requires native fallback in all thirty-five learned workflows. Its case-weighted geometric mean complete-time ratio to native choices is 1.252, corresponding to 25.2% more time on these batches. Summing one median per batch gives 18.867 seconds for the native control and 22.256 seconds for the experimental route. This sum weights the work differently from the geometric mean.

The symbolic nonplanar box has median times 5.438 and 6.321 seconds for native and experimental reductions, respectively. The symbolic pentagon has median times 1.032 and 1.390 seconds for native and experimental reductions, respectively. The box with symbolic external virtuality has median times 0.723 and 0.958 seconds for native and experimental reductions, respectively. The largest sampled memory of Kira and its child processes is 61.14 MiB, and the largest sampled solver scratch space is 1.16 MiB. These measurements include the unsuccessful filtered attempts. A separate application receives only the family, targets and prescribed masters. All 10 application checks agree with the reference coefficients after execution, including the explicitly requested experimental-filter cases. These checks do not supply target answers to the reducer.

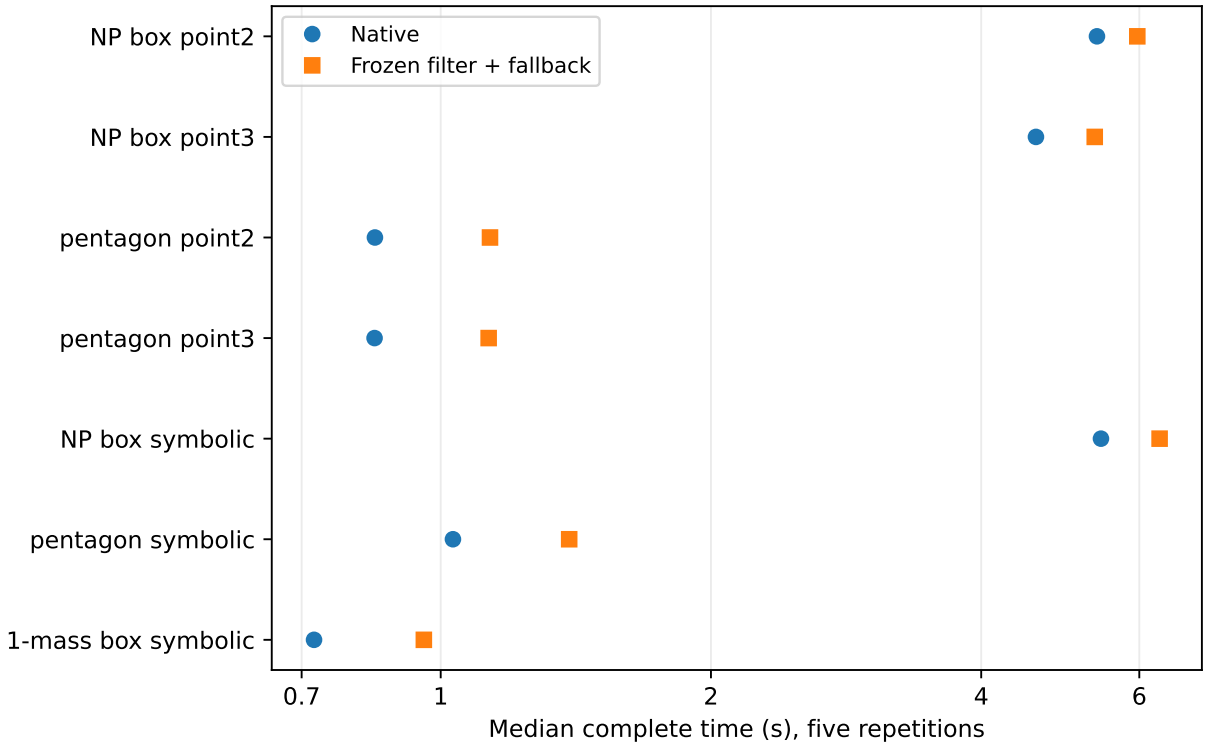


Figure 52: Complete native and experimental learned-filter times on the new scattering batches. Each point is a median of five repetitions. The experimental time includes native fallback. Symbolic batches retain all invariants specified in the text.

The new application accepts symbolic family definitions and a supplied master basis without reference target coefficients. It defaults to native choices for these domains. The transfer measurements do not change the earlier speedup estimates or activate a new learned route. They provide explicit examples for later training on broader scattering inputs [46].

19.3 Physical IBP systems with a finite optimum

A reverse construction supplies a valid path, while an optimality claim also requires a defined cost and a complete comparison with the allowed alternatives. We carry out that comparison on two small systems with symbolic coefficients. For the massless one-loop bubble, define

$$B_{a,b} = \int \frac{d^d\ell}{\mathrm{i}\pi^{d/2}} \frac{1}{(\ell^2+\mathrm{i}0)^a[(\ell-p)^2+\mathrm{i}0]^b}, \qquad p^2 = s. \tag{135}$$

Differentiation with the vectors ℓ and $\ell - p$ gives

$$0 = (d-2a-b)B_{a,b} - bB_{a-1,b+1} + bsB_{a,b+1}, \tag{136}$$
$$0 = (d-a-2b)B_{a,b} - aB_{a+1,b-1} + asB_{a+1,b}. \tag{137}$$

The massless one-denominator integrals vanish in dimensional regularization. Selecting both operators at seed $(1,1)$, the first at $(2,1)$ and the second at $(1,2)$ gives the closed system

$$\begin{pmatrix} 0 & s & 0 & d-3 \\ s & 0 & 0 & d-3 \\ d-5 & -1 & s & 0 \\ -1 & d-5 & s & 0 \end{pmatrix} \begin{pmatrix} B_{2,1} \\ B_{1,2} \\ B_{2,2} \\ B_{1,1} \end{pmatrix} = 0. \tag{138}$$

We protect $B_{1,1}$ as the master and reduce the other three integrals over $\mathbb{Q}(d,s)$. Their exact coefficients are

$$B_{2,1} = B_{1,2} = \frac{3-d}{s} B_{1,1}, \qquad B_{2,2} = \frac{(d-6)(d-3)}{s^2} B_{1,1}. \tag{139}$$

An independent check obtains the same ratios from the gamma-function expression for the bubble.

For a one-loop massive tadpole, define

$$T_a = \int \frac{d^d \ell}{\mathrm{i}\pi^{d/2}} \, (\ell^2 - m^2 + \mathrm{i}0)^{-a}.$$

The IBP identity with vector ℓ is

$$(d-2a)T_a - 2am^2 T_{a+1} = 0. \tag{140}$$

Seeds $a = 1, 2, 3$ give three equations for T_2, T_3, T_4 with master T_1 over $\mathbb{Q}(d, m^2)$. Native Kira reductions generated from the physical denominators provide an additional check of both systems.

At a matrix state S, an allowed action chooses any nonzero entry in an unprocessed row and an unprocessed nonmaster column. It moves that row to the next pivot position, normalizes it and eliminates the chosen column from every other row. The master column remains protected, and each action removes one nonmaster column from future pivot choices. There are three pivot stages in each system.

We count one division for each nonzero nonpivot source entry when the pivot differs from one. Each update of a nonpivot source entry counts one multiplication and one subtraction. Assignments, comparisons, structural cancellation of the pivot, and polynomial operations inside a rational-function operation are outside this cost. Thus the objective measures specified coefficient operations, with no claim that each operation takes the same time.

For this action set, define the remaining minimum cost by

$$V(S) = \min_{a \in \mathcal{A}(S)} \big[c(S,a) + V(S_a) \big], \qquad V(S_{\mathrm{done}}) = 0. \tag{141}$$

Here $\mathcal{A}(S)$ is the complete set of allowed pivots, $c(S,a)$ is the counted cost, and S_a is the exact next matrix. We evaluate every alternative and retain the state graph, using the finite recurrence of dynamic programming [47]. An independent checker reconstructs every row operation, checks all outgoing actions, verifies the cost recurrence, and checks the final master coefficients. Since each action reduces the number of remaining columns, induction from the terminal states proves the minimum for the full stated path space.

To include initial equation selection, we also examine every subset $\mathcal{E}$ of the saved source pool whose nonmaster matrix has rank three. Each subset starts its own complete pivot search, giving

$$C^*_{\mathrm{pool}} = \min_{\substack{\mathcal{E} \subseteq \mathcal{E}_{\mathrm{full}} \\ \operatorname{rank} A_{\mathcal{E}} = 3}} V(S_{\mathcal{E}}). \tag{142}$$

Here $A_{\mathcal{E}}$ contains the nonmaster coefficients, and $S_{\mathcal{E}}$ is the starting matrix for that subset. Initial row selection carries no coefficient-operation cost in this objective. The certificate records every sufficient subset and every subsequent pivot choice.

The massless bubble certificate contains 113 distinct states and checks 234 transitions. It covers 152 complete paths across 5 sufficient source subsets. The minimum is 7 coefficient operations when initial equation selection is allowed, and 13 when the full source pool is retained.

The massive tadpole certificate contains 24 distinct states and checks 32 transitions. It covers 15 complete paths across one sufficient source subset. The minimum is 7 coefficient operations when initial equation selection is allowed, and 7 when the full source pool is retained.

The fixed control chooses the first available nonmaster column and then the first available source row. It attains the full-pool minima of thirteen and seven operations. A control that minimizes only the immediate step cost uses thirteen operations for the bubble and nine for the tadpole. Thus immediate cost can give a larger complete cost even in a three-column physical system. The minimum sufficient equation count is three for each pool, with two distinct bubble seeds and three tadpole seeds. All exact state transitions and terminal coefficients pass independent checks, and six native Kira master coefficients agree with the two symbolic systems.

The subset checks also certify the smallest sufficient equation count and the smallest number of distinct source-seed labels within each saved pool. These records give exact cost labels for later model tests. The certificates apply to the displayed systems, protected masters, allowed operations and cost definition. They do not establish an optimum over additional IBP identities, changes of master basis, other algebraic algorithms or unrestricted scattering reductions. General high-loop scattering and wall-time optimality therefore remain separate questions.

20 Learning from complete reduction time

The scattering test shows why a useful equation label is an incomplete training objective for acceleration. A filter can retain sufficient equations and still add more overhead than it saves. We therefore train a model to predict complete reduction time and let it choose the initial seed region, an optional seed or equation filter, and an integral ordering. The model makes one choice for the whole target batch before Kira begins its exact calculation.

To evaluate that choice fairly, we also need a baseline with the same surrounding implementation. We combine the earlier equation filter with an existing improvement in temporary data handling, which reduces execution cost without changing the equations or arithmetic. The *integrated native control* uses this implementation with Kira's own decisions. Comparing the learned strategy with that control measures the benefit of the prediction beyond the shared implementation changes.

For a target batch, let $B = (r_{\max}, s_{\max}, q_{\max})$ contain the largest positive index sum, numerator rank, and dot count. A dot is one denominator power above one. Table 1 lists the strategies considered, and Appendix B defines the quantities and derives the cost models. The seed limits and ordering options are provided by Kira [8]; ordering five changes the priority of sectors while retaining the within-sector preference of ordering one. All strategies must return the same prescribed master basis.

Table 1: Strategies available to the complete-time model. Negative seed bounds are replaced by zero. Native row choices remain active.

Strategy	Initial seed region and selection
Integrated native	Bounds B; native choices
Smaller region	Bounds $B - (1,1,1)$; native choices
Fewer dots	Bounds $B - (1,0,1)$; native choices
Equation filter	Bounds B; saved equation tree
Seed filter	Bounds B; saved seed tree where available
Different ordering	Bounds B; Kira ordering five
Larger region	Bounds $B + (1,1,1)$; native choices

A smaller region can avoid unnecessary equation generation, while a larger region can avoid a failed first attempt when the target's own index limits are insufficient. If the initial strategy fails the basis check, the calculation tries native choices at B and then at $B + (1,1,1)$ when needed. The native control also enlarges an insufficient region, and every failed attempt contributes to the training and test time. For the four earlier multiloop graphs, the seed-filter option uses native choices because this implementation has no separate seed tree for them. Its equation-filter option uses the earlier trees for those graphs and the newly trained trees for the three additional scattering structures.

The model receives eighteen features describing the graph, symbolic variables, number of targets, index powers, and sectors. For each strategy, its training label is the logarithm of the complete-time ratio to integrated native choices. We compare a constant predictor, six regression trees, three ridge-regression models, and two nearest-neighbour models. Regression trees follow the squared-error construction of Ref. [30]; Appendix B specifies all candidates. Validation selects ridge regression with penalty $\lambda = 10$ and decision margin $\gamma = 0$, giving validation ratio 0.993.

The best fixed strategy selected from training is integrated native reduction. Consequently, the integrated-native and best-fixed controls use the same settings in independent trials, and differences between their times indicate execution variation. The seed and equation trees retain their earlier weights throughout this study, and learned row selection remains disabled.

The data contain 31 training batches, 17 validation batches, and 31 test batches from the

sunrise, planar box, three-loop vacuum, three-loop ladder, nonplanar box, pentagon, and box with external virtuality. Rational batches use different masses or invariants across the splits, while additional symbolic batches use different target lists. These are new inputs within seven known graph structures at one through three loops. The recurrence study through twelve loops remains a separate result for its restricted vacuum family.

Each training and validation cost uses three repetitions. Testing uses five repetitions to compare the preceding native implementation, integrated native choices, the best fixed strategy, and the learned strategy. The controller and models are loaded before timing, but every attempt starts a fresh Kira process. Timers include input preparation, prediction, native IBP generation, algebra, exact checks, cleanup, and retries. They exclude training, enlarged reference reductions, and later recording of results. Additional application checks measure cold startup and compare returned coefficients after a reduction supplied only with its targets and basis. For an unknown graph or target complexity outside the training range, the learned application uses native choices; the same guard applies to combined requests.

20.1 Results for individual target batches

All 620 timed calculations pass exact checks of 59,640 coefficient entries, and all 31 applications supplied without reference coefficients pass their subsequent comparisons. Each timed method makes fifteen fallback attempts. Thus this model changes some initial strategies but does not remove the remaining cases that need a second attempt.

Relative to the preceding native implementation, the learned strategy has time ratio 0.9903, with descriptive interval $[0.9796, 0.9996]$, or about 1.0% less time. Its ratio to integrated native choices is 0.9965 with interval $[0.9875, 1.0070]$, and its ratio to the best-fixed control is 1.0026 with interval $[0.9979, 1.0081]$. Both comparisons with the matched integrated controls include equal time. They therefore provide no reliable additional saving from the learned choices. The two identically configured controls have ratios 0.9938 and 0.9878 to the preceding native implementation, illustrating the scale of execution variation.

Table 2 reports the results by graph. The aggregate averages logarithmic ratios within each graph and then gives all seven graphs equal weight, as derived in Appendix B. The smallest learned-to-native ratio is 0.9654 for the three-loop vacuum, while the ladder and nonplanar box take slightly more time. These results motivate testing more specific strategies for each graph rather than enabling the common model for all inputs.

Table 2: Complete-cost results. Native seconds are geometric means of batch median times. The other entries are geometric mean time ratios; a value below one means less time.

Graph	Native (s)	Integrated/native	Learned/native	Learned/integrated
Sunrise	0.502	1.0123	0.9932	0.9811
Planar box	1.621	0.9906	0.9771	0.9864
Three-loop vacuum	0.726	0.9745	0.9654	0.9907
Three-loop ladder	3.535	1.0146	1.0013	0.9869
Nonplanar box	3.119	1.0075	1.0073	0.9998
Pentagon	0.599	0.9758	0.9942	1.0189
Box with virtuality	0.484	0.9824	0.9944	1.0122

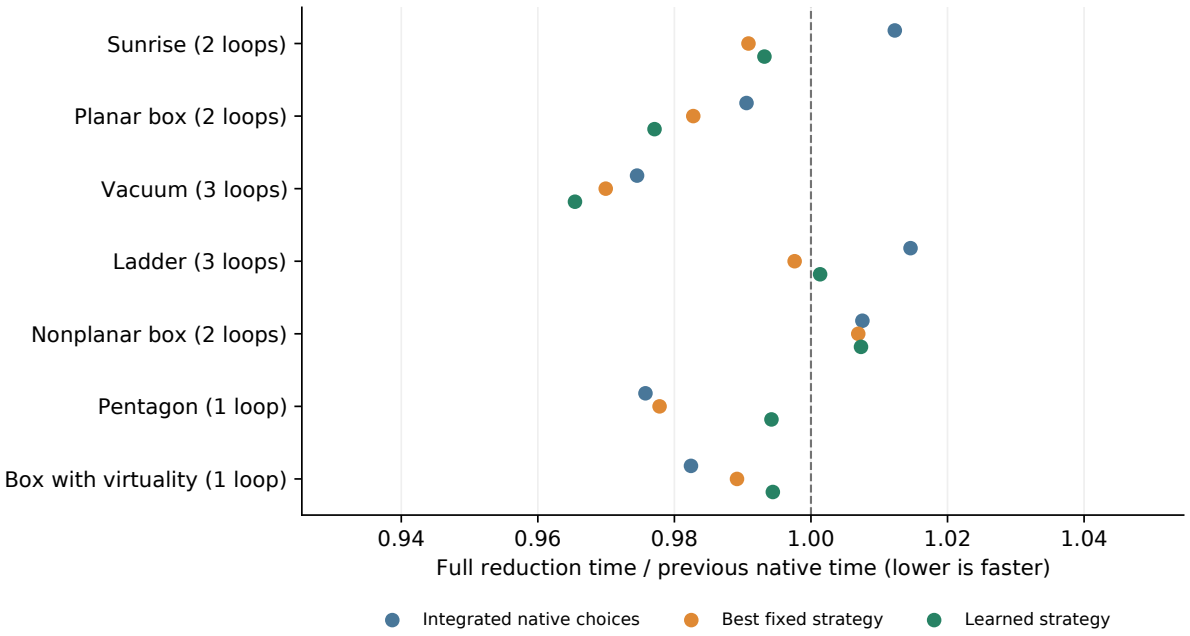


Figure 53: Learned choices and the native controls have similar complete times. The best-fixed and integrated controls use the same configuration in separate trials. The plot shows variation across graph structures; the text gives the aggregate uncertainty intervals. The axis focuses on the measured range and retains the equal-time line.

20.2 Sharing a reduction between compatible requests

Before changing the predictor, we test whether repeated work can be removed from the way Kira receives targets. When two available requests have the same family and kinematics, their target lists can be combined and reduced together. The resulting coefficient rows are then returned to the appropriate request. This shares family preparation and algebra between the lists. Appendix B defines the ten request groups and derives this separation from linearity; the model and its settings remain fixed.

Five repetitions of four methods give 200 complete calculations, all of which pass exact checks. Joint native reduction has ratio 0.5252 to two separate native reductions. With the integrated implementation and native choices, the ratio is 0.5175 with interval $[0.5073, 0.5346]$, giving 48.3% less time or speedup 1.93 with interval $[1.87, 1.97]$. Every tested graph group benefits from the shared reduction. Its integrated-native ratios range from 0.5040 to 0.5679, corresponding to savings of 43.2–49.6%.

The joint learned strategy has ratio 0.5152 to separate native requests, but its ratio to joint integrated native choices is 0.9955 with interval $[0.9810, 1.0058]$. As Table 3 and Figure 54 show, the main gain comes from sharing the calculation; the predictor adds no reliable improvement over the matched joint control. The standard application therefore uses the fixed strategy selected from training, which retains native choices. Ten checks of each joint application and ten checks of the single-batch application supply only a basis and verify the returned coefficients independently, in addition to the 31 application checks above.

Table 3: Joint-request time ratios and descriptive intervals over the seven graph groups. Each group contains two compatible target requests.

Comparison	Time ratio	95% interval
Joint native / separate native	0.5252	$[0.5130, 0.5420]$
Joint integrated / separate native	0.5175	$[0.5073, 0.5346]$
Joint learned / separate native	0.5152	$[0.5022, 0.5322]$
Joint learned / joint native	0.9810	$[0.9657, 0.9946]$
Joint learned / joint integrated	0.9955	$[0.9810, 1.0058]$

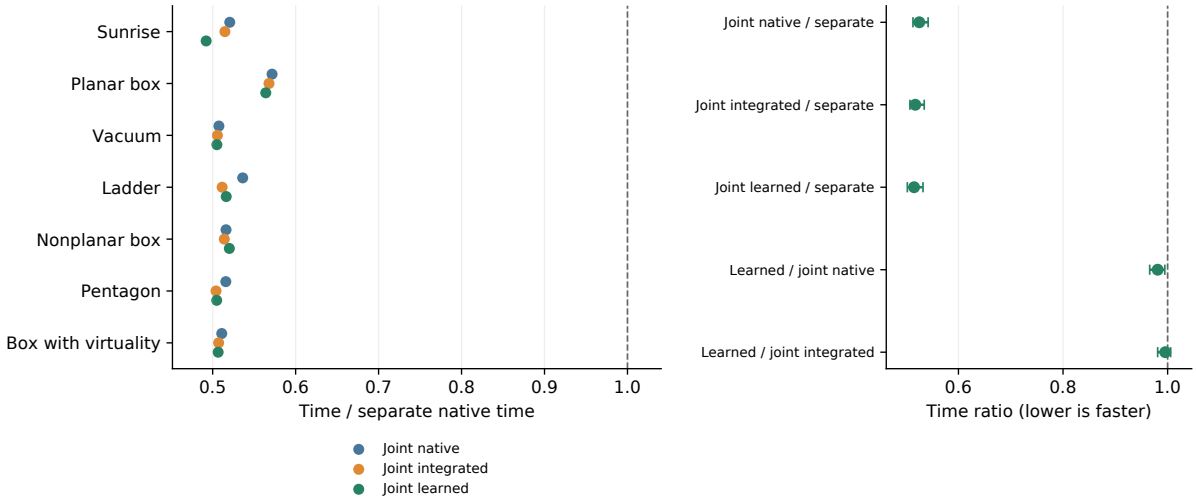


Figure 54: Combining compatible requests gives the main saving. Left: ratios by graph relative to two separate native reductions. Right: aggregate ratios and descriptive intervals. Comparing learned choices with joint integrated native choices gives no reliable extra saving. The panels retain the equal-time reference.

The joint comparison reuses the main test's reserved parameter points, so it provides an additional comparison on that population. Its saving requires requests that can share one calculation. Transfer to new graph structures, isolated arbitrary integrals, and general scattering beyond three loops remains outside this result. The next study returns to individual batches and tests whether separate models and a wider set of seed strategies give a measurable gain.

21 Choosing a strategy separately for each graph

A common predictor may miss differences in the work required by different graph structures. We therefore fit separate predictors for the seven graphs and enlarge the available seed strategies. The added choices can restrict seeding to maximal target sectors or reduce numerator seeding in lower sectors through Kira's truncation option [9]. Each method first enforces a valid positive-index bound for its declared sectors, and the final basis check and fallback remain in place.

For each graph, ten candidate models predict the complete-time ratios of nine strategies from nine target and family features. The candidates include regression trees, a bagged ensemble of trees, regularized linear models, and nearest-neighbour models. Their cost labels include unsuccessful attempts and retries. The data contain 36 training batches, 18 validation batches, and 36 reserved test batches at new rational parameter values, with d symbolic throughout. Targets include propagator powers up to three, lower sectors, and auxiliary numerator indices -2. Appendix C gives the exact input definitions and selection procedure.

Training and validation contain 2,798.5 seconds of measured reductions, while fitting all seventy candidates takes 0.86 seconds. These offline costs exclude reference construction, development, and replaced measurements and are not included in application time. Validation enables a new predictor only if it saves at least five percent against the fastest available control. The controls are native choices, a deterministic sector-and-truncation rule, and a fixed strategy for each graph selected from training; the fixed strategy may itself use the older equation filter. This requirement activates predictors for three graphs and retains a control for the others. Unknown graphs and features outside the training range use native choices.

Five repetitions of four methods give 720 test calculations, all of which pass exact coefficient checks. With equal weight for each graph, the selected method has ratio 0.9368 to native choices, with descriptive interval $[0.8867, 0.9878]$, corresponding to 6.3% less time. Its ratio to a fixed strategy for each graph is 0.9441 with interval $[0.8992, 0.9863]$. This second comparison isolates the benefit of selecting a strategy from the targets beyond choosing one setting for the whole graph.

Table 4 shows lower time for five of the seven groups. The enabled trees save 14.2% for the planar box, 12.4% for the ladder, and 13.8% for the nonplanar box. The vacuum and box with external virtuality take 3.7% and 0.8% more time, respectively, while retaining control strategies throughout. In particular, the vacuum uses native settings, so its difference includes the extra selection overhead and execution variation. Across 180 timed trials, the selected method needs no retries and native choices require 40; these costs are included in the ratios.

Table 4: Reduction time with a separate strategy for each graph. Times are geometric means of batch medians in seconds, and the ratio divides selected time by native time. The strategy column identifies the enabled model or retained control.

Graph	Strategy	Native	Selected	Ratio
Sunrise	Stronger truncation	0.825	0.811	0.983
Planar box	Depth-two tree	3.410	2.925	0.858
Vacuum	Native	1.033	1.072	1.037
Ladder	Depth-one tree	5.144	4.506	0.876
Nonplanar box	Depth-two tree	6.234	5.376	0.862
Pentagon	Target sectors	1.040	0.989	0.951
Virtuality box	Sectors and truncation	0.808	0.815	1.008

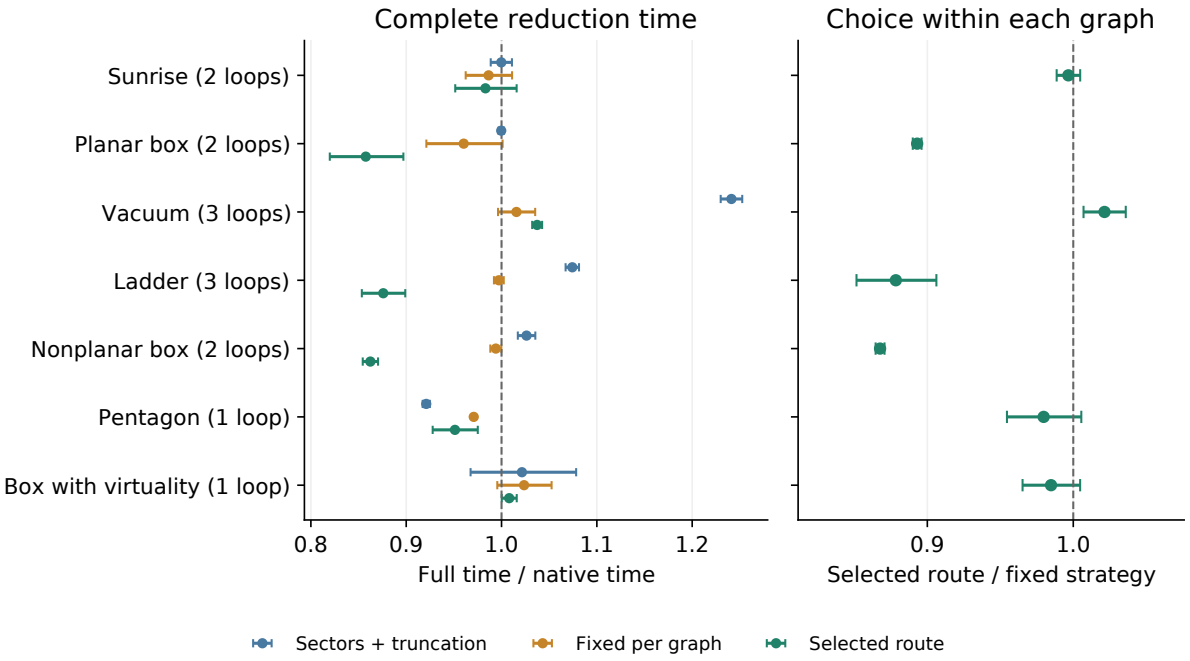


Figure 55: The effect of choosing a separate policy for each graph. Left: complete-time ratios to native choices. Right: the selected method relative to one fixed strategy per graph. Bars show the range across two reserved parameter profiles; they are not significance intervals. Both panels retain the equal-time reference.

The timed trials check 14,040 coefficient entries. A further 36 applications supplied only with the master basis and seven additional application checks verify 945 coefficients independently. These checks establish correct completion for the measured inputs. The graph structures and target patterns are known during training, and each graph has only two training parameter profiles. The reserved profiles therefore test new kinematic values within these inputs, leaving new target structures and general scattering beyond three loops for further study.

22 Expanding seeds selectively and using an intermediate basis

The graph-specific study identifies symbolic equation regeneration as the largest measured solver stage. Some target batches need seed limits beyond those of the targets themselves, and a general enlargement can generate many unnecessary equations. We therefore test whether selective increases in the limits, or a reduction through an intermediate master basis, can lower this remaining cost.

The candidate set contains six ways to increase selected seed limits and two ways to reduce first to a temporary basis. In the latter case, Kira reduces the remaining temporary masters to the prescribed basis, and exact multiplication combines the two coefficient matrices. The final basis requirement is the same for every method. Appendix D defines the candidates and derives this conversion, which allows the first target reduction to use a smaller system when the remaining basis conversion is inexpensive.

For development batch c, let $B_c = (r_c, s_c, q_c)$ denote valid initial limits on the positive index sum, numerator rank, and number of extra denominator powers. We retain the fastest new candidate for a development batch only if it saves at least ten percent against the graph-specific method in Section 21. The resulting candidate set contains that preceding method, full-sector basis completion, and the selective increments $B_c + (1,0,0)$, $B_c + (0,1,0)$, and $B_c + (1,1,0)$. Development results select the options but do not enter the reserved comparison.

The new data contain 36 training batches, 18 validation batches, and 36 test batches from the same seven graphs and target patterns, with new rational kinematics and symbolic d. Ten predictors per graph use the same nine features to predict time relative to the preceding method. Validation compares each predictor with that method and with a fixed new strategy selected from training for each graph. A predictor is enabled only if it saves at least five percent against the faster control; otherwise that control is used. A fixed candidate may itself call the preceding learned method, so this comparison tests the additional choice among the new options.

Validation enables new predictors for three graphs. The retained training and validation records contain 1,731.7 seconds of measured reductions, and fitting the seventy candidates takes 0.88 seconds. These costs exclude reference construction and development and remain outside the reserved reduction timer.

The test compares native choices, the preceding graph-specific method, the new fixed strategy, and the selected strategy. Five repetitions give 720 calculations, all of which pass exact checks. A further 43 applications also pass when the reduction receives no target reference coefficients. With equal weight for each graph, the selected method has ratio 0.9115 to the preceding method, with descriptive interval $[0.8105, 0.9980]$, giving 8.8% less time. Its ratio to native choices in the same experiment is 0.8516 with interval $[0.7282, 0.9647]$, giving 14.8% less time. Relative to the new fixed strategy, the ratio is 0.9264 with interval $[0.8760, 0.9747]$.

The sum of batch medians answers a different question: how long would this set of requests take when each is counted once? For the 36 reserved batches, this sum is 106.24 seconds for native choices, 96.32 seconds for the preceding method, and 77.03 seconds for the selected method. The geometric ratios above instead give every graph equal weight. Both measures show a saving,

although their magnitudes differ because the longer calculations contribute more to the workload total.

The stage measurements explain where part of the saving occurs. Across 180 runs per method, symbolic equation regeneration falls from 138.02 to 80.58 seconds. Neither the preceding nor the selected method makes a fallback attempt, compared with 40 for native choices. Planned second-stage reductions of temporary masters are included in complete time and are counted separately from fallback. The comparison with the fixed strategy supports an additional benefit from the selection procedure, which combines validation-based control choices with predictions for individual target batches.

Table 5: Selective seed expansion compared with the graph-specific method in Section 21. Times are geometric means of batch medians in seconds. "Previous" denotes that graph-specific method and "New" denotes the seed-expansion method. Each graph has two reserved parameter profiles.

Graph	Policy	Previous (s)	New (s)	New/previous	New/native
Sunrise	Add s	0.596	0.593	0.995	0.979
Planar box	Tree depth 1	2.422	2.062	0.851	0.735
Vacuum	Complete basis	0.928	0.941	1.013	1.016
Ladder	Tree depth 1	4.477	3.007	0.672	0.584
Nonplanar box	Tree depth 1	4.091	3.550	0.868	0.759
Pentagon	Previous	0.723	0.754	1.043	0.998
Virtuality box	Previous	0.580	0.582	1.003	1.004

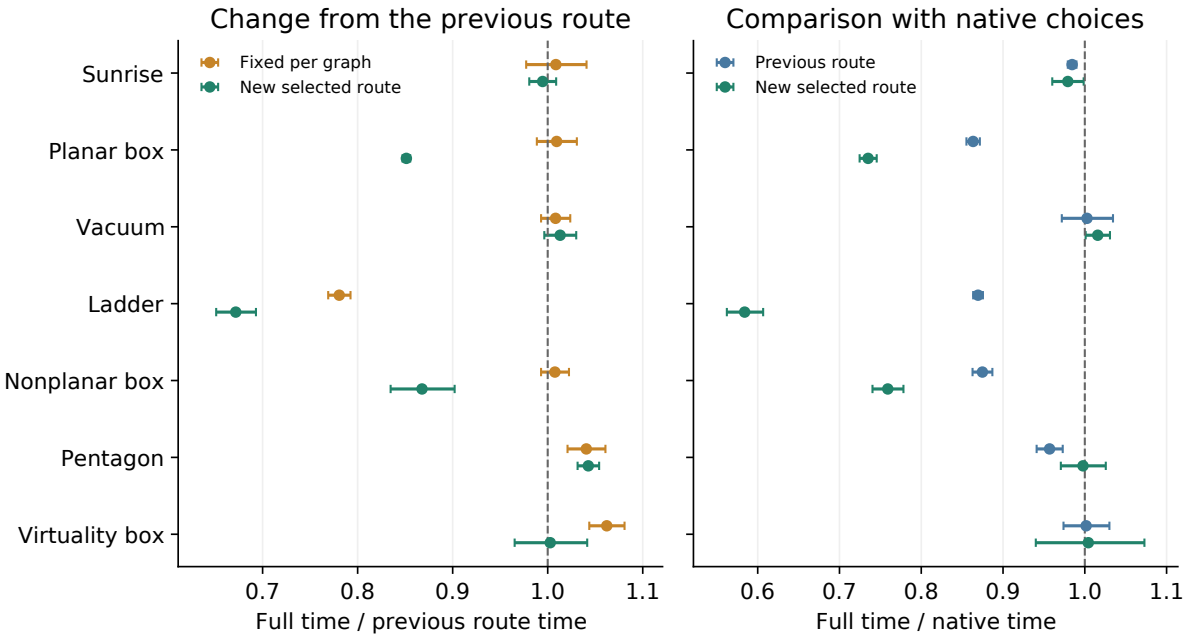


Figure 56: Selective choices change complete reduction time. Left: the new fixed strategy and selected method relative to the preceding method. Right: the preceding and new methods relative to native choices in the same test. Values below one are faster. Bars show the range over the two reserved profiles; they are not significance intervals.

The gain is not uniform across the graphs. Relative to the preceding method, the vacuum takes 1.3% more time, the pentagon 4.3% more, and the box with external virtuality 0.3% more. All reported times include prediction, native IBP generation, basis conversion, exact algebra, checks, cleanup, and fallback. These results measure the additional method directly on new parameter profiles; they do not multiply gains from earlier experiments. Their scope remains

known graphs and target patterns with rational kinematics, rather than unrestricted symbolic kinematics or arbitrary high-loop scattering.

23 Testing integral and propagator orderings

Selective seed expansion reduces equation-generation work, but the ordering of the resulting integrals can still affect elimination. Kira provides eight integral priority rules and four automatic propagator-ordering schemes [8, 9]. We test whether a model can choose among these settings from the target indices before equation generation, while retaining the seed strategy from Section 22. The model makes one decision for each batch, and a failed basis check triggers a repeat with the preceding settings. Appendix E defines the candidates and shows why consistent relabelling preserves the integrals.

Development first compares eleven candidates and then tests the pair of best nondefault orderings for each batch. We keep the preceding settings and any batch winner that saves at least ten percent of complete time. If ω is the integral-order number and χ the propagator-scheme number, the retained new settings are $(\omega, \chi) = (2, 1)$, $(7, 1)$, $(5, 4)$, and $(8, 3)$. The preceding method also remains available without a change in its settings.

We fit ten predictors per graph on 36 training batches, select their use on 18 validation batches, and test on 36 reserved batches. The features, complete-time labels, candidate model types, and five-percent validation requirement are the same as in the seed-expansion study. A fixed ordering still uses the preceding learned seed strategy, so the fixed-setting comparison measures the additional ordering decision. Training and validation contain 1,432.8 seconds of reductions, and fitting the seventy predictors takes 0.93 seconds. These offline costs exclude development and reference construction.

None of the predictors passes the validation requirement. The tested selected method therefore uses fixed ordering settings chosen by validation, with no newly enabled ordering predictor. Any difference in its time reflects those settings, their interaction with the retained seed strategy, and execution variation. This distinction is necessary when interpreting the results as evidence about machine learning.

All 720 timed reductions and 43 additional applications pass exact checks; the additional applications receive no target reference coefficients during reduction. With equal weight for each graph, the selected-to-preceding ratio is 1.0141 with descriptive interval $[0.9986, 1.0303]$, giving 1.4% more time. The selected-to-native ratio is 0.8499 with interval $[0.7306, 0.9642]$, giving 15.0% less time, while the ratio to the new fixed strategy is 1.0070 with interval $[0.9811, 1.0327]$. The saving against native choices therefore does not show an improvement over the already learned seed-expansion method.

The sum of median times over the 36 test batches is 110.09 seconds for native choices, 79.61 seconds for the preceding method, 79.53 seconds for the new fixed settings, and 79.68 seconds for the selected settings. On this workload, the selected method takes 0.1% more time than its predecessor. Its additional ordering decision takes a median of 271.1 microseconds per application, and both this cost and the retained seed-policy cost are included in complete time.

Table 6: Ordering settings compared with the seed-expansion method in Section 22. Times are geometric means of batch medians in seconds. "Previous" retains the earlier ordering, and "New" uses the selected setting. The pair (ω, χ) specifies the integral order and propagator scheme.

Graph	Choice	Previous (s)	New (s)	New/previous	New/native
Sunrise	$(7, 1)$	0.610	0.633	1.037	1.034
Planar box	$(2, 1)$	2.407	2.390	0.993	0.759
Vacuum	Previous	0.975	0.982	1.007	0.983
Ladder	Previous	3.122	3.088	0.989	0.591
Nonplanar box	$(2, 1)$	3.698	3.681	0.995	0.745
Pentagon	$(2, 1)$	0.788	0.820	1.041	0.987
Virtuality box	$(5, 4)$	0.601	0.624	1.039	0.955

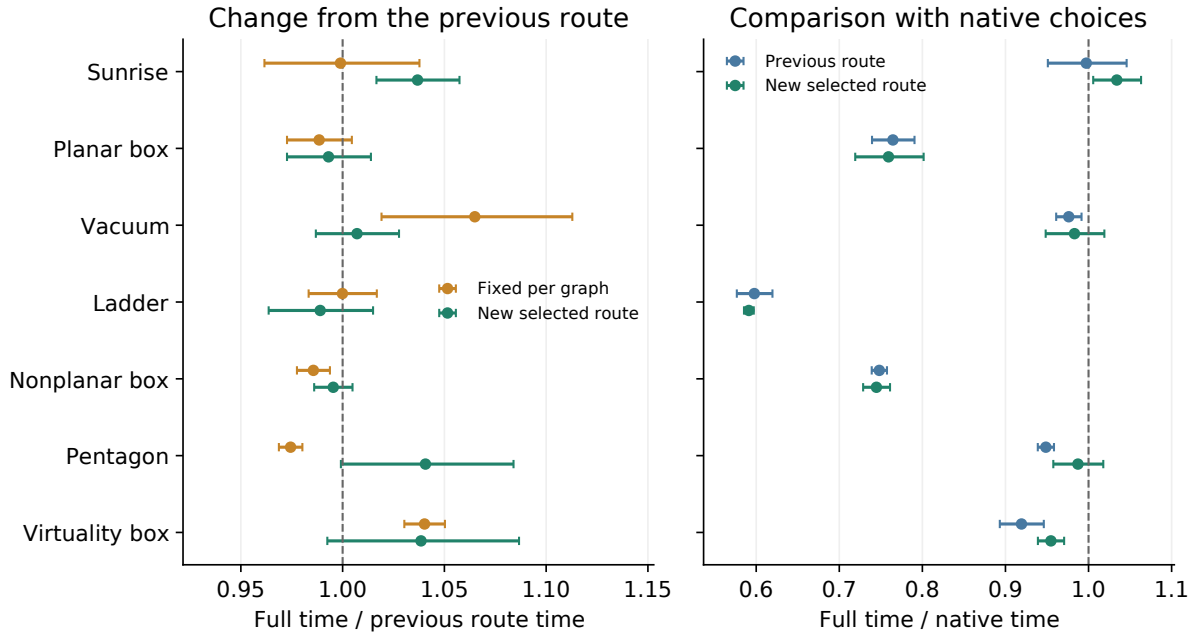


Figure 57: The effect of choosing Kira settings before reduction. Left: new fixed settings and selected settings relative to the preceding method. Right: preceding and selected methods relative to native choices. Values below one indicate less complete time. Bars give the range across two parameter profiles; they are not significance intervals.

The sunrise, vacuum, pentagon, and box with external virtuality take 3.7%, 0.7%, 4.1%, and 3.9% more time, respectively. No selected calculation requires fallback. Across 180 runs per method, symbolic equation regeneration changes from 79.88 to 80.58 seconds, while back-substitution changes from 37.99 to 37.35 seconds. These stage measurements show small changes in opposite directions rather than a consistent overall improvement.

The timers include prediction, preparation, native generation, exact checks, basis conversion, retries, and cleanup. As in the preceding comparisons, the caller and models are already loaded, and each attempt starts a fresh Kira process. The evidence therefore supports retaining the earlier ordering for this measured workload of known one- through three-loop graphs at new rational parameter values. The next comparison keeps that choice and changes the amount of parallel work instead.

24 Choosing how many Kira workers to use

The ordering study gives no further saving, so we retain the seed-expansion method and its earlier ordering. We next test the number of Kira workers, which determines how much of the symbolic calculation can run in parallel [2]. Development measurements show that extra workers add overhead on small batches, while some larger box batches finish sooner. This suggests choosing the worker count from the requested targets before equation generation.

The candidate counts are one, two, and four workers in the same integrated executable. The native control uses Kira’s own seed and reduction decisions together with the common earlier implementation changes. For the learned comparison, the nine target features used above predict each worker count’s complete time relative to one worker. We fit ten predictor types per graph on 36 training batches, select settings on 18 validation batches, and reserve 36 more batches for testing. Validation minimizes the sum of batch median times. A predictor must save at least two percent against the faster control and give lower summed time in at least two of three validation repetitions; otherwise the control remains active. Appendix F gives the labels, checks, and comparison rules.

Validation enables a depth-two tree for the planar box and a depth-one tree for the nonplanar box. The other five graph types retain the preceding one-worker strategy. Both trees split only on the number of targets in the batch and choose at most two workers. Since the tested target patterns repeat across the parameter splits, these results establish a useful decision within those patterns; they do not establish target count as a sufficient description of arbitrary integral structures. Training and validation contain 1,098.1 seconds of reductions, and fitting all seventy candidates takes 0.82 seconds. These offline costs exclude reference construction and development.

All 900 timed reductions and 50 additional applications pass exact checks. The additional applications receive targets and a prescribed basis without reference coefficients. Seven run consecutively through a caller that keeps the models loaded, in a separate two-core allocation; each native attempt still starts a fresh Kira process. The timing experiment reserves four cores for every method so that the always-four-worker control can use its full setting. Thus every timed method has the same available resources, even when it uses fewer workers.

For the 36 reserved batches, the sum of median complete times falls from 82.21 seconds with the preceding one-worker method to 78.73 seconds with worker selection, a reduction of 4.2%. In the same experiment, native one-worker choices take 110.67 seconds, always using four workers takes 156.95 seconds, and a fixed count for each graph takes 82.45 seconds. Selection therefore gives 4.5% less time than the fixed-per-graph control. Table 7 shows where these differences arise; changes for the five graphs that retain one worker include execution variation and the

additional decision cost.

Table 7: Time with and without worker selection. Entries sum the batch median times within each graph, in seconds. “Previous” uses the seed-expansion method with one worker; “Selected” uses the validation-selected count. The final column divides selected time by previous time.

Graph	Choice	Previous (s)	Selected (s)	Selected/previous
Sunrise	1 worker	5.271	5.215	0.989
Planar box	Tree depth 2	15.643	14.997	0.959
Vacuum	1 worker	4.813	4.620	0.960
Ladder	1 worker	23.197	23.602	1.017
Nonplanar box	Tree depth 1	26.768	23.819	0.890
Pentagon	1 worker	3.558	3.523	0.990
Virtuality box	1 worker	2.963	2.955	0.997

Giving each graph equal weight yields a smaller selected-to-preceding ratio of 0.9817, with descriptive interval $[0.9682, 0.9946]$, or 1.8% less time. The selected-to-native ratio is 0.8456 with interval $[0.7346, 0.9502]$, or 15.4% less time. These averages give equal importance to each graph, whereas the sum of medians measures the time for this particular collection of requests. Figure 58 also shows why simply increasing every calculation to four workers is ineffective on these small cases.

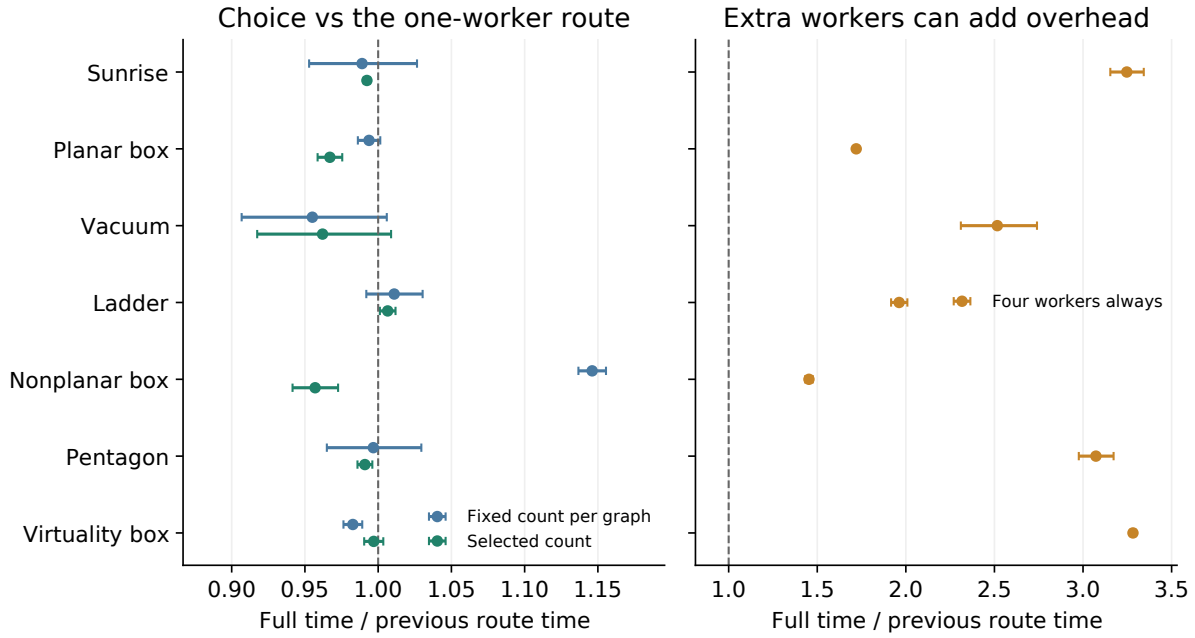


Figure 58: The effect of selecting the worker count. Left: selected and fixed worker counts compared with the preceding one-worker method. Right: always using four workers adds overhead on many small cases. Each panel has its own horizontal scale. Each point is the geometric mean of batch time ratios for one graph. Bars show the range across two parameter profiles and are not significance intervals.

Variation between repetitions matters for the size of the gain. Across all five repetitions, the selected method takes 404.84 seconds and the preceding method 414.08 seconds, giving 2.2% less elapsed time. Selection has lower summed time in four of the five repetitions, with selected-to-preceding ratios ranging from 0.9346 to 1.0555. This range describes the observed repetitions and is not a confidence interval. The ladder’s sum of median times increases by 1.7%.

The saving in elapsed time also uses more CPU time. The extra prediction takes a median of 276.6 microseconds and is included in the complete-time measurements. Across 180 reductions per method, the preceding strategy uses 385.9 CPU-seconds, the always-four-worker strategy 1024.1, and the selected strategy 458.9. Selection therefore increases CPU use by 18.9%. These values measure CPU time consumed by the calculation, rather than the number of cores allocated to it.

The selected method has no failed attempts and requires no fallback. The always-four-worker method has ten unsuccessful native attempts, whose subsequent retries succeed; their costs remain in the reported time. Some native attempts report allocation failures under the imposed process limits. All comparisons include prediction, preparation, exact algebra, basis conversion, checks, retries, and cleanup, with the caller and models loaded before timing and a fresh Kira process for each attempt.

Worker selection is retained for this measured workload because it lowers elapsed time while preserving the exact results. Its demonstrated scope is seven known graph structures at one through three loops, known target patterns, and new rational kinematics with symbolic dimension. The preceding one-worker method remains available when the resource conditions or inputs differ. The result identifies a measured improvement among the tested choices, with the CPU cost stated alongside it.

25 Checking the contribution of each component

The previous models form a sequence of choices, so a faster final calculation does not identify which choice supplies the saving. We compare ten methods on fourteen fresh batches from seven known graph structures and then disable individual learned decisions while keeping the other stages unchanged. Every method uses the same targets and prescribed basis. All 420 reductions in the first comparison and all 96 additional decision controls pass exact coefficient checks.

The native-choice total is 66.035 seconds, using sums of three-run batch medians. Table 8 shows the complete strategies relative to that control. The result through worker selection takes 46.777 seconds, but this saving includes fixed seed settings as well as learned decisions. The equation, seed, and row filters increase time on this particular target pattern. Earlier savings on other targets remain separate results.

Table 8: Component comparison on the same fourteen batches. Complete times include decisions, exact checks, and recovery. The first three rows retain preceding stages.

Method	Complete time (s)	Change from native
Graph-strategy model	61.779	6.4% less time
Through seed expansion	49.309	25.3% less time
Through worker selection	46.777	29.2% less time
Equation filter	72.446	9.7% more time
Seed filter	70.241	6.4% more time
Row model	105.500	59.8% more time

The isolated controls in Table 9 show why complete-time gains need a fixed comparison. On these batches, graph choice saves a small amount, while the added seed and worker choices increase time. Figure 59 presents both comparisons. These results motivate replacing the sequence of selectors with the single cost policy in the next section.

Table 9: The added learned decision, with other stages held fixed. Each row uses the subset on which that decision is enabled. Fixed settings were selected before this test.

Decision	Batches	Fixed (s)	Learned (s)	Change
Graph choice	6	51.448	50.188	2.4% less time
Seed expansion	6	41.030	41.093	0.2% more time
Worker count	4	24.232	25.555	5.5% more time

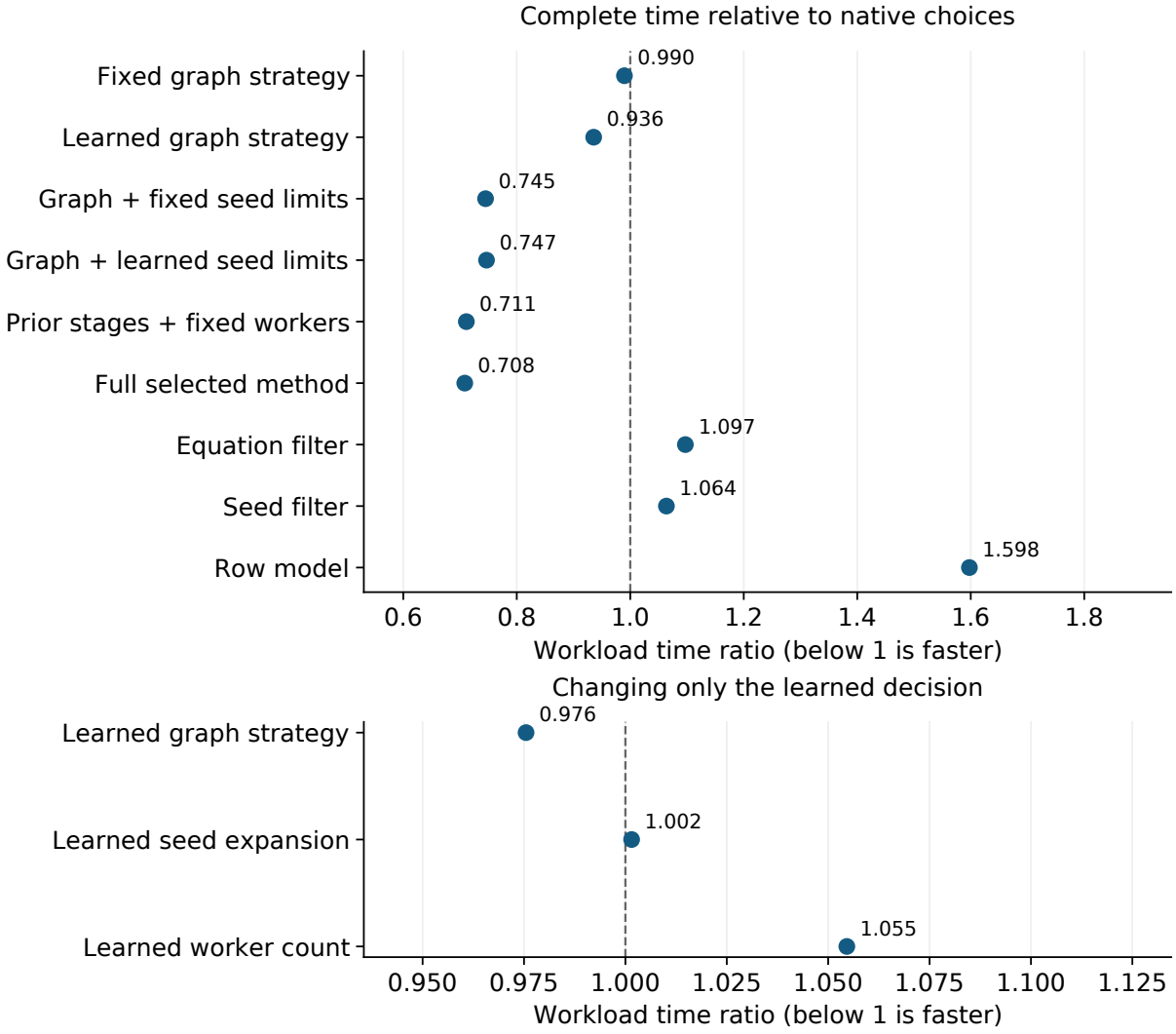


Figure 59: Complete strategies and their added learned decisions. The upper panel uses all fourteen batches. The lower panel changes one learned decision on its enabled subset. Both show ratios of summed batch medians and retain the equal-time reference.

25.1 Connected scattering and bounded higher-loop tests

We also construct connected four-point ladders and boxes with bubble subgraphs, defined in Appendix G. The two completed three-loop bubble-box profiles take 10.538 seconds with restricted initialization and native equation choices, compared with 7.468 seconds with the frozen equation filter. This is 29.1% less time under the same initialization. The complete strategy selectors otherwise retain native choices on these previously unsupported graph structures.

The bounded four- through six-loop probes identify separate costs and limits. Four-loop initialization exceeds the initial short time allowance. Five-loop attempts exceed the memory allowance, and the six-loop four-point representation requires 39 inverse propagators, above the tested executable's 31-entry limit. Later runs produce a four-loop bubble-box reference and reproduce its coefficients with enlarged seeds. Three native repetitions have median time 465.329 seconds. Two completed filtered repetitions take 520.336 and 551.161 seconds, each requiring native recovery after an insufficient equation set. Only these two filtered repetitions complete, so this diagnostic supplies no matched three-run speedup estimate. The incomplete attempt is excluded from timing results.

The new work therefore addresses both preparation and the learned choice. A complete four-loop reduction is established for the stated bubble-box family, while the failed larger-family probes supply no speedup claim. The vacuum recurrence used elsewhere in this paper does not determine the scattering coefficients, as explained in Appendix G.

26 Learning one policy for complete reduction time

The component controls show that adding another learned decision can increase time when a fixed setting already suits the targets. We therefore replace the sequence of strategy selectors with one policy that compares complete calculations before equation generation. Its purpose is to choose a small sufficient calculation when possible and to avoid an expensive failed attempt when more seeds are needed. Kira continues to generate and solve the exact identities.

26.1 Model, training data, and comparison

The policy uses the target indices and the family's loop, momentum, mass, and denominator structure. It compares seven choices: the target-based native seed limits, three smaller seed regions, integral ordering 5, the existing equation filter, and direct expansion of the seed limits. Each training label is the measured time of a complete reduction, including retries and exact coefficient checks. The new model therefore learns the cost of an insufficient equation set as well as the cost of a successful first attempt. Appendix H gives the inputs, cost function, prediction rule, and recovery steps.

We fit regression trees, random forests, and ensembles of extremely randomized trees [30, 48, 39]. The comparison uses nine candidate models, with validation selecting both the model and a minimum predicted saving required to depart from its fixed control. The selected model is a regression tree with maximum depth 8 and 1 tree. Training supplies 47 complete cases, validation supplies 26 cases, and the reserved test contains 26 completed batches. Constructing and validating the cost data takes 4703.1 seconds of recorded reduction time, while fitting the nine models takes 0.409 seconds. These are offline costs; a saved policy can serve later requests without repeating them. The tests cover the earlier seven graph structures and connected bubble-box scattering families at three and four loops. Rational kinematics change between splits, while dimension remains symbolic. The symbolic scattering tests use disjoint target

indices within each graph. These are tests within known structures, with complete definitions in the appendices.

Before fitting the models, we also remove two deterministic costs from Kira's preparation. The first reads the existing sector files directly in their original order. The second simplifies the test for zero sectors, whose integrals vanish in dimensional regularization. That test uses parametric polynomials formed from the propagator denominators [49, 9]. We set the parameters of propagators absent from every declared top sector to zero before constructing the test. Equation (189) proves that this change preserves the test within the declared domain. The complete polynomials remain available for other operations. Nine coefficient comparisons against earlier independent references pass before the new training begins. Both exact changes are present in every method in Table 10, so that table measures the policy after the common implementation improvement.

26.2 Results on the reserved targets

Table 10 gives sums of three-run batch median times. Native choices take 111.269 seconds, the validation-selected fixed choice takes 111.305 seconds, and the learned policy takes 110.693 seconds. The selected fixed choice is the target-based native seed region. The learned result uses 0.5% less time than native choices and 0.5% less time than the fixed control. Summing all repetitions gives learned/native and learned/fixed ratios of 0.9945 and 0.9940, respectively. These additional ratios show the effect of keeping every measured time. The same targets and prescribed master basis are used in each comparison, and every attempt uses one Kira worker within the same one-core, 8 GiB allocation. Figure 60 shows how the added effect varies across graph structures.

Table 10: Complete times on reserved targets, in seconds. Each graph row sums its batch medians. All three methods use the same improved executable; the fixed choice is selected on validation data.

Graph	Loops	Batches	Native	Fixed	Learned
Three-loop bubble box	3	1	2.628	2.599	2.654
Four-loop bubble box	4	1	29.580	29.746	30.454
Planar double box	2	3	9.450	9.664	9.195
Three-loop two-point ladder	3	3	24.190	24.626	23.381
Nonplanar box	2	4	31.250	30.932	31.167
Virtuality box	1	4	3.175	3.197	3.385
Pentagon	1	4	4.934	4.778	4.833
Sunrise	2	3	2.068	2.041	1.982
Three-loop vacuum	3	3	3.993	3.721	3.642
Total		26	111.269	111.305	110.693

All 234 timed reductions and 26 further applications agree exactly with their reference coefficients. The additional applications receive the master basis but no target coefficients before execution. A separate check of the public application interface completes 9 batches. Native choices require 15 recovery attempts across the timed tests, compared with 9 for the learned policy. The median decision time across the test batches is 0.519 milliseconds, including feature construction and tree evaluation. The prediction cost, all failed attempts, and recovery are included in complete time. Model loading, offline reference construction, and basis discovery are outside this repeated-reduction comparison. 1 planned reserved batch(es) lack a completed reference or enlarged check and receive no speedup estimate.

The four-loop confirmation follows a condition registered before fitting the model. When the

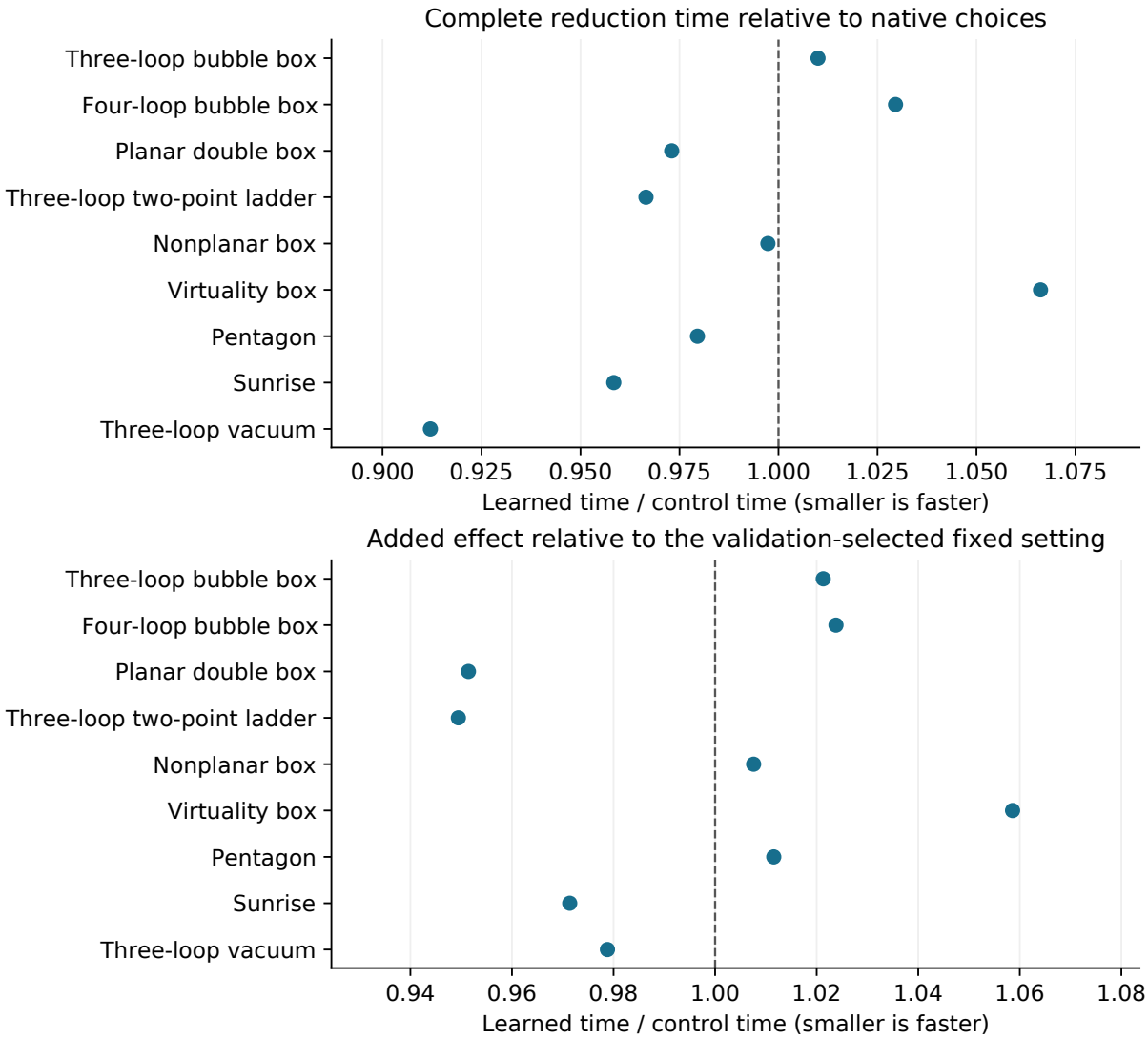


Figure 60: The upper panel compares the learned policy with native choices after the common exact improvements. The lower panel compares it with the best fixed validation choice. Each point divides sums of batch medians within one graph structure. Values below one indicate less time; the equal-time line remains visible. Three repetitions support descriptive comparisons, rather than a general speed guarantee.

original reserved reference check does not complete, a fresh profile uses the two nested seed regions already verified during development. The model remains fixed, and the original incomplete case remains in the failure count.

26.3 A stronger fixed control and the deployed policy

One fixed setting for all graphs can be a weak control because different denominator structures can favour different seed regions. We therefore select a separate fixed setting for each graph using validation data only. We also fit 21 small regression trees separately for the graphs with sufficient training cases. They use the same measured cost labels and can specialize in the effect of target powers and numerator rank within one denominator structure. Validation compares these trees with the global model and selects a predictor and gate for each graph. Fitting the additional trees takes 0.723 seconds, with no additional label-generation cost.

We compare the graph-specific fixed settings, the original global model, and a validated policy that uses its selected predictor only where validation supports an additional saving. This procedure was specified before any reserved timings were collected. The final rule requires at least two percent less summed validation time and a saving in both validation repetitions; otherwise the graph's fixed setting is used. All choices remain fixed during the matched comparison. Appendix H defines the local models and selection rule.

Validation enables target-dependent predictions for planar double box, three-loop two-point ladder, nonplanar box. Table 11 gives a separate matched comparison on the same reserved targets, with three repetitions per method. The graph-specific fixed choices take 105.909 seconds, the cost model takes 103.896 seconds, and the validated policy takes 103.983 seconds, using sums of batch medians. The validated policy uses 1.8% less time than the graph-specific control. Across all repetitions, its time ratio is 0.9830. All 234 reductions pass exact coefficient checks. The fixed choices and validated policy require 21 and 9 recovery attempts, respectively. Figure 61 shows where predictions add value beyond a fixed choice for each graph.

Table 11: Matched complete times in seconds against the stronger fixed control. The validated policy keeps fixed choices where validation does not support the model.

Graph	Fixed per graph	Cost model	Validated policy
Three-loop bubble box	2.197	2.190	2.201
Four-loop bubble box	28.777	29.170	28.961
Planar double box	9.932	9.275	9.333
Three-loop two-point ladder	23.542	22.620	22.518
Nonplanar box	30.572	29.734	29.839
Virtuality box	2.207	2.160	2.213
Pentagon	3.124	3.295	3.371
Sunrise	1.855	1.873	1.791
Three-loop vacuum	3.702	3.576	3.756
Total	105.909	103.896	103.983

The public application uses the validated policy and completes the separate exact checks reported above. It keeps native choices for requests outside the recorded family structures and target ranges. An additional application check confirms this fallback for an unsupported graph label. This guard limits the domain in which the learned prediction is used; it does not establish performance on unseen graph structures.

26.4 Separating preprocessing and prediction

We repeat the connected three- and four-loop bubble-box calculations with each exact change enabled in turn, followed by the validated policy. Table 12 reports matched complete times from these separate trials. Every method receives the same 600-second allowance per attempt. The previous-preparation control retains declared-sector initialization and uses native equation choices in the same executable. It disables the two new exact changes. Thus the overall factor includes deterministic preprocessing as well as any subsequent benefit from its selected settings. The final change from projected preparation includes the validated policy and its graph-specific fixed settings; Figure 62 keeps these contributions separate.

The 4-loop previous control does not complete within its bounded attempts, after 1190.6 seconds of recorded work. Its remaining 2 repetitions are omitted after this failure, while the other prespecified settings are completed. It receives no median completion time or speedup factor.

The new policy makes one inexpensive decision before the costly work begins and retains native row elimination. Its tests cover stated target patterns and kinematics in known families. They

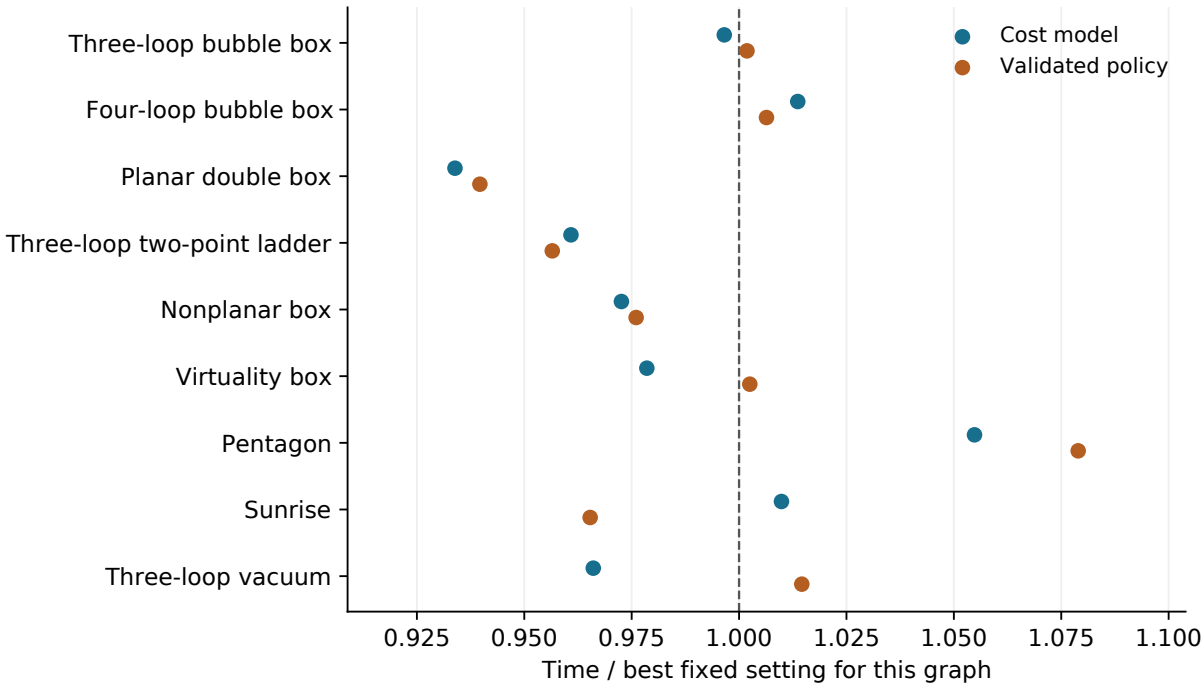


Figure 61: Added effect of target-dependent choices relative to the best fixed validation setting for each graph. Each point divides summed batch medians in the matched comparison. The validated policy uses the model only in the graph structures selected by its validation rule. Timing differences also remain when two methods choose the same setting.

Table 12: Matched preparation comparison for the connected bubble boxes. Times are medians of three complete reductions, in seconds. The final column divides previous-preparation time by validated-policy time.

Loops	Previous	File index	Projection	Validated policy	Overall factor
3	6.087	5.525	2.906	2.939	2.07
4	Incomplete	98.408	28.197	28.666	–

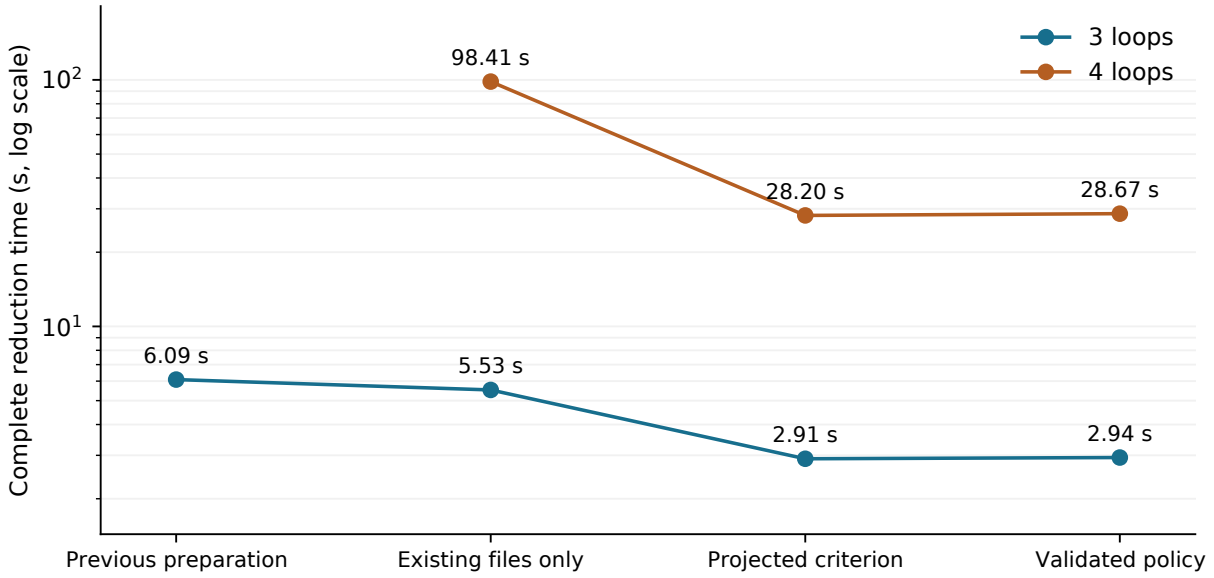


Figure 62: Complete time for the matched preparation study. The logarithmic vertical axis displays the large deterministic change and the final change from the validated policy. The separate graph-specific comparison measures the added effect of prediction. All plotted points represent completed reductions into the same prescribed basis with exact coefficient checks. Incomplete controls are omitted from the curve and reported in the table.

do not establish a gain for every integral type, a minimal master basis, or a globally fastest reduction sequence. Reference failures and bounded higher-loop attempts remain part of the evidence. These limits determine where the current measurements support use of the method.

27 A learned portfolio of complete reduction strategies

The preceding comparisons show that the useful decision depends on both the denominator structure and the target powers. A seed region that suffices for one batch can leave another batch unreduced, while expanding every seed limit can generate equations that the calculation does not need. We therefore train a model to choose a complete calculation from several strategies before Kira generates the equations. This choice includes the previous workflow as a candidate, so the new policy can retain that workflow when its measured cost is lower.

27.1 What the model chooses

For a target batch, let r_0 be the largest sum of positive indices, s_0 the largest total numerator power, and d_0 the largest number of extra denominator powers, or dots. Kira uses limits of this kind to define the seed region [9]. Our earlier complete-cost policy chooses among seven regions or equation settings, as described in Appendix 26. A development comparison reveals a useful additional choice: increase s_0 by one while leaving the positive-index and dot limits unchanged. This selective expansion can supply missing identities without the cost of increasing all three limits.

The final portfolio contains five choices: the earlier cost policy with one or two workers, selective numerator-rank expansion with one or two workers, and the previous workflow from Appendix F. A worker carries out tasks in Kira's existing parallel symbolic solver [2]. The previous workflow combines its existing graph, seed, and worker choices; its parameters remain fixed in this com-

parison. Each choice uses the same exact preprocessing and a supplied master basis. The three- and four-loop bubble boxes have no trained policy in the previous workflow, so its control uses the documented native fallback. We retain Kira's native row elimination because the learned row model has not reduced complete time reliably.

Each attempt must express every target in the prescribed basis. If it cannot do so, the calculation retries with native seeds and then an expanded seed region. We skip a repeated attempt when its effective seed bounds, ordering, and equation filter are identical to those already tried. The model does not predict reduction coefficients. It predicts the relative time of these complete strategies, including checks, failed attempts, and recovery. Appendix I defines the model inputs, objective, guard, and exact recovery procedure.

27.2 Training and independent tests

We fit 30 candidate predictors using 47 training batches and select the predictor, fixed control, and decision threshold using 26 separate validation batches. The candidates include regression trees, random forests, and extremely randomized trees [30, 48, 39]. Their twenty inputs describe the graph structure, target powers, numerator ranks, number of targets, and distribution of dots. They contain no reference coefficients or measured test times. The recorded label calculations take 1858.3 seconds, and fitting takes 2.621 seconds. These costs occur before deployment.

For each graph, validation also selects the fastest fixed portfolio choice. The model replaces this choice only if it reduces summed validation time by at least two percent and gives a saving in both validation repetitions. This rule enables target-dependent portfolio predictions for planar double box, three-loop two-point ladder, nonplanar box, sunrise. All enabled outer predictors are single regression trees. A retained portfolio choice can itself contain an earlier learned policy; therefore the fixed-portfolio control isolates the added outer decision, rather than removing all earlier learned decisions. Requests outside the recorded graph structures and feature ranges use native settings.

The reserved test uses rational kinematic profile $h = 306$ and symbolic targets with a dot position absent from the earlier symbolic training and validation sets. The earlier reserved profiles do not enter fitting or selection. The completed comparison contains 26 batches from nine known graph structures through four loops, with one to three targets per batch. Appendix I specifies the targets and reference checks. These tests change kinematics and target positions within known structures; some rational target patterns recur across profiles. They do not test transfer to arbitrary new graphs or larger target batches.

Every method receives the same two-core allocation and an 8 GiB memory allowance. The native control and earlier cost-policy control use one Kira worker. The previous workflow, fixed portfolio, and selected portfolio can use one or two workers according to their chosen strategy. We rotate method order and repeat each method three times, with a fresh Kira process and temporary files for every attempt. Complete time includes prediction, preparation, exact algebra, coefficient checks, cleanup, and recovery. Model imports, initial model loading, reference construction, and basis discovery are separate. No coefficient cache supplies the test answers.

27.3 Complete-time results

Table 13 sums the three-run median times of the individual batches. The selected portfolio takes 85.497 seconds, compared with 108.029 seconds for native choices and 95.382 seconds for the previous workflow in the same improved executable. These results correspond to 20.9% less time than native choices and 10.4% less time than the previous workflow. The earlier cost policy takes

102.561 seconds on these same inputs. The graph-specific fixed portfolio takes 86.809 seconds, so the added portfolio decision gives 1.5% less time relative to this stronger control. Figure 63 shows where these differences occur.

Summing every timed repetition gives 19.4% less time than native choices, 9.4% less time than the previous workflow, and 0.4% more time than the fixed portfolio. The small difference from the fixed portfolio changes sign between the two summaries. These measurements establish no stable aggregate saving from the added outer prediction. The practical improvement over the previous workflow includes the revised candidate strategies and worker choices, which also benefit the fixed portfolio.

Table 13: Complete reduction times in seconds. Each row sums batch medians from matched tests with common exact preprocessing. “Fixed” selects a portfolio choice per graph using validation only.

Graph	Native	Previous	Fixed	Selected
Three-loop bubble box	2.280	2.300	2.274	2.388
Four-loop bubble box	32.420	29.726	28.981	29.129
Planar double box	9.478	8.226	8.179	7.406
Three-loop two-point ladder	23.326	17.584	14.314	14.108
Nonplanar box	28.380	25.414	20.793	20.236
Virtuality box	2.585	2.756	2.763	2.608
Pentagon	3.895	3.922	4.072	3.966
Sunrise	1.941	1.677	1.661	1.770
Three-loop vacuum	3.724	3.777	3.772	3.886
Total	108.029	95.382	86.809	85.497

All 390 timed reductions and 26 further applications reproduce the reference coefficients exactly. The applications receive only the prescribed master basis before reduction; the comparison with reference coefficients follows execution. The selected portfolio requires 3 recovery attempts across the timed tests. Its median outer decision time is 0.575 milliseconds. The CPU-time totals are 115.370 seconds for native choices and 114.666 seconds for the selected portfolio. Reporting both wall and CPU time separates elapsed-time savings from changes in CPU use, as shown in Figure 64. Every planned batch completes both reference calculations.

The dotted three-loop two-point ladder illustrates the change in the equation system. Native choices use seed limits $(10, 0, 2)$, then $(11, 1, 3)$, while the selected portfolio uses $(10, 1, 2)$, in the order $(r, s, d_{\mathrm{dots}})$. Summing the reported generated-equation counts across all attempts gives medians of 287,558 equations for native choices and 94,380 for the selected portfolio. Their complete times are 14.637 and 7.345 seconds. This count measures equation generation; symbolic simplification can require different work per equation. The CPU-time comparison also accounts for the effect of worker count.

The application also avoids loading the previous workflow until a selected strategy requires it. On the dotted sunrise batch, three rotated comparisons of a new application instance give median request times of 3.258 seconds with immediate loading and 0.727 seconds with deferred loading. These times include construction, one reduction, an exact coefficient check, and closure, while common Python imports remain outside the timer. All six requests choose the same strategy and return the same exact coefficients. This separate interface saving applies when the request does not need the previous workflow. A request that first selects that workflow pays its loading cost within the application time. The repeated portfolio timings above already keep the models loaded, so they receive no additional saving from this change.

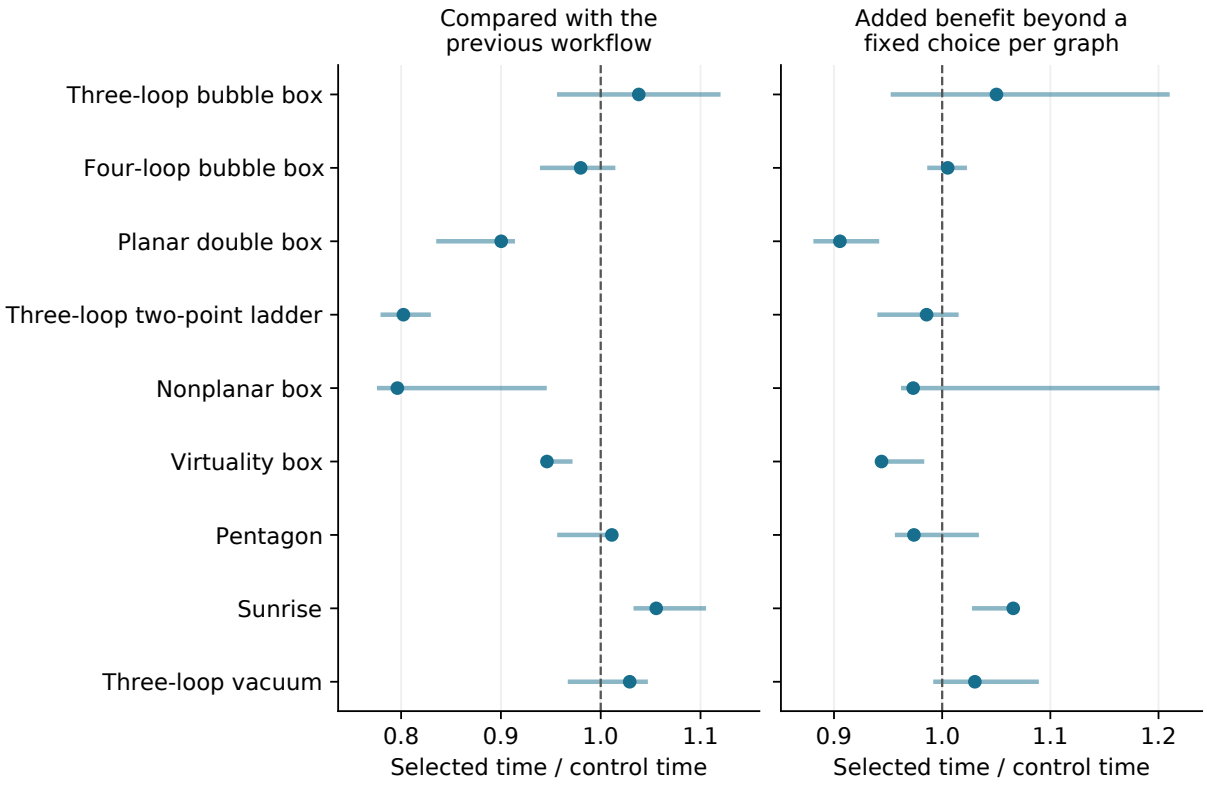


Figure 63: Selected-portfolio time relative to the previous workflow and the graph-specific fixed portfolio. Points divide sums of batch median times. Horizontal segments show the range of the three corresponding repetition totals; they are descriptive ranges, not confidence intervals. Values below one indicate a saving.

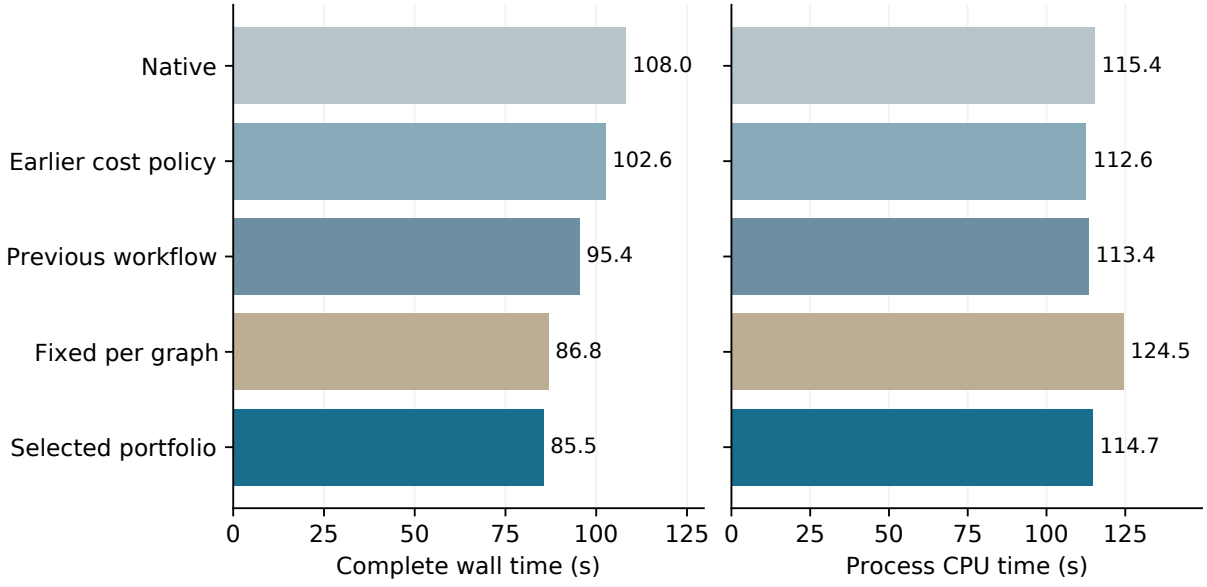


Figure 64: Complete wall time and recorded process CPU time for the same reserved batches. The five methods share exact preprocessing and a two-core allocation. CPU time counts the main driver, Kira child processes, and the timed part of the previous-workflow service. Its message conversion lies outside that service's CPU counter but remains in complete wall time. Initial loading lies outside the repeated timings.

27.4 Separating the exact implementation changes

The portfolio tests use two exact improvements in every method. We index the sector files that exist instead of searching for absent files, while retaining their original processing order. We also simplify the criterion for zero sectors by setting Feynman parameters absent from all declared top sectors to zero before forming that criterion. The complete parametric polynomials remain available for the other operations. Appendix H proves why this projection preserves the criterion in the declared sector domain, following the parametric treatment of zero sectors [49].

Separate matched tests enable these changes in sequence for connected bubble boxes. For the 3-loop batch, the complete time changes from 6.087 to 2.939 seconds with the first study's validated policy, an overall factor of 2.07. The 4-loop previous-preparation control does not complete after 1190.6 seconds across its bounded attempts. Further repetitions of this failed control are omitted, while the remaining settings are completed. The fully improved calculation takes 28.666 seconds, but the incomplete control supplies no measured overall speedup factor. With file indexing already enabled, projection changes its complete time from 98.408 to 28.197 seconds, a factor of 3.49. These comparisons include exact implementation changes and therefore do not measure an isolated ML gain. Figure 62 and Table 12 preserve the intermediate controls in the earlier cost study. The common-preprocessing comparison in Table 13 measures the subsequent portfolio choices separately.

The measurements support the selected strategies in the stated domain. Timing differences can remain when two methods choose the same strategy, and the three repetitions do not establish a significant gain for every graph. General higher-loop scattering, arbitrary symbolic kinematics, automatic discovery of a minimal master basis, and globally optimal elimination paths remain outside the demonstrated scope.

28 Reducing preparation around the learned choices

The previous comparison leaves a substantial cost even after the model has chosen a useful seed region. On a four-loop development batch, the actual row operations take a small fraction of the total time, while sector symmetry preparation and repeated equation generation dominate. We therefore retain the trained policies and change the exact work that surrounds their choices. This comparison tests whether better integration can make the same predictions more useful without adding another model or changing its weights.

28.1 Avoiding unused polynomial terms and repeated equations

Kira uses parametric polynomials to identify zero sectors and sector symmetries [2, 49]. Each inverse propagator has a Schwinger parameter, an auxiliary variable in this representation. An auxiliary inverse propagator can occur in a numerator while its index remains nonpositive throughout the declared sector domain. Its Schwinger parameter then vanishes in the sector polynomials used for these checks. The preceding implementation first constructs the full polynomials and removes the unused parameter terms from the zero-sector criterion. We now apply this substitution before polynomial construction and use the resulting restricted polynomials for both zero-sector and symmetry checks. Appendix J proves that this change gives the same polynomial in every admitted sector. The full propagator list remains available when Kira forms IBP identities and transforms numerators.

Kira also constructs selected symbolic equations to check their remaining masters, then constructs those equations again for the final reduction. We keep the unmodified exact rows from

the first construction. When the next selection returns the identical ordered list of source equations, the calculation uses those saved rows. If the selection changes or the storage allowance is exceeded, Kira regenerates the equations normally. The saved rows exist only within that reduction; no previous target answer supplies a later result. This change removes a repeated calculation of an established identity. It does not replace equation selection or symbolic elimination.

The application uses the same trained portfolio, graph-specific predictors, thresholds, seed settings, worker choices, and recovery rules as Section 27. We fit no model in this comparison. Thus any change relative to the previous application measures the exact integration changes and timing variation. A matched fixed-portfolio control measures the additional effect of the outer prediction. The earlier separate service remains available for the few requests assigned to it, and its full cost remains in the measurement.

28.2 Fresh targets and matched controls

We reserve 26 new batches from the same nine graph structures, with one to three targets per batch and loop orders one through four. The rational cases use a fresh parameter profile. The symbolic nonplanar and pentagon cases move the raised denominator power to a new position, while the virtuality box has two singly dotted denominators. Appendix J specifies these inputs. The frozen range guard can select native settings when a new target pattern lies outside its training range.

The first reference reduction uses expanded seeds with the revised preparation. A second calculation uses the previous executable and a larger numerator-rank limit to check the resulting coefficients in the same basis. Each timed method then receives the target and that prescribed basis. Reference coefficients enter only the comparison after the calculation. The test checks reduction to this basis, rather than proving that the basis is minimal.

We compare four methods: the previous application; native choices with revised preparation; a graph-specific fixed portfolio with revised preparation; and the selected portfolio with revised preparation. The fixed portfolio can contain an earlier learned policy, so it isolates the added outer choice. All methods use the same two-core allocation, memory limits, and exact checks. Native choices use one Kira worker, while the portfolios retain their chosen one or two workers. The revised controls share the application's input checks and portfolio proposal; the native and fixed controls then override that proposal. We report the effect of removing this common overhead below. Each attempt starts a fresh Kira process with fresh temporary files. Three repetitions rotate method order. Complete time includes decisions, input preparation, algebra, coefficient checks, cleanup, and unsuccessful attempts. Initial model loading, reference construction, and basis discovery remain outside these repeated timings.

28.3 Results and attribution

Table 14 sums the median time of each batch. The revised selected application takes 64.183 seconds, compared with 79.784 seconds for the previous application. This is 19.6% less time on the measured collection. The selected actions agree with the previous application for every batch and repetition, so this improvement comes from the exact preparation changes around the existing models. Summing all repetitions gives 19.6% less time than the previous application.

With each candidate implemented in the same way for all controls, the selected method uses 26.9% less time than native choices and 4.7% less time than the fixed portfolio. Across all repetitions, its difference from the fixed portfolio is 5.0% less time. These controls keep the effect of learned decisions separate from exact reuse. The graph results in Figure 65 show the

Table 14: Complete times in seconds, summed over batch medians. The previous column uses the earlier preparation. The other columns enable both exact changes for direct Kira attempts; the earlier separate service retains its preparation. Fixed denotes the validation-selected portfolio choice for each graph.

Graph	Previous	Native	Fixed	Selected
Sunrise (2 loops)	1.742	1.706	1.806	1.793
Planar double box (2)	6.155	8.501	7.123	5.953
Vacuum (3)	2.788	2.678	2.678	2.636
Two-point ladder (3)	13.393	23.400	12.917	11.581
Nonplanar box (2)	19.794	28.662	20.478	20.052
Pentagon (1)	3.094	2.989	3.003	3.022
Virtuality box (1)	2.172	2.216	2.232	2.229
Bubble box (3)	2.225	1.617	1.735	1.647
Bubble box (4)	28.421	16.078	15.409	15.271
Total	79.784	87.847	67.381	64.183

measured savings and regressions; a smaller total does not imply that every batch becomes faster.

The common interval for input checks and the outer proposal is at most 2.854 milliseconds per revised request. Removing this entire measured interval from the native and fixed controls gives optimistic totals of 87.828 and 67.363 seconds, respectively. The selected method then uses 4.7% less time than this optimistic fixed control. This subtraction removes useful checks as well as the unused proposal and therefore serves as a sensitivity bound, rather than a separately timed implementation.

All 312 timed reductions and 26 further basis-only applications reproduce their checked reference coefficients exactly. The reference checks, timed tests, and applications compare 8,086 target–master coefficients. The revised selected method needs 6 recovery attempts across its timed tests. The recorded process-tree memory peak among the timed reductions is 961.8 MiB. The selected method takes 92.782 seconds in summed batch median CPU time, compared with 107.054 seconds for the previous application. The bounded replay mechanism activates 117 times across the timed controls and retains at most 0.221 MiB of accounted row data in an activation.

These results support the revised application for this measured collection. They test new parameters and target patterns within known graph structures, with supplied master bases. They do not establish general high-loop scattering performance or a globally shortest reduction. The earlier row-model result remains unchanged, so the application continues to use native row elimination.

29 How far the improvement extends

The preceding gain was measured on known graph structures with a supplied master basis. To test its wider use, we now vary the physical structure as well as the loop order and include calculations that find their own basis. We keep the deployed models fixed so that these measurements test the current method on new inputs. The study contains 90 native-family requests, of which 54 fit the application's current index limit. It also tests a separate restricted vacuum construction at every loop order from one through twelve. This is a finite coverage study; a loop-order label alone does not establish coverage of all graphs at that order.

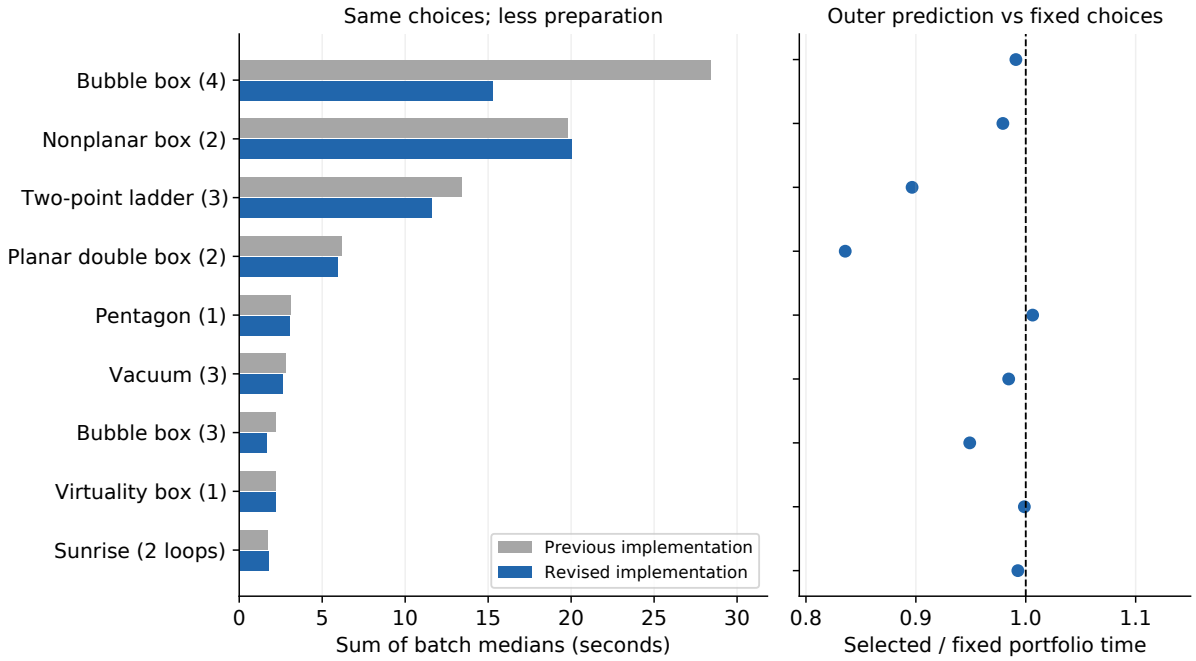


Figure 65: Left: the same learned actions run with the previous and revised preparation. Right: the selected-to-fixed time ratio with revised preparation; the dashed line marks equal time. Each point or bar uses sums of batch medians within a graph. The right panel isolates the additional outer prediction.

29.1 Physical families and complete-time controls

The native grid includes vacuum families with one or several masses, massless and massive two-point families, four-point ladders, and boxes with bubble subgraphs. We add planar, nonplanar, and five-point examples with independent symbolic kinematic parameters. Appendix K gives the denominator definitions, parameters, and targets. The 82 initial requests contain a sector-corner integral, one raised propagator power, and one numerator integral when an auxiliary denominator is available. Some massive two-point batches return every target unchanged as a master. We therefore preserve those results and add 8 separate batches containing every distribution of up to two dots over the physical denominators. The additional batches cover all six loop orders accepted for that two-point family, plus symbolic cases at one and two loops. Their inputs are fixed before their own comparison, and no model is refitted from either stage.

We first calculate a reference with expanded seeds and the revised implementation. A second calculation uses the previous executable and a larger numerator-rank bound, with the reference basis prescribed. Both must complete and reproduce the same coefficients before a case enters the repeated comparisons. This checks the reductions in the stated basis; it does not prove that the basis has the smallest possible size. Every native attempt has a 120-second allowance, a 6 GiB address-space limit, and a 5,200 MiB process-tree memory limit. An attempt that reaches a limit remains a failed result. Inputs with more than 31 indices exceed the current application's accepted domain and receive no timing estimate.

The first comparison runs the saved original Kira executable, the previous improved implementation, and the revised implementation with the same expanded seed region. These calculations receive no master list, so their times include basis discovery, equation generation, elimination, coefficient checks, and cleanup. This comparison measures the complete effect of the implementation. It makes no learned decision. The second comparison supplies the checked basis

and runs the previous learned application, the revised learned application, and a native-choice control with revised preparation. Its times also include prediction and unsuccessful attempts. Both comparisons use three repetitions with rotated method order and fresh Kira processes. Initial caller imports and model loading remain separate from the repeated application times.

An input audit finds two box-with-bubble definitions that match saved training structures exactly but carry new family names. The initial application treats these names as unknown. We add a lookup that resolves a unique match of the ordered momentum forms, sector definition, and zero or nonzero mass pattern before prediction. Numerical mass and kinematic values remain free, as in the existing model signature, and all original feature-range checks remain active. Unmatched structures retain the native fallback. The lookup uses no measured time or reference coefficient. We repeat the paired application comparison for both names and use these corrected pairs in the application totals. The initial unknown-name measurements remain available separately. This correction allows the existing models to recognize their own domain; it does not train a new model or infer equivalence between different momentum representations.

29.2 Coverage and measured time

Figure 66 shows the rational grid, including its failures and unsupported inputs. The reference checks pass for 41 requests, and 41 requests complete all three revised full-calculation repetitions. Of these, 39 contain a nontrivial reduction and 2 contain only identity outputs. We exclude identity-only requests from speedup ratios. The supplied-basis selected application completes all repetitions for 41 requests. The remaining requests retain their stated failure or support-limit classification. The timings therefore do not establish a nontrivial reduction for every entry in the grid.

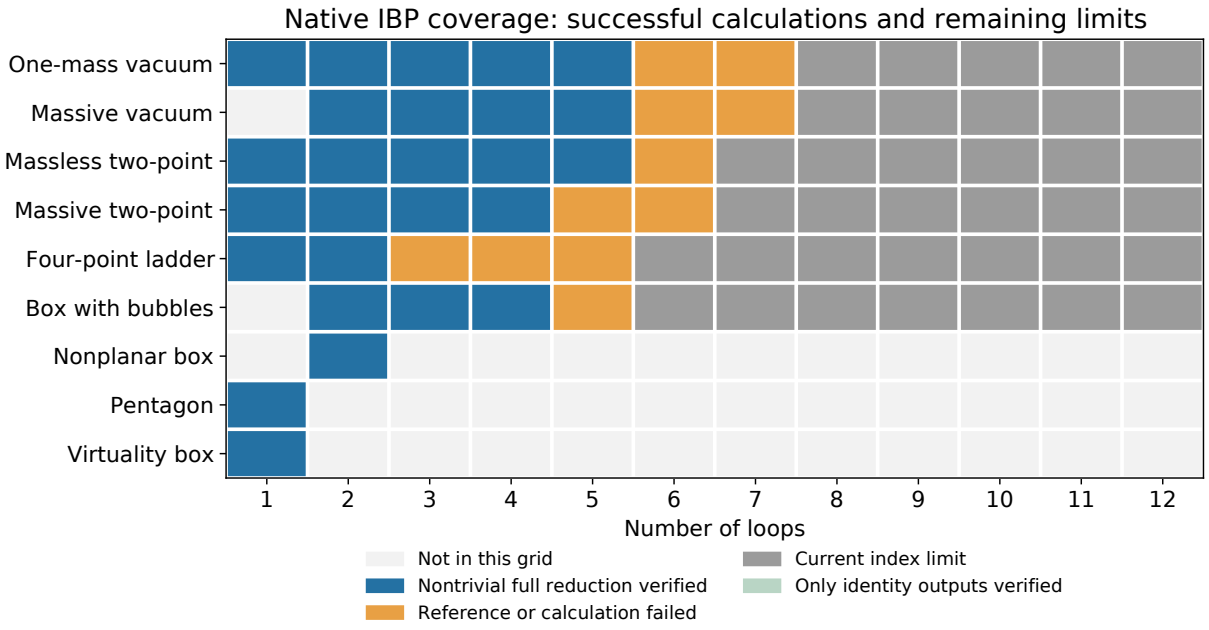


Figure 66: Native IBP coverage in the rational grid. Blue requires a checked reference, a nontrivial reduction, and three exact revised full calculations including basis discovery. Pale green records only identity outputs. Orange records a failed reference or calculation within the stated limits. Gray records the current index limit. A cell takes the strongest verified status across initial and additional batches. Blank cells are absent from this grid; the two coincident one-loop definitions appear only once. Symbolic requests enter the timing table and the text totals.

Table 15 uses a separate matched set for each comparison. A request enters a row only when both methods complete all repetitions and the reduction contains a nontrivial coefficient. We sum the median time of each request, so a smaller ratio means less total time on that completed set. The revised full calculation takes 88.2% less time than the saved original Kira executable and 54.3% less time than the preceding improved implementation on their respective matched sets. The supplied-basis selected application takes 54.1% less time than the previous application and 0.1% less time than the revised native control. These aggregates retain slower individual requests and do not turn unsuccessful cases into speedup estimates.

Table 15: Summed request median times in seconds. The revised column uses full revised calculations for the first two rows and the selected application for the last two. Each row names its own control and completed request count.

Control	Requests	Control time	Revised time	Ratio
Full: original Kira	37	369.363	43.750	0.118
Full: previous implementation	39	146.541	66.897	0.457
Basis supplied: previous application	39	146.342	67.195	0.459
Basis supplied: revised native control	39	67.269	67.195	0.999

The revised native control passes through the same input checks and portfolio proposal before overriding the proposed action. For a conservative comparison, we subtract that entire recorded interval and the family-name lookup from its time, while retaining every selected-application cost. The largest removed interval is 2.124 milliseconds. This gives an optimistic native total of 67.259 seconds, against which the selected application takes 0.1% less time. The subtraction also removes useful input checks, so it supplies a sensitivity bound rather than a separately timed implementation.

The selected application and revised native control differ by only 0.1% in the summed medians. With only three repetitions, this small difference does not establish a reliable additional learned gain on the wider grid. The larger improvement against the preceding application includes the exact preparation changes.

The repeated native comparisons contain 752 recorded requests, with 750 exact completions and 2 unsuccessful requests. The largest recorded process-tree memory use among these timings is 961.8 MiB. The grid includes new graph definitions, so the existing domain guard often retains native choices. This is part of the deployed method and prevents us from describing the full implementation gain as a gain from newly learned decisions.

Table 16 also includes unsuccessful first timed requests. Its elapsed totals describe the cost actually incurred on that common reference-verified collection. They are not completed-reduction speedups, since a failed request supplies no accepted answer. This view complements the paired timing table and prevents completion failures from disappearing in an aggregate improvement claim.

29.3 Every tested loop order in the restricted vacuum class

The existing vacuum recurrence has a different domain: unit powers on the massless lines and several massive denominators with the same total loop momentum. We construct three new mass profiles with two, three, or four distinct masses and six target exponent vectors per profile. We compare the saved original Kira, dense and selected supplied-identity systems, the frozen learned identity selector, and the deployed exact recurrence. Each method receives the same targets and passes the independent coefficient check derived in Section 6.12. The individual requests cover every loop order from one through twelve. Separate shared requests ask for all

Table 16: First timed requests on the reference-verified collection. Elapsed seconds include failed requests. Later repetitions are excluded from this table so that each method receives one request per case.

Method	Requested	Exact completions	Elapsed seconds
Full original Kira	41	39	623.836
Full previous implementation	41	41	148.606
Full revised implementation	41	41	68.568
Previous application	41	41	150.733
Selected application	41	41	68.497
Revised native control	41	41	68.721

twelve orders together.

The study records 585 exact completions in 585 vacuum requests. Table 17 separates individual-loop requests from shared requests. In the individual-loop set, learned identity selection takes 8.2% more time than the fixed identity rule. The deployed recurrence takes 99.8% less time than the saved original Kira calculation. This gain uses the exact restricted-family relations and their implementation; the deployed recurrence uses its existing fixed policy. The native study and this supplied-identity study therefore establish different forms of coverage.

Table 17: Restricted vacuum timings in seconds, summed over request medians. There are 36 individual-loop requests and three shared requests, each repeated three times. The original control solves separately for each requested loop order. The other methods can use the exact shared variable $z = L(d-2)$. All times include coefficient verification.

Method	Individual loops	Shared loops
Original Kira, separate loop orders	122.179	129.306
Improved Kira, dense identities	14.626	1.310
Fixed identity selection	13.311	1.398
Learned identity selection	14.399	1.327
Deployed exact recurrence	0.189	0.069

The broader tests identify where the current application completes an exact reduction, where its cost changes, and where its support or resource limits stop the calculation. They retain the earlier known-graph gain while restricting its interpretation to measured inputs. General high-loop scattering, arbitrary numerator rank, cuts, symbolic propagator powers, and every possible mass or kinematic assignment remain outside a universal verified claim.

30 Conclusion

We integrated seed and source-row policies with exact native Kira reduction and reduced their evaluation cost while retaining the same decisions. The latest seed implementation evaluates independent neural outputs together, and the row implementation combines Euclid's modular inverse with direct integer evaluation and shared coefficient storage. All probability vectors agree in the thirty-six development contexts. Checks on 100,000 inverses and 1,000 sparse-row groups also agree.

The new seed component has a time ratio of 0.423 to the preceding shared implementation. The row-feature component has a ratio of 0.803. These component savings require separate complete-reduction measurements because model evaluation is only one part of the calculation.

The seed method has a complete-time ratio of 0.990 to its preceding implementation, with

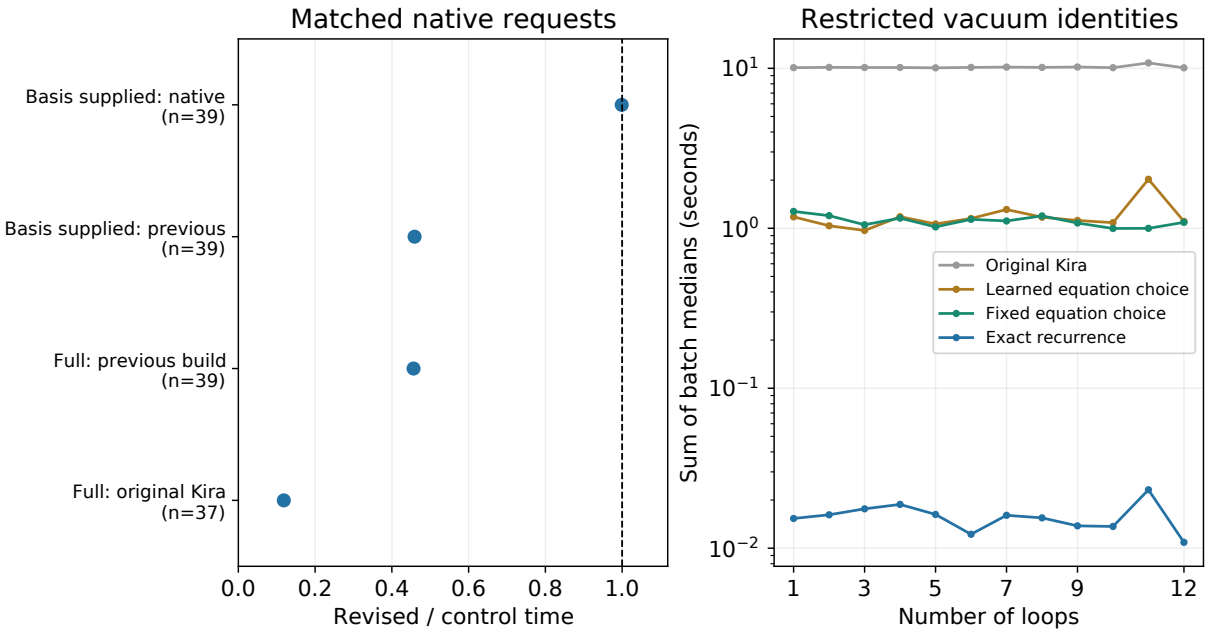


Figure 67: Left: revised-to-control time ratios on matched completed native requests; the dashed line marks equal time. The request count changes with each pair's completed coverage. Right: restricted vacuum time at each loop order, summed over the three mass profiles. The logarithmic vertical axis keeps the exact recurrence and Kira-based calculations visible. These panels use different integral definitions and cannot be combined into a universal speedup.

descriptive interval $[0.977, 1.003]$. Its ratio to the new native control is 1.096, with interval $[0.963, 1.275]$.

The row method has a complete-time ratio of 0.990 to its preceding implementation, with descriptive interval $[0.971, 1.010]$. Its ratio to the new native control is 0.982, with interval $[0.959, 1.003]$.

The combined method has a complete-time ratio of 0.984 to its preceding implementation, with descriptive interval $[0.965, 1.007]$. Its ratio to the new native control is 1.100, with interval $[0.966, 1.279]$.

The descriptive complete-time intervals include one for all three comparisons with the preceding implementation. Component evaluation is faster, and the complete-workflow gain remains uncertain at this sample size. The seed and combined methods still have aggregate costs above the native control. The row-only comparison with native choices also includes one in its interval.

All 144 validation workflows and 640 confirmation workflows pass the exact checks and preserve the recorded decisions and operation counts. These times include IBP generation, model evaluation, exact algebra, checks and fallback. The implementation comparison reuses eight physical profiles and retains the weights, thresholds and routing selected before confirmation [35].

The data construction provides 17,100 family instances, 57,979 family-local master entries and 2,301,272 targets with recorded identities and reduction paths. These paths provide valid reference constructions and costs. On 690,382 held-out targets, the seed model reaches 96.318% F_1, and its predicted stored equations supply complete master-span reductions for 94.515% of targets. The full candidate pool supplies the remaining reductions. These label and sufficiency measures require separate runtime tests; the earlier cost comparisons and their negative results remain part of the study [15, 19, 22, 26].

For production use, the interface needs a prescribed master basis and can calculate target coefficients without their reference answers. Native exact IBP operations determine the coefficients,

and explicit output-basis admission detects incomplete seed sets. Eight application checks pass independent coefficient comparisons after execution with reference answers withheld from the reducer. The report keeps this native algebraic assurance separate from the additional reference comparisons used in the benchmark.

The original collection through twelve loops uses restricted coupled vacuum families. The native performance comparisons cover physical two- and three-loop inputs at fixed rational kinematics with symbolic d. The ladder topology is absent from that row-policy training and validation sets. Those native timing comparisons retain their fixed-kinematic scope. Section 19 extends the tests to new scattering topologies and independent symbolic invariants and supplies finite optimality certificates. General high-loop scattering and optimality over unrestricted identity pools remain open.

The third model selects generated identities for the requested targets. The broad test found that incomplete vacuum filters added fallback cost. A restricted route was then fixed and tested on six fresh sunrise profiles. It gives a complete-time ratio of 0.994, with descriptive interval $[0.991, 0.997]$ and an equation-preparation ratio of 0.785, with descriptive interval $[0.776, 0.794]$. All 162 confirmation comparisons, 36 checks with the two earlier models and six applications pass exact coefficient checks. The preceding deployed equation policy uses the recorded 21-target sunrise domain and retains native choices elsewhere [36]. The measured three-model composition is slower, with time ratio 1.317 to the earlier two-model configuration. That preceding interface therefore retains native equation choices when either earlier learned policy is active.

Later training moves the equation decision before coefficient construction. The trees use 17 seed, operator and target features and train on 153,124 native candidates in eight physical profiles. Validation selects early filtering with native seed and row choices for the recorded double-box, vacuum and ladder batches and retains the earlier sunrise route. On eight fresh profiles, the deployed policy gives a complete-time ratio of 0.943 to native choices, with descriptive interval $[0.915, 0.970]$, and a preparation ratio of 0.638, with interval $[0.546, 0.734]$. Its complete-time ratio to the preceding release is 0.943. All 280 comparisons and eight basis-only applications pass independent exact checks. These results include inference and fallback costs and support the measured fixed-kinematic domain. The validation comparisons with the seed and row models remain recorded, and their added costs keep those combinations out of the selected route [37].

The higher-loop extension uses the exact variable $z = L(d-2)$ for the restricted coupled vacuum family. The trained equation selector and a shared symbolic reduction cover every tested order from one through twelve, with exact applications at 13, 16, 24 and 32. The learned route with separate reductions at each loop order has complete-time ratio 0.911 to the supplied full pool. Validation selects a simple rule for the mass pair for deployment; its shared route has complete-time ratio 0.155 to separate reductions with the full equation pool. Before Python startup and test-record archiving, the learned calculation has time ratio 0.943; time inside Kira has ratio 0.965. All 96 workflows pass exact checks. These results establish the stated coefficient reuse within this family and cost comparisons. General native IBP generation at higher loop orders remains outside this test domain [38].

The next extension learns shared equation choices for entire target batches and reduces repeated exact checks in the common variable z. Its four-tree cost model trains on 420 batches and 6,300 candidate orderings. Combining independent profiles in one Kira call preserves separate master bases and reduces repeated startup work. On the final twelve reserved profiles, the selected combined route has complete-time ratio 0.220 to the preceding separate workflow, with descriptive interval $[0.171, 0.270]$. All 460 reserved applications and the additional integration checks pass exact comparisons; the integration tests also verify the full-system fallback. Compiled and Python inference produce identical equation systems. The comparisons separate model decisions, shared exact arithmetic and shared Kira calls, and apply to the stated restricted vacuum

family [40].

The direct-reduction extension evaluates selected triangular identities with exact integer polynomial arithmetic and checks every master coefficient through an independent Taylor construction. Validation selects the simple direct route. On eight fresh three-profile batches, its calculation-time ratio to the preceding combined workflow is 0.306, with descriptive interval $[0.246, 0.373]$. The full worker-time ratio is 0.430. All 200 reserved calculations pass exact checks, and an injected failure recovers through Kira. The comparison does not establish a further complete-time advantage from learned ordering over both direct controls. The direct saving follows mainly from the exact family-specific recurrence; the native IBP-generation results retain their earlier scope [41].

The polynomial-reuse extension shares the exact transformations for $z = L(d-2)$ and the Taylor terms in the independent check. On eight new reserved batches, both changes together have calculation-time ratio 0.372 to the preceding direct route and full worker-time ratio 0.682. Keeping one process for several batches gives combined full-time ratio 0.365 across the three specified request streams. These streams share inputs, so this last ratio describes those sequences. All 352 reserved applications pass exact checks, including an independent audit of 28860 expanded coefficient positions. The request-interface checks also pass, including recovery through Kira after an injected failure. The changes reuse exact mathematical work and process startup; the previous model and ordering choice remain fixed. The supported domain remains the restricted coupled vacuum family, with native IBP-generation results retaining their earlier scope [42].

The compiled-arithmetic extension retains integer polynomial coefficients, compiles exact loop-order expansion and supplies lossless binary coefficient output. A 19-node regression tree learns batch strategies from 1,440 complete warm-worker measurements on 96 training batches. The first frozen test finds no established complete-time gain from the initial selected integer route once output reading is included. Compiling the binary reader then permits a new validation and independent test. Validation selects compiled expansion and binary output with a fixed recurrence rule. Its full process-time ratio to the preceding arithmetic is 0.850, and its ratio including output reading is 0.858, with descriptive interval $[0.800, 0.914]$. The deployed persistent-worker ratio including reading is 0.725 across three disjoint test streams. All 948 reserved batch calculations across both studies pass exact checks. The learned-to-fixed time ratio is 1.017, with descriptive interval $[0.969, 1.070]$. This interval includes one, so the reserved comparison does not establish a further time reduction from learned choices. The deployed default uses the fixed validation choice. The new tree, fixed-rule controls and earlier results remain preserved, and the supported domain remains the stated coupled vacuum family [44].

The direct cumulative comparison uses the saved original Kira executable and the final deployed route on identical restricted vacuum targets at every loop order from one through twelve. Eight independent mass-profile groups and five repetitions of three methods give 120 complete workflows. Including process startup, output saving and output reading, the final-to-original time ratio is 0.001839, corresponding to 543.815 times faster execution and 99.82% less time. The descriptive 95% speedup interval is $[488.134, 605.064]$. All matched outputs agree exactly, and the independent audit checks 27960 nonzero target–master coefficient polynomials after loop-order substitution. This directly measures the cumulative change without multiplying earlier ratios. The gain includes shared symbolic reductions, direct checked recurrences, compiled arithmetic and output changes. The selected route uses a fixed recurrence rule and makes no Kira call on these successful cases, so the result retains the restricted supplied-identity scope [45].

The scattering extension adds new nonplanar four-point and five-point inputs and exact reductions with independent symbolic invariants. All seventy comparison workflows pass, while the frozen equation filter requires native fallback in all thirty-five learned workflows and has complete-time ratio 1.252 to native choices. The new symbolic-capable application therefore de-

faults to native choices for these domains. Separate complete certificates prove seven-operation minima for two small physical systems when equation-subset selection and the stated pivot actions are allowed. The certificates cover finite source pools with three nonmaster columns; their costs and native coefficient checks provide exact reference labels without asserting a general minimum for scattering reductions [46].

The complete-cost study in Section 20 compares twelve fitted predictors and preserves the selected model during testing. All 620 timed reductions and 31 basis-only applications pass exact checks, but the learned choice adds no reliable gain over integrated native controls. Combining two compatible target requests gives 48.3% less time, or 1.93 times faster reduction, in 200 exact comparisons across seven graph structures. The standard application retains the best fixed training strategy, and the learned route remains optional. Thirty further command checks verify the single-batch, learned joint and default joint applications without supplying target coefficients. These measurements cover native calculations at one through three loops. The earlier restricted vacuum result through twelve loops remains a separate result.

The graph-specific study in Section 21 fits seventy candidates and gives 6.3% less time relative to native choices on 720 reserved trials. Its ratio to a fixed strategy per graph is 0.9441. All trials and 43 additional applications pass exact checks. The study covers the seven known graphs at one through three loops with new rational profiles and repeated target patterns. The latest comparison keeps every trained model fixed and reduces exact preparation around its decisions. On 26 new batches, complete time changes from 79.784 to 64.183 seconds, or 19.6% less time. Section 28 gives the matched controls and graph results. With the revised preparation held fixed, the selected portfolio uses 4.7% less time than the fixed portfolio by summed medians and 5.0% less time across all repetitions. The previous portfolio study in Section 27 remains a separate comparison on different targets.

The broader coverage study in Section 29 defines 90 native-family requests through twelve loops. The current interface accepts 54 of them, and 39 complete three checked nontrivial full reductions with revised preparation. On 37 matched completed requests, the revised full calculation takes 88.2% less time than the saved original Kira executable. The supplied-basis selected application takes 0.1% less time than the revised native control on its separate matched set. Unsupported, failed, and identity-only requests remain outside these timing ratios. The small difference from the revised native control establishes no reliable additional learned gain on the wider set.

31 Learning seed bounds from complete reduction time

The broader test found that the earlier models often kept native choices and provided little additional benefit. We therefore train a new model for a decision that directly changes equation generation: the seed limits for the requested target batch. The model chooses these limits before Kira starts. All earlier seed, row, and equation models are disabled in this comparison, and every method uses the same exact preparation. The fixed-control comparison measures the extra benefit of a learned choice after tuning the seed settings.

31.1 What the model learns and how Kira uses it

A target batch can need a larger numerator bound even when its denominators require no further increase. Other batches can use fewer seeds in lower sectors. Kira provides bounds on positive powers, numerator rank, and extra denominator powers, as well as a separate restriction for lower sectors [9]. These settings define ten candidate configurations in the present study. They include unshifted bounds, selective expansion, reduced bounds, full expansion, and four

configurations with lower sector truncation. Appendix L specifies every configuration and its recovery sequence.

The model receives 27 quantities that describe the graph and target indices. For each target $\boldsymbol{a}$, the extra denominator powers are $\delta(\boldsymbol{a}) = \sum_i \max(a_i - 1, 0)$, and the numerator rank is $\rho(\boldsymbol{a}) = \sum_i \max(-a_i, 0)$. The first sum counts denominator powers beyond one, often called dots, and the second is the total numerator power. Together with the loop count and denominator structure, these quantities help the model predict the relative complete time of each configuration. Its training labels come from actual Kira reductions, including any failed first attempt and the subsequent recovery. The prediction therefore concerns the cost of reaching a checked answer, rather than the number of selected seeds alone. Related work uses learned search to discover sparse seed patterns [50, 51]; the present experiment uses Kira's existing seed-region controls and does not implement tube seeding. The application checks the ordered momentum structure and the target-count and overall index-power ranges recorded in training and validation. Inputs outside this domain use native bounds and the same exact recovery procedure.

We compare 30 fitted models: regression trees, random forests, extremely randomized trees, and multilayer perceptrons [30, 48, 39, 52]. Each model predicts ten costs. Validation selects a random forest with a gate on predicted log cost of 0.0. The selected predictor uses 48 trees, maximum depth 8, and at least 2 training cases per leaf. The model receives no reference coefficients, case identifier, or kinematic profile number. Its inputs and fitted parameters remain fixed during the reserved test. Details of the objective, model choices, and selection rule appear in Appendix L.

For each request, the application loads the fitted parameters, calculates the features, and selects a configuration. Kira then generates IBP equations and performs exact algebra in the supplied master basis. If the chosen seed region cannot reach that basis, the application tries the unshifted bounds and then full expansion, skipping duplicate configurations. A request that exhausts these attempts remains unsuccessful. Coefficient comparisons occur after Kira has produced its result and are included in the reported time.

31.2 Independent inputs and controls

The input grid contains ten graph structures at one through four loops, with one to three target integrals per batch. It includes vacuum and two-point integrals, planar and nonplanar boxes, a pentagon, a box with an external virtuality, and boxes with bubble subgraphs. We define 74 training requests, 37 validation requests, and 74 reserved requests at separate rational kinematic profiles, keeping the dimension symbolic. After reference checks, complete candidate comparisons, and removal of identity-only cases, 68 training batches and 34 validation batches supply the fitted comparison. Target positions change with the profile, but graph structures and target patterns are shared. This tests transfer to new requests within known structures, not arbitrary new topologies or general symbolic kinematics.

Each reserved request first receives a reference reduction and a second exact check with a larger numerator bound. These preparation calculations provide a common basis and comparison coefficients. Agreement after one enlarged seed region does not prove that this basis has the smallest possible size. They remain outside the timed application, whose purpose is reduction in an available basis. The tests therefore include native IBP generation but do not include finding the master basis from the beginning. Some target patterns coincide when a family has no auxiliary numerator. Each declared batch remains a separate workload request, and the uncertainty calculation across graphs keeps such related requests together.

The first control uses Kira's native equation and elimination choices with the unshifted target bounds and the common expansion fallback. This is a specified seed policy, not a claim that

Kira has a universal default seed region. The stronger fixed control chooses the cheapest of the same ten configurations separately for each graph, using both training and validation costs. It performs no model prediction and can use lower sector truncation. The learned method may change its choice between target batches in the same graph. All three methods use the same executable, exact optimizations, one Kira worker, basis, coefficient checks, and resource limits. Five repetitions rotate their order on the same node. The timer includes model loading, prediction, equation generation, exact elimination, retries, verification, and cleanup. Initial Python imports occur before this timer.

31.3 Measured benefit and remaining limits

Of the 74 reserved requests, 68 supply matched nontrivial reductions for all three methods and all repetitions. The record also contains 4 failed reference checks, 0 incomplete timed requests, and 2 matched identity-only requests. The full timed record contains 1050 exact completions in 1050 requests and 23295 coefficient comparisons. Failed or identity-only requests do not enter the paired speedup estimate.

Summing the median time of each matched batch gives 164.926 seconds for the control with native bounds, 119.717 seconds for the tuned fixed control, and 119.081 seconds for the learned choice. The learned method therefore takes 27.8% less time than the control with native bounds and 0.5% less time than the tuned fixed control. The corresponding speedup factors are 1.385 and 1.005. Table 18 gives the graph results, including slower groups. These are complete application-time comparisons on the same inputs; they are separate from the earlier exact-code speedups.

Table 18: Sum of batch median times in seconds for the new seed-bound comparison. Every row uses matched nontrivial reductions. The fixed control uses one configuration per graph.

Graph	Loops	Batches	Native bounds	Fixed	Learned
Massive sunrise	2	8	3.90	3.87	3.93
Planar double box	2	8	15.22	13.56	13.40
Massive vacuum	3	8	6.35	6.42	6.55
Massive three-loop ladder	3	8	37.72	21.21	21.18
Nonplanar box	2	8	31.74	27.77	28.00
Pentagon	1	8	5.13	5.14	5.17
Box with virtual leg	1	8	3.95	3.96	4.02
Box with bubbles, 3 loops	3	6	12.20	6.57	6.59
Box with bubbles, 4 loops	4	2	43.32	27.16	26.68
Massless two-point, 4 loops	4	4	5.40	4.06	3.55

The learned-to-control time ratios have descriptive 95% intervals $[0.625, 0.885]$ against native bounds and $[0.980, 1.006]$ against the tuned fixed settings when whole graphs are resampled. Resampling repetitions within the fixed workload instead gives $[0.716, 0.727]$ and $[0.984, 1.006]$. The first calculation describes variation across this small graph collection; the second describes observed timing variation on these particular inputs. Neither interval proves a speedup on all integrals.

The summed generated-equation count changes from 2,029,994 under native bounds to 1,089,074 under learned choices, including equations from failed attempts. Average parameter loading and prediction take 7.067 and 0.836 milliseconds per learned request. Figure 69 shows both the time ratios for each batch and the equation counts. The domain check and its time remain part of every learned request. The largest observed process-tree memory use is 946.8 MiB within the common limit.

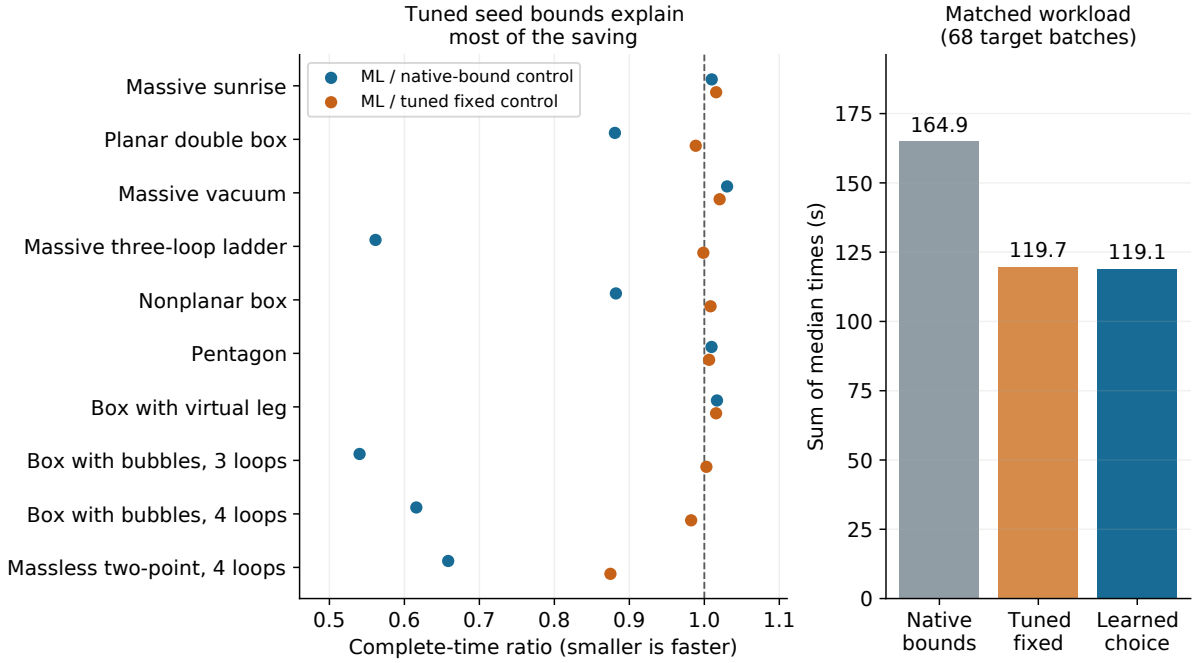


Figure 68: The left panel shows how the learned choice changes complete time for each graph; a ratio below one means a saving. The right panel shows the summed matched workload time. Both controls disable all learned choices. The graph labels refer to the explicit denominator definitions in the appendices.

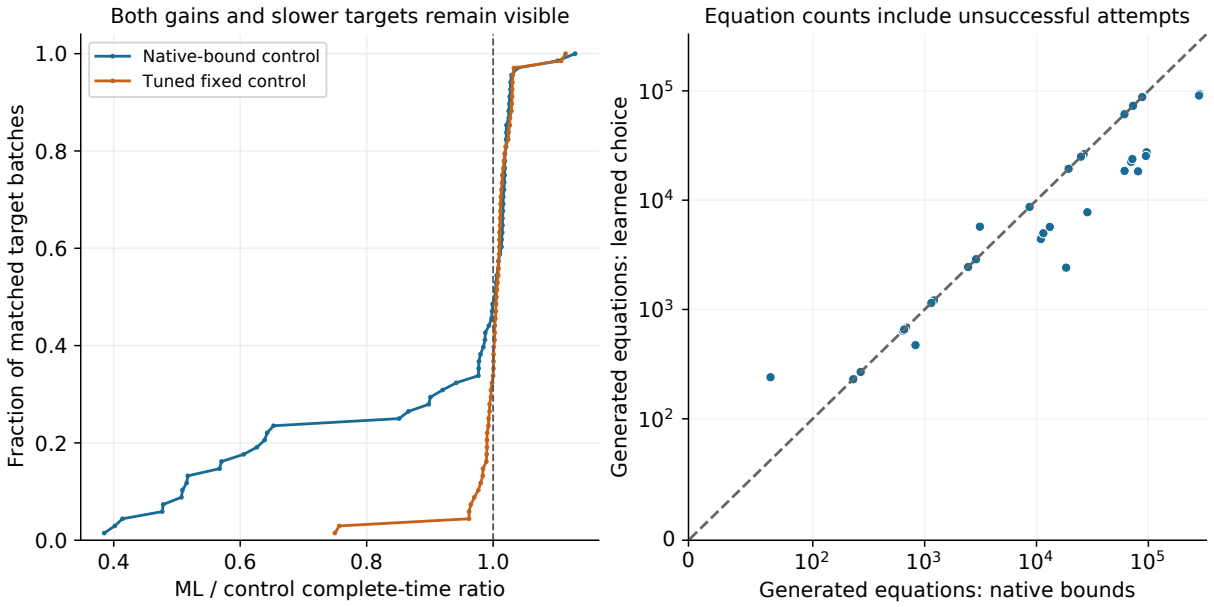


Figure 69: Left: the distribution of batch time ratios retains every matched batch, including regressions. Right: generated equations under the learned and native-bound choices, with retries included. Points below the diagonal use fewer equations. The axes use a linear scale near zero and a logarithmic scale above 100 equations. Equation count explains part of the timing change but does not replace the complete-time measurement.

Development, training-cost collection, validation, and their reference calculations take 1.37 hours of recorded calculation time, including 5.68 seconds for model fitting and selection. If future requests have the same mean saving as this matched workload, approximately 7295 requests would recover that cost relative to native bounds and 525989 relative to the fixed control. This extrapolation uses the measured workload and the derivation in Appendix L; it is not a prediction for a different workload. The interval against the tuned fixed control includes equal time. Thus, this test establishes a saving from tuned seed settings but does not establish a reliable extra saving from learning which setting to use. The wider high-loop and symbolic coverage results retain the separate scope given in Section 29.

31.4 Fresh application comparison

A separate comparison starts a new application process for every request. It uses one fixed target pattern from each graph at the first reserved profile and three repetitions. The timer includes Python imports, input parsing, model loading, calculation, result storage, process exit, and coefficient checks. On 9 matched nontrivial batches, summed median time is 29.648 seconds for native bounds, 20.913 seconds for the previous published application, and 20.797 seconds for the new model. This is 29.9% less time than the fresh native-bound application and 0.6% less time than the previous application. The previous application retains its own validated portfolio and worker choices. Its comparison therefore measures the deployed workflow as a whole; the primary comparison above isolates the new learned choice.

32 Learning which equations to construct after seed tuning

The seed-bound experiment shows that tuning a fixed configuration explains most of its time saving. We therefore keep those tuned seed limits and train a model for a finer decision: whether to construct each equation inside the selected region. This comparison asks whether learned equation selection supplies an additional saving after the coarse seed region has already been improved.

32.1 Model and integration with exact reduction

Each candidate consists of a seed integral and one IBP or Lorentz-invariance operator. A classification tree uses the seventeen quantities in Table 28, including seed powers, target powers, distance to a target, and operator index. All these quantities are available before symbolic coefficient construction. The tree follows feature-threshold tests to a terminal node, called a leaf. The leaf score is the weighted positive fraction of its training examples, and Kira constructs the equation when that score meets the selected threshold. We also test a learned seed choice that retains every operator from a seed when at least one operator receives a score above the threshold. This uses the same trained scores and keeps more identities from each retained seed, which can prevent a failed reduction. Validation compares both uses of the model. Native symmetries, exact elimination, and back substitution remain in use. A failed attempt triggers the same tuned seed configuration with the filter disabled, followed by native and expanded bounds if needed. Thus the reported cost includes the consequences of an insufficient equation set.

The training label records whether the equation's integral support appears in Kira's final selected system. Support means the set of integrals with nonzero coefficients. This label is a proxy for usefulness and does not identify a unique necessary equation set. We collect new labels with the tuned seed settings because changing those settings changes which equations Kira can select.

Collection uses a Kira build that records selected supports, and every collected reduction must agree with the independent exact reference. The timed comparisons use the same current build for both methods. The earlier filters frequently need a complete retry on these inputs, so their previous performance does not justify using them without new validation.

The four-loop box with bubbles has no completed label collection within the resource allowance, so it uses unfiltered reduction in this test. We preserve those collection failures. For the nine remaining structures, we fit eight shared classification trees and eight separate trees per graph [30, 52], with maximum depths 6, 10, 14, or 18 and minimum leaf populations 4 or 16, and a limit of 4,096 leaves. The data contain 137015 sampled equation records, including 39285 positive labels, from 66 training batches. Sampling weights preserve the native class proportions and give each batch equal total weight. The model receives no final reduction coefficients. Validation uses 33 separate batches, first to find filters that retain nearly all positive labels and then to measure complete reduction cost. A graph uses a learned seed or equation filter only when its median validation cost is at least 5% below the same tuned calculation with all ML disabled. Appendix M specifies the fitting, threshold selection, and recovery rules.

32.2 Reserved comparison and the additional learned gain

We freeze the trees, thresholds, graph choices, and domain limits before evaluating two new rational kinematic profiles, $h = 525, 526$. The dimension remains symbolic. The ten graph structures and target patterns are the same as in Appendix L; the new profiles supply distinct kinematic requests. Some index patterns repeat training patterns, including the planar and nonplanar boxes at these two profiles. This is a test within known structures at one through four loops. A separate comparison below changes the target-index batches as well. As in the seed-bound study, each timed request receives a common master basis, and independent reference coefficients enter only the final exact comparison.

Both methods use the same executable, tuned seed limits, native row choices, exact implementation improvements, worker count, and recovery limits. The control disables every learned selector. Five repetitions alternate the order of the two methods. Time includes policy loading, the domain check, equation prediction and generation, exact algebra, verification, retries, and cleanup. Initial Python imports and construction of the common reference basis remain outside this repeated timer. A domain check disables filtering for an unrecognized ordered graph or target complexity outside the saved ranges.

The reserved grid has 74 requests. It supplies 68 matched nontrivial batches, with 4 failed reference checks, 0 incomplete timed batches, and 2 identity-only batches kept separately. All successful outputs pass exact coefficient comparisons: 700 completions among 700 timed requests, with 15430 comparisons. Table 19 retains each graph, including any slower result.

The full matched workload changes from 153.937 to 146.586 seconds, a time saving of 4.8%. For the 32 batches on which the frozen policy enables a filter, the corresponding times are 91.707 and 84.384 seconds, giving 8.0% less time. The 95% intervals from resampling paired repetitions are $[0.946, 0.959]$ for the full workload ratio and $[0.916, 0.927]$ for the active subset. The active subset follows the pre-test policy and is not selected from the observed test speedups. These intervals describe timing variation on the tested requests, not generalization to arbitrary graphs.

On active requests, the filters reject 36.8% of candidate construction calls, with a mean recorded filtering cost of 16.972 milliseconds per request. Repeated generation passes contribute to this count. Recovery can construct some rejected equations later, and its full cost remains in the timing comparison. The learned policy makes 5 recovery attempts in the reserved comparison. Recorded development, label collection, validation, and fitting take 39.2 minutes, excluding earlier seed-tuning costs. This expense precedes application and must be recovered over repeated

Table 19: Additional effect of the equation model after fixed seed tuning. Times are sums of batch medians in seconds. "Active" counts batches on which the frozen policy enables filtering. A ratio below one means less complete time.

Graph	Loops	Batches	Active	ML off	Policy	Ratio
Massive sunrise	2	8	0	4.70	4.69	0.997
Planar double box	2	8	8	18.78	15.79	0.841
Massive vacuum	3	8	8	8.07	7.55	0.937
Massive three-loop ladder	3	8	8	26.12	23.97	0.918
Nonplanar box	2	8	8	38.74	37.07	0.957
Pentagon	1	8	0	6.75	6.64	0.983
Box with virtual leg	1	8	0	4.85	4.83	0.996
Box with bubbles, 3 loops	3	6	0	9.48	9.50	1.002
Box with bubbles, 4 loops	4	2	0	30.99	31.16	1.005
Massless two-point, 4 loops	4	4	0	5.44	5.38	0.990

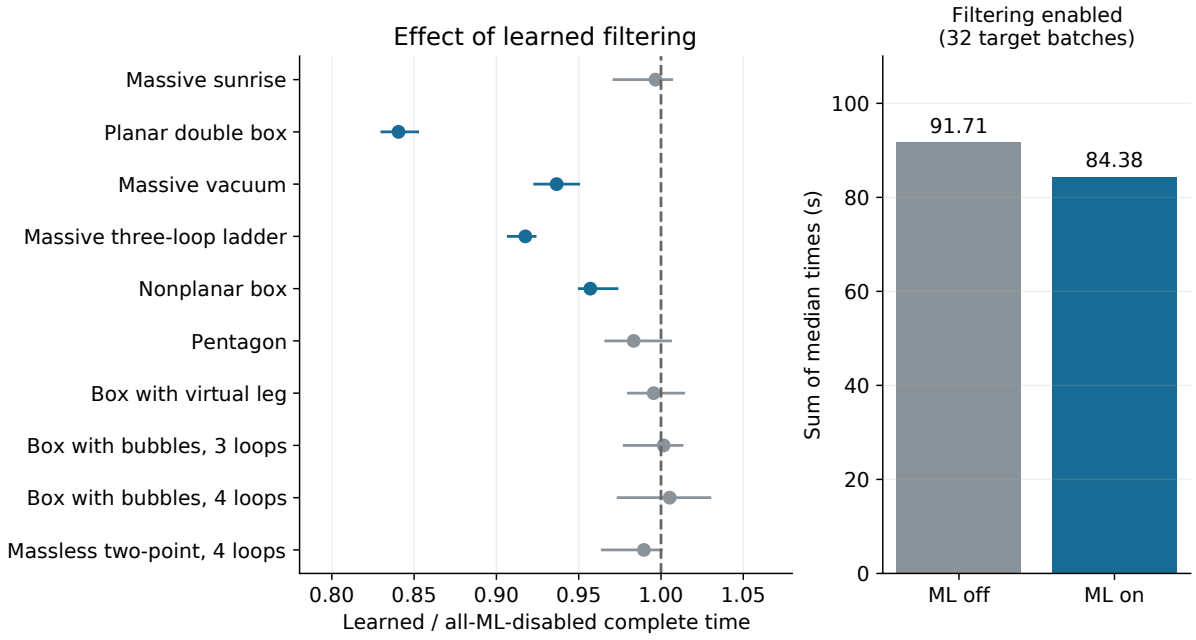


Figure 70: The left panel shows the complete-time ratio for every matched graph. Blue marks graphs with filtering enabled; grey marks graphs using the unchanged exact calculation. Horizontal intervals show the 95% range from resampling paired repetitions within each graph. The right panel isolates the batches on which the preselected policy uses the trained filter. The control shares all exact optimizations and tuned seed settings, so the difference includes only the learned policy and its consequences.

use. A useful model on this collection does not establish a benefit for an unsupported topology, symbolic kinematics beyond the dimension, or all high-loop integrals. The separate wider coverage results retain their stated scope.

32.3 Complete application launches

A separate check starts a fresh application process for each request and supplies only the target definition and master basis. It uses the first supported reserved request for each graph where validation enabled filtering, with three rotated repetitions. These are a subset of the same reserved physical inputs. The timer includes Python startup, input parsing, model loading, output storage, process exit, and the exact coefficient check. Across 4 matched requests, summed median time changes from 21.835 seconds with all ML disabled to 18.946 seconds with the filter, giving a time saving of 13.2%. This comparison checks the live application overhead; it does not add new graph structures to the physical scope.

32.4 Transfer to new target-index batches

The primary comparison changes kinematics but partly repeats the index patterns seen in training. To test index transfer separately, we keep the original fitted models, thresholds, and graph choices and define profiles $h = 605, 606$. We retain a batch only if its set of index vectors is absent from both training and validation and contains at least one individual index vector absent from those sets. We also remove repeated batches within each profile. These conditions select inputs from the graph structures enabled by the original validation; no primary test timing enters this selection. The selection rules and all resulting inputs are fixed before their reductions begin. The test therefore changes the indices within known graph structures and degree ranges, while transfer to new topologies remains untested.

Each request receives the same reference reduction and enlarged-region check as the primary comparison. Three repetitions rotate the two methods. The live application reads only a target definition and a master basis. Time includes basis reading, policy loading, prediction, exact reduction, recovery, and coefficient checks; it excludes initial Python imports and reference construction. Both methods retain the same seed settings and exact implementation.

Table 20: The unchanged learned policy on new index batches. Times are sums of batch medians in seconds. The control disables every learned selector.

Graph	Batches	ML off	Policy	Ratio
Planar double box	6	14.644	14.614	0.998
Massive three-loop ladder	6	21.815	19.833	0.909
Nonplanar box	6	29.771	27.317	0.918
Massive vacuum	6	6.066	5.714	0.942

The structural rule defines 24 requests, of which 24 give matched nontrivial reductions. The record retains 0 failed reference checks, 0 incomplete comparisons, and 0 identity-only batches. All 144 successful outputs among 144 timed requests pass exact comparison, with 5106 coefficient checks. The summed median time changes from 72.295 to 67.478 seconds, giving 6.7% less time. The 95% paired-repetition interval for the learned-to-control ratio is $[0.928, 0.940]$. Table 20 retains every matched graph, including any slower result. This check provides separate evidence about changes in target indices and is not combined with the primary timings to form a cumulative speedup.

33 Learning which targets to reduce together

The equation filters reduce work inside a seed region, but the targets grouped in one request also determine that region. A batch can contain targets with high denominator powers and other targets with high numerator rank. Their combined limits can generate equations that neither group needs separately. Dividing the batch can avoid these equations, while repeating preparation and shared algebra can make an unnecessary division slower. We therefore train a model to choose the division from measured complete reduction time. Recent tube-seeding work also uses target groups to control reduction cost [51]; the present method uses ordinary Kira seed regions and the already checked equation filters.

33.1 Model and integration with Kira

The model compares five ways to divide a batch: keep all targets together, group equal numerator ranks, group equal extra denominator powers, group equal sectors, or use consecutive groups of at most three targets. A sixth candidate keeps the batch together and disables the equation filter, allowing the new model to avoid an expensive filter or retry. It receives 32 quantities that describe the physical family, the requested powers, and their distribution across the batch. It predicts the complete relative cost of each choice before Kira generates any equations. Each selected group then uses the same fixed seed policy and native row elimination as the preceding method. The existing equation filter acts only when enabled and when the group lies inside its original checked domain. We merge the resulting exact master coefficients after checking that every target has completed.

We compare regression trees, random forests, and extremely randomized forests [30, 48, 39]. A tree compares the input quantities with learned thresholds and returns predicted costs from the final group of training examples. A forest averages the predictions of several trees. Here each prediction contains six log-cost ratios: the logarithm of a candidate’s complete time divided by the time of the undivided batch. Training uses 49 batches, and separate validation uses 49 batches. The candidate with the smallest validation cost is an extremely randomized forest with 32 trees, at most 6 threshold levels, and at least one training example in each final group. Its chosen prediction margin is 0.02 in log-cost units, as defined in Appendix N, and its selected validation costs give 0.7% less time than fixed partitions. No predictor reaches the required three-percent validation saving, so deployment uses the fixed partitions. The appendix specifies every partition, input, training objective, recovery rule, and physical request, and derives how the independently reduced groups give the complete target reduction.

The training labels include unsuccessful attempts and recovery. Reference construction supplies a common basis and checked coefficients before timing, but the solver receives only the basis and the target definitions. Reference coefficients enter the exact check after reduction. Recorded development, training, and validation reductions take 92.6 minutes, while fitting takes 8.86 seconds; these costs precede deployment and exclude reference construction. The median loading and decision cost in the timed test is 6.73 milliseconds.

Selecting the smallest recorded cost separately for every validation batch would lower the fixed-partition cost by 4.5%. This comparison uses timing information available only after the calculations and describes the remaining opportunity within these six candidates. It does not describe a deployable predictor. Appendix N defines this retrospective bound.

33.2 Controls and results

The reserved comparison uses new rational profiles in the same ten structures at one through four loops, with symbolic dimension. The matched requests contain 1 to 11 targets each. Five repetitions rotate the methods through execution positions on the same worker. The previous method keeps each batch together. One fixed control chooses a partition per graph with the equation filters available; a second fixed control tunes its partitions independently with every ML model disabled. The comparison with the first fixed control measures the added batch-selection application, including its domain checks and loading cost. The second compares the combined workflow with tuned choices that perform no ML inference. All methods share the same exact implementation, basis, worker count, and checks.

Of 108 requests, 96 give matched nontrivial completions. There are 10 failed reference requests, 0 incomplete timed batches, and 2 matched identity-only batches. The comparison record contains 2450 successful results in 2450 calculations, with 90275 exact coefficient comparisons. Every successful result passes its reference check. A total of 34 requested index batches recur in this study's training or validation sets at different rational profiles. The test therefore measures new kinematics and a defined mixture of target batches within known graph structures.

The selected workflow actually calls an equation filter on 4 matched batches. On this subset it uses 7.0% less time than the separately tuned ML-disabled control. This subset records filter use; its controls can also choose different fixed partitions. Its paired-repetition ratio interval is $[0.834, 1.018]$. This interval includes equal time and does not establish a reliable saving on this subset.

Two cluster allocations reached their time limits before completing all requests. We preserve the 76 complete comparison batches and resume the 22 remaining batches with unchanged models and timing rules. For an interrupted batch, we repeat every method and repetition on one worker. The 4 saved timings from its earlier partial comparison remain available separately and do not enter the ratios. Work assignment uses validation costs only.

Summed batch median time changes from 383.760 seconds for the previous undivided workflow to 389.368 seconds for the selected workflow, giving 1.5% more time. The fixed control with filters takes 386.251 seconds, and the independently tuned all-ML-disabled control takes 389.826 seconds. Relative to these controls, the selected workflow uses 0.8% more time and 0.1% less time, respectively. The selected-to-fixed ratio has a paired-repetition interval of $[0.988, 1.035]$, and the ratio to the all-ML-disabled control has interval $[0.986, 1.025]$. Validation disables the new predictor, so this comparison establishes no additional ML gain from batch selection. Table 21 shows the separate graph results, including regressions.

The recorded generation counts sum to 2,429,859 equations for the previous workflow, 2,514,300 for the tuned ML-disabled control, and 2,494,576 for the selected workflow. These are sums of per-batch median counters over reported generation passes, including repeated passes and retries. They describe generated equations rather than the total number of arithmetic operations. Complete time remains the criterion for retaining a model.

Table 21: Sum of median complete times in seconds on matched batches. Previous keeps each batch together. Fixed uses one partition per graph with filters. ML off tunes partitions separately with every model disabled. Selected uses the frozen deployment choice.

Graph	Batches	Previous	Fixed	ML off	Selected
Three-loop bubble box	12	27.83	27.69	28.03	28.06
Four-loop bubble box	2	18.69	19.61	18.99	19.75
Planar double box	12	37.29	39.87	40.36	40.05
Massive ladder	12	98.23	103.36	102.32	102.17
Massless two-point	10	30.38	26.66	26.76	26.56
Nonplanar box	12	138.39	135.97	139.94	139.29
Virtuality box	8	5.31	5.62	5.64	5.70
Pentagon	8	6.20	6.07	6.12	5.99
Sunrise	12	13.12	12.97	12.91	13.55
Massive vacuum	8	8.31	8.41	8.76	8.23

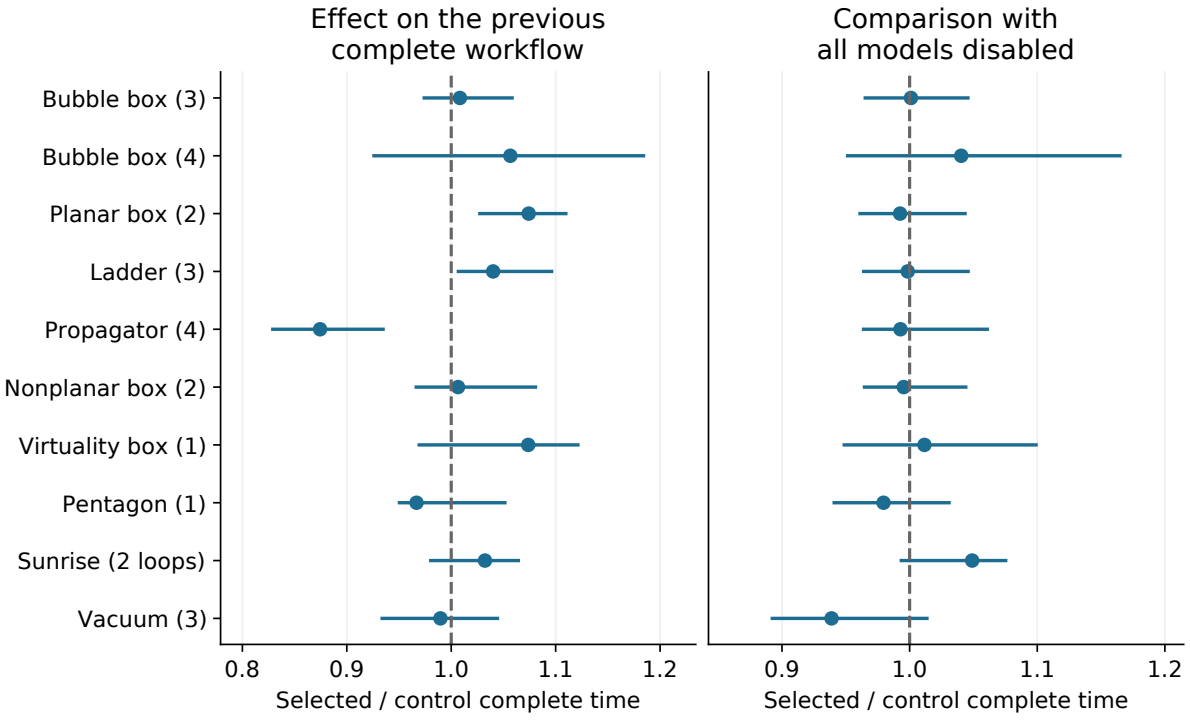


Figure 71: The left panel compares the selected complete workflow with the previous undivided workflow. The right panel compares it with independently tuned fixed partitions with every model disabled. Values below one indicate less time; the dashed line marks equal time. Segments describe paired repetition variation on this workload. Validation disables the new batch predictor; the selected workflow uses the retained fixed partitions and equation-filter settings.

A separate application check starts a new Python process for each request and includes imports, model loading, input and output, and exact checks. Deployment loads only the parts of the frozen policy that it uses: when validation disables the predictor, it retains the fixed choices and domain limits and omits the unused tree arrays. We check that this compact form gives the same choices on every training, validation, and reserved input, without reading reserved timing outcomes. The repeated comparison above retains the original frozen application; this separate check uses the compact form. It completes 36 timed requests over 4 matched batches and gives 1.2% less time than the previous workflow. These batches are a predeclared subset of the reserved inputs, so this check measures application overhead without adding evidence for new topologies.

The new study extends the batch sizes tested by the earlier equation-filter comparison. It still supplies a master basis and keeps the kinematic invariants rational. General symbolic scattering, arbitrary higher-loop structures, and globally optimal partitions remain outside this evidence. The earlier restricted vacuum recurrence covers its stated one-through-twelve-loop family separately, and its speedup cannot be multiplied by the ratios measured here.

34 Learning sparse seed regions from reduction time

The batch study leaves little additional saving among its predefined partitions. We therefore change the equations available to Kira more directly. A learned boundary selects individual seeds around the requested numerator powers before equation generation. Kira then applies its usual IBP operators to those seeds. This follows the motivation of sparse target-dependent seeding [51], while the boundary parameters in this study are fitted through complete reductions with symbolic dimension.

34.1 The seed model and its training data

The model describes a surrounding region around each target's numerator-power box. Its width can depend on the seed rank and on how many physical propagators are absent from its sector. It also controls the largest seed rank in each sector. Appendix P defines these boundaries, derives the distance used by the model, and gives a counting example that explains why a narrow region can avoid many seeds at higher numerator rank.

We first fit the boundary parameters through evolutionary search. Each proposal runs Kira and receives a score from the measured time needed to reach the prescribed masters and pass an exact coefficient check. The score includes unsuccessful attempts and native recovery. Thus the labels describe completed calculations rather than membership in a previously selected equation set. Three training requests per graph start this search; separate validation selects the first fitted boundary. An independent planar-box comparison then tests that boundary on eight new kinematic requests.

The second stage adds twelve training requests per graph and fits models that predict the cost of different seed boundaries. These requests contain single-direction and distributed numerators of ranks two, three, and four. The candidate boundaries include those found by the first search, independent changes to the rank allowance, fixed surrounding widths, and unfiltered Kira. Trees and forests predict the log complete-time ratio of each candidate [30, 48, 39]. Ten validation requests per graph select the model, its decision margin, and a fixed sparse control. The predictor is enabled only when validation gives at least three percent less recorded cost than that fixed control. When validation disables prediction, the deployed policy uses the selected fixed boundary. Table 22 gives the selected models. The independent timing tests include the cost of

loading and applying them.

Table 22: Cost predictors selected before the reserved comparisons. Each graph also has a separately selected fixed sparse control.

Graph	Predictor (depth)	Training requests	Deployment
Planar box	Tree (2)	12	Fixed boundary retained
Nonplanar box	Extra trees (2)	12	Enabled

Validation enables request-dependent cost prediction for the nonplanar box. Other graph choices use their fixed sparse boundary.

34.2 Independent reductions and controls

The first evolved planar-box boundary gives 22.3% less time, with a paired-repetition ratio interval of $[0.751, 0.813]$ relative to tuned native Kira. Relative to the fixed surrounding-width control, it gives 6.5% less time, with a paired-repetition ratio interval of $[0.899, 0.980]$. This comparison contains 168 timed reductions, and all successful outputs pass exact checks. The boundary remains fixed throughout this test.

Table 23: First independent planar-box comparison. Times are sums of per-request medians over eight requests, with seven repetitions per method. The learned boundary is fitted before this comparison.

Method	Complete time (s)
Tuned native Kira	37.893
Fixed surrounding width	31.465
Evolved seed boundary	29.424

The second comparison uses 20 matched requests from the planar and nonplanar two-loop boxes, with seven repetitions per method. All methods use the same Kira executable, exact preparation, master basis, and native row elimination. The native control disables every learned choice. The fixed sparse control uses one validation-selected boundary per graph, including boundaries discovered during training, and performs no request-dependent cost prediction. The previous application retains the earlier published batch and equation policies. These controls separate the effect of sparse seeding from the added benefit of predicting which boundary to use for a particular target.

The complete selected workflow gives 22.2% less time, with a paired-repetition ratio interval of $[0.764, 0.792]$ relative to tuned native Kira and 22.6% less time, with a paired-repetition ratio interval of $[0.762, 0.789]$ relative to the previous application. Relative to the fixed sparse control, it gives 3.6% less time, with a paired-repetition ratio interval of $[0.946, 0.981]$. Table 24 reports the measured times for both graphs. The comparison contains 560 timed reductions and 5600 exact coefficient comparisons. Failed reference requests number 0, and failed timed reductions number 0; these remain in the evidence record.

Table 24: Sum of per-request median complete times, in seconds. Native uses the tuned outer bounds with all ML disabled. Fixed uses one sparse boundary per graph. Selected uses the frozen deployment policy.

Graph	Requests	Native	Previous	Fixed	Selected
Planar box	10	20.986	21.072	15.742	15.802
Nonplanar box	10	54.857	55.100	45.462	43.169
All matched	20	75.843	76.172	61.203	58.972

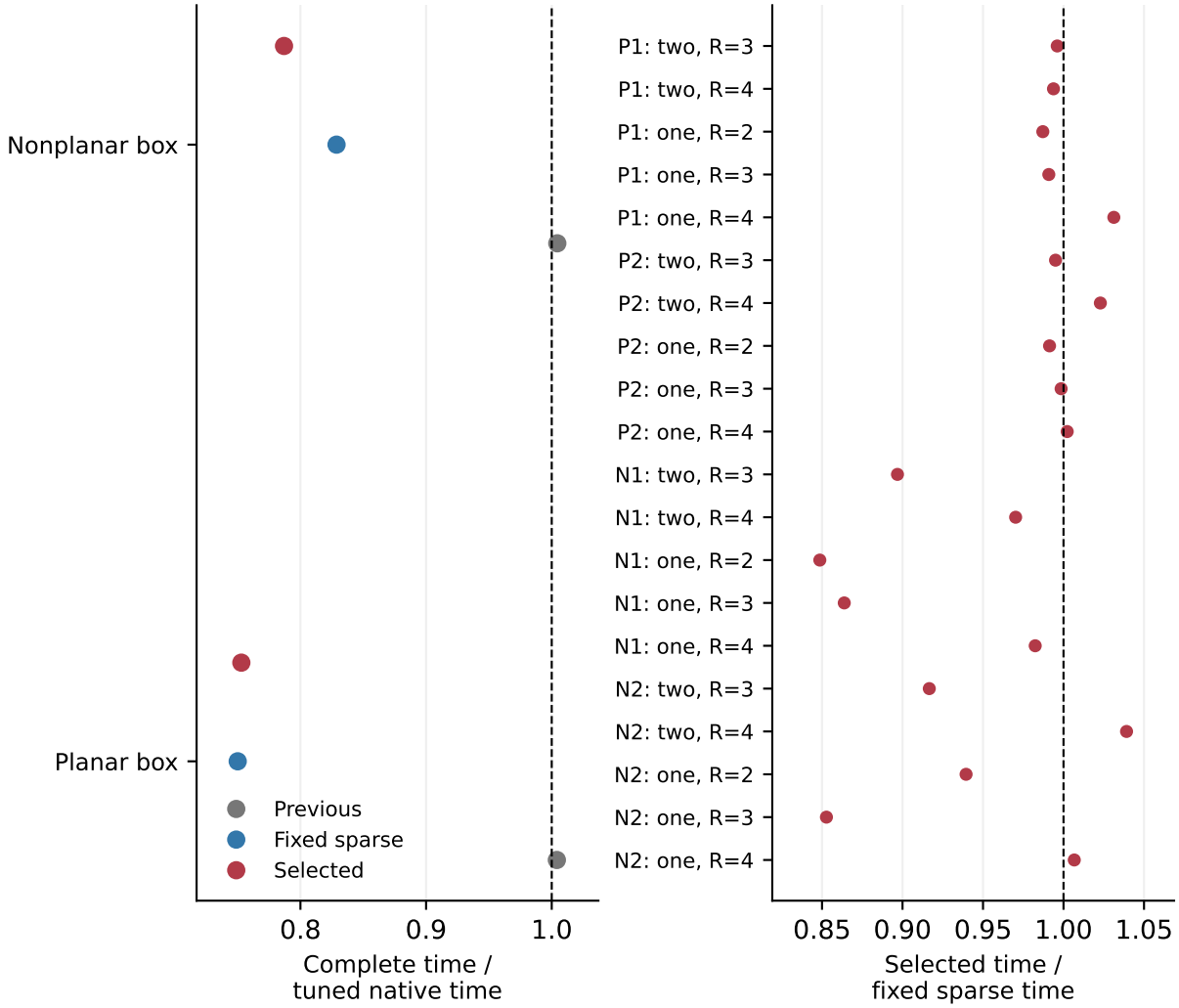


Figure 72: Left: complete-time ratios to tuned native Kira, summed within each graph. Right: the selected-to-fixed-sparse ratio for each reserved request. Labels P and N identify the planar and nonplanar graphs; 1 and 2 identify the two reserved kinematic profiles. Each label gives the number of powered numerator directions and the rank R. The dashed lines mark equal time. Values below one indicate a saving. The right panel shows the measured difference from using the fixed sparse rule.

A separate application check starts a fresh process for each request and supplies only the physical definition, target indices, and master basis. It includes imports, model loading, input and output, process exit, and a subsequent exact check. It completes 48 runs over 4 matched requests. Its selected total is 15.774 seconds, compared with 27.897 seconds for native choices and 16.187 seconds for the fixed sparse control. These requests are a predefined subset of the reserved test, so they check application overhead without adding graph coverage.

The complete integrated application also passes 24 fresh-process checks. These use two of the reserved two-loop requests, one previously tested four-loop native request, and an eight-loop massless banana request. For the two-loop requests, the summed median complete time is 8.091 seconds, compared with 17.088 seconds for the preceding complete application. The other inputs check that the existing native and analytic methods remain available and exact. They do not extend the sparse model's fitted domain.

These results concern numerator ranks two through four, unit positive physical powers, rational kinematic values, symbolic dimension, and supplied bases in two known two-loop structures. The reported production times exclude offline training and construction of the independent reference basis. Some index patterns recur between training and testing. Development probes at other loop orders do not extend the fitted model's validated domain. The application retains its earlier exact methods and native fallback outside this domain. The comparison establishes the measured effects of the new seed policy; it does not establish a universal speedup or a globally optimal reduction path.

35 An exact higher-loop specialization alongside the learned method

The learned choices above apply to the physical structures represented in their training and validation data. A useful reduction workflow can also use a proved integral identity when the requested family permits one. We implement this choice for massless two-point banana integrals, which have two vertices connected by $L+1$ massless lines at L loops. Repeated one-loop convolution gives their coefficients directly [53]. This saving comes from the analytic identity and is measured separately from the learned decisions.

Let p be the external momentum, with $p^2 \neq 0$, and let the physical denominators be $\ell_1^2, \ldots, \ell_L^2$ and $(p - \sum_j \ell_j)^2$. Each denominator has a positive integer power a_i. There are no numerator insertions in this specialization. Define $n = L+1$, $b_i = a_i - 1$, $K = \sum_i b_i$, and the corner integral $M_L = B_L(1, \ldots, 1)$. Appendix O defines the measure and derives

$$B_L(\boldsymbol{a}) = \frac{(n - Ld/2)_K (n + 1 - nd/2)_K}{(p^2)^K \prod_{i=1}^n (2 - d/2)_{b_i} b_i!} \, M_L, \tag{143}$$

where $(z)_k = \prod_{j=0}^{k-1} (z + j)$ and $(z)_0 = 1$. Thus all scalar targets in this class have rational reduction coefficients in d and p^2.

Before applying Eq. (143), the workflow checks the propagator momenta, zero masses, nonzero external squared momentum, and allowed indices. It forms and cancels exact linear factors in d. Other families pass to the native Kira workflow with the frozen batch and equation choices. Both the ML-enabled and ML-disabled comparison paths use the same analytic identity. The identity therefore contributes no saving to the reported ML comparison.

35.1 Checks and measured time

The first native comparison asks Kira to use the corner master with seed limits set directly by the targets. It completes the one-loop requests and fails in eight higher-loop requests. These failures include rejection of the supplied basis and an allocation failure during topology preparation. We retain all of them, then compare four fixed seed choices on a separate development point and freeze the fastest successful choice for each loop count and kinematic form. A new point supplies the matched comparison in Table 25. Appendix O gives the seed definitions and repetitions.

The follow-up completes 9 matched requests and leaves 1 request without a completed comparison. Six-loop development reports allocation failures during topology preparation under the imposed memory limit. Since no fixed seed choice passes, the six-loop reserved request does not run and receives no timing ratio. The completed requests check 81 native coefficients exactly. The complete-time speedup ranges from 21.3 to 357.8 times across the completed requests. Both paths include an exact coefficient check, and the direct path also includes its domain check. These timings exclude shared Python imports.

Table 25: Complete times in seconds for the defined scalar banana targets. Native uses the development-selected fixed seeds with all ML disabled. Direct uses the proved formula. Each native time is the median of three repetitions.

Loops	p^2	Native	Direct	Speedup
1	Rational	0.4223	0.0067	62.8×
2	Rational	0.4425	0.0194	22.8×
3	Rational	0.5373	0.0218	24.7×
4	Rational	1.4625	0.0249	58.8×
5	Rational	8.7544	0.0245	357.8×
1	Symbolic	0.4693	0.0081	58.0×
2	Symbolic	0.4966	0.0233	21.3×
3	Symbolic	0.5555	0.0246	22.6×
4	Symbolic	1.4615	0.0243	60.2×

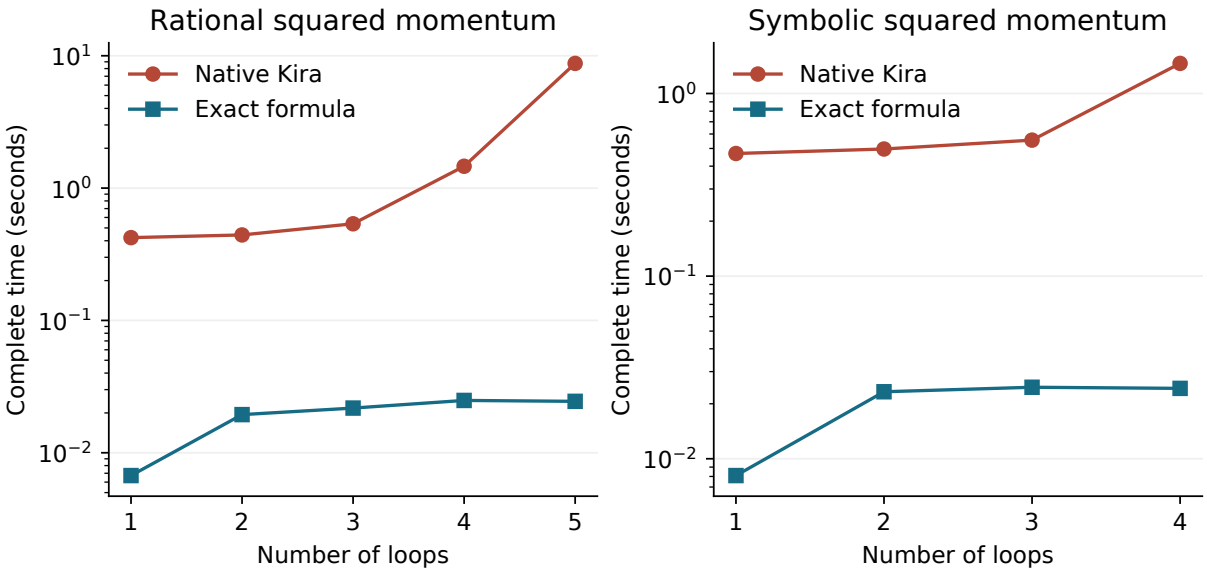


Figure 73: The direct coefficient formula avoids the native equation system for the displayed scalar family. Times include exact coefficient checks. The logarithmic time axis shows both methods, and only completed native comparisons appear. These are analytic gains, separate from ML.

An additional 240 numerical checks compare the finite rational formula with the uncancelled Gamma-function expression at one through twelve loops and at 16, 24, and 32 loops. They use 70-digit arithmetic at two noninteger dimensions. The general derivation supports this scalar family at every positive loop count; the numerical checks test its implementation at the stated counts. General higher-loop scattering remains outside this analytic specialization.

A separate test starts a new application process for each request. All 48 executions use the combined routing interface, and every successful output passes an exact check. It also passes 49 domain checks. The four analytic requests use one, four, eight, and twelve loops; their summed median application time is 0.6458 seconds. The same formula serves both comparison paths. The 4 matched native requests take 38.161 seconds with the frozen learned workflow and 38.318 seconds with the separately tuned ML-disabled workflow. These native requests repeat the predeclared application subset of the batch study, so they test integration and startup rather than new physical coverage. The new seed-bound study in Section 31 isolates a learned choice with all earlier models disabled. On 68 matched nontrivial batches, complete application time changes from 164.926 seconds under native bounds and 119.717 seconds under tuned fixed settings to 119.081 seconds with the new model. This gives 27.8% less time than native bounds and 0.5% less time than the tuned fixed control. The comparison includes model loading, native IBP generation, exact reduction, coefficient checks, and recovery, with a supplied basis. It uses known structures at one through four loops and separate rational profiles; the additional saving over tuned fixed settings remains within timing variation.

A Integral families for the native cost comparisons

The denominator convention is $D_i = q_i^2 - m_i^2 + \mathrm{i}0$, with the measure in Eq. (3). This appendix collects the ordered momenta used by the later cost studies so that the notes specify their physical inputs independently of the companion paper. Appendix H gives the masses, kinematic profiles, and target indices for the new comparisons. Appendix G adds the connected higher-loop scattering families.

The sunrise uses

$$\boldsymbol{q}_{\mathrm{S}} = (\ell_1, \ell_2, p - \ell_1 - \ell_2, p - \ell_1, p - \ell_2). \tag{144}$$

Its first three entries are physical denominators and its last two entries are auxiliary. Their indices remain nonpositive.

A.1 Native multiloop families

The sunrise comparisons allow independent masses on the same three physical propagators. The other native comparison families use

$$\begin{aligned}
\boldsymbol{q}_{\mathrm{DB}} = (&\ell_1, \ell_1 - p_1, \ell_1 - p_1 - p_2, \ell_2, \ell_2 - p_1 - p_2, \ell_2 - p_1 - p_2 - p_3, \\
&\ell_1 - \ell_2, \ell_1 - p_3, \ell_2 - p_1), && (145)\\
\boldsymbol{q}_{\mathrm{V}} = (&\ell_1, \ell_2, \ell_3, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3), && (146)\\
\boldsymbol{q}_{\mathrm{L}} = (&\ell_1, \ell_1 - p, \ell_2, \ell_2 - p, \ell_3, \ell_3 - p, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3). && (147)
\end{aligned}$$

The double box has two loops and seven physical denominators; its last two entries are auxiliary. The vacuum has three loops and six physical denominators. The ladder has three loops and eight physical denominators; its last entry is auxiliary. Auxiliary indices remain nonpositive. The massless double box has $p_i^2 = 0$ and uses $2p_1 \cdot p_2 = s$, $2p_1 \cdot p_3 = t$, $2p_2 \cdot p_3 = -s - t$. This local definition of t differs from the initial box convention because the routing differs.

A.2 Scattering structures and symbolic invariants

The massless nonplanar box uses the routing of Ref. [2]:

$$\begin{aligned}
\boldsymbol{q}_{\mathrm{NP}} = (&\ell_1, \ell_2, p_1 - \ell_1, p_2 - \ell_2, p_1 + p_2 - p_3 - \ell_1, \\
&p_1 - \ell_1 + \ell_2, p_1 - p_3 - \ell_1 + \ell_2, \ell_1 - p_3, \ell_2 - p_1 - p_2). && (148)
\end{aligned}$$

The first seven entries are physical propagators, and the final two are auxiliary entries. Here $p_1^2 = p_2^2 = p_3^2 = (p_1 + p_2 - p_3)^2 = 0$, $s = (p_1 + p_2)^2$ and $t = (p_1 - p_3)^2$. Expanding these squares gives $2p_1 \cdot p_2 = s$, $2p_1 \cdot p_3 = -t$ and $2p_2 \cdot p_3 = s + t$.

The massless pentagon uses $\sum_{i=1}^{5} p_i = 0$, $p_i^2 = 0$ and

$$\boldsymbol{q}_{\mathrm{P}} = (\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_1 - p_2 - p_3, \ell - p_1 - p_2 - p_3 - p_4). \tag{149}$$

All five denominators are physical. The independent invariants are $(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})$, where $s_{ij} = (p_i + p_j)^2$. Squaring $p_1 + p_2 + p_3 = -(p_4 + p_5)$ gives $2p_1 \cdot p_3 = s_{45} - s_{12} - s_{23}$; the other nonadjacent products follow by cyclic relabelling. The box with an external virtuality uses $\boldsymbol{q} = (\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_3)$, $p_1^2 = p_2^2 = p_3^2 = 0$, and $v = (p_1 + p_2 - p_3)^2$. With the nonplanar convention for s, t, expanding v gives $2p_2 \cdot p_3 = s + t - v$.

B Learning the cost of a complete reduction

This appendix defines the complete-time prediction problem, derives the fitted models, and specifies the physical inputs used in Section 20. The cost model receives a feature vector $\boldsymbol{f}_c$ for target batch c. Its entries are the number of loops, external momenta, inverse propagators, physical propagators and symbolic invariants; $\log(1+n_c)$, where n_c is the number of targets; the maximum dot count, maximum numerator rank, mean dot count and mean numerator rank; the number of target sectors; and seven indicators for the graph structures. An indicator is one for its graph and zero for the other graphs. Together these quantities give eighteen inputs, all available from the requested targets and family before reduction.

For an index vector $\boldsymbol{a}$, define

$$r(\boldsymbol{a})=\sum_i \max(a_i,0), \qquad s(\boldsymbol{a})=\sum_i \max(-a_i,0), \qquad q(\boldsymbol{a})=\sum_i \max(a_i-1,0). \tag{150}$$

Here r is the positive index sum, s is the numerator rank, and q is the number of extra denominator powers, called dots. These definitions follow directly by counting the denominator and numerator factors. The initial bounds $B_c=(r_c,s_c,q_c)$ are the componentwise maxima over the targets. The trial seed regions use B_c, $B_c-(1,1,1)$, $B_c-(1,0,1)$, or $B_c+(1,1,1)$. Negative bounds are set to zero. The choices change the region in which equations are generated. Since a smaller region can omit required relations, every attempt must still pass the prescribed-basis check.

Let t_{cj} be the median complete time of strategy j on training batch c, and let t_{c0} be the time of the integrated native strategy. The training response is

$$y_{cj}=\log\frac{t_{cj}}{t_{c0}}. \tag{151}$$

Each time includes unsuccessful attempts and the eventual successful fallback, so a strategy that omits necessary equations is penalized by the extra calculation. The logarithm preserves the ordering of positive time ratios; minimizing the response therefore also minimizes the complete time relative to the common baseline.

We fit constant, tree, linear and nearest-neighbour predictors of this response. The constant predictor averages each strategy's training responses. A tree splits the feature space and uses a constant vector within each leaf, following regression-tree construction [30]. For a leaf containing training set A, minimizing $\sum_{c\in A}(y_{cj}-v_j)^2$ gives

$$\frac{\partial}{\partial v_j}\sum_{c\in A}(y_{cj}-v_j)^2=2|A|v_j-2\sum_{c\in A}y_{cj}=0, \qquad v_j=\frac{1}{|A|}\sum_{c\in A}y_{cj}. \tag{152}$$

Each split minimizes the sum of squared errors over the seven strategies. The tested maximum depths are two, four and six. Each depth is tested with a minimum of either one or three training batches per child.

For the linear models, we first subtract the training mean of each feature and divide by its training population standard deviation, computed with divisor equal to the number of training batches. A constant feature uses divisor one. Let Z contain these standardized feature rows with a leading intercept column, and let Y contain the responses. Ridge regression minimizes

$$J(W)=\|ZW-Y\|_F^2+\lambda\,\mathrm{tr}(W^TPW), \qquad P=\mathrm{diag}(0,1,\dots,1). \tag{153}$$

Here W is the matrix of fitted coefficients, $\|\cdot\|_F^2$ is the sum of squared matrix entries, and P leaves the intercept unpenalized. Differentiating each entry of W gives

$$\nabla_W J=2Z^T(ZW-Y)+2\lambda PW=0, \qquad (Z^TZ+\lambda P)W=Z^TY. \tag{154}$$

We solve this system for $\lambda = 0.1, 1, 10$.

The nearest-neighbour models estimate a cost from similar training batches. Eligible neighbours must have the same graph, loop count, external-momentum count, propagator counts, and number of symbolic invariants. They choose up to two or four training batches with the smallest squared distance in standardized features. The predicted response is the mean of their response vectors. The leaf-mean derivative above also derives this local constant estimate. If no eligible neighbour exists, the predictor returns equal costs and the selector uses native choices.

Let j_* be the strategy with the smallest predicted logarithmic cost $\widehat{y}_j(\boldsymbol{f})$. Validation selects a margin $\gamma \in \{0, 0.05, 0.15\}$, and the model chooses the strategy as follows:

$$\widehat{\jmath}(\boldsymbol{f}) = \begin{cases} j_*, & \widehat{y}_{j_*}(\boldsymbol{f}) < \widehat{y}_0(\boldsymbol{f}) - \gamma, \\ 0, & \text{otherwise.} \end{cases} \tag{155}$$

Subtracting the two predicted logarithmic costs shows that the condition requires a predicted relative cost below $e^{-\gamma}$. Validation selects the model and margin with the smallest geometric mean time ratio. The selected settings remain fixed during reserved tests. This finite comparison establishes the best measured candidate within the study. It does not prove a globally optimal reduction strategy.

B.1 Parameter points and target lists

The comparison uses the sunrise, planar box, three-loop vacuum, three-loop ladder, nonplanar box, pentagon, and box with external virtuality, with the same denominator routings and kinematic conventions as the preceding tests. The definitions below specify the parameter values and target lists for each split. Let h identify a rational parameter profile. Training uses $h = 41, 42$, validation uses $h = 43$, and testing uses $h = 44, 45$. In a common squared-momentum unit, the sunrise has squared masses $(3, 7, 13)$ and $p^2 = -1009 - 12h$. The planar box has $(s, t) = (-1009 - 12h, -1021 - 14h)$. The three-loop vacuum has squared masses $(1, 1, 1, 13, 13, 53 + 2h)$. The ladder keeps squared masses $(1, 0, 0, 0, 0, 0, 0, 0)$ and has $p^2 = -1009 - 12h$. The scattering assignments are

$$(s, t)_{\mathrm{NP}} = (-113 - 18h, -127 - 10h), \tag{156}$$

$$(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})_{\mathrm{P}} = (-11 - 20h, -13 - 20h, -17 - 20h, -19 - 20h, -23 - 20h), \tag{157}$$

$$(s, t, v)_{\mathrm{box}} = (-113 - 18h, -127 - 10h, -149 - 12h). \tag{158}$$

These assignments fix the rational parameter values in each profile while leaving the dimension symbolic. The additional symbolic batches instead retain all independent invariants of their scattering graph as symbols.

For the four earlier multiloop graphs, we retain the ordered target list that starts with the corner, raises each physical denominator power once, and then introduces each single auxiliary numerator. We repeat these steps after removing one, two and three physical denominators, retain at least two, and remove duplicates. Each rational profile supplies a four-target batch and a batch of the first 32 targets, or all 21 targets for the sunrise. The three scattering graphs use the same construction with at most one removed denominator. Their larger batches have 32, 31 and 21 targets for the nonplanar box, pentagon and virtuality box.

For the symbolic nonplanar box and pentagon, training takes target entries 12–15, validation takes 16–19, and testing takes 20–23, with numbering starting at zero. These lists do not overlap. For the symbolic virtuality box, the respective single targets are $(3, 1, 1, 1)$, $(1, 3, 1, 1)$ and $(1, 1, 3, 1)$. These targets have two extra powers on one physical denominator. The rational batches reuse target patterns at different parameter points.

The joint-request test uses the larger rational batch at $h = 44$ for each graph and the three symbolic test batches. The symbolic virtuality box also includes $(1, 1, 1, 3)$, so it can supply two disjoint requests. A group of n targets is divided after entry $\lfloor n/2 \rfloor$, with the first request containing that many entries. The two requests share the same family definition and are available together, and their union contains each target once. By linearity, selecting the corresponding target rows from the joint reduction returns the coefficients for each original request. This separation leaves every coefficient unchanged and requires no new integral identity. The test reuses reserved parameter points and keeps the learned model fixed.

B.2 Combining the measured times

Let m_{cj} be the median of the five test times for batch c and method j, and let method zero be its stated native control. For graph g with batch set C_g, we define

$$R_{gj} = \exp\left[\frac{1}{|C_g|}\sum_{c\in C_g}\log\frac{m_{cj}}{m_{c0}}\right], \qquad R_j = \exp\left[\frac{1}{7}\sum_{g=1}^{7}\log R_{gj}\right]. \tag{159}$$

The logarithm of a product equals the sum of the logarithms, so each expression is the corresponding geometric mean of the time ratios. The second expression assigns equal weight to each graph even when the numbers of batches differ. A ratio $R_j < 1$ means less time. The percentage saving is $100(1 - R_j)$, and the reciprocal $1/R_j$ is the speedup.

For the descriptive intervals, we sample seven graph labels with replacement, calculate their mean logarithmic ratio, and exponentiate. We repeat this 10,000 times and report the 2.5th and 97.5th percentiles. This procedure describes variation among the seven tested graphs. It does not provide a population guarantee for other integral families. The joint-request comparison uses the same calculation, with separate native requests as its first control. Its parameter points overlap the main reserved test, so it is an additional comparison on the same test population.

C Separate cost models for each graph

The graph-specific study changes the target batches and available strategies while retaining the seven denominator definitions and parameter formulas in Appendix B. This appendix specifies those changes and the rule for deciding whether a new predictor is used. Development uses profile $h = 61$, training uses $h = 62, 63$, validation uses $h = 64$, and the reserved test uses $h = 65, 66$. The dimension remains symbolic. Related target batches share the same split, so their rational kinematic values cannot pass from the training set into the reserved set. The graph structures and target patterns occur in both sets.

Let P be the number of physical propagators and let $\boldsymbol{b} = (1, \ldots, 1, 0, \ldots, 0)$ have P entries equal to one. Let $\boldsymbol{e}_j$ be the unit vector for index j. The first batch contains $\boldsymbol{b} + 2\boldsymbol{e}_j$ for $j = 1, \ldots, \min(P, 3)$. Thus one denominator has power three in each target. For the second batch, define the lower-sector corner

$$\boldsymbol{b}^{\mathrm{low}} = \begin{cases} \boldsymbol{b} - \boldsymbol{e}_1, & P < 5, \\ \boldsymbol{b} - \boldsymbol{e}_1 - \boldsymbol{e}_3, & P \geq 5. \end{cases} \tag{160}$$

Include this corner and up to three single-dot targets, $\boldsymbol{b}^{\mathrm{low}} + \boldsymbol{e}_j$ with $b_j^{\mathrm{low}} = 1$, taking the first available indices in increasing order. The third batch uses the original corner $\boldsymbol{b}$ and contains $\boldsymbol{b} - 2\boldsymbol{e}_j$ for the first three auxiliary entries, or all auxiliary entries when fewer than three exist. When a family has no auxiliary entry, this construction repeats the first batch and we remove

the duplicate. These formulas specify the requested integral indices. Their master coefficients are obtained by exact reduction and are not used to construct the model inputs.

C.1 The allowed changes to Kira

For target batch $\mathcal{T}$ and a target $\boldsymbol{a}$, let $S(\boldsymbol{a}) = \{i : a_i > 0\}$ be its positive-index set. One strategy supplies only maximal target sectors:

$$\mathcal{S}_{\max} = \{S(\boldsymbol{a}) : \boldsymbol{a} \in \mathcal{T},\ S(\boldsymbol{a}) \not\subseteq S(\boldsymbol{a}') \text{ for every } \boldsymbol{a}' \in \mathcal{T}\}. \tag{161}$$

Kira then includes their subsectors. To see why this is a useful candidate, consider the shifted integrals produced by the IBP derivative. A raised index $a_i + 1$ carries a factor a_i, and the other shifts lower an index. If $a_i = 0$, the raising term vanishes; if $a_i < 0$, raising it once leaves it nonpositive. Thus an IBP equation seeded in one sector stays in that sector or its subsectors. The sector property makes this restriction a useful candidate, but it does not establish that the restricted pool contains every relation needed for the prescribed basis. We therefore retain the basis check and the option to repeat the calculation with the full sector set.

Another strategy uses Kira's numerator truncation [9]. In a sector with t positive indices, Kira restricts the inherited numerator bound by

$$s \le \max(1, t - \tau + 1, s_{\text{explicit}}), \tag{162}$$

where τ is the chosen integer cutoff and s_{explicit} is a bound explicitly supplied for that sector. We test $\tau = P - s_c$ and $\tau = P - s_c + 1$, where s_c is the largest target numerator rank. The top-level sector retains its explicit target bound. The first cutoff combined with maximal target sectors forms the deterministic control. The development probes also test $\tau = P - s_c - 1$.

The remaining choices use the original bounds B_c, the smaller bounds $B_c - (1, 0, 1)$ with negative entries replaced by zero, the larger bounds $B_c + (1, 1, 1)$, Kira integral ordering five [8], or the preceding trained equation filter. Together these choices give nine strategies. Before each attempt, every strategy enforces a nonempty seed region in its declared top-level sectors. If $\mathcal{S}_0$ is that sector list and r_{req} is the requested positive-index bound, the actual bound is

$$r_{\text{run}} = \max\left(r_{\text{req}}, \max_{S \in \mathcal{S}_0} |S|\right). \tag{163}$$

Each positive index is at least one, so a seed in sector S has $\sum_{i \in S} a_i \ge |S|$. This proves the lower bound used in Eq. (163). Enforcing this bound prevents an invalid configuration from causing a failed startup before any meaningful reduction can occur. Identical input configurations share one training or validation measurement. If a proposed reduction fails, it retries with native choices at B_c and then at $B_c + (1, 1, 1)$. The recorded cost includes every attempt.

C.2 Fitting and selecting the models

Each graph has its own predictors of $y_{cj} = \log(t_{cj}/t_{c0})$, where t_{cj} is the median complete cost of strategy j on batch c and zero denotes native choices. The nine input features are the target count; maximum dot count and numerator rank; minimum and maximum positive-index counts; mean dot count and numerator rank; number of maximal target sectors; and number of physical propagators. The features are calculated from the requested indices and the family definition, so no reduced coefficients enter the prediction.

We fit ten candidates per graph: one constant predictor, trees of maximum depth one, two and three, a bagged ensemble of sixteen depth-three trees, ridge regressions with penalties $0.1, 1, 10,$

and nearest-neighbour predictors with one and three neighbours. Trees use squared-error splits with at least one row in each child. Each ensemble tree receives a bootstrap sample of the training batches and uses all features at each split. This follows bootstrap aggregation [54]. For fitted trees $f_1, \ldots, f_{16}$, the ensemble prediction is

$$\widehat{y}_j(\boldsymbol{f}) = \frac{1}{16} \sum_{b=1}^{16} f_{bj}(\boldsymbol{f}). \tag{164}$$

This average also follows by minimizing $\sum_b (f_{bj}(\boldsymbol{f}) - v_j)^2$ with respect to v_j, using the leaf-mean derivative in Appendix B. That appendix derives the ridge fit and the local-mean estimates used here. Neighbour distances use the training standard deviations; a constant feature uses divisor one. We use two timing repetitions for training and three for validation. Within each batch comparison, every strategy uses the same computer. Different training batches can use different computers, and the training response normalizes each cost to its own batch baseline.

Validation considers decision margins $0, 0.05, 0.15$ in Eq. (155). It chooses the candidate and margin with the lowest complete cost separately for each graph. The controls are native choices, the deterministic sector-and-truncation rule, and the single strategy with the lowest geometric mean training cost for that graph. A fixed strategy can include the older equation filter. We enable the new model only when its validation cost ratio to the fastest control is at most 0.95. Otherwise, the method uses that control. The threshold determines when the model is enabled; statistical interpretation comes from the separately reported test measurements. Unknown graph structures and features outside the fitted range use native choices.

The reserved comparison uses five repetitions of native choices, the deterministic rule, the fixed strategy per graph, and the selected method. The order rotates across batches and repetitions, giving every method the same number of appearances in each run position over the full test. Each run uses the same targets, prescribed basis, executable, and one processor. All methods retain the preceding exact implementation improvements, so the native control uses that same implementation with native selection rules. Full time includes input preparation, prediction, native IBP generation, exact reduction, coefficient checks, cleanup, and retries. Offline fitting, reference discovery, and saving measurement records stay outside the timer. Separate application checks receive the basis without target coefficients and compare their answers only after completion. The calling program and fitted parameters are loaded before the timing trials, and each trial starts a fresh Kira process. The additional application checks also record time with a newly started calling program; they do not supply a matched speed comparison for that condition.

We combine five-run median ratios within each graph and then give the seven graphs equal weight, as derived in Appendix B. The aggregate intervals resample graph groups. Per-graph plot bars show the range across the two reserved kinematic profiles. These ranges show the variation observed in two profiles. The sample is too small to support a general significance claim for the integral type.

D Selective seed expansion and exact basis completion

The seed-expansion study asks whether a smaller first calculation and an exact conversion of its remaining integrals can be cheaper than one large reduction. It uses the seven families in Appendix B and the target definitions in Appendix C. Development uses profile $h = 71$, training uses $h = 72, 73$, validation uses $h = 74$, and the reserved test uses $h = 75, 76$. These are separate rational parameter profiles. The graph structures and target index patterns occur in all splits, and d remains symbolic.

D.1 Changing individual seed bounds

The bounds $B_c = (r_c, s_c, q_c)$ retain the definitions in Appendix B: r_c bounds the sum of positive indices, s_c bounds the numerator rank, and q_c bounds the number of extra denominator powers. Each component starts from the largest value among the requested targets. We first enforce the sector lower bound in Eq. (163). The six new direct reductions use

$$B_c + \boldsymbol{\delta}, \qquad \boldsymbol{\delta} \in \{(1,0,0),(0,1,0),(0,0,1),(1,0,1),(1,1,0),(0,1,1)\}. \tag{165}$$

For a seed with t positive indices, summing $a_i = 1+(a_i-1)$ over those indices gives $r = t+q$. The positive-index and extra-power limits can therefore overlap, and different requested triples can give the same seed region after all limits are applied. The labels therefore identify requested limits rather than necessarily distinct equation sets. Their purpose is to allow selective expansion, such as requesting more numerator identities without also increasing both denominator limits. Sufficiency remains a property to check for the resulting system. Each direct reduction keeps the prescribed-basis check described in Ref. [9]. If it fails, the calculation calls the preceding fixed graph-specific method and includes both attempts in its time.

D.2 Completing a temporary basis

We also test two ways to complete a smaller reduction. The first stage uses B_c and supplies the preferred masters, while allowing Kira to return additional integrals. Kira supports this behaviour when its early master-basis check is disabled [9]. Let M_m denote prescribed masters and let U_u denote additional integrals that remain in the first-stage output. For target I_t, this stage returns exact relations

$$I_t = \sum_m A_{tm} M_m + \sum_u C_{tu} U_u. \tag{166}$$

Here A_{tm} is the coefficient of prescribed master M_m, and C_{tu} is the coefficient of unresolved integral U_u. These coefficients are rational functions of the kinematics and d. The integrals U_u are unresolved in the smaller first-stage system and need not be independent masters of the full family. Their index vectors define the targets of the second reduction, allowing that calculation to focus on the integrals that actually remain. That request uses its own componentwise target bounds plus $(1,1,1)$ and again supplies the prescribed basis. One candidate uses the maximal sectors of the U_u and their subsectors; the other keeps the original top-level sectors. Both retain the positive-index lower bound.

If the second reduction succeeds, it gives

$$U_u = \sum_m H_{um} M_m. \tag{167}$$

The coefficient H_{um} multiplies master M_m in the reduction of U_u. Substitution of Eq. (167) into Eq. (166) yields

$$I_t = \sum_m A_{tm} M_m + \sum_u C_{tu} \sum_m H_{um} M_m \tag{168}$$

$$= \sum_m \left(A_{tm} + \sum_u C_{tu} H_{um} \right) M_m. \tag{169}$$

The final coefficient matrix is therefore $A + CH$. Computing its entries exactly in the rational-function field, cancelling common factors, and checking the final integral labels gives a reduction in the prescribed basis. When either reduction fails, the preceding method supplies a full fallback. If no additional integral occurs, the second reduction is unnecessary. All basis conversion and fallback costs enter the timer. Independent reference coefficients check the final result only; the policy and the two reduction stages do not receive them.

D.3 Training, model selection, and comparison

The development set compares the preceding method, six direct expansions, and two basis-completion choices. We retain the preceding method and each case's fastest candidate when it lowers that case's complete time by at least ten percent. This rule selects candidates before training and does not inspect reserved data.

We then fit the ten model types and nine target features defined in Appendix C, separately for each graph. The new response is $\log(t_{cj}/t_{c\mathrm{prev}})$, so it directly measures improvement over the preceding method. Here t_{cj} is the median complete time of candidate j on batch c, and $t_{c\mathrm{prev}}$ is the corresponding time of the preceding method. Two repetitions supply each training label, and three repetitions supply each validation cost. A fixed strategy minimizes geometric mean training time for each graph. Validation first selects the faster control between that fixed strategy and the preceding method. It then tests the fitted predictors at margins $0, 0.05, 0.15$ and enables one only when it lowers the control's validation time by at least five percent. The fitting formulas and their derivations remain those in Appendices B and C.

The fixed control may call the learned method retained from the preceding study. Its purpose is to isolate the benefit of the new choice among candidates while preserving the earlier improvements in every method.

The chosen graph policy and candidate set are fixed before the reserved tests. Inputs with another graph structure or target features outside the training range use the preceding method. That method retains its own native fallback. Each reserved batch compares native choices, the preceding method, the new fixed strategy, and the selected new method on the same computer. Five repetitions use a rotating order. All methods use the same executable and exact final coefficient checks. The time includes preparation, prediction, Kira, any basis conversion, checking, cleanup, and every retry. Offline training, independent reference generation, and writing benchmark records remain outside these times. The selection program and fitted parameters are loaded before these trials; each attempt starts a fresh Kira process. Separate application tests start a new caller and supply only the targets, family, and prescribed basis. They check the coefficients after the reduction.

We first take the median time for each batch and method, then combine ratios within each graph and give all seven graphs equal weight. The geometric-mean derivation in Appendix B also applies here. Descriptive aggregate intervals resample the seven graph groups. The per-graph bars show the range over the two reserved parameter profiles. These ranges describe the tested profiles and do not establish a gain for all integrals with the same graph.

E Learned choices of integral and propagator order

This appendix specifies the ordering choices and explains why consistent relabelling leaves the integrals unchanged. The comparison uses the family definitions and target patterns in Appendix C. We use parameter profile 81 for development, profiles 82 and 83 for training, profile 84 for validation, and profiles 85 and 86 for the reserved comparison. Each profile uses the rational parameters defined there, while the dimension d remains symbolic. These are new parameter values of known graphs and target patterns.

E.1 What the choice changes

Kira provides eight integral orderings [8]. Order one treats numerator powers as simpler than extra denominator powers, called dots; order two reverses that preference. Orders three and

four first compare their sum and then favour numerator powers or dots, respectively. Orders one through four compare sectors by sector number. Orders five through eight use the corresponding index rules but first compare the number of positive indices and favour subsectors of the declared top sectors. We also test the four automatic propagator permutation schemes of Kira [9]. The final coefficients use the original index convention in every comparison.

The development candidates retain the preceding method, test each integral order from two through eight with propagator scheme one, and test propagator schemes two through four with integral order one. These are eleven initial choices. For each development batch, we also combine its fastest nondefault integral order with its fastest nondefault propagator scheme and time that pair. This adds one development calculation per batch. The preceding seed policy remains active in each new choice. It can use truncation, selected seed bounds, or temporary-basis completion as defined in the preceding appendices. A changed ordering is applied to each Kira attempt in that method. If it fails to reduce to the prescribed basis, we repeat the preceding method with its original settings. Both attempts enter the recorded time. This finite comparison does not search every joint ordering or every possible elimination sequence.

To see why changing labels preserves an integral, let π be a permutation of the N propagator positions. Set $D'_i = D_{\pi(i)}$ and $a'_i = a_{\pi(i)}$. Reindexing the product gives

$$\prod_{i=1}^{N} (D'_i)^{-a'_i} = \prod_{i=1}^{N} D_{\pi(i)}^{-a_{\pi(i)}} = \prod_{j=1}^{N} D_j^{-a_j}, \qquad I_{D'}(\mathbf{a}') = I_D(\mathbf{a}). \tag{170}$$

Here I_D denotes the integral defined by propagators D_i. Since the relabelling also leaves the measure unchanged, it preserves the integral. Changing the priority of integrals instead changes the sequence in which Kira solves the IBP equations. If $A\mathbf{I} = 0$ denotes those equations and P is the permutation matrix that reorders the integral columns, then

$$\mathbf{J} = P\mathbf{I}, \qquad AP^{-1}\mathbf{J} = A\mathbf{I} = 0. \tag{171}$$

The permutation therefore represents the same equations with a different column order. In a finite calculation, however, the ordering can also affect which equations are selected, so the equivalence of representations does not replace the checks of completeness and exact output coefficients. We require every target to reduce to the prescribed master basis and compare every output coefficient with an independent expanded-seed reduction. The predictor receives neither those coefficients nor the reference reduction steps.

E.2 Training, validation, and timing

For each development batch, we retain the fastest candidate if its full time is at least ten percent below the preceding method. We also retain the preceding method. The resulting candidate set is fixed before training. We fit the same ten predictor types per graph as in Appendix D, using the same nine features and complete-time log-ratio labels. Training has two measurements per candidate and batch; validation has three. Validation chooses a fixed control and enables a predictor only when it saves at least five percent against that control. We freeze the models, controls, and selection requirements before using the reserved profiles.

Each application selects its ordering before Kira creates integral weights and generates equations. The prediction is made once per requested batch. The selected method retains the earlier seed policy, and Kira performs all algebraic operations. The decision selects a setting for the complete reduction before it starts. Kira continues to choose individual row operations, as required by the earlier row-model comparison.

The reserved comparison measures native choices, the preceding method, the training-selected fixed setting for each graph, and the new selected method. It uses five repetitions with rotating

execution order. The same worker handles all methods for each batch. The timer includes prediction, input preparation, native equation generation, exact algebra, basis conversion where needed, coefficient checks, fallback, and cleanup. The caller and models are already loaded, and each attempt starts a new Kira process. Offline development, reference construction, training, and record writing are excluded. Additional applications receive the target indices and master basis without reference coefficients. Their results are checked independently after the reduction.

F Selecting the number of Kira workers

The worker-count study retains the seed strategy from Section 22 and varies only the number of Kira workers used for a requested batch. The families and target patterns are those in Appendix C. Parameter profile 91 supplies the development test, profiles 92 and 93 supply training, profile 94 supplies validation, and profiles 95 and 96 supply the reserved comparison. The dimension remains symbolic. Target patterns overlap across these sets; the rational kinematic values differ.

F.1 Choices and exactness

The preceding learned method chooses seed bounds and can reduce through a temporary basis. We retain that method and let a new policy choose one, two, or four Kira workers for each requested batch. The choice applies to each Kira attempt, including a temporary-basis conversion. Kira supports parallel symbolic reduction [2]. Changing the worker count changes how operations are scheduled, while each operation continues to use exact rational arithmetic. Every output must reach the prescribed master basis and agree with an independent expanded-seed reduction. If a new choice fails, its time and the repeat with the preceding one-worker method both enter the complete time.

Let $A\mathbf{I} = 0$ denote the IBP system. An exact elimination step left-multiplies it by an invertible elementary matrix E, so

$$A\mathbf{I} = 0 \quad \Longleftrightarrow \quad EA\mathbf{I} = 0. \tag{172}$$

Multiplication by E proves the forward implication, and multiplication by E^{-1} proves the reverse. Repeating valid steps therefore preserves the system, provided parallel execution respects their dependencies. Kira manages those dependencies through its existing workers, while the predictor sets only their number. This argument preserves relations but does not prove that a finite seed set is sufficient. The basis and coefficient checks therefore remain part of every measured reduction.

F.2 Cost labels and selection

Write x for the nine target features defined in Appendix D, and $q \in \{1, 2, 4\}$ for the worker count. For training batch b, let $t_{b,q}$ be the median complete time of two measurements. The training label is

$$y_{b,q} = \log(t_{b,q}/t_{b,1}). \tag{173}$$

Because the logarithm is strictly increasing and $t_{b,1} > 0$ is common to all choices for that batch, minimizing this label also minimizes $t_{b,q}$. We fit the same ten predictor types per graph as in Appendix D. They include constants, trees, bagged trees, regularized linear fits, and nearest-neighbour fits.

The training-selected fixed control minimizes $\sum_b t_{b,q}$. Validation uses three measurements per choice and batch. It first selects the faster of that fixed control and the preceding method,

using the sum of batch medians. It then compares each predictor with decision margins 0, 0.05, and 0.15 in log relative cost. A predictor is enabled only if its summed median cost is at least two percent lower than the control and its summed cost is lower in at least two of the three validation repetitions. Otherwise the control stays active. This objective measures the time for the workload by summing batch costs. It therefore gives a different weighting from the geometric mean that treats graphs equally. The selected controls and models remain fixed throughout the reserved tests. Unknown graph structures or features outside the training range use the preceding method.

F.3 Complete-time comparisons

The reserved comparison has 36 batches and five repetitions. It measures native choices with one worker, the preceding learned method with one worker, that same method with four workers throughout, a training-selected fixed worker count per graph, and the final selected method. All methods use the same integrated executable and its common earlier optimizations. The native control uses Kira's native seed and reduction choices in that executable. All methods run sequentially in the same four-core allocation. Execution order rotates between repetitions. Thus the always-four-worker control measures the cost of using the extra cores without target-dependent selection.

Full time includes model decisions, input preparation, Kira startup, equation generation, exact algebra, any basis conversion, coefficient checks, fallback, and cleanup. The caller and models are already loaded, while every attempt starts a new Kira process. Offline reference construction, training, and record writing are excluded. CPU time sums user and system time of the caller and its completed child processes. This measures the CPU time actually consumed. Every method has four reserved cores, including the methods that use only one worker. The comparison reports both the sum of batch medians and the graph-weighted geometric ratios defined in the earlier appendices. Additional applications receive the targets and basis without reference coefficients and are checked after reduction.

G Connected scattering families and the initialization test

The restricted vacuum recurrence uses a particular denominator structure. To examine a different physical problem, we construct connected massless four-point families with three independent external momenta. Let $p_1+p_2+p_3+p_4=0$, $p_i^2=0$, $P=p_1+p_2$, and $Q=p_1+p_2+p_3=-p_4$. We define

$$\begin{aligned} s=P^2, \qquad t=(p_2+p_3)^2,\\ 2p_1\cdot p_2=s, \qquad 2p_2\cdot p_3=t,\\ 2p_1\cdot p_3=-s-t. \end{aligned} \tag{174}$$

The first two scalar-product relations follow by expanding the squared momenta and using $p_i^2=0$. Expanding $Q^2=p_4^2=0$ then gives the third relation. Dimension d remains symbolic. Rational profiles use $s=-1009-12h$ and $t=-1021-14h$ in a common squared-momentum unit. The initial probes use $h=211$, the short comparison uses $h=223,224$, and the longer comparison uses $h=225$. An attempted symbolic probe produces no retained measurement and is excluded from the numerical results.

For an L-loop four-point ladder, the physical inverse propagators are

$$\begin{aligned} D_i=\ell_i^2, \qquad D_{L+i}=(\ell_i-P)^2 \quad (1\le i\le L),\\ D_{2L+i}=(\ell_i-\ell_{i+1})^2 \quad (1\le i<L),\\ D_{3L}=(\ell_1-p_1)^2, \qquad D_{3L+1}=(\ell_L-Q)^2. \end{aligned} \tag{175}$$

The Feynman prescription of Eq. (3) is implicit. There are $2L + (L-1) + 2 = 3L + 1$ physical propagators. The differences between adjacent loop momenta connect the loops, while the two end propagators carry the external momentum dependence. This four-point ladder differs from the three-loop two-point ladder in the earlier comparisons.

The second family is a box with massless bubble subgraphs. Its physical inverse propagators are

$$\begin{aligned} D_1 &= \ell_1^2, \qquad D_2 = (\ell_1 - p_1)^2, \\ D_3 &= (\ell_1 - P)^2, \qquad D_4 = (\ell_1 - Q)^2, \\ D_{2j+1} &= \ell_j^2, \qquad D_{2j+2} = (\ell_j - \ell_1)^2 \quad (2 \le j \le L). \end{aligned} \tag{176}$$

There are $4 + 2(L-1) = 2L + 2$ physical propagators. Each additional loop couples to the box momentum ℓ_1. The family admits subloop integration, so it is a restricted scattering class; it does not represent an arbitrary L-loop graph.

For three independent external momenta, a complete scalar-product representation has

$$N = L + \frac{L(L-1)}{2} + 3L = \frac{L(L+7)}{2} \tag{177}$$

entries. The three terms count loop squares, distinct products of two loop momenta, and loop–external products. This is the usual counting used to complete Kira families [9]. Thus $N = 15, 22, 30, 39$ for $L = 3, 4, 5, 6$. We append independent auxiliary inverse propagators until this rank is reached. The candidates are ℓ_i^2 and $(\ell_i - v_j)^2$, where the ordered list of v_j contains the remaining loop momenta followed by p_1, p_2, p_3. Exact row reduction of their scalar-product coefficients retains a candidate only when it increases the rank.

Each batch in the earlier scattering comparison contains the full-sector integral, the integral with D_1 raised to power two, and the integral with one power of the first auxiliary inverse propagator in the numerator. All other physical indices are one and all remaining auxiliary indices are zero. A reference reduction uses positive-index limit $n + 2$, numerator limit one, and dot limit two, where n is the number of physical propagators. A second reduction increases the numerator limit to two and must reproduce all coefficients in the reference basis. At least one target must have a nonzero reduction that differs from the identity. A failed reference or failed enlargement prevents a completed-speedup claim for that case.

G.1 Why the declared sector domain closes under IBP

The initialization experiment restricts zero-sector classification to the declared top sectors and their subsectors. This restriction requires a mathematical justification because an omitted equation can otherwise prevent a complete reduction. Consider an index $a_j \le 0$ for a propagator outside the declared sector. Differentiating its factor gives

$$\frac{\partial}{\partial \ell_k^\mu} D_j^{-a_j} = -a_j D_j^{-a_j - 1} \frac{\partial D_j}{\partial \ell_k^\mu}. \tag{178}$$

The only increase of a_j is therefore multiplied by a_j. At $a_j = 0$ this term vanishes, and at $a_j < 0$ the increased index remains nonpositive. Rewriting scalar products in the numerator as linear combinations of inverse propagators can lower indices, but cannot introduce a new positive index. Consequently, IBP equations seeded in a sector remain within that sector and its subsectors. The union of several such domains also closes under these equations.

The test applies the existing zero-sector criterion within this domain. It does not declare omitted integrals to be zero. The experiment uses a single family, no cut propagators, and no symbolic propagator powers. It retains Kira's exact elimination and admits only the prescribed final

basis. Comparisons with the unchanged initialization check the coefficients on the seven earlier graph types before the new scattering tests are interpreted. This establishes the stated restricted implementation on the tested inputs; it does not prove that every optional symmetry or extra-relation procedure can be restricted in the same way.

For a single top sector containing n positive indices, its subsectors number 2^n, since each propagator is either present or absent. Classifying all N family entries instead considers 2^N sectors. At four loops this changes the candidate count from 2^{22} to 2^{13} for the ladder and to 2^{10} for the bubble box. These are counts of initialization candidates, not predictions of total runtime.

G.2 Separating the contributions to complete time

The component comparison uses the seven earlier graphs at profiles $h = 221, 222$ and the three full-sector targets with one physical propagator raised from power one to power three. The construction uses the first three physical propagators. It compares ten methods on the same targets and prescribed basis: native choices, fixed and learned graph strategies, fixed and learned seed expansion, fixed and learned worker counts, the equation filter, the seed filter, and the row model. Fixed settings come from the earlier training data, and all model parameters remain frozen.

Every completed method has three timing repetitions. The method order rotates within each case and repetition, and every method receives the same two-core allocation. The complete timer includes decisions, preparation, exact algebra, coefficient checks, retries, and cleanup. It excludes offline reference construction, training, and the initial loading of the caller and models. A larger reference reduction must reproduce the coefficients before a case enters the timing comparison. Ratios compare batch medians on matching completed inputs, using the complete-time definitions of this study. Cases that cannot complete remain in the failure table and receive no completed-time ratio.

The first scattering comparisons allow 120 seconds per native attempt, including a ten-second reserve for export checking and cleanup. A follow-up increases this allowance to 600 seconds. Both initially limit solver address space to 2,400 MiB and summed resident memory to 2,100 MiB. When the larger reference checks reach the memory limit, a further comparison reserves more memory through the scheduler and raises these limits to 6,144 and 5,200 MiB, respectively. Its native and learned methods use identical limits. Temporary files remain bounded by 32 MiB, and all earlier failed attempts remain in the results. A completed earlier reference can be reused as an offline coefficient check; every reported follow-up time comes from a new reduction.

The fixed control at each stage can retain choices from earlier stages. The first controls use the best fixed training settings. A further 96 paired reductions disable only the current learned decision and use its fixed validation control, while retaining earlier stages. This second comparison isolates the added choice more directly, while comparison with native choices measures the combined strategy. These effects need not add or multiply across stages, since a changed seed region alters the equations and later algebra. The new scattering comparison additionally tests the frozen equation and row models, the complete strategy selector with its domain checks, and two separate requests against their combined reduction. The separated requests use the same frozen seed limits as the combined request, which isolates repeated work under those settings. Failed filtered attempts retry with native choices and retain that cost.

G.3 Why the vacuum formula does not establish a scattering reduction

For the vacuum family, the derivation above gives closed recurrences in $z = L(d-2)$ under its specific denominator and mass assumptions. The families in Eqs. (175) and (176) also depend

on two external invariants. Dimensional scaling alone therefore permits dependence on their ratio t/s; it cannot determine that dependence from z. Applying the vacuum formula would require a new identity relating these scattering coefficients, which the vacuum derivation does not supply. We consequently exclude that formula from the scattering comparison. Combining compatible target requests remains a separate, generally applicable use of the linear reduction map, as derived in Appendix B.

H A single policy for complete reduction cost

The new policy selects a calculation before Kira generates its equations. This appendix specifies its inputs, training objective, recovery rule, and exact preprocessing changes. The measured results appear in Section 26.

H.1 Inputs and allowed calculations

For a target set $\mathcal{T}$, define

$$r_{\mathcal{T}} = \max_{\boldsymbol{a}\in\mathcal{T}} \sum_i \max(a_i, 0), \qquad s_{\mathcal{T}} = \max_{\boldsymbol{a}\in\mathcal{T}} \sum_i \max(-a_i, 0), \qquad d_{\mathcal{T}} = \max_{\boldsymbol{a}\in\mathcal{T}} \sum_i \max(a_i - 1, 0). \tag{179}$$

These definitions give the largest sum of positive indices, numerator rank, and number of extra denominator powers among the targets. The last quantity counts the powers above one, often called dots. The policy chooses from seven calculations. Four use integral ordering 1 and seed limits $(r_{\mathcal{T}}, s_{\mathcal{T}}, d_{\mathcal{T}})$, $(r_{\mathcal{T}} - 1, s_{\mathcal{T}} - 1, d_{\mathcal{T}} - 1)$, $(r_{\mathcal{T}} - 1, s_{\mathcal{T}}, d_{\mathcal{T}} - 1)$, or $(r_{\mathcal{T}}, s_{\mathcal{T}} - 1, d_{\mathcal{T}})$. Negative limits are replaced by zero, and the positive-index limit is at least the number of positive indices in the largest declared top sector. A fifth calculation uses the first set of limits with Kira ordering 5. A sixth applies the previously fitted equation filter with the first set of limits. The seventh calculation starts directly with each native limit increased by one. The equation-filter parameters remain fixed during this study.

The input vector contains 18 quantities obtained from the family and targets. They describe the number of loops, external momenta, inverse propagators, physical propagators, and symbolic invariants; the natural logarithm of one plus the target count; maximum and mean dots and numerator ranks; minimum and maximum positive-index counts; target-sector count; massive-denominator count; and three measures of loop incidence in physical denominators. The final input is the mean number of external momenta present in a denominator momentum. A loop occurs in a denominator when its momentum appears in that denominator's momentum expression. The incidence measures count denominators involving several loops, the largest number of loops in one denominator, and the largest number of denominators containing one loop. Target coefficients and reference reductions are absent from these inputs.

H.2 Physical inputs and target construction

The seven earlier graph structures retain the ordered denominator momenta and scalar-product conventions of Appendix A. In a common squared-momentum unit, the sunrise has squared masses $(3, 7, 13)$ and the vacuum has squared masses $(1, 1, 1, 13, 13, 53 + 2h)$. The two-point ladder has squared masses $(1, 0, 0, 0, 0, 0, 0, 0)$ on its physical denominators. The planar double box, nonplanar box, pentagon, and virtuality box have massless internal denominators. The

parameter assignments are

$$\begin{aligned} p^2_{\text{sunrise}} = p^2_{\text{ladder}} &= -1009 - 12h, \\ (s,t)_{\text{DB}} &= (-1009 - 12h, -1021 - 14h), \\ (s,t)_{\text{NP}} &= (-113 - 18h, -127 - 10h), \\ (s_{12}, s_{23}, s_{34}, s_{45}, s_{51})_{\text{P}} &= (-11 - 20h, -13 - 20h, -17 - 20h, -19 - 20h, -23 - 20h), \\ (s,t,v)_{\text{box}} &= (-113 - 18h, -127 - 10h, -149 - 12h). \end{aligned} \tag{180}$$

Here h is the integer profile number, v is the external virtuality of the box, and the remaining invariants have their family-specific definitions in Appendix A. These assignments leave the dimension symbolic. The connected bubble boxes instead use Eq. (174) and its rational profile formula.

The seven earlier graph structures use profiles $h = 301, 302$ for training, $h = 303$ for validation, and $h = 304$ for testing. Each profile supplies three target patterns: two extra powers on a denominator; a dot with an auxiliary numerator, or two dots when no auxiliary entry exists; and a removed denominator with a dot on an adjacent denominator. Each batch has up to three targets, with positions shifted by the profile. The three- and four-loop bubble boxes of Eq. (176) use profiles $301, 303, 304$ and the full-sector, one-dot, and one-numerator targets. The symbolic nonplanar box, pentagon, and external-virtuality box use one target per split, with power three on the first, second, or third physical denominator. Their target lists are disjoint. Thus the tests measure new kinematics and targets within known graph structures.

The following rule specifies the rational target lists. Let n be the number of physical denominators, let N be the total number of family entries, and let $\boldsymbol{a}^{(0)}$ contain n ones followed by $N - n$ zeros. For $j = 0, 1, 2$, set $i_j = 1 + [(h + j) \bmod n]$ and $i_j^+ = 1 + (i_j \bmod n)$. With $\boldsymbol{e}_i$ the unit index vector, the dotted and pinched targets have indices

$$\boldsymbol{a}_j^{\text{dots}} = \boldsymbol{a}^{(0)} + 2\boldsymbol{e}_{i_j}, \qquad \boldsymbol{a}_j^{\text{pinched}} = \boldsymbol{a}^{(0)} - \boldsymbol{e}_{i_j} + \boldsymbol{e}_{i_j^+}. \tag{181}$$

For the mixed targets with $N > n$, set $k_j = n+1+[(h+j) \bmod (N-n)]$ and use $\boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j} - \boldsymbol{e}_{k_j}$. When $N = n$, use $\boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j} + \boldsymbol{e}_{i_j^+}$. Repeated vectors are kept once. The target patterns are shared across the rational splits, and some index sets recur; their kinematic profiles remain separate. The comparison therefore does not test arbitrary unseen index patterns.

The common basis for each batch comes from a native reduction with seed limits $(r_{\mathcal{T}} + 1, s_{\mathcal{T}} + 1, d_{\mathcal{T}} + 1)$. For a reserved batch, a second native reduction uses $(r_{\mathcal{T}} + 1, s_{\mathcal{T}} + 2, d_{\mathcal{T}} + 1)$ and must reproduce the reference coefficients in the same basis. Both calculations keep the positive-index floor specified above. A failed reference or enlarged check excludes that batch from speedup estimates. The conditional four-loop confirmation, when needed, uses the separately stated pair of nested regions.

H.3 Training from measured cost

Let T_{ca} be the measured complete time for training case c and initial choice a, including every recovery attempt. Let $a = 0$ denote native equation choices with the unshifted target limits. We fit the log time ratio

$$y_{ca} = \log \frac{T_{ca}}{T_{c0}}, \qquad \mathcal{L} = \sum_c \sqrt{T_{c0}} \sum_a \left[f_a(\boldsymbol{x}_c) - y_{ca} \right]^2 . \tag{182}$$

Here $\boldsymbol{x}_c$ is the input vector and f_a predicts the cost of choice a relative to the native choice. The square-root weight gives longer calculations more influence without making the training loss proportional to their full duration. This weighting is a design choice of this study.

We compare a regression tree, a random forest, and an ensemble of extremely randomized trees [30, 48, 39]. Each ensemble has 48 trees. The candidate depths are 3, 5, and 8, and each leaf requires at least two training cases. Every tree predicts all seven costs. For an ensemble with B trees, the prediction is the arithmetic mean

$$f_a(\boldsymbol{x}) = \frac{1}{B}\sum_{b=1}^{B} f_{ba}(\boldsymbol{x}), \tag{183}$$

where f_{ba} is tree b's prediction for choice a. The production evaluator follows the fitted tree splits directly, and its predictions are checked against the training library before validation.

For any fixed case, $T_{c0} > 0$ is independent of the selected choice. Since the logarithm is strictly increasing,

$$\operatorname*{argmin}_a T_{ca} = \operatorname*{argmin}_a \frac{T_{ca}}{T_{c0}} = \operatorname*{argmin}_a y_{ca}. \tag{184}$$

This identity explains the use of relative log time as the training target; it does not make a fitted prediction exact. Validation selects the ensemble and a gate $g \in \{0, 0.05, 0.15\}$. The predicted best choice is used only when its predicted log cost is at least g below that of the best fixed validation choice. Otherwise the fixed choice is used. Equivalently, the required predicted time ratio is at most e^{-g}. Selection minimizes the sum of validation batch median times, and the model then remains fixed during testing. The fixed comparison uses the same validation-selected setting for every test case.

If the selected calculation cannot reduce the targets into the supplied master basis, the driver tries native choices and then increases each native seed limit by one. The original policy removes repeated action names; different names can still produce equal effective bounds. The later portfolio also removes these equivalent attempts. If the last attempt fails, the request is recorded as unfinished. In timed tests, exact rational-function comparison checks every returned coefficient against the reference. Additional applications withhold the reference coefficients until the selected reduction has finished. Reference construction and training have separate cost records. Initial loading of the caller and model occurs outside the repeated timer, so these measurements do not describe the latency of starting a new application. The comparisons concern repeated reductions with an available basis, rather than the full cost of discovering that basis for a new problem.

H.4 Comparison with a fixed setting for each graph

Let $\mathcal{V}_H$ be the validation cases for graph structure H, and let $\widetilde{T}_{ca}$ be the median of the two validation times for case c and initial choice a. The fixed setting for that graph is

$$a_H = \operatorname*{argmin}_a \sum_{c \in \mathcal{V}_H} \widetilde{T}_{ca}. \tag{185}$$

This definition chooses the least measured validation cost subject to using one setting for the whole graph. It supplies a stronger fixed control than a setting shared by all graphs.

For each graph with at least four training cases, we also fit three regression trees with maximum depths one, two, and three and a minimum of two training cases per leaf. Two additional inputs describe how extra denominator powers are distributed. For case c with target set $\mathcal{T}_c$, define

$$A_c = \max_{\boldsymbol{a} \in \mathcal{T}_c} \max_i a_i, \qquad B_c = \frac{1}{|\mathcal{T}_c|} \sum_{\boldsymbol{a} \in \mathcal{T}_c} \sum_i \mathbf{1}_{a_i > 1}, \qquad \boldsymbol{z}_c = (\boldsymbol{x}_c, A_c, B_c). \tag{186}$$

The indicator $\mathbf{1}_{a_i>1}$ equals one when the propagator power exceeds one and zero otherwise. Thus A_c is the largest individual propagator power and B_c is the mean number of dotted propagators

per target. The patterns $(3, 1, \ldots)$ and $(2, 2, \ldots)$ both contain two dots, while these inputs record their different distributions. The local trees use Eq. (182) with the 20-component vector $\boldsymbol{z}_c$ and the same labels and weights, restricted to that graph's training cases. The global model continues to use the first 18 components.

Validation compares these local trees with the selected global model and considers gates $0, 0.05, 0.15$, now measured against a_H. It selects the predictor and gate with the least summed batch median cost. The graph-specific trees use no new reduction labels and remain fixed during the comparison.

Write $\pi_H(\boldsymbol{z}_c)$ for the resulting prediction in graph H. The deployed policy uses π_H only when

$$\sum_{c \in \mathcal{V}_H} \widetilde{T}_{c,\pi_H(\boldsymbol{z}_c)} \le 0.98 \sum_{c \in \mathcal{V}_H} \widetilde{T}_{c,a_H}, \tag{187}$$

and the predicted-choice total is strictly smaller in each of the two validation repetitions. Otherwise it uses a_H. The factor 0.98 is a chosen two-percent margin. The local-model fitting and selection procedure was specified before any reserved timings were collected. This rule compares measured validation cost; it is not a probability that a future request will be faster. It preserves the global model's weights and the exact recovery sequence.

Three rotated repetitions compare graph-specific fixed settings, the frozen cost model, and this validated policy on each reference-checked reserved batch. These are separate matched measurements from the native and global-fixed comparison. The public interface also checks the denominator structure and target-feature ranges against the training definitions. Unknown structures or values outside those ranges retain native choices. These checks restrict deployment to the recorded domain without asserting that the model generalizes beyond it.

Validation selects from the measured candidate-reduction costs. The reserved comparisons additionally time the actual feature construction, prediction, and deployment checks. They therefore measure any overhead that the cost-based validation selection omits.

H.5 An exact simplification of sector initialization

Kira's zero-sector test uses the parametric polynomials U and F [49, 9]. A sector is zero when its integrals vanish in dimensional regularization. The present change simplifies the input polynomial of the existing test within the declared sector domain. It preserves the full polynomials used by other Kira operations.

Let $\mathcal{A}$ contain all propagator indices present in at least one declared top sector, and let $R_{\mathcal{A}}$ set the Feynman parameter b_j to zero for every $j \notin \mathcal{A}$. For a polynomial $P(\boldsymbol{b})$, write the weighted Euler expression used in the criterion as

$$C_{\boldsymbol{\kappa}}[P] = \sum_j \kappa_j b_j \frac{\partial P}{\partial b_j} - P. \tag{188}$$

The unknown constants κ_j are the coefficients determined by the sector test. Apply $R_{\mathcal{A}}$ to each term. If $j \notin \mathcal{A}$, the explicit factor b_j makes the term zero. If $j \in \mathcal{A}$, substitution in the other variables commutes with differentiation in b_j. Therefore

$$\begin{aligned} R_{\mathcal{A}} C_{\boldsymbol{\kappa}}[P] &= \sum_{j \in \mathcal{A}} \kappa_j b_j R_{\mathcal{A}} \frac{\partial P}{\partial b_j} - R_{\mathcal{A}} P \\ &= \sum_{j \in \mathcal{A}} \kappa_j b_j \frac{\partial (R_{\mathcal{A}} P)}{\partial b_j} - R_{\mathcal{A}} P. \end{aligned} \tag{189}$$

For every admitted sector $S \subseteq \mathcal{A}$, its further substitution satisfies $R_S R_{\mathcal{A}} = R_S$. Constructing the criterion from $R_{\mathcal{A}} U$ and $R_{\mathcal{A}} F$ therefore gives the same polynomial constraints as constructing the full criterion and then restricting it to S. The implementation applies this simplification only together with declared-sector initialization, without cut propagators or symbolic propagator powers. The derivation concerns the zero-sector criterion; it does not remove auxiliary numerator factors from the integrals.

A second exact change enumerates the sector files that exist before triangular reduction and back-substitution. It reads those files in the original sector order with the original loading functions. Missing files contain no equations, so removing their repeated existence checks leaves the loaded system unchanged. Both preprocessing changes are deterministic. Their timing benefit is measured separately from the policy's choice of equations and seed limits.

H.6 Conditional four-loop reference confirmation

The generic enlarged-reference check does not complete for the original reserved four-loop bubble box. A rule registered before model fitting therefore adds profile $h = 305$, with reference seed limits $(r, s, d_{\mathrm{dots}}) = (12, 1, 2)$ and check limits $(12, 2, 2)$. Here d_{dots} is the dot limit, distinct from the spacetime dimension. The development study had already verified this pair of nested regions. The new profile uses the same frozen model and the same three timing repetitions for native, fixed, and learned choices. It supplies a separate bounded confirmation; the unfinished original profile is retained in the failure count.

I Complete-cost portfolio: definitions and derivation

I.1 Targets and seed choices

For a batch $\mathcal{T}$ of index vectors $\mathbf{a}$, define

$$r_0 = \max_{\mathbf{a}\in\mathcal{T}} \sum_i \max(a_i, 0), \qquad s_0 = \max_{\mathbf{a}\in\mathcal{T}} \sum_i \max(-a_i, 0), \qquad d_0 = \max_{\mathbf{a}\in\mathcal{T}} \sum_i \max(a_i - 1, 0). \tag{190}$$

These definitions count positive powers, numerator rank, and dots separately. Selective expansion uses $(r_0, s_0 + 1, d_0)$, with the positive-index limit floored at the largest declared top-sector size. The full expansion uses $(r_0 + 1, s_0 + 1, d_0 + 1)$. The five portfolio choices combine the earlier validated policy or selective expansion with one or two workers, and retain the previous workflow as a fifth choice. A failed new-policy attempt proceeds to native seeds and then full expansion. Skipping identical effective configurations removes a repeated calculation with the same equations, ordering, basis, and filter; it introduces no new algebraic relation.

The training and validation cases are the 47 and 26 cases defined in Appendix H. The reserved rational cases use the same family definitions at $h = 306$ and the target constructions with $j = 0, 1, 2$. The three symbolic graphs instead use the corner target with $a_4 = 3$, so the fourth physical denominator has power three. For the four-loop bubble box, the reference uses $(r, s, d_{\mathrm{dots}}) = (12, 1, 2)$ and its independent check uses $(12, 2, 2)$. All other reserved references use full expansion and then raise its numerator-rank bound by one more unit. These limits were fixed before any new reserved calculation. A common supplied basis permits exact coefficient comparisons; these checks do not prove its minimality.

I.2 Learning relative complete cost

Let $T_{ba} > 0$ denote the measured complete time for training batch b and portfolio choice a. Let a_0 denote the earlier cost policy with one worker. We define

$$y_{ba} = \log \frac{T_{ba}}{T_{ba_0}}. \tag{191}$$

For the same batch, subtracting two labels gives $y_{ba} - y_{ba'} = \log(T_{ba}/T_{ba'})$. Since the logarithm is increasing, the choice with the smallest exact label also has the smallest measured time. A regression model f estimates the five labels from the twenty features x_b given in Appendix H. Its fitted objective is

$$\mathcal{L}(f) = \sum_{b \in \mathcal{B}_{\text{train}}} \sqrt{T_{ba_0}} \sum_{a} \big[f_a(x_b) - y_{ba}\big]^2. \tag{192}$$

The positive weight increases the influence of costly training batches without changing their preferred action. It is a chosen training objective, not a proof that the fitted model minimizes future wall time. Each label includes native recovery, so an unsuccessful first attempt carries its measured cost.

We fit nine global models: a regression tree, a 48-tree random forest, and a 48-tree extremely randomized ensemble at depths 3, 5, and 8. Each leaf has at least two training cases. Graphs with at least four training cases also receive three local regression trees at depths 1, 2, and 3. Tree methods follow the cited constructions [30, 48, 39]. The saved evaluator reproduces the fitted predictions before validation begins.

I.3 Validation and the deployment guard

For graph g, let c_g be the fixed portfolio choice with the smallest sum of validation batch median times. If $a_\star = \arg\min_a f_a(x)$, a predictor with gate γ proposes $a_\star$ only when

$$f_{a_\star}(x) < f_{c_g}(x) - \gamma, \qquad \gamma \in \{0, 0.05, 0.15\}. \tag{193}$$

Otherwise it uses c_g. Under exact labels, exponentiation gives $T_{ba_\star}/T_{bc_g} < e^{-\gamma}$; with fitted labels this is a predicted saving, not a guarantee. Validation selects the predictor and gate for each graph. Deployment enables this outer prediction only if its summed validation median time is at most 0.98 times the fixed-control time and it wins both validation repetitions. The alternative is c_g. Graph structure and feature-range checks send unsupported inputs to native settings. A fixed portfolio choice can contain the earlier policy; the outer guard evaluates only the added decision.

I.4 Correctness and timing

The input to each reduction is the family, target batch, and prescribed master basis. Kira generates the exact identities and returns symbolic coefficients in that basis. The test then checks each coefficient by exact rational-function subtraction against the independently checked reference. The caller does not provide those reference coefficients to the deployed predictor or to the application. Thus the learned choice affects which calculation is attempted, while the algebraic identities determine the returned relation. Failure to reach the supplied basis triggers recovery; exhausting the bounded attempts returns failure.

For method m, the reported aggregate is $\sum_b \text{median}_{r=1,2,3}\, T_{bmr}$. The speedup relative to control c is the ratio of its aggregate to the selected aggregate. This follows by dividing the times required for the same collection of batches. We also report totals over all repetitions and process CPU

times. These summaries describe the measured collection; they are not statistical confidence intervals or guarantees for unseen integral families. All methods use the same exact executable and two-core allocation. The earlier preparation study uses a separate one-core allocation, so its speedup factors are not multiplied by these portfolio ratios.

J Exact restrictions and selected-equation reuse

J.1 Restricting before constructing the polynomials

Let b_i be the Schwinger parameter associated with inverse propagator D_i. For algebraic preparation, suppress its infinitesimal prescription and write the quadratic form with the same Minkowski sign convention as the integral definition:

$$-\sum_i b_i D_i = -\boldsymbol{\ell}^{\mathsf{T}} H(b) \boldsymbol{\ell} + 2w(b)^{\mathsf{T}} \boldsymbol{\ell} + J(b). \tag{194}$$

Here H is a symmetric $L \times L$ matrix of scalar coefficients, w is a vector whose entries are external-momentum combinations, and J contains no loop momentum. Lorentz contractions are implicit in the quadratic form. Completing the square gives

$$-(\boldsymbol{\ell} - H^{-1}w)^{\mathsf{T}} H (\boldsymbol{\ell} - H^{-1}w) + J + w^{\mathsf{T}} H^{-1} w. \tag{195}$$

The standard parametric construction [55], with an inessential normalization scale suppressed, therefore uses the polynomials

$$U = \det H, \qquad F = J \det H + w^{\mathsf{T}} \operatorname{adj}(H) w, \tag{196}$$

where $\operatorname{adj}(H)$ is the adjugate matrix. The inverse serves only in the derivation; Eq. (196) remains a polynomial identity when a substituted matrix is singular.

Let B contain the propagator positions that occur in at least one declared top sector. Define R_B by setting $b_i = 0$ for $i \notin B$. Substitution preserves addition and multiplication. Expanding the determinant over permutations π gives

$$R_B(\det H) = R_B\left(\sum_\pi \operatorname{sgn}(\pi) \prod_{j=1}^{L} H_{j,\pi(j)}\right) \tag{197}$$

$$= \sum_\pi \operatorname{sgn}(\pi) \prod_{j=1}^{L} R_B(H_{j,\pi(j)}) = \det(R_B H). \tag{198}$$

Each entry of the adjugate is a signed minor, so the same argument gives $R_B \operatorname{adj}(H) = \operatorname{adj}(R_B H)$. Applying these equalities to Eq. (196) shows that constructing U, F after the substitution produces exactly $R_B U, R_B F$.

Every admitted sector S is a subset of B. Setting the remaining absent parameters to zero then gives

$$R_S R_B P = R_S P \tag{199}$$

for any polynomial P. The sector polynomials used in zero-sector and symmetry checks are consequently identical to those obtained from the full construction. The Euler criterion also agrees because substitution commutes with differentiation in every retained variable, as derived in Appendix H. This proof concerns sector polynomials. It does not set an auxiliary inverse propagator to zero in an integral numerator. The implementation retains those propagators in all IBP and numerator operations. It enables the restriction only for declared-sector calculations without cuts or symbolic propagator powers.

J.2 Reusing an unchanged set of exact equations

For one generated system, let $Q = (q_1, \ldots, q_k)$ be its ordered list of selected source-equation numbers and let $E_q = \sum_i c_{qi} I_i$ be the corresponding unmodified symbolic row. The first retrieval supplies $E_{q_1}, \ldots, E_{q_k}$ to the basis check. If the subsequent selection returns $Q' = Q$, then

$$(E_{q'_1}, \ldots, E_{q'_k}) = (E_{q_1}, \ldots, E_{q_k}). \tag{200}$$

Passing the saved rows to the final reduction therefore supplies the same exact identities. Copies of the original rows preserve them when the basis check sorts or alters its working rows. We require equality of the full ordered source list, keep the same system and coefficient treatment, and use ordinary regeneration when that condition fails. Storage allows at most 4,096 rows and 4 MiB of accounted row payload, plus bounded container overhead. A development check forces a one-byte allowance and confirms that ordinary regeneration returns the same coefficients. This is within-calculation reuse, with no transfer of coefficients between target requests.

J.3 Reserved inputs and comparison definitions

The rational families and target construction follow Appendix H with $h = 307$ and $j = 0, 1, 2$. The planar double box and connected bubble boxes use $s = -4693$ and $t = -5319$, and the two-point families use $p^2 = -4693$. The nonplanar and virtuality boxes use $s = -5639$ and $t = -3197$, with virtuality $v = -3833$ for the latter. The pentagon invariants $(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})$ are $(-6151, -6153, -6157, -6159, -6163)$. The three-loop vacuum uses the squared masses stated there with the last value $53 + 2h = 667$. Graph-specific momentum conventions remain those of Appendix A; the connected bubble boxes retain Appendix G's convention. Each dots, mixed, or pinched batch contains up to three distinct indexed targets. The symbolic nonplanar and pentagon targets have power three on the fifth physical denominator. The symbolic virtuality box has power two on its first and fourth denominators. Other physical powers are one and other auxiliary powers are zero.

References use (r_0+1, s_0+1, d_0+1), followed by an independent check with numerator-rank limit increased by one more unit. The four-loop bubble reference instead uses $(12, 1, 2)$ and $(12, 2, 2)$, fixed before testing. The first calculation uses the new exact preparation, and the second uses the previous executable. The model files and decision rules remain frozen throughout the comparison. The definitions of complete-time aggregation and relative speedup are those in Appendix I. All repetitions and individual batch results remain available; no confidence interval is inferred from three runs.

For the common-dispatch sensitivity calculation, let D_{bcr} be the measured input-check and outer-proposal interval for batch b, control c, and repetition r. We subtract that interval before taking the median:

$$T_c^{\text{optimistic}} = \sum_b \text{median}_{r=1,2,3}(T_{bcr} - D_{bcr}). \tag{201}$$

The unused proposal occupies only part of D_{bcr}. Holding the remaining measured work fixed, removing all of D_{bcr} therefore overstates the saving available from removing that proposal alone. This is a sensitivity calculation on the recorded timings, not a prediction of how a different executable would interact with the operating system.

K Definitions and timing for the coverage grid

The new grid uses the integral convention in Eq. (3); every denominator includes the same implicit Feynman prescription. Let $Q = \sum_{i=1}^{L} \ell_i$. For $L \geq 2$, the vacuum families have physical

denominators

$$D_i = \ell_i^2 - m_i^2 \quad (1 \le i \le L), \qquad D_{L+1} = Q^2 - m_{L+1}^2. \tag{202}$$

The one-mass case sets $m_1^2 = \cdots = m_L^2 = 0$ and $m_{L+1}^2 = 835$. The fully massive case sets $m_i^2 = 837 + 6(i-1)$ for each physical position. At one loop we instead use the single massive tadpole denominator $D_1 = \ell_1^2 - 835$, since the two putative vacuum denominators would have dependent scalar-product coefficients. These numbers use a common squared-momentum unit.

The two-point families replace the final vacuum denominator by

$$D_{L+1} = (p - Q)^2 - m_{L+1}^2, \qquad p^2 = -847. \tag{203}$$

The massless case sets every mass to zero; the massive case uses the same distinct squared masses as the fully massive vacuum. The four-point ladders and boxes with bubble subgraphs use Eqs. (175) and (176), with $s = -5917$ and $t = -6747$. The main grid uses profile $h = 409$. It defines these families through twelve loops before applying the current support limit. The two coincident one-loop definitions are included only once.

For E independent external momenta, the scalar products consist of L loop squares, $L(L-1)/2$ products of distinct loop momenta, and LE loop–external products. Their total is therefore

$$N = L + \frac{L(L-1)}{2} + LE = \frac{L(L+1)}{2} + LE. \tag{204}$$

This is the complete representation required for native IBP generation [9]. We append candidate squared momentum differences only when their exact coefficient vector increases the rank of the physical denominator list. The resulting $N \times N$ coefficient matrix has rank N for every declared family. This construction defines numerator coordinates even for cases that cannot enter the current application. In this grid the 31-index limit permits vacuum definitions through seven loops, two-point definitions through six loops, and four-point definitions through five loops. Permitted input size does not imply completion within the common limits.

Let n be the number of physical denominators and let $\boldsymbol{a}^{(0)}$ have entries one on these denominators and zero on the auxiliary ones. The target batch is

$$\mathcal{T} = \{\boldsymbol{a}^{(0)},\ \boldsymbol{a}^{(0)} + \boldsymbol{e}_1\} \cup \{\boldsymbol{a}^{(0)} - \boldsymbol{e}_{n+1} \text{ if } N > n\}, \tag{205}$$

where $\boldsymbol{e}_i$ has unit entry at position i. The full comparison and first reference use positive-index bound $r = n + 1$, numerator bound $s = 1$, and dot bound one. The independent reference check raises the numerator bound to two. The full comparisons receive no preferred master list. The application comparisons receive the checked basis and retain their frozen learned choices, domain guards, and expansion recovery.

The symbolic additions are the one-loop tadpole, the one-loop massless two-point family, massive two-point families at one and two loops, and four-point ladders at one and two loops. Their nonzero squared masses and external invariants remain independent symbols. The nonplanar box, pentagon, and virtuality-box definitions in Appendix A add rational and symbolic requests. Their rational profile is $h = 411$ in the previously defined parameter formulas. Each of these three structures uses the corner target and one extra power on its last physical denominator. These finite cases do not span arbitrary symbolic kinematics.

The additional massive two-point batches use rational profile $h = 419$, so $p^2 = -867$ and $m_i^2 = 857 + 6(i-1)$. Their symbolic variants retain independent masses and p^2. The target set includes $\boldsymbol{a}^{(0)}$, every $\boldsymbol{a}^{(0)} + \boldsymbol{e}_i$, every $\boldsymbol{a}^{(0)} + 2\boldsymbol{e}_i$, and every $\boldsymbol{a}^{(0)} + \boldsymbol{e}_i + \boldsymbol{e}_j$ with $1 \le i < j \le n$. It contains $1 + 2n + n(n-1)/2$ targets. The last count follows from choosing an unordered pair of distinct physical denominators. The expanded region uses $r = n + 2$, numerator bound one, and dot bound two; the independent check raises the numerator bound to two. These batches

supply a separate test of nontrivial reductions after the initial small requests, without replacing the original outcomes.

For the separate restricted vacuum study, the squared-mass ratios are the first R entries of $(823, 829, 853, 857)$, with $R = 2, 3, 4$. We choose six distinct exponent vectors $\boldsymbol{b}$ with $\sum_j b_j = 6$ and at least two positive entries. The vectors are fixed before the comparisons. Each request uses either one loop order or all twelve orders together. The exact recurrence and the Taylor check use the derivations in Section 6.12; this study does not infer those identities from a high-loop native reduction. The original Kira control generates and solves the full supplied identity system separately at each loop order. Other identity-based controls and the recurrence can substitute $z = L(d-2)$ after their shared calculation.

Table 26: The six target exponent vectors for each restricted vacuum profile.

R	Target vectors $\boldsymbol{b}$
2	$(5,1),\ (1,5),\ (4,2)$
	$(2,4),\ (3,3)$
3	$(2,2,2),\ (3,2,1),\ (0,1,5)$
	$(1,1,4),\ (0,2,4),\ (1,5,0)$
4	$(1,2,3,0),\ (0,2,2,2),\ (1,0,2,3)$
	$(3,0,1,2),\ (0,2,3,1),\ (4,1,1,0)$

For method m, request c, and repetition j, let T_{mcj} denote measured complete time. The matched total and reported ratio are

$$S_m(\mathcal{C}) = \sum_{c \in \mathcal{C}} \mathrm{median}_{j=1,2,3}\, T_{mcj}, \qquad R_{A/B} = \frac{S_A(\mathcal{C})}{S_B(\mathcal{C})}. \tag{206}$$

The set $\mathcal{C}$ contains only requests with three exact completions for both compared methods. It is specified separately for each pair; no failed case receives an imputed completed time. The fractional time saving follows directly by subtracting the revised total from the control total and dividing by the control total:

$$\frac{S_B - S_A}{S_B} = 1 - \frac{S_A}{S_B} = 1 - R_{A/B}. \tag{207}$$

The coverage table retains every excluded case. This separates a conditional timing gain from the probability that a request completes. Three timing repetitions supply the stated medians, not a confidence interval or a proof of global optimality.

The supplied-identity Kira workers use a 90-second solver limit, a 1,800 MiB address-space limit, a 1,500 MiB process-tree memory limit, and a 64 MiB temporary-file limit. The recurrence works within the same scheduler allocation and checks the expanded coefficients independently. Native and vacuum calculations run sequentially, so their temporary storage does not overlap.

L Learning seed bounds: inputs, objective, and timing definitions

L.1 Integral definitions and target batches

The new study uses the family definition of Eq. (3). The sunrise, planar double box, three-loop vacuum, and massive three-loop two-point ladder retain the ordered denominator momenta in Appendix A, with the masses and kinematic assignments in Eq. (180). The nonplanar box,

pentagon, and external-virtuality box use that equation's corresponding assignments and the denominator definitions in the same appendix. The three- and four-loop boxes with bubbles use Eq. (176). The four-loop massless two-point family uses $D_i = \ell_i^2$ for $i = 1, \ldots, 4$ and $D_5 = (p - \ell_1 - \ell_2 - \ell_3 - \ell_4)^2$, with $p^2 = -2h - 29$. Auxiliary squared momentum differences complete its scalar-product coordinates as in Appendix K. The training profiles are $h = 511, 512$, validation uses $h = 513$, and the reserved test uses $h = 515, 516$. Development uses $h = 501$ and supplies no reserved labels.

Write n for the number of physical denominators, N for the total number of family entries, and $\boldsymbol{a}^{(0)} = (1, \ldots, 1, 0, \ldots, 0)$ with n leading ones. For $j = 0, 1, 2$, define $i_j = 1 + [(h + j) \bmod n]$ and $i_j^+ = 1 + (i_j \bmod n)$. When $N > n$, define $k_j = n + 1 + [(h + j) \bmod (N - n)]$. The first seven structures use four target patterns:

$$\begin{aligned} \boldsymbol{a}_j^{\text{dots}} &= \boldsymbol{a}^{(0)} + 2\boldsymbol{e}_{i_j}, & \boldsymbol{a}_j^{\text{pinch}} &= \boldsymbol{a}^{(0)} - \boldsymbol{e}_{i_j} + \boldsymbol{e}_{i_j^+}, \\ \boldsymbol{a}_j^{\text{mixed}} &= \boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j} - \boldsymbol{e}_{k_j}, & \boldsymbol{a}_j^{\text{rank2}} &= \boldsymbol{a}^{(0)} - 2\boldsymbol{e}_{k_j}. \end{aligned} \tag{208}$$

Here $\boldsymbol{e}_i$ is the unit index vector. For families without auxiliary entries, the last two patterns become $\boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j} + \boldsymbol{e}_{i_j^+}$ and $\boldsymbol{a}_j^{\text{dots}}$. The remaining three structures use the corner, $\boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j}$, and $\boldsymbol{a}^{(0)} - \boldsymbol{e}_{k_j}$. Repeated vectors within a batch appear once. These equations define the requested integrals directly, with no assumed optimal reduction path.

L.2 The ten candidate configurations

Let (r_0, s_0, d_0) be the target maxima in Eq. (179), and let t_{top} be the largest number of positive indices in a declared top sector. Table 27 specifies the ten candidates as changes to these maxima. Negative bounds become zero, and the positive-index bound is at least t_{top}. The word "native" in the table denotes the unshifted target-bound configuration. Kira's lower sector restriction

Table 27: Candidate seed settings. A dash means that the lower sector truncation is disabled.

Choice	Δr	Δs	Δd_0	l
Native bounds	0	0	0	—
Numerator expansion	0	1	0	—
Tight bounds	−1	−1	−1	—
Reduced dots	−1	0	−1	—
Reduced numerator rank	0	−1	0	—
Full expansion	1	1	1	—
Truncated native	0	0	0	$t_{\text{top}} - s_0$
Truncated numerator expansion	0	1	0	$t_{\text{top}} - s_0$
Truncated reduced dots	−1	0	−1	$t_{\text{top}} - s_0$
Relaxed truncation	0	0	0	$t_{\text{top}} - s_0 - 1$

has the form [9]

$$s \le \max\{1, t - l + 1, s_{\text{max,sector}}\}, \tag{209}$$

where t is the number of positive indices in that sector, l is the chosen truncation parameter, and $s_{\text{max,sector}}$ is an explicitly assigned sector limit. This bound supplements the candidate seed limits. Every unsuccessful candidate is followed by native bounds and full expansion, with repeated effective bound-and-truncation pairs removed. All attempts use integral ordering 1 and one worker.

L.3 Features and fitted objective

The first eighteen features are those defined in Appendix H. The remaining nine are the minimum dots and numerator rank, mean number of positive indices, largest absolute index, mean dots on massive entries, mean numerator-weighted loop incidence, mean dot-weighted loop incidence, maximum sum of positive indices, and maximum number of absent physical denominators. All means run over the target batch. If $B_{i\ell} = 1$ when loop momentum ℓ appears in denominator momentum q_i and zero otherwise, the two weighted incidence quantities are

$$\frac{1}{|\mathcal{T}|}\sum_{\boldsymbol{a}\in\mathcal{T}}\sum_{i}\max(-a_i,0)\sum_{\ell}B_{i\ell}, \qquad \frac{1}{|\mathcal{T}|}\sum_{\boldsymbol{a}\in\mathcal{T}}\sum_{i}\max(a_i-1,0)\sum_{\ell}B_{i\ell}. \tag{210}$$

These definitions describe index structure; they do not encode a target's reference reduction or measured runtime.

For a training case c and candidate a, let $T_{ca} > 0$ be its measured time with recovery, and let $a = 0$ denote native bounds. We fit $y_{ca} = \log(T_{ca}/T_{c0})$. Subtracting two such labels gives

$$y_{ca} - y_{cb} = \log T_{ca} - \log T_{cb} = \log(T_{ca}/T_{cb}). \tag{211}$$

The logarithm is increasing, so minimizing the exact label selects the least measured time for that case. This statement concerns the labels; prediction error can change the selected configuration. The tree models fit weighted squared errors with $w_c = \sqrt{T_{c0}/\operatorname{median}_{c'} T_{c'0}}$. The twelve single trees have maximum depths $2, 3, 5, 8$ and minimum leaf sizes $1, 2, 4$. Six ensembles combine random forests or extremely randomized trees with depths $3, 5, 8$, 48 trees, and minimum leaf size two [30, 48, 39].

The twelve neural candidates use hidden widths (16), (32), or $(32, 16)$, a rectified-linear or hyperbolic-tangent activation, and regularization parameter 0.01 or 0.1. They standardize features using training means and standard deviations, fit unweighted squared errors with a squared-weight penalty, and use L-BFGS with a 1,500-iteration limit [52]. Their hidden states satisfy $\boldsymbol{h}^{(j+1)} = \sigma(W_j^{\mathsf{T}}\boldsymbol{h}^{(j)} + \boldsymbol{b}_j)$, while the output layer is linear. This equation defines the implemented network. Validation compares actual candidate costs, so differences between the fitting objectives do not determine the final choice by themselves. Every stochastic fit uses random seed 41.

Let b_H be the fixed candidate with the smallest summed training-plus-validation time for graph H, and let $a_\star = \arg\min_a f_a(\boldsymbol{x}_c)$ be the predicted best candidate. A model with gate g uses $a_\star$ when

$$f_{a_\star}(\boldsymbol{x}_c) < f_{b_H}(\boldsymbol{x}_c) - g, \qquad g \in \{0, 0.03, 0.10, 0.20\}, \tag{212}$$

and otherwise uses b_H. Under exact labels, this inequality becomes $T_{ca_\star}/T_{cb_H} < e^{-g}$ by exponentiation. With fitted labels it expresses a predicted saving. Validation selects the model and gate with the lowest sum of recorded configuration times. No reserved result enters that choice. The saved evaluator is checked against the training library for every validation prediction before the model is frozen.

L.4 Measured ratios, uncertainty, and training cost

Let $\widetilde{T}_{cm}$ be the median of five complete times for case c and method m. For a matched set $\mathcal{C}$, define

$$Q_m = \sum_{c\in\mathcal{C}}\widetilde{T}_{cm}, \qquad R_b = Q_{\mathrm{ML}}/Q_b, \qquad S_b = 1/R_b. \tag{213}$$

Summing accounts for the time required by the same batch collection; division gives its relative time, and inversion gives its speedup factor. The percentage time saving is $100(1-R_b)$. Matched

cases must complete all methods and repetitions with a nontrivial reduction. We also retain the total elapsed time and completion counts for every timed request, including failures and identity-only outputs.

For the graph interval, we sample the observed graph groups with replacement and recompute R_b, preserving each selected graph's complete batch block. For the repetition interval, each batch samples five paired repetition positions with replacement and recomputes its medians and then R_b. Both calculations use 5,000 draws and report the 2.5th and 97.5th percentiles. These are descriptive resampling intervals for the specified collection. The graph count is small, and repeated index patterns remain correlated within a graph.

Let T_{off} be the recorded development, training, validation, and associated reference cost, including fitting. If the future mean saving equals the estimate from the matched workload $\Delta_b = (Q_b - Q_{\text{ML}})/|\mathcal{C}| > 0$, then N requests recover this cost when $N\Delta_b \geq T_{\text{off}}$. Solving this inequality gives

$$N_{\text{break}} = \lceil T_{\text{off}}/\Delta_b \rceil \,. \tag{214}$$

This calculation excludes reserved-test expense and assumes that an appropriate master basis is already available. It states the cost of collecting the model's evidence as well as its application saving. It does not prove that future workloads will have the same mean saving.

M Equation refinement: fitting and evaluation

M.1 Inputs available before coefficient construction

Let $\boldsymbol{\sigma}$ be a seed with N indices, and let $\mathcal{T}$ be the target batch. Define the number of positive indices, the extra denominator powers, and the numerator rank by

$$n_+(\boldsymbol{\sigma}) = \sum_i \mathbf{1}[\sigma_i > 0], \qquad \delta(\boldsymbol{\sigma}) = \sum_i \max(\sigma_i - 1, 0), \qquad \rho(\boldsymbol{\sigma}) = \sum_i \max(-\sigma_i, 0). \tag{215}$$

The target maxima are $\delta_{\max} = \max_{\boldsymbol{a}\in\mathcal{T}} \delta(\boldsymbol{a})$ and $\rho_{\max} = \max_{\boldsymbol{a}\in\mathcal{T}} \rho(\boldsymbol{a})$. For the seed's sector S, let b_S be one if the batch contains a target in that sector and zero otherwise. Define δ_S, ρ_S as the corresponding maxima over targets in S, or zero when that sector has no target. The minimum target distance is

$$\Delta(\boldsymbol{\sigma}, \mathcal{T}) = \min_{\boldsymbol{a}\in\mathcal{T}} \sum_i |\sigma_i - a_i|. \tag{216}$$

These quantities depend only on indices. Table 28 specifies the complete input vector. The power differences are signed; a negative value means that the seed has fewer powers than the stated target maximum. Here L is the number of loops, E is the number of independent external momenta, and o is the operator index in Kira's candidate list.

Table 28: The seventeen inputs to the new equation classifiers. A whole-seed choice uses the same inputs with the operator index removed.

Group	Quantities	Number
Family and request	$L, E, N, \log(1 + \lvert\mathcal{T}\rvert)$	4
Seed powers	$n_+(\boldsymbol{\sigma}), \delta(\boldsymbol{\sigma}), \rho(\boldsymbol{\sigma})$	3
Target differences	$\delta(\boldsymbol{\sigma}) - \delta_{\max}$ and $\rho(\boldsymbol{\sigma}) - \rho_{\max}$	2
Sector information	b_S, $\delta(\boldsymbol{\sigma}) - \delta_S$, and $\rho(\boldsymbol{\sigma}) - \rho_S$	3
Target proximity	$\Delta(\boldsymbol{\sigma}, \mathcal{T})$ and $\mathbf{1}[\boldsymbol{\sigma} \in \mathcal{T}]$	2
Operator	Index o for an IBP or Lorentz-invariance identity	1
Overall limits	$\delta_{\max}, \rho_{\max}$	2

M.2 Sampling and tree scores

The physical families, target patterns, and tuned seed configurations are those in Appendix L. Training uses $h = 511, 512$, validation uses $h = 513$, and the new reserved test uses $h = 525, 526$. The earlier reserved profiles $515, 516$ supply no training or validation labels for this filter. Within a batch c, let N_{cy} be the number of native equation records with label $y \in \{0, 1\}$ and let $N_c = N_{c0} + N_{c1}$. We sample $n_{cy} = \min(2000, N_{cy})$ records without replacement from each nonempty class, using a deterministic batch-specific random seed. For fitting, every sampled record in that class receives weight

$$w_{cy} = \frac{N_{cy}}{n_{cy} N_c}. \tag{217}$$

For fitting, its class has total sampled weight $n_{cy} w_{cy} = N_{cy}/N_c$, so summing over classes gives unit weight per batch. This calculation explains both the correction for class sampling and the equal batch contribution. Validation recall and retained fractions instead use N_{cy}/n_{cy}, which reconstructs native record counts across the validation batches.

For a tree leaf ℓ, let $W_{\ell 1}$ and $W_{\ell 0}$ be the sums of these weights for positive and negative training records. Its score is

$$p_\ell = \frac{W_{\ell 1}}{W_{\ell 0} + W_{\ell 1}}. \tag{218}$$

This is the weighted positive fraction in the leaf. It does not assert calibrated probabilities for new equations. The tree chooses splits through weighted Gini impurity [30], using the classification-tree implementation of Ref. [52]. At a node, let W_0 and W_1 be the negative and positive total weights, $W = W_0 + W_1$, and $p = W_1/W$. The weighted impurity is

$$G = W[1 - p^2 - (1 - p)^2] = 2Wp(1 - p) = \frac{2W_0 W_1}{W_0 + W_1}. \tag{219}$$

The equalities follow by expanding the two squares and substituting the weighted class fractions. For each allowed split, subtract the two child impurities from the parent impurity. The tree selects the largest decrease, subject to its depth, leaf-count, and minimum-population limits. The eight depth and leaf-size choices appear in the main text. We compare shared fits with separate fits for each graph before the complete-cost validation. All fits use random seed 611.

M.3 Validation and frozen deployment

For each graph, candidate thresholds include 10^{-12}, 0.01, 0.05, 0.1, 0.25, and 0.5, together with the 0, 0.1, 0.5, 1, and 2 percentiles of positive validation scores. For each required weighted recall of 0.9999, 0.995, and 0.98, we choose the tree and threshold with the smallest retained fraction among candidates that meet that recall. Repeated choices appear once. These thresholds concern reference labels; they do not guarantee a complete reduction.

We run each shortlisted filter and the all-ML-disabled control three times for every validation batch. For each graph, we sum the per-batch median times and choose the cheapest candidate, retaining it only if it beats the control by at least 5%. This threshold for deployment is fixed before the reserved experiment. For the shortlisted source trees, we also calculate the maximum score over operators and select seed thresholds using the same recall levels and retained-fraction criterion. The seed threshold grid adds 0.75, 0.9, and 1 to the equation grid. We compare up to three seed variants with the equation filters in the same complete-cost validation. The selected filters are The model tests 17 features before symbolic coefficient construction. It retains a candidate when $p_\ell \geq \tau$, where τ is the graph's threshold. If that attempt fails, the next attempt uses the tuned seed setting without the model. Native and fully expanded bounds follow if necessary, with identical configurations removed. Every attempt must express

Table 29: Filters fixed by complete-time validation. The columns give the filter type, compiled node count, source-tree depth limit, minimum leaf population, and original score threshold.

Graph	Filter	Nodes	Depth limit	Leaf size	Threshold
Planar double box	Equation	1161	18	16	1e-12
Massive vacuum	Equation	369	14	4	1e-12
Massive three-loop ladder	Seed	60	14	16	0.02232
Nonplanar box	Seed	57	14	4	0.05

all mandatory targets in the prescribed basis. Independent exact coefficient comparisons then check the exported answer. Every failed attempt remains in the timing record.

M.4 A learned choice of whole seeds

For a seed $\boldsymbol{\sigma}$, target batch $\mathcal{T}$, and candidate operator o, let $p_\theta(\boldsymbol{\sigma}, o, \mathcal{T})$ be the equation score. The equation filter retains the candidate when this score is at least τ. Its whole-seed variant instead uses

$$K_\theta(\boldsymbol{\sigma}, \mathcal{T}) = \mathbf{1}\left[\max_{o \in \mathcal{O}} p_\theta(\boldsymbol{\sigma}, o, \mathcal{T}) \geq \tau\right]. \tag{220}$$

Here $\mathcal{O}$ is the finite list of candidate operators, and $\mathbf{1}$ is one when its condition holds and zero otherwise. A maximum reaches τ exactly when at least one member reaches it. Therefore Eq. (220) is the logical union of the original equation decisions. If $K_\theta = 1$, all operators from that seed remain available. The resulting set contains every equation that the source filter would retain, although it can contain more. This inclusion does not prove that the reduction will complete.

For L loop momenta and E independent external momenta, Kira's candidate list contains $L(L+E)$ IBP operators and $E(E-1)/2$ external Lorentz-invariance operators [2]. The first count follows by choosing a loop derivative and a contraction vector from the loop and external momenta. The second counts unordered pairs of external vectors for antisymmetric contractions. These are candidate counts before redundancies are removed. The seed filter takes the maximum over that full list. Its decision no longer depends on which operator is being considered, so it uses sixteen of the seventeen features. We evaluate the same logical decision directly during generation and check its equivalence to the original operator scores. Kira still enumerates the configured seed region, applies its native symmetries, and performs exact algebra.

M.5 Ratios and uncertainty

For each complete matched batch, take the median of five times for each method, then sum the medians as in Eq. (213). This gives the learned-to-control ratio and the percentage saving $100(1-R)$. The active subset consists of batches for which the frozen domain and graph policy enable a filter. It uses the same ratio definition. We resample paired repetition positions within each batch 5,000 times and report the 2.5th and 97.5th percentiles. Failed reference checks, incomplete method comparisons, and identity-only targets remain outside the nontrivial paired ratios. Their counts remain in the main results.

N Target partitions, cost learning, and exact checks

N.1 Why target grouping changes the seed region

Let $\mathcal{T}$ be a finite batch of distinct index vectors from one family. For a vector $\boldsymbol{a}$, define its extra denominator powers and numerator rank as $\delta(\boldsymbol{a}) = \sum_i \max(a_i - 1, 0)$ and $\rho(\boldsymbol{a}) = \sum_i \max(-a_i, 0)$, as in Appendix M. A partition $\Pi = \{\mathcal{T}_1, \dots, \mathcal{T}_K\}$ consists of disjoint nonempty subsets whose union is $\mathcal{T}$. The number K is the number of separate reductions. For each part, the corresponding maxima obey

$$\max_{\boldsymbol{a}\in\mathcal{T}_k} \delta(\boldsymbol{a}) \le \max_{\boldsymbol{a}\in\mathcal{T}} \delta(\boldsymbol{a}), \qquad \max_{\boldsymbol{a}\in\mathcal{T}_k} \rho(\boldsymbol{a}) \le \max_{\boldsymbol{a}\in\mathcal{T}} \rho(\boldsymbol{a}). \tag{221}$$

These inequalities follow because each maximum on the left runs over a subset of the indices on the right. They explain how a partition can lower one or both seed limits. They do not imply a time saving: separate parts can generate the same intermediate equations and repeat the preparation. For example, one part can contain targets with large δ and zero ρ, while another contains targets with zero δ and large ρ. Their union requires both large bounds simultaneously. Kira's lower-sector truncation remains active in every comparison where the fixed seed policy requests it [9].

The five candidate partitions keep the batch together, group equal values of ρ, group equal values of δ, group equal sets of positive indices, or divide the given target sequence into consecutive groups of at most three. The first three group definitions depend on integral powers, and the fourth uses the set of positive indices that defines a sector. The last definition also depends on the order supplied by the caller. Every target occurs in exactly one part. The learned method selects one partition before equation generation and processes its parts sequentially with one Kira worker. An additional candidate keeps the batch together and disables the equation filter. Thus the cost predictor compares six calculations. This candidate allows a target-dependent decision to avoid a filter whose retry cost exceeds its saving. When a batch has at most three targets and all targets have equal rank, equal extra denominator powers, and the same sector, all five partitions contain just that batch. If the earlier policy also disables its equation filter on this batch, the application skips loading the cost model and uses the undivided unfiltered request. This follows directly from the partition definitions and preserves the available reduction choice.

If a completed part gives $I_{\boldsymbol{a}} = \sum_m r_{\boldsymbol{a}m} M_m$ for every $\boldsymbol{a} \in \mathcal{T}_k$, then the union of these relations supplies the reduction of every target in $\mathcal{T}$. Indeed, disjointness assigns each target to one part, and the union property ensures that no target is missing. All parts use the same prescribed master basis. For a target expression $T = \sum_{\boldsymbol{a}\in\mathcal{T}} c_{\boldsymbol{a}} I_{\boldsymbol{a}}$, substitution and interchange of the finite sums gives

$$T = \sum_{k=1}^{K} \sum_{\boldsymbol{a}\in\mathcal{T}_k} c_{\boldsymbol{a}} \sum_m r_{\boldsymbol{a}m} M_m = \sum_m \left(\sum_{k=1}^{K} \sum_{\boldsymbol{a}\in\mathcal{T}_k} c_{\boldsymbol{a}} r_{\boldsymbol{a}m} \right) M_m. \tag{222}$$

The model changes the division of the calculation. Equation (222) shows how the exact part reductions provide the same requested expression.

N.2 Physical requests and reference construction

The denominator momenta, masses, and rational kinematic profiles are those specified in Appendix L. The ten structures are the sunrise, planar double box, massive three-loop vacuum, massive three-loop ladder, nonplanar box, pentagon, external-virtuality box, three- and four-loop boxes with bubbles, and four-loop massless two-point family. The dimension d remains symbolic. For the first seven structures, let D, R, X, and P denote the three-target sets with

extra denominator powers, rank two, mixed powers, and a pinched propagator in Eq. (208). The requested batches are D, R, X, $D \cup R$, $P \cup R$, and $D \cup R \cup X \cup P$. The two bubble-box structures and the massless two-point family use the corresponding one-dot, one-numerator, and corner sets from that appendix, with the corner set also taking the place of P. Repeated index vectors and identical complete batches at a profile appear once. Development uses $h = 701$, training uses $h = 711$, validation uses $h = 712$, and the reserved comparison uses $h = 721, 722$.

Each component first receives a native reduction with expanded seed limits. We collect the resulting master indices into a common basis, reduce the components in that basis, and check each coefficient again after increasing the numerator limit by one. The checked component relations form the reference for each union. A failure in discovery, common-basis reduction, or the enlarged check excludes that batch from the timed comparison and remains in the record. Agreement with this finite enlargement does not prove a minimal basis. Reference construction remains outside the application timer; the timed request receives the basis and generates its own IBP equations.

N.3 Predictor inputs and objective

The predictor uses the 27 family and target quantities specified in Appendix L, together with five descriptions of the distribution of powers within the batch:

$$|\{\delta(\boldsymbol{a}) : \boldsymbol{a} \in \mathcal{T}\}|, \quad |\{\rho(\boldsymbol{a}) : \boldsymbol{a} \in \mathcal{T}\}|, \quad \delta_{\max} - \delta_{\min}, \quad \rho_{\max} - \rho_{\min}, \quad \frac{1}{|\mathcal{T}|} \sum_{\boldsymbol{a} \in \mathcal{T}} \delta(\boldsymbol{a})\rho(\boldsymbol{a}). \tag{223}$$

Here $|\cdot|$ denotes the number of members, and the maxima and minima run over the batch. The last quantity distinguishes targets that carry both kinds of power from a batch in which different targets carry them separately. No reference coefficient, measured test time, or kinematic profile number enters the features.

Candidate definitions can produce identical ordered parts on a particular batch. Such candidates share the same measured training or validation cost, and the record identifies the common measurement. The reserved comparison still executes every complete application method separately, including its own selection and loading costs.

For training batch b and partition a, let $\bar{T}_{ba}$ be the median of two complete timings when both reductions succeed. A failed candidate receives a training cost equal to the larger of 300 seconds and the sum of its measured attempt times. This chosen penalty makes a bounded failure costly to the predictor; it is not an observed completion time. With $a = 0$ denoting the undivided batch, the labels and normalized weights are

$$y_{ba} = \log \frac{\bar{T}_{ba}}{\bar{T}_{b0}}, \qquad u_b = \sqrt{\min_a \bar{T}_{ba}}, \qquad w_b = \frac{u_b}{B^{-1} \sum_{b'=1}^{B} u_{b'}}. \tag{224}$$

The positive integer B is the number of usable training batches. The normalization gives $B^{-1} \sum_b w_b = 1$ by direct substitution. The fitted multi-output regressors use weighted squared error,

$$\mathcal{L}(\theta) = \sum_{b=1}^{B} w_b \sum_a \big[f_{\theta,a}(x_b) - y_{ba}\big]^2. \tag{225}$$

Here x_b is the feature vector, θ denotes the fitted regressor parameters, and $f_{\theta,a}$ is the predicted log-cost ratio for partition a. Since $y_{ba} - y_{ba'} = \log(\bar{T}_{ba}/\bar{T}_{ba'})$ and the logarithm is increasing, the smallest exact label corresponds to the smallest assigned cost. Fitting Eq. (225) approximates these labels; it supplies no guarantee that a prediction is optimal on a new batch.

We fit eighteen candidates: regression trees, random forests with 32 trees, and extremely randomized forests with 32 trees, each with maximum depth 2, 4, or 6 and minimum leaf population 1 or 3 [30, 48, 39]. The implementation uses Ref. [52]. Three complete validation repetitions select a fixed partition c_g for each graph g. A model proposes $a_\star = \arg\min_a f_{\theta,a}(x)$ when

$$f_{\theta,a_\star}(x) < f_{\theta,c_g}(x) - \gamma, \qquad \gamma \in \{0, 0.02, 0.05, 0.1\}; \tag{226}$$

otherwise it uses c_g. Validation chooses the model and margin together. Deployment requires at least a three-percent saving in summed validation cost relative to the graph-specific fixed partitions. A separate validation run with every ML model disabled selects the fixed control for that condition. Ordered physical structures and target-count and power ranges constrain deployment; an unmatched input retains an undivided batch.

To measure the remaining opportunity within this candidate pool, we also sum the smallest recorded cost for each validation batch:

$$T_{\mathrm{min,recorded}} = \sum_b \min_a \bar{T}_{ba} \le \sum_b \bar{T}_{b,c_{g(b)}} = T_{\mathrm{fixed}}. \tag{227}$$

Here $g(b)$ is the graph of batch b, and the sums run over validation batches. Each minimum is no greater than the chosen fixed candidate's cost, which proves the inequality after summation. This retrospective quantity uses measured costs that a new request does not supply. It is neither an achieved prediction time nor a lower bound on all possible reduction methods.

N.4 Timed scope and failures

The application loads the policy for each request. Each part uses the fixed seed configuration for its physical graph. The previous equation filter is available only inside its original checked domain, with its weights and threshold unchanged. An insufficient filtered system retries without the filter, then uses unshifted and expanded seed limits where distinct. A failed partition can retry the undivided unfiltered calculation if time remains. The common request budget is 150 seconds, and each native attempt has at most 60 seconds. A request that exhausts this budget remains a failure.

The reserved test rotates five methods through five repetition positions: the previous undivided workflow, that workflow with every ML model disabled, the new graph-specific fixed partitions with filters available, independently tuned fixed partitions with every ML model disabled, and the learned partition choice with filters available. Every method uses the same exact implementation, worker count, basis, and coefficient checks. Initial Python imports precede these repeated timers. The separate application check starts a new Python process and includes imports, output, and the exact comparison.

For deployment, we retain the frozen domain limits, fixed choices, and prediction margin. We retain the tree arrays only when validation enables the predictor. When the predictor is disabled, its choice rule returns before consulting those arrays, so removing them leaves each choice unchanged. We check equality of the returned choices on every training, validation, and reserved input. This check uses no reserved timing or coefficient outcome. The repeated test retains its original frozen implementation; the fresh-process application check uses the compact deployment form. All ML-disabled controls keep their existing behavior.

We sum per-batch median times over requests for which all methods and all repetitions succeed and whose output is not entirely the identity. Failed references, incomplete timed batches, and identity-only batches remain separate. The ratio of two such sums compares the same completed workload. Paired resampling of the five repetition positions within each batch gives 5,000 ratios; the 2.5th and 97.5th percentiles describe timing variation for this specified collection. They do

not establish transfer to untested topologies or loop orders. Some index sets recur across rational profiles, and the report records this overlap.

Independent graph groups run on two workers during data collection and four workers during the reserved comparison. This shortens the experiment without parallelizing an individual timed reduction. Every reserved batch keeps all methods and repetitions on the same worker, with the same rotated order described above. Complete training measurements made before this scheduling change remain in the training record.

If a cluster allocation ends during a comparison, we retain each complete batch and repeat every method and repetition for an unfinished batch on one replacement worker. The saved partial comparison remains in a separate archive and does not enter the timing ratios. We assign the remaining whole batches from validation costs and keep the frozen models, global method rotation, exact checks, and timing definitions unchanged. The audit checks the worker identity for every completed comparison batch.

O An exact reduction for massless banana integrals

O.1 Definition and convolution

Consider the massless two-point family

$$B_L(\boldsymbol{a}; p^2, d) = \int \prod_{j=1}^{L} \frac{d^d \ell_j}{i\pi^{d/2}} \frac{1}{\prod_{j=1}^{L} (\ell_j^2 + i0)^{a_j} [(p - \sum_{j=1}^{L} \ell_j)^2 + i0]^{a_{L+1}}}. \tag{228}$$

Here $L \geq 1$, all a_i are positive integers, and $p^2 \neq 0$. The family has two vertices joined by $L+1$ massless lines and is often called a banana or watermelon diagram. The implemented specialization covers the displayed scalar integrals. Auxiliary denominator entries may be present in the Kira definition, but their target indices must be zero. The proof does not cover arbitrary numerator insertions or other propagator structures.

The one-loop convolution underlying this calculation is standard [53]. For completeness, write its Euclidean form as

$$E_1(a, b; p_E^2) = \int \frac{d^d k_E}{\pi^{d/2}} (k_E^2)^{-a} [(p_E - k_E)^2]^{-b}. \tag{229}$$

Here k_E and p_E are Euclidean loop and external momenta, and a, b are the two denominator powers. The Euler Gamma function is $\Gamma(z) = \int_0^\infty \eta^{z-1} e^{-\eta} d\eta$ for $\operatorname{Re} z > 0$. Changing the integration variable to $\eta = tD$ gives the Schwinger representation $D^{-a} = \Gamma(a)^{-1} \int_0^\infty t^{a-1} e^{-tD} dt$ for positive D in its convergence domain. Applying this representation to both denominators and completing the square gives

$$tk_E^2 + u(p_E - k_E)^2 = (t+u) \left(k_E - \frac{u}{t+u} p_E \right)^2 + \frac{tu}{t+u} p_E^2. \tag{230}$$

For $\alpha > 0$, squaring $\int_{\mathbb{R}} e^{-\alpha x^2} dx$ gives a two-dimensional Gaussian integral. Polar coordinates evaluate it as $2\pi \int_0^\infty r e^{-\alpha r^2} dr = \pi/\alpha$. The positive square root and the product over momentum components give the normalized Gaussian momentum integral $(t+u)^{-d/2}$, continued in d from integer dimension. Thus the Schwinger parameters $t, u > 0$ give

$$E_1 = \frac{1}{\Gamma(a)\Gamma(b)} \int_0^\infty dt\, du\, t^{a-1} u^{b-1} (t+u)^{-d/2} \exp\left[-\frac{tu}{t+u} p_E^2 \right]. \tag{231}$$

Set $t = \lambda x$, $u = \lambda(1-x)$, with Jacobian λ, $\lambda > 0$, and $0 < x < 1$. The λ integral gives $\Gamma(a+b-d/2)[x(1-x)p_E^2]^{d/2-a-b}$. The remaining integral is $\int_0^1 x^{d/2-b-1}(1-x)^{d/2-a-1}dx$. To evaluate it, apply the same sum-and-fraction change of variables to the two integrals defining $\Gamma(z)\Gamma(w)$. The sum integral gives $\Gamma(z+w)$, which yields

$$\int_0^1 x^{z-1}(1-x)^{w-1}dx = \frac{\Gamma(z)\Gamma(w)}{\Gamma(z+w)} \quad (\mathrm{Re}\, z, \mathrm{Re}\, w > 0). \tag{232}$$

Taking $z = d/2-b$ and $w = d/2-a$ then gives

$$E_1 = (p_E^2)^{d/2-a-b}G(a,b), \qquad G(a,b) = \frac{\Gamma(a+b-d/2)\Gamma(d/2-a)\Gamma(d/2-b)}{\Gamma(a)\Gamma(b)\Gamma(d-a-b)}. \tag{233}$$

The parameter integrations first hold in a common convergence region. Analytic continuation defines the dimensionally regulated expression outside that region, as in the usual one-loop formula [53].

Let E_L be the Euclidean version of Eq. (228), with measure $\prod_j d^d\ell_{j,E}/\pi^{d/2}$, and write $n = L+1$ and $A = \sum_{i=1}^n a_i$. Integrating the last loop combines the last two indices into $b = a_L + a_{L+1} - d/2$. Applying Eq. (233) and then the $(L-1)$-loop formula cancels the factors $\Gamma(b)$ and $\Gamma(d-a_L-a_{L+1})$ between the two stages. Induction from $L = 1$ therefore gives

$$E_L(\boldsymbol{a}; p_E^2, d) = (p_E^2)^{Ld/2-A}\frac{\Gamma(A-Ld/2)}{\Gamma(nd/2-A)}\prod_{i=1}^n \frac{\Gamma(d/2-a_i)}{\Gamma(a_i)}. \tag{234}$$

Wick rotation to the denominator convention in Eq. (228) gives one factor of i per loop measure, cancelling its normalization, and changes the sign of each quadratic denominator. It therefore replaces the power factor by $(-1)^A(-p^2-i0)^{Ld/2-A}$. The remaining Gamma factors stay unchanged. This sign convention matters when reducing an integral with an additional denominator power.

O.2 Rational coefficients in a single master

Choose $M_L = B_L(1,\ldots,1;p^2,d)$ and define $b_i = a_i - 1 \geq 0$ and $K = \sum_i b_i = A-n$. The Gamma function recurrence gives

$$\frac{\Gamma(z+K)}{\Gamma(z)} = (z)_K, \qquad \frac{\Gamma(z)}{\Gamma(z-K)} = (-1)^K(1-z)_K, \tag{235}$$

where $(z)_K = \prod_{j=0}^{K-1}(z+j)$ and $(z)_0 = 1$. Integration by parts in the defining Gamma integral gives $\Gamma(z+1) = z\Gamma(z)$ when the boundary terms vanish; analytic continuation extends the relation. Applying this recurrence repeatedly gives the first identity. Reversing the finite product and factoring out -1 from each factor gives the second. In the ratio of Eq. (234) to its corner value, the dimension-independent index factors give $\Gamma(1)/\Gamma(a_i) = 1/b_i!$. The shifted factors satisfy

$$\prod_i \frac{\Gamma(d/2-1-b_i)}{\Gamma(d/2-1)} = \frac{(-1)^K}{\prod_i(2-d/2)_{b_i}}. \tag{236}$$

This sign cancels the sign from the inverse Gamma ratio. The remaining Minkowski power factor is $(-1)^K(-p^2-i0)^{-K} = (p^2+i0)^{-K}$ because K is an integer. Thus the required reduction is

$$\boxed{B_L(\boldsymbol{a}; p^2, d) = \frac{(n-Ld/2)_K\,(n+1-nd/2)_K}{(p^2)^K\prod_{i=1}^n(2-d/2)_{b_i}\,b_i!}\,M_L.} \tag{237}$$

We understand the rational coefficient by meromorphic continuation in d and generic nonzero p^2. The corner value is generically nonzero, and Eq. (237) expresses every scalar target in this class through it. This establishes a one-element basis for the stated scalar class without a new master-discovery calculation.

The implementation forms finite products of linear factors in d and cancels common factors with exact rational arithmetic before formatting the result. It uses no trained model to predict a coefficient. The domain check verifies the physical denominator momenta, zero masses, the single external squared momentum, positive physical indices, and zero auxiliary indices. If the caller supplies a basis, that basis must contain M_L. The implementation caps the loop count at 64 and the total extra powers at 256 to bound work; the mathematical identity holds for every positive integer loop count under the stated assumptions.

O.3 Independent comparisons

The native comparison uses three targets at each of one through six loops, with rational squared momentum $p^2 = -1651$, and repeats the calculation three times. A second comparison keeps p^2 symbolic at one through four loops. The requested indices raise one physical power by one or two. Kira receives the corner basis, generates native IBP equations with all ML disabled, and returns rational coefficients. Exact subtraction checks these coefficients against Eq. (237). Both reported timings include their coefficient check; the direct formula also includes its domain check. Shared Python imports precede these timings.

Eight initial higher-loop requests fail, including corner-basis rejections and an allocation failure in the six-loop preparation. We retain them and make a separate fixed-seed comparison. Let r be the largest sum of positive target indices, s the largest numerator rank, and q the largest number of extra denominator powers. Development at $p^2 = -1649$ compares seed bounds $(r-1, s, q-1)$, (r, s, q), $(r, s+1, q)$, and $(r+1, s+1, q+1)$; the positive-power limit never falls below the number of physical denominators. Symbolic development changes the raised propagator positions at the same index profile. For each loop count and kinematic form, the fastest successful complete development calculation fixes the seed choice. A new comparison at $p^2 = -1655$, or symbolic p^2 with the new index profile, uses three repetitions of that fixed choice. Timed failures remain separate from matched speedup ratios.

Further checks use one through twelve loops and loop counts 16, 24, and 32, with eight distributions of one through eight extra powers at each loop count. The direct rational result is compared with the uncancelled Gamma ratio from Eq. (234) at $d = 257/101$ and $d = 401/97$. The evaluation uses 70-digit arithmetic and a relative error threshold of 10^{-55} with denominator $\max(1, |r|)$, where r is the Gamma-ratio value. These numerical checks supplement the derivation. Native timing ratios are reported only where the native calculation completes; no native speedup is assigned to the higher-loop cases checked by the formula alone.

The combined-interface application check uses a fresh process for each request. Four scalar banana requests at one, four, eight, and twelve loops use an independent symbolic implementation of Eq. (237) as their reference. It constructs rising factorials directly and does not call the live factor-cancellation routine. The native application requests use the independently checked IBP references of the target-batch study. All reference construction precedes application timing. Both comparison strategies use the same analytic route; their difference on native requests comes from the frozen learned or fixed reduction choices. Exact rational checks at $d = 257/101$ also test the implemented power limit: one- and 64-loop targets have 256 extra powers on one denominator. A direct product calculation checks the factored result without expanding its polynomials. Further checks verify the corner identity and reject requests beyond the implemented loop or power limits.

P Cost-trained seed regions

P.1 The selected integral lattice

We specify a seed by its integer index vector $\boldsymbol{\sigma}$. Let P be its positive-index set, let n_{top} be the largest number of propagators in a declared top sector, and let $h = n_{\mathrm{top}} - |P|$ be the number of absent propagators relative to that count. The number of extra denominator powers and the numerator rank are

$$D(\boldsymbol{\sigma}) = \sum_i \max(0, \sigma_i - 1), \qquad R(\boldsymbol{\sigma}) = \sum_i \max(0, -\sigma_i). \tag{238}$$

For a target $\boldsymbol{a}$, write $v_i = \max(0, -a_i)$, while $u_i = \max(0, -\sigma_i)$ denotes a seed numerator power. We measure how far the seed lies outside the target's numerator-power box by

$$q(\boldsymbol{\sigma}, \mathcal{T}) = \min_{\boldsymbol{a} \in \mathcal{T}} \sum_{i \notin P} \max(0, u_i - v_i). \tag{239}$$

Here $\mathcal{T}$ is the requested target set. To obtain this expression, project each coordinate u_i onto the interval $[0, v_i]$. Since $u_i \geq 0$, the shortest coordinate distance is $\max(0, u_i - v_i)$. Adding these distances gives the distance in the sum-of-absolute-values norm, and minimizing over the targets gives the distance to their union of boxes. With several numerator directions this union can contain a box with a nonzero interior; the construction does not impose the zigzag paths of Ref. [51].

Let R_T and D_T be the maximum target rank and extra denominator power. A parameter vector $\boldsymbol{\theta} = (\theta_0, \ldots, \theta_6)$ defines the additional inequalities

$$q(\boldsymbol{\sigma}, \mathcal{T}) \leq \max(0, \lfloor \theta_0 + \theta_1 h + \theta_2 D(\boldsymbol{\sigma}) + \theta_6 R(\boldsymbol{\sigma}) \rfloor), \tag{240}$$

$$R(\boldsymbol{\sigma}) \leq \max(0, \lfloor R_T + \theta_3 - \theta_4 h \rfloor), \tag{241}$$

$$D(\boldsymbol{\sigma}) \leq \max(0, \lfloor D_T + \theta_5 \rfloor). \tag{242}$$

The outer Kira bounds and its fixed subsector truncation also apply. The generator adds admissible target and master seeds and retains sector corners. These additions preserve useful starting identities but do not prove that the selected system is sufficient. The comparisons keep all positive target powers equal to one, so $D_T = 0$. The dot-dependent extension therefore remains untested; five boundary parameters affect the numerator-only search, while $\theta_5 = 1$ remains fixed.

The geometric restriction can reduce the growth with numerator rank. For one numerator direction of degree R_T, suppose a surrounding width w stays fixed and there are $n_\perp$ other numerator directions. Distributing at most w powers over these transverse directions gives $\binom{n_\perp + w}{w}$ assignments. This count follows by adding a nonnegative slack variable to make the sum equal to w and counting the resulting compositions. The parallel coordinate has at most $R_T + w + 1$ values, so

$$N_{\mathrm{seed}} \leq (R_T + w + 1) \binom{n_\perp + w}{w} \tag{243}$$

for this fixed-sector, fixed-denominator-power example. Thus the bound grows linearly in R_T when w and $n_\perp$ remain fixed. A changing learned width or several simultaneously powered numerator directions need not have this scaling. The timing results use measured reductions and do not infer a general speedup from this counting argument.

P.2 Training on completed reductions

For each graph, the first search starts from complete Kira reductions at three training targets. It evaluates twenty-four boundary proposals through three populations. The four proposals with

the lowest mean log-time ratio determine the mean and component standard deviations for the next Gaussian population, with a minimum standard deviation of 0.2. We round parameters to quarter units and clip them to the chosen ranges: $\theta_0 \in [0,3]$, $\theta_1, \theta_2 \in [-1,1]$, $\theta_3 \in [-1,2]$, $\theta_4 \in [0,2]$, and $\theta_6 \in [-1,1/2]$, with $\theta_5 = 1$. These choices define a finite search and do not establish a globally optimal seed set. Evolutionary optimization of IBP seed decisions is also used in Ref. [51]; the present search directly measures complete Kira time with symbolic dimension.

For training request b, let $T_b(\boldsymbol{\theta})$ include seed prediction, every reduction attempt, recovery, and the exact reference comparison. Let $T_b^{(0)}$ be the corresponding time with the tuned native seed settings. The objective is

$$J(\boldsymbol{\theta}) = \frac{1}{B} \sum_{b=1}^{B} \log \frac{T_b(\boldsymbol{\theta})}{T_b^{(0)}}. \tag{244}$$

Exponentiating the mean gives the geometric mean of the complete-time ratios, which explains the equal weight assigned to each request. A failed bounded reduction receives a cost of 300 seconds when forming the objective. This is a training penalty, not a successful measured reduction time. Successful recovery contributes its actual elapsed time.

The first planar-box fit selects

$$\boldsymbol{\theta}^* = (7/4,\ 1/4,\ 1/4,\ 3/4,\ 0,\ 1,\ -1/4). \tag{245}$$

The fixed control uses $(1,0,0,1,0,1,0)$, and the fitted vector allows the seed-distance bound to change with rank through its last component. Its second component changes the allowed width after a physical propagator is removed. Both choices retain the same exact Kira algebra and recovery procedure.

The second stage adds twelve training requests per graph, covering ranks two, three, and four with one or two powered auxiliary entries at two new rational profiles. It compares the preceding fitted boundaries, independently varied rank allowances, two fixed surrounding widths, and unfiltered Kira. Six regression models comprise trees, random forests, and extremely randomized forests with depths two or four; forests contain 32 trees and every leaf has at least two training examples [30, 48, 39]. Their targets are the measured log-time ratios for each boundary. Inputs include the physical family and index summaries of Appendix L, the individual numerator and extra denominator powers, and the number of powered numerator directions. Reference coefficients do not enter these inputs.

Ten separate validation requests per graph cover ranks two through four. Repeated complete reductions select a fixed sparse boundary for each graph and compare it with the cost predictors. If $\widehat{y}_j$ is a predicted log cost and f is the fixed choice, the model selects candidate j only when

$$\widehat{y}_j < \widehat{y}_f - \eta, \qquad \eta \in \{0, 0.03, 0.1\}. \tag{246}$$

Exponentiating the difference gives the equivalent predicted ratio condition $\exp(\widehat{y}_j - \widehat{y}_f) < \exp(-\eta)$. Validation selects the margin and enables the cost predictor only when its total recorded time is at least three percent lower than the separately selected fixed sparse choice. Otherwise the deployment retains that fixed choice. The validation ranking uses recorded candidate costs; the independent application comparisons also include model loading and forecasting. This gate is a selection rule, rather than a proof of a future saving.

P.3 Exactness, controls, and physical coverage

Removing seeds removes rows from an exact IBP system. Every remaining row still follows from an integration-by-parts identity, and exact row combinations preserve those identities.

The final coefficients therefore remain valid if the selected system reaches the prescribed basis. Kira checks basis membership, and the experiments compare every returned coefficient with a separately computed unfiltered reduction. An insufficient sparse system first repeats the same outer region without the sparse restriction; native and enlarged bounds follow when required. All repeated work enters the complete-time measurement.

The first independent test uses eight planar-box requests at new rational profiles. The second uses ten requests each for the planar and nonplanar boxes. Both retain symbolic d and a supplied master basis. The physical denominators follow Appendix A; each target has unit positive physical indices and total auxiliary numerator rank two, three, or four. A single-direction target puts the full rank on one auxiliary entry. A distributed target splits it into $\lceil R_T/2 \rceil$ and $\lfloor R_T/2 \rfloor$ powers on two entries. The chosen auxiliary entry changes between the two profiles. Some index patterns recur across training and testing, so these comparisons test new kinematics within known structures and target patterns. Master discovery precedes the repeated timer.

Let κ be an integer that labels a kinematic profile. In a common squared-momentum unit, the profiles are defined by

$$\begin{aligned}(s,t)_{\text{planar}} &= (-1009-12\kappa, -1021-14\kappa),\\ (s,t)_{\text{nonplanar}} &= (-113-18\kappa, -127-10\kappa).\end{aligned} \tag{247}$$

These are chosen kinematic inputs, with the graph-specific scalar-product conventions in Appendix A. The first search uses $\kappa = 811, 812$ for training, $\kappa = 813, 814$ for validation, and $\kappa = 821, 822$ for testing. The larger cost study uses $\kappa = 831, 832$, $\kappa = 841, 842$, and $\kappa = 851, 852$ for those three splits. A single-direction target places its numerator on auxiliary index $8+(\kappa \bmod 2)$. A distributed target places the ceiling power on $8 + ((\kappa+1) \bmod 2)$ and the floor power on the other auxiliary index. Together with the seven unit physical powers, these rules specify the tested integrals. The second reserved set contains single-direction ranks two, three, and four, and distributed ranks three and four at each profile.

Seven repetitions rotate method order on the same worker for every request, using one Kira thread. Different requests can use different cluster workers. The native control disables all learned decisions and keeps the tuned outer bounds. The fixed sparse control retains one validation-selected boundary per graph, including boundaries found during training, and performs no cost prediction. It therefore isolates the additional value of request-dependent prediction. The previous application uses the preceding published batch and equation policies. Failed references and incomplete calculations remain in the record and outside matched successful timing ratios. Paired repetition resampling describes timing variation on this fixed workload; it does not establish transfer to arbitrary graphs, masses, symbolic invariants, or higher loop orders.

Citations

[1] Stefan Weinzierl. Introduction to Feynman integrals, 2010. arXiv:1005.1855.

[2] Philipp Maierhöfer, Johann Usovitsch, and Peter Uwer. Kira—a Feynman integral reduction program. *Computer Physics Communications*, 230:99–112, 2018. arXiv:1705.05610.

[3] Stefan Weinzierl. *Feynman Integrals: A Comprehensive Treatment for Students and Researchers*. UNITEXT for Physics. Springer, 2022. arXiv:2201.03593; publisher DOI.

[4] K. G. Chetyrkin and F. V. Tkachov. Integration by parts: The algorithm to calculate β-functions in 4 loops. *Nuclear Physics B*, 192:159–204, 1981. publisher DOI.

[5] Thomas Gehrmann and Ettore Remiddi. Differential equations for two-loop four-point functions. *Nuclear Physics B*, 580:485–518, 2000. arXiv:hep-ph/9912329.

[6] A. V. Smirnov and A. V. Petukhov. The number of master integrals is finite, 2010. arXiv:1004.4199.

[7] Stefano Laporta. High-precision calculation of multiloop Feynman integrals by difference equations. *International Journal of Modern Physics A*, 15:5087–5159, 2000. arXiv:hep-ph/0102033.

[8] Philipp Maierhöfer and Johann Usovitsch. Kira 1.2 release notes, 2018. arXiv:1812.01491.

[9] Fabian Lange, Johann Usovitsch, and Zihao Wu. Kira 3: Integral reduction with efficient seeding and optimized equation selection, 2025. arXiv:2505.20197.

[10] Philipp Kant. Finding linear dependencies in integration-by-parts equations: A Monte Carlo approach. *Computer Physics Communications*, 185:1473–1476, 2014. arXiv:1309.7287.

[11] Jonas Klappert, Fabian Lange, Philipp Maierhöfer, and Johann Usovitsch. Integral reduction with Kira 2.0 and finite field methods. *Computer Physics Communications*, 266:108024, 2021. arXiv:2008.06494.

[12] Rushil Saraswat. Master-integral bases for sixteen one-loop and two-loop families. Accompanying family definitions, master bases and validation data, 2026.

[13] Rushil Saraswat. Target integrals constructed from master bases with seed and elimination records. Accompanying target integrals, source identities and validation data, 2026.

[14] Rushil Saraswat. Master bases and constructed targets for one thousand integral families. Accompanying family definitions, master bases, target reductions and validation data, 2026.

[15] Rushil Saraswat. Coupled vacuum master bases and reverse targets through twelve loops. Family definitions, source identities, grouped splits and independent checks, 2026.

[16] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1703.06114.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1706.03762.

[18] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. arXiv:1711.05101.

[19] Rushil Saraswat. Seed-selection model after twenty expanded-data passes: Held-out tests through twelve loops. Frozen predictions, metrics, plots and validation records, 2026.

[20] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.

[21] scikit-learn developers. Average precision score: Definition and treatment of score thresholds. scikit-learn 1.5 documentation, 2024.

[22] Rushil Saraswat. Predicted-seed sufficiency through exact master-span tests in Kira. Frozen protocol, selected identities, symbolic results and independent checks, 2026.

[23] Rushil Saraswat. Reduction-propagation model trained on integral families through twelve loops. Model, optimizer lineage, state features and training records, 2026.

[24] Rushil Saraswat. Expanded-data reduction model: Held-out next-step and complete-path tests. Frozen predictions, exact row replays, plots and validation, 2026.

[25] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[26] Rushil Saraswat. Exact reduction costs with learned seeds, learned rows and native Kira baselines. Frozen cases, exact reductions, repeated costs and independent checks, 2026.

[27] Rushil Saraswat. Frozen learned row choices inside the evolving symbolic matrix of Kira. Source changes, frozen protocol, exact row records and repeated comparisons, 2026.

[28] Rushil Saraswat. Learned seed selection and row choices in exact Kira reductions. Frozen policies, native and supplied-equation comparisons, and exact checks, 2026.

[29] Rushil Saraswat. Responsive fermat shutdown and learned seed selection in exact Kira reductions. Frozen code, paired solver comparisons, exact checks and complete timings, 2026.

[30] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Chapman & Hall, 1984. Publisher record.

[31] Rushil Saraswat. Complete native multiloop reductions, learned seed selection and row choices trained on measured Kira cost. Physical families, frozen models, complete timings, exact checks and reproducible comparison code, 2026.

[32] SQLite Developers. PRAGMA statements: Synchronization and journal modes. `https://www.sqlite.org/pragma.html`. Accessed 10 October 2026.

[33] Rushil Saraswat. Integrated learned seed inference and native row policies with complete multiloop cost tests. Frozen methods, complete workflows, retained failures and independent exact checks, 2026.

[34] Rushil Saraswat. Shared model calculations with unchanged native kira reductions. Frozen implementations, complete timing pairs and independent exact checks, 2026.

[35] Rushil Saraswat. Parallel seed evaluation and exact modular inverses in native kira. Frozen arithmetic implementations, timing pairs and independent exact checks, 2026.

[36] Rushil Saraswat. Target-dependent equation selection: native training, frozen tests and exact checks. GitHub repository, commit `63d87b1e863e`, 2026. Code, model, measurements and retained development results.

[37] Rushil Saraswat. Learned equation selection before native kira identity construction. Frozen models, native integration, complete timings and independent exact checks, 2026. GitHub repository, commit `2e970c1c448c`.

[38] Rushil Saraswat. Equation selection and exact coefficient reuse across loop orders. Frozen models, identities, complete timings and exact higher-loop checks, 2026. GitHub repository, commit `0d088383aff7`.

[39] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63:3–42, 2006.

[40] Rushil Saraswat. Learned batch equation reuse with shared exact checks and kira calls. Models, complete timings, exact checks and integration, 2026. Frozen data commit `61b97195b486`.

[41] Rushil Saraswat. Exact pre-reduction with learned ordering and a checked kira fallback. Frozen comparisons, exact outputs and implementation, 2026. Data commit `3c830972713b`.

[42] Rushil Saraswat. Exact polynomial reuse and persistent reduction workers. Frozen arithmetic comparisons, stream measurements and exact checks, 2026. Data commit `0dec744f90d6`.

[43] GNU MP developers. GNU MP manual: Integer functions. Integer arithmetic documentation. The recorded cluster implementation uses GMP 6.0.0.

[44] Rushil Saraswat. Exact integer expansion, lossless coefficient output and learned batch strategies. Frozen implementations, runtime-trained tree, full comparisons and exact checks, 2026. Data commit `94f9e0a9df8b`.

[45] Rushil Saraswat. Direct cumulative speedup comparison on identical restricted vacuum integrals. Frozen original and final implementations, identical inputs, timing records and exact audits, 2026.

[46] Rushil Saraswat. Native scattering reductions with symbolic kinematics and certified finite ibp paths. Physical definitions, frozen comparisons, exact symbolic checks and complete finite certificates, 2026.

[47] Richard Bellman. On the theory of dynamic programming. *Proceedings of the National Academy of Sciences*, 38(8):716–719, 1952.

[48] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.

[49] Roman N. Lee. LiteRed 1.4: a powerful tool for the reduction of the multiloop integrals. *Journal of Physics: Conference Series*, 523:012059, 2014.

[50] Zhuo-Yang Song, Tong-Zhi Yang, Qing-Hong Cao, Ming-xing Luo, and Hua Xing Zhu. Explainable AI-assisted optimization for Feynman integral reduction, 2025. arXiv:2502.09544.

[51] Justin Berman, Francois Charton, Andres Luna, Matthias Wilhelm, and Mao Zeng. Efficient ai-inspired reduction of feynman integrals via tube seeding, 2026. arXiv:2606.10698.

[52] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. Journal article.

[53] A. G. Grozin. Lectures on multiloop calculations. *International Journal of Modern Physics A*, 19:473–520, 2004.

[54] Leo Breiman. Bagging predictors. *Machine Learning*, 24:123–140, 1996. Publisher record.

[55] Christian Bogner and Stefan Weinzierl. Feynman graph polynomials, 2010. arXiv:1002.3458.